

R Course: Beginner to Expert

Sri Ram

2025-12-12

Table of contents

1	Welcome	14
2	Welcome to R for SAS Users	15
2.1	Course Overview	15
2.2	Learning Objectives	15
2.3	Course Structure	15
2.3.1	Module 1: R Fundamentals for SAS Users	15
2.3.2	Module 2: Data Import and Export	16
2.3.3	Module 3: Data Manipulation - The dplyr Way	16
2.3.4	Module 4: Statistical Analysis	16
2.3.5	Module 5: Data Visualization	17
2.3.6	Module 6: Reporting and Documentation	17
2.4	SAS to R Translation Guide	17
2.5	Prerequisites	18
2.6	Course Format	18
2.7	Getting Started	18
2.7.1	Required Software	18
3	Base R Functions & Apply Family	19
4	Introduction	20
5	Common Utility Functions	21
5.1	Statistical Functions	21
5.2	Quantile Function	22
5.3	Sequence and Repetition Functions	23
5.3.1	seq() - Generate Sequences	23
5.3.2	rep() - Repeat Values	23
6	The Apply Family of Functions	25
6.1	apply() - For Matrices and Arrays	25
6.2	lapply() - For Lists and Vectors	26
6.3	sapply() - Simplified lapply()	28
6.4	vapply() - Type-Safe sapply()	29
6.5	mapply() - Multivariate Apply	30
6.6	tapply() - Apply by Groups	31

7	Data Subsetting and Indexing	33
7.1	Vector Subsetting	33
7.2	Data Frame Subsetting	34
7.3	Matrix Subsetting	36
8	Advanced Function Applications	38
8.1	Custom Functions with Apply	38
8.2	Nested Apply Operations	38
9	Performance Considerations	40
9.1	Vectorization vs Apply vs Loops	40
10	Practical Examples	41
10.1	Data Analysis with Base R Functions	41
11	Exercises	43
11.1	Exercise 1: Matrix Operations	43
11.2	Exercise 2: IQR Calculation	43
11.3	Exercise 3: lapply vs sapply Comparison	43
11.4	Exercise 4: Custom Function with mapply	44
11.5	Exercise 5: Data Filtering and Aggregation	44
12	Summary	46
13	Data Types & Data Structures	47
14	Introduction	48
14.1	Learning Objectives	48
14.2	R's Memory Model: Foundation Concepts	48
15	Atomic Vectors: The Building Blocks	49
15.1	Understanding Atomic Vectors	49
15.1.1	The Six Atomic Types	49
15.1.2	Vector Properties and Inspection	50
15.1.3	Type Coercion Hierarchy	51
15.1.4	Special Values in R	52
15.2	Vector Creation Methods	53
15.2.1	Basic Creation Functions	53
15.2.2	Advanced Creation Patterns	54
15.3	Vector Indexing and Subsetting	55
15.3.1	Positive Integer Indexing	55
15.3.2	Negative Integer Indexing (Exclusion)	56
15.3.3	Logical Indexing (Conditional Selection)	57
15.3.4	Named Indexing	58

15.4	Vector Operations and Arithmetic	59
15.4.1	Scalar Operations (Broadcasting)	59
15.4.2	Vector-to-Vector Operations	60
15.4.3	Mathematical Functions	61
15.5	Set Operations and Comparisons	63
15.5.1	Set Operations	63
15.5.2	Advanced Comparison Operations	64
16	Factors: Categorical Data Excellence	67
16.1	Understanding Factors	67
16.1.1	Basic Factor Creation	67
16.1.2	Controlling Factor Levels	68
16.1.3	Factor Level Manipulation	69
16.1.4	Advanced Factor Operations with forcats	70
17	Lists: The Swiss Army Knife	72
17.1	Understanding Lists	72
17.1.1	Basic List Creation and Structure	72
17.1.2	List Indexing Methods	73
17.1.3	List Modification and Manipulation	75
17.1.4	List Apply Functions	77
17.1.5	Advanced List Operations with purrr	78
18	Matrices: Linear Algebra Powerhouse	81
18.1	Understanding Matrices	81
18.1.1	Matrix Creation Methods	81
18.1.2	Matrix Properties and Attributes	83
18.1.3	Matrix Indexing and Subsetting	85
18.1.4	Matrix Arithmetic Operations	87
18.1.5	Advanced Matrix Operations	89
18.1.6	Matrix Apply Operations	91
19	Data Frames: The Data Analysis Workhorse	93
19.1	Understanding Data Frames	93
19.1.1	Data Frame Creation	93
19.1.2	Data Frame Properties and Inspection	94
19.1.3	Data Frame Indexing and Access	97
19.1.4	Data Frame Filtering and Subsetting	98
19.1.5	Data Frame Modification	101
19.1.6	Advanced Data Frame Operations	102
19.1.7	Data Frame Aggregation and Grouping	104

20 Arrays: Multi-Dimensional Data	107
20.1 Understanding Arrays	107
20.1.1 Array Creation and Structure	107
20.1.2 Array Indexing and Slicing	108
20.1.3 Array Operations with Apply	110
21 Advanced Data Structures and Concepts	112
21.1 Environments: R's Scoping System	112
21.2 Object-Oriented Programming: S3 Classes	113
22 Performance Optimization and Best Practices	116
22.1 Memory Efficiency and Performance	116
22.1.1 Pre-allocation vs Growing Objects	116
22.1.2 Data Structure Selection for Performance	117
22.1.3 Factors vs Characters for Memory	118
22.2 Best Practices Summary	119
22.2.1 1. Data Structure Selection Guide	119
22.2.2 2. Common Pitfalls and Solutions	120
22.2.3 3. Debugging and Validation Functions	122
23 Summary and Quick Reference	125
23.1 Data Structure Comparison Table	125
23.2 Key Takeaways	125
23.2.1 For Beginners	125
24 Manipulating Vectors, Data Frames, and Lists	127
25 Chapter 1: Vector Operations in Pharmaceutical Context	128
25.1 1.1 Creating Vectors - The Foundation	128
25.1.1 Understanding Vector Naming in Clinical Data	128
25.2 1.2 Vector Indexing - Accessing Patient Data	129
25.3 1.3 Vector Modification - Updating Clinical Data	130
25.4 1.4 Arithmetic Operations - Statistical Calculations	130
25.5 1.5 The WHICH Function - Clinical Data Subsetting	132
25.6 1.6 The REP Function - Creating Repetitive Clinical Data	133
25.7 1.7 The SEQ Function - Creating Clinical Sequences	134
25.8 1.8 Sequence Helper Functions	135
25.9 1.9 Missing Data Handling - Critical in Clinical Studies	135
25.10 1.10 Vector Element Membership - Clinical Criteria Checking	136
25.11 1.11 String Operations - Essential for Clinical Data	136
25.12 1.12 Advanced String Manipulation for Clinical Data	137
25.13 1.13 Regular Expressions for Clinical Data Cleaning	138
25.14 1.14 String Replacement - Data Standardization	139

26 Chapter 2: Data Frame Manipulation with dplyr - Clinical Data Analysis	141
26.1 2.1 Creating Data Frames - Study Data Structure	141
26.2 2.2 Accessing Data Frame Elements - Patient Data Extraction	142
26.3 2.3 Modifying Data Frames - Clinical Data Updates	143
26.4 2.4 Data Frame Summary and Subsetting	144
27 Chapter 3: Reading SAS Datasets - SDTM and ADaM	146
28 Chapter 4: Advanced dplyr Operations - Clinical Data Manipulation	147
28.1 4.1 Column Selection - Choosing Clinical Variables	147
28.2 4.2 Dropping Columns - Data Cleanup	148
28.3 4.3 Filtering Rows - Patient Population Selection	149
28.4 4.4 Variable Renaming - Standardization	150
28.5 4.5 Sorting Data - Clinical Data Ordering	150
28.6 4.6 Creating New Variables - Derived Variables	151
28.7 4.7 Data Binding - Combining Datasets	152
28.8 4.8 Recoding Variables - Data Standardization	153
28.9 4.9 Factor Creation - Categorical Data Management	154
29 Chapter 5: Advanced Data Operations	155
29.1 5.1 Table Joins - Merging Clinical Datasets	155
29.1.1 Inner Join - Only Patients with Lab Results	156
29.1.2 Left Join - All Patients, Lab Data Where Available	156
29.1.3 Right Join - All Lab Results, Demographics Where Available	157
29.1.4 Full Join - All Records from Both Datasets	157
29.1.5 Semi Join - Patients Who Have Lab Results (No Lab Columns)	158
29.1.6 Anti Join - Patients Without Lab Results	158
29.2 5.2 Data Reshaping with tidyr - Wide vs. Long Format	159
29.3 5.3 Counting and Summarizing - Clinical Summary Tables	162
29.4 5.4 Distinct Values and Unique Records	165
30 Chapter 6: Best Practices and Advanced Techniques	167
30.1 6.1 Efficient Pipeline Construction	167
30.2 6.2 Error Handling and Data Validation	168
31 String Functions	171
32 Learning Objectives	172
33 Introduction	173
34 Setup	174

35 Basic String Functions	175
35.1 1. String Length and Basic Information	175
35.2 2. Case Conversion	176
35.3 3. String Concatenation	176
36 Substring Operations	178
36.1 1. Extracting Substrings	178
36.2 2. Finding and Replacing Text	179
37 Pattern Matching and Regular Expressions	181
37.1 1. Basic Pattern Detection	181
37.2 2. Advanced Regular Expressions	182
38 Working with coded terms	184
38.1 1. ICD-10 Code Processing	184
38.2 2. ATC Code Analysis	185
39 Data Cleaning Applications	187
39.1 1. Standardizing Drug Names	187
39.2 2. Parsing Dosage Information	188
40 Advanced String Operations	191
40.1 1. Working with Patient Identifiers	191
40.2 2. Processing Adverse Event Reports	192
40.3 3. Clinical Trial Data Processing Example	194
41 Performance Tips and Best Practices	197
41.1 1. Efficient String Operations	197
41.2 2. Error Handling in String Operations	198
42 Summary and Key Takeaways	200
42.1 Essential String Functions for Pharmaceutical Analysis	200
42.2 Best Practices	200
42.3 Common Pharmaceutical Applications	201
42.4 Next Steps	201
43 Comparing stringr and base R string functions	203
44 date and time functions	207
44.0.1 Date and Time Base Functions in R	207
44.0.2 Creating Date and Time Objects	207
44.0.3 Year Formats	207
44.0.4 Month Formats	207
44.0.5 Day Formats	208

44.0.6	Weekday Formats	208
44.0.7	Time Formats	208
44.1	1. Date & Time Basics in R	212
44.1.1	1.1 Core classes	212
44.1.2	1.2 Converting numerics to Date (origins matter)	212
44.1.3	1.3 Custom date formats	213
44.1.4	1.4 Datetime strings (ISO 8601 & time zones)	213
44.2	2. Lubridate: Fast, Friendly Parsing & Arithmetic	213
44.2.1	2.1 Intuitive parsers	213
44.2.2	2.2 Accessors and rounding	214
44.2.3	2.3 Timespans	214
44.2.4	3 ADaM-Style Partial Date Imputation (Start & End)	214
44.2.5	Start date (AESTDTC)	214
44.2.6	End date (AEENDTC)	215
44.2.7	Censoring by death / last alive date	215
44.2.8	Imputation flags	215
44.3	4. Durations & Analysis Variables	217
44.3.1	4.1 AE duration (AEDUR) in days	217
45	Using difftime (base) or as.numeric on Date differences	218
45.0.1	6.2 Handy lubridate parsers (subset)	218
46	Reading SAS Datasets and Working with XPT Files	219
47	Reading SAS Datasets and Working with XPT Files	220
47.1	Introduction	220
47.2	Required Packages	220
47.3	Reading SAS Datasets	220
47.3.1	Reading SAS7BDAT Files	220
47.3.2	Reading XPT (Transport) Files	221
47.3.3	Handling Different SAS Formats	221
47.4	Working with Labels and Attributes	221
47.4.1	Understanding SAS Labels in R	221
47.4.2	Adding and Modifying Labels	222
47.4.3	Working with Formats	223
47.5	Writing XPT Files	223
47.5.1	Basic XPT File Creation	223
47.5.2	Advanced XPT Writing with SASxport	224
47.5.3	Handling Variable Names and Lengths	224
47.6	Adding Dataset Attributes	225
47.6.1	Dataset-Level Attributes	225
47.6.2	Variable-Level Attributes	225

47.7	Quality Checks and Validation	227
47.7.1	Validating XPT Files	227
47.7.2	Creating a Data Dictionary	228
47.8	Best Practices	229
47.8.1	Recommended Workflow	229
47.8.2	Performance Tips	229
47.9	Summary	230
48	Base R Functions & Apply Family	231
49	Common Utilities	232
50	Apply Family	233
51	Subsetting Essentials	234
52	Exercises	235
53	Custom Functions & Validation	236
54	Introduction to Functions in R	237
55	Function Fundamentals	238
55.1	Anatomy of a Function	238
55.2	Basic Function Syntax	239
55.2.1	Simple Examples	239
55.3	Function Arguments and Parameters	240
55.3.1	Default Arguments	240
55.3.2	The ... (Dots) Parameter	241
55.4	Return Values	241
55.4.1	Explicit vs Implicit Returns	241
55.4.2	Returning Multiple Values	243
56	Intermediate Function Concepts	245
56.1	Function Scope and Environments	245
56.1.1	Lexical Scoping	246
56.2	Input Validation and Error Handling	246
56.2.1	Basic Input Validation	246
56.2.2	Advanced Error Handling with tryCatch	248
57	Advanced Function Concepts	250
57.1	Higher-Order Functions	250
57.2	Anonymous Functions and Lambda Functions	251

57.3	Working with Data Frames	252
57.3.1	Single Input, Single Output Functions	252
57.3.2	Multiple Input, Multiple Output Functions	253
58	Tidy Evaluation and Advanced dplyr Usage	257
58.1	Understanding <code>{{}}</code> , <code>!!</code> , and <code>!!!</code>	257
58.2	Advanced Data Processing Function	259
59	Advanced Topics	263
59.1	Function Factories and Closures	263
59.2	Memoization for Performance	264
59.3	Method Dispatch and S3 Classes	265
60	Testing and Documentation	268
60.1	Unit Testing with <code>testthat</code>	268
60.2	Documentation with <code>roxygen2</code>	269
61	Performance and Optimization	273
61.1	Vectorization vs Loops	273
61.2	Memory Management	274
62	Best Practices and Common Patterns	276
62.1	Function Design Principles	276
62.2	Error Handling Patterns	279
63	Practical Exercises	282
63.1	Exercise 1: Data Validation Function	282
63.2	Exercise 2: Advanced Statistical Function	286
64	Summary and Best Practices	291
64.1	Key Takeaways	291
64.2	Function Writing Checklist	291
65	R Package Development	292
65.1	1. Prerequisites	292
65.2	2. Create the Package Skeleton	292
65.2.1	2.1 Create a new package directory	292
65.2.2	2.2 Initialize Git and a README	293
65.2.3	2.3 The minimal structure (what you get)	293
65.3	3. DESCRIPTION: Package Metadata	293
65.4	4. NAMESPACE: Exports and Imports (auto-generated)	294
65.5	5. Write Functions + <code>roxygen2</code> Headers	294
65.6	6. Generate Documentation	295

65.7	7. Add Data, Vignettes, Tests (Optional but Recommended)	296
65.7.1	7.1 Package data	296
65.7.2	7.2 Vignettes	296
65.7.3	7.3 Tests with testthat	296
65.8	8. Manage Dependencies	297
65.9	9. Check Your Package Thoroughly	297
65.10	10. Build & Install Locally	298
65.11	11. Continuous Integration (optional)	298
65.12	12. Package Website (optional)	298
65.13	13. Release Checklist	299
65.13.1	Appendix A — Common Files (at a glance)	299
65.13.2	Appendix B — Troubleshooting Snippets	299
66	Git in RStudio (Setup & Auth)	301
66.1	One-Time Setup	301
66.2	Initialize Git for the Current Project	301
66.3	Connect to GitHub	301
66.4	Typical Workflow	301
66.5	Remove Git from a Project (macOS/RStudio)	302
67	Creating ADaM: ADSL from SDTM-like Inputs	303
68	ADVS program	311
69	TLFs: Table, Figure, Listing	318
69.1	Create demographic table from ADSL	318
69.2	Create adverse event table from ADAE	324
70	ggplot2 in R with CDISC ADaM Examples	327
71	Introduction	328
72	Creating Simulated ADaM-Style Data	329
72.1	ADSL: Subject-Level Data	329
72.2	ADLB: Laboratory Data (e.g. ALT)	329
72.3	ADTTE: Time-to-Event Data (e.g. PFS)	329
73	Grammar of Graphics with ADaM	331
74	Basic Plots from ADSL (Beginner Level)	332
74.1	Scatterplot: AGE vs BMIBL	332
74.1.1	Mapping vs Setting Aesthetics	332
74.2	Bar Plot: Subject Counts per Arm	334
74.3	Histogram and Density: Age Distribution	336

74.4	Boxplot: BMIBL by ARM	338
75	Longitudinal Plots from ADLB (Intermediate)	341
75.1	Line Plot: Mean ALT Over Time by ARM	341
75.2	Individual Profiles (Spaghetti Plot)	343
76	Faceting with ADaM Data	344
76.1	Example: Facet ALT by Sex	344
77	Scales, Labels, and Themes in ADaM Context	346
77.1	Changing Axis Labels and Titles	346
77.2	Controlling Color Scales	347
77.3	Transforming Scales (e.g., log10 ALT)	348
78	Themes and Publication-Style Graphics	350
78.1	Using Built-In Themes	350
78.2	Custom Theme for Clinical Reports	352
79	Coordinates and Positions	354
79.1	Flipping Coordinates: Boxplot of BMI by Sex	354
79.2	Stacked vs Dodged Bar Plots: TRTA by SEX	355
80	Adding Statistical Layers	357
80.1	Linear Regression: BMIBL vs AGE	357
80.2	Summaries: Mean ALT $\pm$ SD per Visit and TRTA	358
81	Tidy Workflow: dplyr + ggplot2 with ADaM	359
82	Advanced: Simple Kaplan–Meier Plot from ADTTE	361
83	Advanced: Annotations and Secondary Axes	363
83.1	Annotations on an ADaM Plot	363
83.2	Secondary Axis (Use with Care)	364
84	Writing Reusable Plot Functions for ADaM	366
85	Saving Plots for Reports	368
86	Introduction	369
87	Figure 1 – Bar Graph with Individual Lesion Lines	370
87.1	1.1 Create bar-graph data	370
87.2	1.2 Reshape to long format for lines	371
87.3	1.3 Build the bar + line plot	372

88	Figure 2 – Waterfall Plot of Best Percentage Change from Baseline	375
88.1	2.1 Create waterfall data	375
88.2	2.2 Sort patients by change	376
88.3	2.3 Build the waterfall plot	377
89	Figure 3 – Swimmer Plot: Duration of Response	380
89.1	3.1 Create swimmer data	380
89.2	3.2 Build the swimmer plot	381
90	Figure 4 – Spider Plot: Tumor Trajectories Over Time	384
90.1	4.1 Create spider data	384
90.2	4.2 Build the spider plot	385
91	Figure 5 – Kaplan–Meier Survival Curve	387
91.1	5.1 Create KM data	387
91.2	5.2 Fit the KM model and plot	388
92	Figure 6 – Forest Plot for Subgroup Hazard Ratios	390
92.1	6.1 Load example data and prepare	390
92.2	6.2 Define forest theme and build the plot	392
93	Summary and Next Steps	395
94	Capstone: End-to-End Mini Workflow	396
94.1	Parameters	396
94.2	1) Read (or Synthesize) SDTM	396
94.3	2) Derive ADSL (Minimal Demo)	397
94.4	3) TLFs	398
94.5	4) Save Outputs	399
95	Appendix: Tips, Profiles, .libPaths	400
95.1	Useful Profiles	400
95.2	Custom Library Paths	400
95.3	Format vs formatC (quick recap)	400
95.4	POSIXct vs POSIXlt	401
95.5	Recommended Packages	401
95.6	Short Glossary	401

1 Welcome

2 Welcome to R for SAS Users

2.1 Course Overview

This comprehensive course is designed specifically for **SAS programmers** who want to learn R programming. We'll leverage your existing knowledge of data manipulation, statistical analysis, and programming concepts to help you become proficient in R.

2.2 Learning Objectives

By the end of this course, you will be able to:

- **Understand R fundamentals:** Master R syntax, data types, and programming concepts
- **Data manipulation:** Perform complex data transformations using `dplyr` and `base R`
- **Statistical analysis:** Apply statistical methods and create models in R
- **Data visualization:** Create compelling visualizations using `ggplot2`
- **Reporting:** Generate dynamic reports with R Markdown and Quarto
- **Bridge knowledge:** Map your SAS skills to equivalent R functions and workflows
- **Best practices:** Write clean, efficient, and reproducible R code

2.3 Course Structure

2.3.1 Module 1: R Fundamentals for SAS Users

- **Getting Started**
 - R and RStudio installation and setup
 - Understanding the R environment vs SAS environment
 - Package management (CRAN vs SAS procedures)
- **Basic R Syntax**
 - R syntax compared to SAS syntax
 - Variables and assignment operators
 - Data types and structures
 - Functions and help system

2.3.2 Module 2: Data Import and Export

- **Reading Data**
 - CSV, Excel, and database connections
 - SAS dataset import (`haven` package)
 - Data import best practices
- **Data Export**
 - Writing to various formats
 - Creating SAS-compatible datasets

2.3.3 Module 3: Data Manipulation - The dplyr Way

- **Core dplyr Functions**
 - `select()` vs KEEP/DROP statements
 - `filter()` vs WHERE clause
 - `mutate()` vs assignment statements
 - `summarise()` vs PROC MEANS
 - `group_by()` vs BY statement
- **Advanced Data Manipulation**
 - Joins equivalent to PROC SQL joins
 - Reshaping data (`tidyr` vs PROC TRANSPOSE)
 - String manipulation (`stringr` vs SAS string functions)

2.3.4 Module 4: Statistical Analysis

- **Descriptive Statistics**
 - Summary statistics (equivalent to PROC UNIVARIATE)
 - Frequency tables (equivalent to PROC FREQ)
 - Cross-tabulations and chi-square tests
- **Statistical Modeling**
 - Linear regression (equivalent to PROC REG)
 - Logistic regression (equivalent to PROC LOGISTIC)
 - ANOVA (equivalent to PROC ANOVA)
 - Mixed models and advanced techniques

2.3.5 Module 5: Data Visualization

- **Base R Graphics**
 - Basic plots and customization
 - Comparison with SAS/GRAPH
- **ggplot2 - Grammar of Graphics**
 - Understanding the layered approach
 - Creating publication-ready plots
 - Advanced visualization techniques

2.3.6 Module 6: Reporting and Documentation

- **R Markdown and Quarto**
 - Creating dynamic reports (equivalent to ODS output)
 - Integrating code, results, and narrative
 - Output formats: HTML, PDF, Word
- **Reproducible Research**
 - Project organization
 - Version control with Git
 - Best practices for code documentation

2.4 SAS to R Translation Guide

SAS Concept	R Equivalent	Package
DATA step	<code>dplyr::mutate()</code>	dplyr
PROC SQL	dplyr verbs	dplyr
PROC MEANS	<code>dplyr::summarise()</code>	dplyr
PROC FREQ	<code>table(), xtabs()</code>	base R
PROC REG	<code>lm()</code>	base R
PROC LOGISTIC	<code>glm()</code>	base R
PROC TRANSPOSE	<code>tidyr::pivot_*()</code>	tidyr
ODS OUTPUT	R Markdown/Quarto	rmarkdown/quarto

2.5 Prerequisites

- **SAS Experience:** Familiarity with SAS programming, data steps, and procedures
- **Statistical Knowledge:** Basic understanding of statistical concepts
- **Programming Basics:** Understanding of programming logic and data structures

2.6 Course Format

- **Interactive Learning:** Hands-on exercises with real datasets
- **Comparative Examples:** Side-by-side SAS and R code comparisons
- **Practical Projects:** Real-world scenarios mimicking typical SAS workflows
- **Reference Materials:** Quick reference guides and cheat sheets

2.7 Getting Started

2.7.1 Required Software

1. **R** (version 4.3+): Download from [CRAN](#)
2. **RStudio**: Download from [Posit](#)
3. **Essential Packages:** We'll install these as needed

```
install.packages(c("tidyverse", "haven", "readxl", "rmarkdown"))
```

Tip: If you don't have sample SDTM/ADaM data yet, the chapters generate **small synthetic data** as a fallback so everything runs end-to-end. ## contact For questions or feedback, reach out to **r2sas2025@gmail.com**

3 Base R Functions & Apply Family

4 Introduction

Base R provides a rich set of built-in functions for data manipulation, statistical calculations, and programming. The apply family of functions is particularly powerful for applying operations across different dimensions of data structures. This chapter covers essential base R functions and the apply family in detail.

5 Common Utility Functions

Base R includes numerous utility functions for statistical calculations, data generation, and manipulation.

5.1 Statistical Functions

```
# Create sample data  
x <- 1:10  
print(x)
```

```
[1] 1 2 3 4 5 6 7 8 9 10
```

```
# Basic statistical functions  
sum(x)      # Sum of all elements
```

```
[1] 55
```

```
mean(x)     # Arithmetic mean
```

```
[1] 5.5
```

```
median(x)   # Median value
```

```
[1] 5.5
```

```
sd(x)       # Standard deviation
```

```
[1] 3.02765
```

```
var(x)          # Variance
```

```
[1] 9.166667
```

```
min(x)          # Minimum value
```

```
[1] 1
```

```
max(x)          # Maximum value
```

```
[1] 10
```

```
range(x)        # Returns min and max
```

```
[1] 1 10
```

```
length(x)       # Number of elements
```

```
[1] 10
```

5.2 Quantile Function

The `quantile()` function calculates sample quantiles corresponding to given probabilities:

```
x <- 1:10  
quantile(x)          # Default quartiles (0%, 25%, 50%, 75%, 100%)
```

```
  0%   25%   50%   75%  100%  
1.00  3.25  5.50  7.75 10.00
```

```
quantile(x, probs = 0.9)  # 90th percentile
```

```
90%  
9.1
```

```
quantile(x, probs = c(0.1, 0.5, 0.9)) # Custom percentiles
```

```
10% 50% 90%  
1.9 5.5 9.1
```

5.3 Sequence and Repetition Functions

5.3.1 seq() - Generate Sequences

The `seq()` function creates sequences of numbers:

```
# Different ways to create sequences  
seq(0, 1, by = 0.1) # From 0 to 1, increment by 0.1
```

```
[1] 0.0 0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9 1.0
```

```
seq(0, 1, length.out = 11) # From 0 to 1, 11 equally spaced values
```

```
[1] 0.0 0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9 1.0
```

```
seq(from = 5, to = 50, by = 5) # From 5 to 50, increment by 5
```

```
[1] 5 10 15 20 25 30 35 40 45 50
```

```
seq_along(letters[1:5]) # Sequence along the length of an object
```

```
[1] 1 2 3 4 5
```

5.3.2 rep() - Repeat Values

The `rep()` function repeats values:

```
# Different repetition patterns  
rep(5, times = 3) # Repeat 5 three times
```

```
[1] 5 5 5
```

```
rep(c(1, 2), times = 3)      # Repeat vector three times
```

```
[1] 1 2 1 2 1 2
```

```
rep(c(1, 2), each = 3)      # Repeat each element three times
```

```
[1] 1 1 1 2 2 2
```

```
rep(1:3, length.out = 10)   # Repeat to reach specified length
```

```
[1] 1 2 3 1 2 3 1 2 3 1
```


6 The Apply Family of Functions

The apply family provides efficient ways to apply functions across different dimensions of data structures. These functions are alternatives to writing explicit loops.

6.1 `apply()` - For Matrices and Arrays

The `apply()` function applies a function over the margins of an array or matrix.

Syntax: `apply(X, MARGIN, FUN, ...)` - X: Array or matrix - MARGIN: 1 = rows, 2 = columns, c(1,2) = both - FUN: Function to apply - ...: Additional arguments to FUN

```
# Create a sample matrix
m <- matrix(1:12, nrow = 3, ncol = 4)
print(m)
```

```
      [,1] [,2] [,3] [,4]
[1,]    1    4    7   10
[2,]    2    5    8   11
[3,]    3    6    9   12
```

```
# Apply functions across rows (MARGIN = 1)
apply(m, 1, sum)      # Row sums
```

```
[1] 22 26 30
```

```
apply(m, 1, mean)     # Row means
```

```
[1] 5.5 6.5 7.5
```

```
apply(m, 1, max)      # Row maxima
```

```
[1] 10 11 12
```

```
# Apply functions across columns (MARGIN = 2)
apply(m, 2, sum)      # Column sums
```

```
[1]  6 15 24 33
```

```
apply(m, 2, mean)     # Column means
```

```
[1]  2  5  8 11
```

```
apply(m, 2, sd)       # Column standard deviations
```

```
[1] 1 1 1 1
```

```
# Using built-in optimized functions (when available)
rowSums(m)            # Equivalent to apply(m, 1, sum) but faster
```

```
[1] 22 26 30
```

```
colSums(m)            # Equivalent to apply(m, 2, sum) but faster
```

```
[1]  6 15 24 33
```

```
rowMeans(m)           # Equivalent to apply(m, 1, mean) but faster
```

```
[1] 5.5 6.5 7.5
```

```
colMeans(m)           # Equivalent to apply(m, 2, mean) but faster
```

```
[1]  2  5  8 11
```

6.2 lapply() - For Lists and Vectors

The `lapply()` function applies a function to each element of a list or vector and returns a list.

Syntax: `lapply(X, FUN, ...)`

```
# Create sample data
lst <- list(a = 1:5, b = 6:10, c = 11:15)
print(lst)
```

```
$a
[1] 1 2 3 4 5
```

```
$b
[1] 6 7 8 9 10
```

```
$c
[1] 11 12 13 14 15
```

```
# Apply function to each list element
lapply(lst, sum)      # Sum of each element
```

```
$a
[1] 15
```

```
$b
[1] 40
```

```
$c
[1] 65
```

```
lapply(lst, mean)     # Mean of each element
```

```
$a
[1] 3
```

```
$b
[1] 8
```

```
$c
[1] 13
```

```
lapply(lst, length)   # Length of each element
```

```
$a  
[1] 5
```

```
$b  
[1] 5
```

```
$c  
[1] 5
```

```
# Using lapply with custom functions  
lapply(lst, function(x) x^2)      # Square each element
```

```
$a  
[1] 1 4 9 16 25
```

```
$b  
[1] 36 49 64 81 100
```

```
$c  
[1] 121 144 169 196 225
```

```
lapply(lst, function(x) x[x > 8])      # Filter elements > 8
```

```
$a  
integer(0)
```

```
$b  
[1] 9 10
```

```
$c  
[1] 11 12 13 14 15
```

6.3 sapply() - Simplified lapply()

The `sapply()` function is similar to `lapply()` but attempts to simplify the result to a vector or matrix when possible.

```
# Same data as above
lst <- list(a = 1:5, b = 6:10, c = 11:15)

# sapply returns simplified results
sapply(lst, sum)      # Returns named vector instead of list
```

```
  a  b  c
15 40 65
```

```
sapply(lst, mean)     # Returns named vector
```

```
  a  b  c
 3  8 13
```

```
sapply(lst, range)    # Returns matrix when each result has same length
```

```
      a  b  c
[1,] 1  6 11
[2,] 5 10 15
```

```
# Compare lapply vs sapply
result_lapply <- lapply(lst, mean)
result_sapply <- sapply(lst, mean)
class(result_lapply) # List
```

```
[1] "list"
```

```
class(result_sapply) # Named numeric vector
```

```
[1] "numeric"
```

6.4 vapply() - Type-Safe sapply()

The `vapply()` function is like `sapply()` but with explicit specification of the return type, making it safer and faster.

Syntax: `vapply(X, FUN, FUN.VALUE, ...)`

```
# vapply with explicit return type specification
lst <- list(a = 1:5, b = 6:10, c = 11:15)

# Specify that each function call returns a single numeric value
vapply(lst, mean, FUN.VALUE = numeric(1))
```

```
  a  b  c
3  8 13
```

```
# Specify that each function call returns two numeric values
vapply(lst, range, FUN.VALUE = numeric(2))
```

```
      a  b  c
[1,] 1  6 11
[2,] 5 10 15
```

```
# This would throw an error if the function doesn't return the expected type
# vapply(lst, mean, FUN.VALUE = character(1)) # Error!
```

6.5 mapply() - Multivariate Apply

The `mapply()` function applies a function to multiple arguments element-wise.

Syntax: `mapply(FUN, ..., MoreArgs = NULL, SIMPLIFY = TRUE)`

```
# Apply function to multiple vectors element-wise
vec1 <- 1:3
vec2 <- 10:12
vec3 <- 100:102

# Add corresponding elements from two vectors
mapply(sum, vec1, vec2)
```

```
[1] 11 13 15
```

```
# Equivalent to: c(sum(1, 10), sum(2, 11), sum(3, 12))

# Add corresponding elements from three vectors
mapply(sum, vec1, vec2, vec3)
```

```
[1] 111 114 117
```

```
# Using mapply with custom functions  
mapply(function(x, y) x^y, vec1, c(2, 3, 4))
```

```
[1] 1 8 81
```

```
# mapply with lists  
list1 <- list(a = 1:3, b = 4:6)  
list2 <- list(x = 7:9, y = 10:12)  
mapply(function(a, b) sum(a) + sum(b), list1, list2)
```

```
a b  
30 48
```

6.6 tapply() - Apply by Groups

The `tapply()` function applies a function to subsets of a vector based on grouping factors.

Syntax: `tapply(X, INDEX, FUN, ...)`

```
# Sample data with grouping  
values <- c(1, 2, 3, 4, 5, 6, 7, 8, 9, 10)  
groups <- c("A", "B", "A", "B", "A", "B", "A", "B", "A", "B")
```

```
# Apply function by groups  
tapply(values, groups, mean) # Mean by group
```

```
A B  
5 6
```

```
tapply(values, groups, sum) # Sum by group
```

```
A B  
25 30
```

```
tapply(values, groups, length) # Count by group
```

A B
5 5

```
# Using tapply with built-in datasets  
tapply(mtcars$mpg, mtcars$cyl, mean) # Mean mpg by cylinder count
```

	4	6	8
	26.66364	19.74286	15.10000

```
tapply(mtcars$hp, mtcars$gear, median) # Median horsepower by gear count
```

	3	4	5
	180	94	175

7 Data Subsetting and Indexing

R provides powerful subsetting capabilities for extracting specific elements from data structures.

7.1 Vector Subsetting

```
# Create sample vector
x <- c(10, 20, 30, 40, 50)
names(x) <- letters[1:5]

# Subsetting by position
x[1]          # First element
```

```
a
10
```

```
x[c(1, 3, 5)]  # Elements 1, 3, and 5
```

```
a c e
10 30 50
```

```
x[-2]          # All except second element
```

```
a c d e
10 30 40 50
```

```
x[1:3]         # First three elements
```

```
a b c
10 20 30
```

```
# Subsetting by name
```

```
x["a"]          # Element named 'a'
```

```
a  
10
```

```
x[c("a", "c")]  # Elements named 'a' and 'c'
```

```
a  c  
10 30
```

```
# Logical subsetting
```

```
x[x > 25]       # Elements greater than 25
```

```
c  d  e  
30 40 50
```

```
x[x %in% c(20, 40)] # Elements equal to 20 or 40
```

```
b  d  
20 40
```

7.2 Data Frame Subsetting

```
# Create sample data frame
```

```
df <- data.frame(  
  id = 1:5,  
  name = c("Alice", "Bob", "Charlie", "Diana", "Eve"),  
  age = c(25, 30, 35, 28, 32),  
  salary = c(50000, 60000, 70000, 55000, 65000)  
)  
print(df)
```

	id	name	age	salary
1	1	Alice	25	50000
2	2	Bob	30	60000
3	3	Charlie	35	70000
4	4	Diana	28	55000
5	5	Eve	32	65000

```
# Subsetting columns
```

```
df$name          # Single column by name (returns vector)
```

```
[1] "Alice"  "Bob"    "Charlie" "Diana"  "Eve"
```

```
df["name"]       # Single column by name (returns data frame)
```

```
  name
1  Alice
2    Bob
3 Charlie
4  Diana
5    Eve
```

```
df[c("name", "age")] # Multiple columns
```

```
  name age
1  Alice 25
2    Bob 30
3 Charlie 35
4  Diana 28
5    Eve 32
```

```
# Subsetting rows
```

```
df[1, ]          # First row
```

```
  id name age salary
1  1 Alice 25 50000
```

```
df[c(1, 3), ]    # Rows 1 and 3
```

```
  id name age salary
1  1 Alice 25 50000
3  3 Charlie 35 70000
```

```
df[df$age > 30, ] # Rows where age > 30
```

	id	name	age	salary
3	3	Charlie	35	70000
5	5	Eve	32	65000

```
# Subsetting specific elements
df[1, "name"]      # Specific cell
```

```
[1] "Alice"
```

```
df[2, 3]           # Row 2, column 3
```

```
[1] 30
```

```
df[df$salary > 60000, c("name", "salary")] # Conditional row and column selection
```

	name	salary
3	Charlie	70000
5	Eve	65000

7.3 Matrix Subsetting

```
# Create sample matrix
m <- matrix(1:20, nrow = 4, ncol = 5)
rownames(m) <- paste0("row", 1:4)
colnames(m) <- paste0("col", 1:5)
print(m)
```

	col1	col2	col3	col4	col5
row1	1	5	9	13	17
row2	2	6	10	14	18
row3	3	7	11	15	19
row4	4	8	12	16	20

```
# Subsetting by position
m[1, 2]           # Element in row 1, column 2
```

```
[1] 5
```

```
m[1, ]          # Entire first row
```

```
col1 col2 col3 col4 col5
  1     5     9    13    17
```

```
m[, 3]          # Entire third column
```

```
row1 row2 row3 row4
   9    10    11    12
```

```
m[1:2, 3:4]     # Submatrix: rows 1-2, columns 3-4
```

```
      col3 col4
row1     9   13
row2    10   14
```

```
# Subsetting by name
```

```
m["row1", "col3"] # Element by row and column names
```

```
[1] 9
```

```
m[c("row1", "row3"), ] # Specific rows by name
```

```
      col1 col2 col3 col4 col5
row1     1     5     9    13    17
row3     3     7    11    15    19
```

8 Advanced Function Applications

8.1 Custom Functions with Apply

```
# Define custom functions for use with apply family
# Function to calculate coefficient of variation
cv <- function(x) {
  sd(x) / mean(x) * 100
}

# Function to calculate range
my_range <- function(x) {
  max(x) - min(x)
}

# Apply custom functions
sample_data <- matrix(rnorm(20), nrow = 4)
apply(sample_data, 2, cv)      # CV for each column
```

```
[1] 7640.7659 135.4892 -507.6610 103.6749 -206.4065
```

```
apply(sample_data, 1, my_range) # Range for each row
```

```
[1] 3.527973 3.556245 2.462648 3.234055
```

8.2 Nested Apply Operations

```
# Create nested list structure
nested_list <- list(
  group1 = list(a = 1:5, b = 6:10),
  group2 = list(c = 11:15, d = 16:20)
```

```
)

# Apply function to nested structure
lapply(nested_list, function(group) {
  sapply(group, mean)
})
```

```
$group1
a b
3 8
```

```
$group2
c d
13 18
```

```
# Alternative using nested apply
lapply(nested_list, function(group) {
  lapply(group, summary)
})
```

```
$group1
$group1$a
  Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
    1         2      3      3      4      5
```

```
$group1$b
  Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
    6         7      8      8      9     10
```

```
$group2
$group2$c
  Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
   11      12      13     13     14     15
```

```
$group2$d
  Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
   16      17      18     18     19     20
```

9 Performance Considerations

9.1 Vectorization vs Apply vs Loops

```
# Create large dataset for performance comparison
n <- 10000
x <- rnorm(n)
m <- matrix(rnorm(n * 100), nrow = n)

# Timing different approaches (conceptual - actual timing may vary)
# Vectorized (fastest)
result1 <- colMeans(m)

# Apply family (fast)
result2 <- apply(m, 2, mean)

# Explicit loop (slowest)
result3 <- numeric(ncol(m))
for(i in 1:ncol(m)) {
  result3[i] <- mean(m[, i])
}

# All results should be equivalent
all.equal(result1, result2)
```

```
[1] TRUE
```

```
all.equal(result1, result3)
```

```
[1] TRUE
```


10 Practical Examples

10.1 Data Analysis with Base R Functions

```
# Using mtcars dataset for practical examples
data(mtcars)
head(mtcars)
```

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
Mazda RX4	21.0	6	160	110	3.90	2.620	16.46	0	1	4	4
Mazda RX4 Wag	21.0	6	160	110	3.90	2.875	17.02	0	1	4	4
Datsun 710	22.8	4	108	93	3.85	2.320	18.61	1	1	4	1
Hornet 4 Drive	21.4	6	258	110	3.08	3.215	19.44	1	0	3	1
Hornet Sportabout	18.7	8	360	175	3.15	3.440	17.02	0	0	3	2
Valiant	18.1	6	225	105	2.76	3.460	20.22	1	0	3	1

```
# Summary statistics by group using tapply
tapply(mtcars$mpg, mtcars$cyl, summary)
```

```
$`4`
  Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
 21.40  22.80   26.00   26.66  30.40   33.90
```

```
$`6`
  Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
 17.80  18.65   19.70   19.74  21.00   21.40
```

```
$`8`
  Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
 10.40  14.40   15.20   15.10  16.25   19.20
```

```
# Multiple statistics using sapply
stats_list <- list(
  mean_mpg = mean(mtcars$mpg),
  median_hp = median(mtcars$hp),
  sd_wt = sd(mtcars$wt)
)
sapply(stats_list, round, digits = 2)
```

```
mean_mpg median_hp    sd_wt
    20.09    123.00     0.98
```

```
# Apply functions to multiple columns
numeric_cols <- sapply(mtcars, is.numeric)
sapply(mtcars[numeric_cols], function(x) c(mean = mean(x), sd = sd(x)))
```

```
      mpg      cyl      disp      hp      drat      wt      qsec
mean 20.090625 6.187500 230.7219 146.68750 3.5965625 3.2172500 17.848750
sd   6.026948 1.785922 123.9387  68.56287 0.5346787 0.9784574  1.786943

      vs      am      gear      carb
mean 0.4375000 0.4062500 3.6875000 2.8125
sd   0.5040161 0.4989909 0.7378041 1.6152
```

11 Exercises

11.1 Exercise 1: Matrix Operations

Create a 4x6 matrix of random numbers and use `apply()` to: 1. Calculate the maximum value in each row 2. Calculate the standard deviation of each column 3. Find the median of each row

```
# Solution template:
# set.seed(123)
# m <- matrix(rnorm(24), nrow = 4)
# apply(m, 1, max)      # Row maxima
# apply(m, 2, sd)       # Column standard deviations
# apply(m, 1, median)   # Row medians
```

11.2 Exercise 2: IQR Calculation

Write a base R snippet to compute the Interquartile Range (IQR) for each numeric column of the `mtcars` dataset.

```
# Solution template:
# sapply(mtcars, IQR)
# # or
# apply(mtcars, 2, IQR)
```

11.3 Exercise 3: lapply vs sapply Comparison

Create a list with mixed data types and compare the behavior of `lapply()` vs `sapply()`.

```
# Solution template:
# mixed_list <- list(
#   numbers = 1:5,
#   characters = letters[1:3],
```

```
# logicals = c(TRUE, FALSE, TRUE),
# matrix = matrix(1:4, nrow = 2)
# )
#
# lapply(mixed_list, class) # Returns list
# sapply(mixed_list, class) # Returns character vector
#
# lapply(mixed_list, length) # Returns list
# sapply(mixed_list, length) # Returns named numeric vector
```

11.4 Exercise 4: Custom Function with mapply

Use `mapply()` to calculate the area of rectangles given vectors of lengths and widths.

```
# Solution template:
# lengths <- c(2, 4, 6, 8)
# widths <- c(3, 5, 7, 9)
# mapply(function(l, w) l * w, lengths, widths)
# # or simply:
# mapply(`*`, lengths, widths)
```

11.5 Exercise 5: Data Filtering and Aggregation

Using the built-in `iris` dataset: 1. Filter rows where `Sepal.Length > 6` 2. Calculate mean values for each numeric column by Species 3. Find the species with the highest average `Petal.Width`

```
# Solution template:
# # 1. Filter
# filtered_iris <- iris[iris$Sepal.Length > 6, ]
#
# # 2. Mean by species
# numeric_vars <- c("Sepal.Length", "Sepal.Width", "Petal.Length", "Petal.Width")
# species_means <- sapply(numeric_vars, function(var) {
#   tapply(iris[[var]], iris$Species, mean)
# })
#
# # 3. Species with highest average Petal.Width
```

```
# petal_means <- tapply(iris$Petal.Width, iris$Species, mean)
# names(petal_means)[which.max(petal_means)]
```

12 Summary

Base R functions and the apply family provide powerful tools for data manipulation and analysis:

- **Utility functions** like `sum()`, `mean()`, `seq()`, and `rep()` form the foundation of data analysis
- **`apply()`** works on matrices and arrays across specified dimensions
- **`lapply()`** applies functions to lists, returning lists
- **`sapply()`** simplifies `lapply` results when possible
- **`vapply()`** provides type-safe `apply` operations
- **`mapply()`** handles multiple arguments element-wise
- **`tapply()`** applies functions by groups
- **Subsetting** allows precise data extraction using various indexing methods

Understanding these functions is crucial for efficient R programming and forms the basis for more advanced data manipulation techniques.

13 Data Types & Data Structures

14 Introduction

This comprehensive guide covers R's data types and structures with detailed explanations suitable for all skill levels. We'll explore not just *how* to use each structure, but *why* and *when* to use them effectively.

14.1 Learning Objectives

By the end of this chapter, you will be able to:

- **Beginner:** Understand basic data types and create simple data structures
- **Intermediate:** Master complex data manipulation and understand memory concepts
- **Advanced:** Optimize performance and implement advanced object-oriented concepts

14.2 R's Memory Model: Foundation Concepts

R stores all data as objects in memory. Understanding this is crucial for efficient programming:

:: {.callout-important} ## Key Memory Concepts

- **Copy-on-modify:** R creates copies when objects are modified
- **Reference counting:** R tracks how many variables point to an object
- **Garbage collection:** Automatic memory cleanup of unused objects
- **Object attributes:** Metadata attached to R objects :::

15 Atomic Vectors: The Building Blocks

15.1 Understanding Atomic Vectors

Atomic vectors are the fundamental building blocks of R. Every other data structure is built upon them.

15.1.1 The Six Atomic Types

```
# 1. Logical (boolean)
logical_vec <- c(TRUE, FALSE, TRUE, NA)
cat("Logical:", typeof(logical_vec), "\n")
```

Logical: logical

```
# 2. Integer (whole numbers with L suffix)
integer_vec <- c(1L, 2L, 3L, NA_integer_)
cat("Integer:", typeof(integer_vec), "\n")
```

Integer: integer

```
# 3. Double (real numbers)
double_vec <- c(1.5, 2.7, 3.14159, NA_real_)
cat("Double:", typeof(double_vec), "\n")
```

Double: double

```
# 4. Character (strings)
character_vec <- c("hello", "world", "R", NA_character_)
cat("Character:", typeof(character_vec), "\n")
```

Character: character

```
# 5. Complex (complex numbers)
complex_vec <- c(1+2i, 3+4i, 5+0i, NA_complex_)
cat("Complex:", typeof(complex_vec), "\n")
```

Complex: complex

```
# 6. Raw (bytes - rarely used in typical analysis)
raw_vec <- charToRaw("Hello")
cat("Raw:", typeof(raw_vec), "\n")
```

Raw: raw

15.1.2 Vector Properties and Inspection

```
# Create a sample vector for analysis
sample_vec <- c(10, 20, 30, 40, 50)

# Essential properties every R user should know
length(sample_vec)    # Number of elements
```

[1] 5

```
typeof(sample_vec)    # Internal storage type
```

[1] "double"

```
class(sample_vec)      # Object class (what R thinks it is)
```

[1] "numeric"

```
mode(sample_vec)       # Storage mode (legacy, but still used)
```

[1] "numeric"

```
# Check what kind of object we have
is.vector(sample_vec)    # Is it a vector?
```

```
[1] TRUE
```

```
is.atomic(sample_vec)    # Is it atomic (not a list)?
```

```
[1] TRUE
```

```
is.numeric(sample_vec)   # Is it numeric?
```

```
[1] TRUE
```

```
is.double(sample_vec)    # Specifically double precision?
```

```
[1] TRUE
```

Theory: Vectors are homogeneous - all elements must be the same type. When you mix types, R applies coercion rules.

15.1.3 Type Coercion Hierarchy

```
# R's coercion hierarchy:  logical < integer < double < character
# Mixing types forces coercion to the "highest" type

# Logical to integer
logical_to_int <- c(TRUE, FALSE, 1L)
cat("Result type:", typeof(logical_to_int), "\n")
```

```
Result type: integer
```

```
print(logical_to_int)
```

```
[1] 1 0 1
```

```
# Integer to double
int_to_double <- c(1L, 2.5)
cat("Result type:", typeof(int_to_double), "\n")
```

Result type: double

```
print(int_to_double)
```

```
[1] 1.0 2.5
```

```
# Double to character (everything becomes character)
mixed_all <- c(TRUE, 1L, 2.5, "hello")
cat("Result type:", typeof(mixed_all), "\n")
```

Result type: character

```
print(mixed_all)
```

```
[1] "TRUE"  "1"     "2.5"   "hello"
```

Coercion Warning

Automatic coercion can lead to unexpected results. Always check your data types when debugging!

15.1.4 Special Values in R

```
# R has several special values that beginners often find confusing
special_vals <- c(
  normal = 42,
  missing = NA,           # Not Available (missing value)
  not_number = NaN,       # Not a Number (0/0)
  positive_inf = Inf,     # Positive infinity (1/0)
  negative_inf = -Inf,    # Negative infinity (-1/0)
  null_length = NULL      # NULL has length 0
)
```

```
# Functions to test for special values
test_vec <- c(1, NA, NaN, Inf, -Inf)
cat("Original vector:\n")
```

Original vector:

```
print(test_vec)
```

```
[1] 1 NA NaN Inf -Inf
```

```
cat("\nTesting for special values:\n")
```

Testing for special values:

```
cat("is.na():", is.na(test_vec), "\n") # TRUE for NA and NaN
```

```
is.na(): FALSE TRUE TRUE FALSE FALSE
```

```
cat("is.nan():", is.nan(test_vec), "\n") # TRUE only for NaN
```

```
is.nan(): FALSE FALSE TRUE FALSE FALSE
```

```
cat("is.infinite():", is.infinite(test_vec), "\n") # TRUE for ±Inf
```

```
is.infinite(): FALSE FALSE FALSE TRUE TRUE
```

```
cat("is.finite():", is.finite(test_vec), "\n") # FALSE for NA, NaN, ±Inf
```

```
is.finite(): TRUE FALSE FALSE FALSE FALSE
```

15.2 Vector Creation Methods

15.2.1 Basic Creation Functions

```
# Method 1: c() function (combine/concatenate)
basic_vec <- c(1, 2, 3, 4, 5)

# Method 2: Colon operator (integer sequences)
seq_colon <- 1:10
reverse_seq <- 10:1

# Method 3: seq() function (more flexible sequences)
seq_by_step <- seq(from = 0, to = 100, by = 10)
seq_by_length <- seq(from = 0, to = 1, length.out = 11)
seq_along_other <- seq_along(c("a", "b", "c", "d"))

cat("Basic vector:", basic_vec, "\n")
```

Basic vector: 1 2 3 4 5

```
cat("Colon sequence:", seq_colon, "\n")
```

Colon sequence: 1 2 3 4 5 6 7 8 9 10

```
cat("Sequence by step:", seq_by_step, "\n")
```

Sequence by step: 0 10 20 30 40 50 60 70 80 90 100

```
cat("Sequence by length:", seq_by_length, "\n")
```

Sequence by length: 0 0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9 1

15.2.2 Advanced Creation Patterns

```
# rep() function - repetition patterns
rep_times <- rep(c(1, 2, 3), times = 3)      # Repeat entire vector
rep_each <- rep(c(1, 2, 3), each = 3)       # Repeat each element
rep_length_out <- rep(c(1, 2), length.out = 7) # Repeat to specific length

cat("rep times:", rep_times, "\n")
```

```
rep times: 1 2 3 1 2 3 1 2 3
```

```
cat("rep each:", rep_each, "\n")
```

```
rep each: 1 1 1 2 2 2 3 3 3
```

```
cat("rep length. out:", rep_length_out, "\n")
```

```
rep length. out: 1 2 1 2 1 2 1
```

```
# Creating named vectors
named_vec <- c(first = 1, second = 2, third = 3)
print(named_vec)
```

```
first second  third
      1      2      3
```

```
# Alternative way to add names
numbers <- 1:5
names(numbers) <- c("one", "two", "three", "four", "five")
print(numbers)
```

```
one  two three  four  five
  1    2    3    4    5
```

```
# Using paste() for systematic naming
systematic_names <- paste0("item_", 1:5)
cat("Systematic names:", systematic_names, "\n")
```

```
Systematic names: item_1 item_2 item_3 item_4 item_5
```

15.3 Vector Indexing and Subsetting

Vector indexing in R is extremely powerful and flexible.

15.3.1 Positive Integer Indexing

```
# Create a sample vector with names
fruits <- c("apple", "banana", "cherry", "date", "elderberry", "fig")
names(fruits) <- paste0("fruit_", 1:6)
print(fruits)
```

fruit_1	fruit_2	fruit_3	fruit_4	fruit_5	fruit_6
"apple"	"banana"	"cherry"	"date"	"elderberry"	"fig"

```
# Single element access
cat("First fruit:", fruits[1], "\n")
```

First fruit: apple

```
cat("Third fruit:", fruits[3], "\n")
```

Third fruit: cherry

```
# Multiple specific elements
selected_fruits <- fruits[c(1, 3, 5)]
cat("Selected fruits:", selected_fruits, "\n")
```

Selected fruits: apple cherry elderberry

```
# Range selection (slicing)
fruit_range <- fruits[2:4]
cat("Fruit range (2-4):", fruit_range, "\n")
```

Fruit range (2-4): banana cherry date

```
# Using sequences for complex patterns
odd_positions <- fruits[seq(1, length(fruits), by = 2)]
cat("Odd positions:", odd_positions, "\n")
```

Odd positions: apple cherry elderberry

15.3.2 Negative Integer Indexing (Exclusion)


```
# Negative indexing excludes elements
all_but_first <- fruits[-1]
cat("All except first:", all_but_first, "\n")
```

All except first: banana cherry date elderberry fig

```
# Exclude multiple elements
exclude_multiple <- fruits[-c(2, 4, 6)]
cat("Excluding positions 2,4,6:", exclude_multiple, "\n")
```

Excluding positions 2,4,6: apple cherry elderberry

```
# Exclude ranges
exclude_range <- fruits[-(2:4)]
cat("Excluding range 2-4:", exclude_range, "\n")
```

Excluding range 2-4: apple elderberry fig

15.3.3 Logical Indexing (Conditional Selection)

```
# Create data for logical indexing examples
prices <- c(1.50, 3.20, 0.80, 4.50, 2.10, 6.00)
names(prices) <- fruits

# Simple logical conditions
expensive_items <- prices > 3.00
cat("Expensive items (>3.00):", prices[expensive_items], "\n")
```

Expensive items (>3.00): 3.2 4.5 6

```
# Multiple conditions with logical operators
moderate_prices <- prices >= 1.00 & prices <= 4.00
cat("Moderate prices (1-4):", prices[moderate_prices], "\n")
```

Moderate prices (1-4): 1.5 3.2 2.1

```
# Using %in% operator for membership testing
target_fruits <- c("apple", "cherry", "fig")
selected_by_name <- prices[names(prices) %in% target_fruits]
cat("Selected by name:", selected_by_name, "\n")
```

Selected by name: 1.5 0.8 6

```
# Complex logical expressions
complex_condition <- (prices > 2.00) | (nchar(names(prices)) > 10)
cat("Complex condition:", prices[complex_condition], "\n")
```

Complex condition: 3.2 4.5 2.1 6

15.3.4 Named Indexing

```
# Access by single name
apple_price <- prices["fruit_1"]
cat("Apple price:", apple_price, "\n")
```

Apple price: NA

```
# Access by multiple names
fruit_selection <- prices[c("fruit_1", "fruit_3", "fruit_6")]
cat("Selected fruit prices:", fruit_selection, "\n")
```

Selected fruit prices: NA NA NA

```
# Dynamic name creation
dynamic_names <- paste0("fruit_", c(1, 3, 5))
dynamic_selection <- prices[dynamic_names]
cat("Dynamic selection:", dynamic_selection, "\n")
```

Dynamic selection: NA NA NA

15.4 Vector Operations and Arithmetic

15.4.1 Scalar Operations (Broadcasting)

```
# Create sample data
values <- c(10, 20, 30, 40, 50)
scalar <- 5

# Arithmetic operations with scalars
cat("Original values:", values, "\n")
```

Original values: 10 20 30 40 50

```
cat("Add 5:", values + scalar, "\n")
```

Add 5: 15 25 35 45 55

```
cat("Multiply by 5:", values * scalar, "\n")
```

Multiply by 5: 50 100 150 200 250

```
cat("Divide by 5:", values / scalar, "\n")
```

Divide by 5: 2 4 6 8 10

```
cat("Power of 2:", values^2, "\n")
```

Power of 2: 100 400 900 1600 2500

```
cat("Modulus 3:", values %% 3, "\n")
```

Modulus 3: 1 2 0 1 2

```
cat("Integer division by 3:", values %/% 3, "\n")
```

Integer division by 3: 3 6 10 13 16

15.4.2 Vector-to-Vector Operations

```
# Element-wise operations between vectors of same length
vec1 <- c(1, 2, 3, 4, 5)
vec2 <- c(10, 20, 30, 40, 50)

cat("Vector 1:", vec1, "\n")
```

Vector 1: 1 2 3 4 5

```
cat("Vector 2:", vec2, "\n")
```

Vector 2: 10 20 30 40 50

```
cat("Addition:", vec1 + vec2, "\n")
```

Addition: 11 22 33 44 55

```
cat("Multiplication:", vec1 * vec2, "\n")
```

Multiplication: 10 40 90 160 250

```
cat("Division:", vec2 / vec1, "\n")
```

Division: 10 10 10 10 10

```
# Vector recycling with different lengths
short_vec <- c(1, 2)
long_vec <- c(10, 20, 30, 40, 50, 60)

cat("\nShort vector:", short_vec, "\n")
```

Short vector: 1 2

```
cat("Long vector:", long_vec, "\n")
```

Long vector: 10 20 30 40 50 60

```
cat("Addition (with recycling):", short_vec + long_vec, "\n")
```

Addition (with recycling): 11 22 31 42 51 62

: :: {.callout-note} ## Vector Recycling

When operating on vectors of different lengths, R recycles the shorter vector. This is powerful but can lead to unexpected results if not understood properly. :::

15.4.3 Mathematical Functions

```
# Mathematical functions work element-wise on vectors
x <- c(1, 4, 9, 16, 25)

cat("Original values:", x, "\n")
```

Original values: 1 4 9 16 25

```
cat("Square root:", sqrt(x), "\n")
```

Square root: 1 2 3 4 5

```
cat("Natural log:", log(x), "\n")
```

Natural log: 0 1.386294 2.197225 2.772589 3.218876

```
cat("Log base 10:", log10(x), "\n")
```

Log base 10: 0 0.60206 0.9542425 1.20412 1.39794

```
cat("Exponential:", exp(c(0, 1, 2)), "\n")
```

Exponential: 1 2.718282 7.389056

```
# Trigonometric functions
angles <- c(0, pi/4, pi/2, pi)
cat("Angles (radians):", round(angles, 3), "\n")
```

Angles (radians): 0 0.785 1.571 3.142

```
cat("Sine:", round(sin(angles), 3), "\n")
```

Sine: 0 0.707 1 0

```
cat("Cosine:", round(cos(angles), 3), "\n")
```

Cosine: 1 0.707 0 -1

```
# Rounding functions
messy_numbers <- c(3.14159, 2.71828, 1.41421)
cat("Original:", messy_numbers, "\n")
```

Original: 3.14159 2.71828 1.41421

```
cat("Round to 2 places:", round(messy_numbers, 2), "\n")
```

Round to 2 places: 3.14 2.72 1.41

```
cat("Ceiling:", ceiling(messy_numbers), "\n")
```

Ceiling: 4 3 2

```
cat("Floor:", floor(messy_numbers), "\n")
```

Floor: 3 2 1

```
cat("Truncate:", trunc(messy_numbers), "\n")
```

Truncate: 3 2 1

15.5 Set Operations and Comparisons

15.5.1 Set Operations

```
# Create sets for demonstration
set_a <- c("apple", "banana", "cherry", "date")
set_b <- c("cherry", "date", "elderberry", "fig")

cat("Set A:", set_a, "\n")
```

Set A: apple banana cherry date

```
cat("Set B:", set_b, "\n")
```

Set B: cherry date elderberry fig

```
# Core set operations
union_result <- union(set_a, set_b)
cat("Union (A B):", union_result, "\n")
```

Union (A B): apple banana cherry date elderberry fig

```
intersection_result <- intersect(set_a, set_b)
cat("Intersection (A B):", intersection_result, "\n")
```

Intersection (A B): cherry date

```
difference_result <- setdiff(set_a, set_b)
cat("Difference (A - B):", difference_result, "\n")
```

Difference (A - B): apple banana

```
difference_reverse <- setdiff(set_b, set_a)
cat("Difference (B - A):", difference_reverse, "\n")
```

Difference (B - A): elderberry fig

```
# Set equality and membership
cat("Are sets equal?", setequal(set_a, set_b), "\n")
```

Are sets equal? FALSE

```
cat("Is 'apple' in set A?", "apple" %in% set_a, "\n")
```

Is 'apple' in set A? TRUE

```
cat("Is 'grape' in set A?", "grape" %in% set_a, "\n")
```

Is 'grape' in set A? FALSE

15.5.2 Advanced Comparison Operations

```
# Comparison vectors
scores_1 <- c(85, 92, 78, 96, 88)
scores_2 <- c(88, 90, 78, 94, 91)

cat("Scores 1:", scores_1, "\n")
```

Scores 1: 85 92 78 96 88

```
cat("Scores 2:", scores_2, "\n")
```

Scores 2: 88 90 78 94 91

```
# Element-wise comparisons
cat("Equal elements:", scores_1 == scores_2, "\n")
```

Equal elements: FALSE FALSE TRUE FALSE FALSE


```
cat("Score 1 greater:", scores_1 > scores_2, "\n")
```

Score 1 greater: FALSE TRUE FALSE TRUE FALSE

```
cat("Score difference >= 5:", abs(scores_1 - scores_2) >= 5, "\n")
```

Score difference >= 5: FALSE FALSE FALSE FALSE FALSE

```
# Aggregated logical operations
```

```
cat("Any score1 > score2? ", any(scores_1 > scores_2), "\n")
```

Any score1 > score2? TRUE

```
cat("All scores > 75?", all(scores_1 > 75), "\n")
```

All scores > 75? TRUE

```
cat("How many scores1 > 85?", sum(scores_1 > 85), "\n")
```

How many scores1 > 85? 3

```
# Finding positions where conditions are true
```

```
high_scores <- which(scores_1 > 90)
```

```
cat("Positions with scores > 90:", high_scores, "\n")
```

Positions with scores > 90: 2 4

```
# Finding maximum and minimum positions
```

```
max_position <- which.max(scores_1)
```

```
min_position <- which.min(scores_1)
```

```
cat("Highest score position:", max_position, "value:", scores_1[max_position], "\n")
```

Highest score position: 4 value: 96

```
cat("Lowest score position:", min_position, "value:", scores_1[min_position], "\n")
```

```
Lowest score position: 3 value: 78
```

16 Factors: Categorical Data Excellence

16.1 Understanding Factors

Factors are R's sophisticated way of handling categorical data. They're built on integers with associated labels, making them memory-efficient and statistically meaningful.

16.1.1 Basic Factor Creation

```
# Create a simple factor from character data
colors <- c("red", "blue", "green", "red", "blue", "green", "red")
color_factor <- factor(colors)

print(color_factor)
```

```
[1] red   blue  green red   blue  green red
Levels: blue green red
```

```
cat("Levels:", levels(color_factor), "\n")
```

```
Levels: blue green red
```

```
cat("Number of levels:", nlevels(color_factor), "\n")
```

```
Number of levels: 3
```

```
# Internal representation
cat("Internal structure (integer codes):", as.integer(color_factor), "\n")
```

```
Internal structure (integer codes): 3 1 2 3 1 2 3
```

```
cat("Storage type:", typeof(color_factor), "\n")
```

Storage type: integer

```
cat("Memory usage comparison:\n")
```

Memory usage comparison:

```
cat("  Character:", object.size(colors), "bytes\n")
```

Character: 280 bytes

```
cat("  Factor:", object.size(color_factor), "bytes\n")
```

Factor: 664 bytes

16.1.2 Controlling Factor Levels

```
# Explicitly set levels and their order
sizes <- c("small", "large", "medium", "small", "medium", "large")

# Without specifying levels (alphabetical order)
size_factor_auto <- factor(sizes)
cat("Automatic levels:", levels(size_factor_auto), "\n")
```

Automatic levels: large medium small

```
# With explicitly ordered levels
size_factor_ordered <- factor(sizes,
                             levels = c("small", "medium", "large"))
cat("Custom levels:", levels(size_factor_ordered), "\n")
```

Custom levels: small medium large

```
# Ordered factors (with ranking)
size_factor_ranked <- factor(sizes,
                             levels = c("small", "medium", "large"),
                             ordered = TRUE)
cat("Ordered factor:", size_factor_ranked, "\n")
```

Ordered factor: 1 3 2 1 2 3

```
cat("Is ordered? ", is.ordered(size_factor_ranked), "\n")
```

Is ordered? TRUE

```
# Comparisons work with ordered factors
cat("small < medium? ", size_factor_ranked[1] < size_factor_ranked[3], "\n")
```

small < medium? TRUE

16.1.3 Factor Level Manipulation

```
# Create a factor for manipulation
ratings <- factor(c("good", "bad", "excellent", "good", "fair", "bad"))
print(ratings)
```

```
[1] good      bad      excellent good      fair      bad
Levels: bad excellent fair good
```

```
# View current levels
cat("Original levels:", levels(ratings), "\n")
```

Original levels: bad excellent fair good

```
# Reorder levels logically
levels(ratings) <- c("bad", "fair", "good", "excellent")
cat("Reordered levels:", levels(ratings), "\n")
```

Reordered levels: bad fair good excellent

```
# Add new levels (useful before adding new data)
levels(ratings) <- c(levels(ratings), "outstanding")
cat("After adding level:", levels(ratings), "\n")
```

After adding level: bad fair good excellent outstanding

```
# Remove unused levels
ratings_subset <- ratings[ratings != "bad"]
cat("Before droplevels:", levels(ratings_subset), "\n")
```

Before droplevels: bad fair good excellent outstanding

```
ratings_clean <- droplevels(ratings_subset)
cat("After droplevels:", levels(ratings_clean), "\n")
```

After droplevels: fair good excellent

16.1.4 Advanced Factor Operations with forcats

```
library(forcats)

# Create sample survey data
responses <- factor(c("Agree", "Disagree", "Strongly Agree", "Neutral",
                      "Agree", "Strongly Disagree", "Agree", "Neutral",
                      "Disagree", "Strongly Agree"))

# Reorder by frequency
freq_ordered <- fct_infreq(responses)
cat("By frequency:\n")
```

By frequency:

```
print(table(freq_ordered))
```

```
freq_ordered
      Agree      Disagree      Neutral  Strongly Agree
          3             2             2             2
Strongly Disagree
          1
```

```
# Reorder by another variable
scores <- c(4, 2, 5, 3, 4, 1, 4, 3, 2, 5)
score_ordered <- fct_reorder(responses, scores, mean)
cat("Ordered by mean score:\n")
```

Ordered by mean score:

```
print(tapply(scores, score_ordered, mean))
```

Strongly Disagree	Disagree	Neutral	Agree
1	2	3	4
Strongly Agree			
5			

```
# Collapse rare levels
collapsed <- fct_lump_min(responses, min = 2)
cat("After collapsing rare levels:\n")
```

After collapsing rare levels:

```
print(table(collapsed))
```

collapsed	Agree	Disagree	Neutral	Strongly Agree	Other
	3	2	2	2	1

```
# Recode levels
recoded <- fct_recode(responses,
  "Positive" = "Agree",
  "Very Positive" = "Strongly Agree",
  "Negative" = "Disagree",
  "Very Negative" = "Strongly Disagree"
)
print(table(recoded))
```

recoded	Positive	Negative	Neutral	Very Positive	Very Negative
	3	2	2	2	1

17 Lists: The Swiss Army Knife

17.1 Understanding Lists

Lists are recursive data structures - they can contain any R object, including other lists. This makes them incredibly flexible for complex data structures.

17.1.1 Basic List Creation and Structure

```
# Create a heterogeneous list
student_record <- list(
  name = "Alice Johnson",
  student_id = 12345,
  grades = c(85, 92, 78, 96, 88),
  courses = c("Math", "Science", "English", "History", "Art"),
  is_honors = TRUE,
  graduation_date = as.Date("2024-06-15")
)

print(student_record)
```

```
$name
[1] "Alice Johnson"
```

```
$student_id
[1] 12345
```

```
$grades
[1] 85 92 78 96 88
```

```
$courses
[1] "Math"      "Science" "English" "History" "Art"
```

```
$is_honors
```



```
[1] TRUE
```

```
$graduation_date
```

```
[1] "2024-06-15"
```

```
# Examine list structure  
cat("\nList structure:\n")
```

List structure:

```
str(student_record)
```

List of 6

```
$ name      : chr "Alice Johnson"  
$ student_id : num 12345  
$ grades     : num [1:5] 85 92 78 96 88  
$ courses    : chr [1:5] "Math" "Science" "English" "History" ...  
$ is_honors  : logi TRUE  
$ graduation_date: Date[1:1], format: "2024-06-15"
```

```
# List properties  
cat("Length (number of elements):", length(student_record), "\n")
```

Length (number of elements): 6

```
cat("Names of elements:", names(student_record), "\n")
```

Names of elements: name student_id grades courses is_honors graduation_date

```
cat("Is it a list?", is.list(student_record), "\n")
```

Is it a list? TRUE

17.1.2 List Indexing Methods

Understanding the difference between `[`, `[[`, and `$` is crucial for list manipulation.

```
# Method 1: Single bracket [ ] - returns a list
subset_list <- student_record[1] # Returns list containing first element
cat("Using [1] - class:", class(subset_list), "\n")
```

Using [1] - class: list

```
print(subset_list)
```

```
$name
[1] "Alice Johnson"
```

```
# Multiple elements with single bracket
multiple_elements <- student_record[c("name", "student_id")]
print(multiple_elements)
```

```
$name
[1] "Alice Johnson"
```

```
$student_id
[1] 12345
```

```
# Method 2: Double bracket [[ ]] - returns the actual element
actual_name <- student_record[[1]] # Returns the character vector
cat("Using [[1]] - class:", class(actual_name), "\n")
```

Using [[1]] - class: character

```
print(actual_name)
```

```
[1] "Alice Johnson"
```

```
# Access by name with double bracket
grades <- student_record[["grades"]]
cat("Grades class:", class(grades), "\n")
```

Grades class: numeric

```
print(grades)
```

```
[1] 85 92 78 96 88
```

```
# Method 3: Dollar sign $ - convenient for named elements
student_name <- student_record$name
cat("Using $ - class:", class(student_name), "\n")
```

Using \$ - class: character

```
print(student_name)
```

```
[1] "Alice Johnson"
```

💡 List Indexing Memory Aid

- `my_list[1]` → Returns a list (like a box containing the item)
- `my_list[[1]]` → Returns the actual item (takes item out of box)
- `my_list$name` → Convenient shortcut for `my_list[["name"]]`

17.1.3 List Modification and Manipulation

```
student_record <- list(
  name = "Alice Johnson",
  student_id = 12345,
  grades = c(85, 92, 78, 96, 88),
  courses = c("Math", "Science", "English", "History", "Art"),
  is_honors = TRUE,
  graduation_date = as.Date("2024-06-15")
)
# Start with a copy for modification
student_copy <- student_record

# Add new elements
student_copy$gpa <- mean(student_copy$grades) / 100 * 4 # Simple GPA calculation
student_copy$advisor <- "Dr. Smith"
student_copy$extracurricular <- c("Drama Club", "Science Olympics")
```

```
# Modify existing elements
student_copy$grades[1] <- 90 # Change first grade
student_copy$is_honors <- FALSE

# Remove elements (set to NULL)
student_copy$advisor <- NULL

cat("Modified list structure:\n")
```

Modified list structure:

```
str(student_copy)
```

```
List of 8
 $ name      : chr "Alice Johnson"
 $ student_id : num 12345
 $ grades    : num [1:5] 90 92 78 96 88
 $ courses   : chr [1:5] "Math" "Science" "English" "History" ...
 $ is_honors  : logi FALSE
 $ graduation_date: Date[1:1], format: "2024-06-15"
 $ gpa       : num 3.51
 $ extracurricular: chr [1:2] "Drama Club" "Science Olympics"
```

```
# Adding nested structure
student_copy$contact_info <- list(
  email = "alice.johnson@school.edu",
  phone = "555-0123",
  address = list(
    street = "123 Main St",
    city = "Academic City",
    zip = "12345"
  )
)

cat("After adding nested structure:\n")
```

After adding nested structure:

```
str(student_copy, max.level = 2) # Limit depth for readability
```

List of 9

```
$ name      : chr "Alice Johnson"
$ student_id : num 12345
$ grades    : num [1:5] 90 92 78 96 88
$ courses   : chr [1:5] "Math" "Science" "English" "History" ...
$ is_honors  : logi FALSE
$ graduation_date: Date[1:1], format: "2024-06-15"
$ gpa       : num 3.51
$ extracurricular: chr [1:2] "Drama Club" "Science Olympics"
$ contact_info :List of 3
..$ email  : chr "alice.johnson@school.edu"
..$ phone  : chr "555-0123"
..$ address:List of 3
```

17.1.4 List Apply Functions

The apply family of functions is essential for working with lists efficiently.

```
# Create a list of numeric vectors for demonstration
data_list <- list(
  dataset_1 = c(23, 45, 67, 89, 12),
  dataset_2 = c(34, 56, 78, 90, 23, 45),
  dataset_3 = c(12, 34, 56, 78, 90, 12, 34)
)

# lapply - returns a list
means_list <- lapply(data_list, mean)
cat("lapply result (list):\n")
```

lapply result (list):

```
print(means_list)
```

```
$dataset_1
[1] 47.2
```

```
$dataset_2
```

```
[1] 54.33333
```

```
$dataset_3
```

```
[1] 45.14286
```

```
# sapply - simplifies to vector when possible
means_vector <- sapply(data_list, mean)
cat("sapply result (vector):", means_vector, "\n")
```

```
sapply result (vector): 47.2 54.33333 45.14286
```

```
# mapply - multivariate apply
weights <- c(0.3, 0.5, 0.2)
weighted_means <- mapply(function(x, w) sum(x * w), data_list, weights)
cat("Weighted means:", weighted_means, "\n")
```

```
Weighted means: 70.8 163 63.2
```

```
# vapply - like sapply but with type checking (safer)
lengths_safe <- vapply(data_list, length, integer(1))
cat("Lengths (vapply):", lengths_safe, "\n")
```

```
Lengths (vapply): 5 6 7
```

17.1.5 Advanced List Operations with purrr

```
library(purrr)

# Modern functional programming with purrr
# map family - consistent and predictable
data_list <- list(
  dataset_1 = c(23, 45, 67, 89, 12),
  dataset_2 = c(34, 56, 78, 90, 23, 45),
  dataset_3 = c(12, 34, 56, 78, 90, 12, 34)
)
# map returns a list
summary_stats <- map(data_list, summary)
print(summary_stats[[1]])
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
12.0	23.0	45.0	47.2	67.0	89.0

```
# map_dbl returns numeric vector
means_purrr <- map_dbl(data_list, mean)
cat("Means with map_dbl:", means_purrr, "\n")
```

Means with map_dbl: 47.2 54.33333 45.14286

```
# map_chr returns character vector
length_descriptions <- map_chr(data_list, ~ paste("Length:", length(.x)))
cat("Descriptions:", length_descriptions, "\n")
```

Descriptions: Length: 5 Length: 6 Length: 7

```
# map2 for working with two lists simultaneously
multipliers <- c(2, 3, 4)
scaled_data <- map2(data_list, multipliers, ~ .x * .y)
cat("First scaled dataset:", scaled_data[[1]], "\n")
```

First scaled dataset: 46 90 134 178 24

```
# Complex transformations with map
complex_stats <- data_list |>
  map(~ list(
    mean = mean(.x),
    sd = sd(.x),
    min = min(.x),
    max = max(.x),
    n = length(.x)
  ))

print(complex_stats$dataset_1)
```

```
$mean
[1] 47.2
```

```
$sd
[1] 31.49921
```

```
$min  
[1] 12
```

```
$max  
[1] 89
```

```
$n  
[1] 5
```


18 Matrices: Linear Algebra Powerhouse

18.1 Understanding Matrices

Matrices are 2-dimensional, homogeneous data structures. They're essentially vectors with a dimension attribute, making them perfect for mathematical operations.

18.1.1 Matrix Creation Methods

```
# Method 1: matrix() function
basic_matrix <- matrix(1:12, nrow = 3, ncol = 4)
cat("Basic matrix (filled by columns):\n")
```

Basic matrix (filled by columns):

```
print(basic_matrix)
```

	[,1]	[,2]	[,3]	[,4]
[1,]	1	4	7	10
[2,]	2	5	8	11
[3,]	3	6	9	12

```
# Fill by rows instead
by_rows <- matrix(1:12, nrow = 3, ncol = 4, byrow = TRUE)
cat("Matrix filled by rows:\n")
```

Matrix filled by rows:

```
print(by_rows)
```

	[,1]	[,2]	[,3]	[,4]
[1,]	1	2	3	4
[2,]	5	6	7	8
[3,]	9	10	11	12

```
# Method 2: Combining vectors
row_1 <- c(1, 2, 3, 4)
row_2 <- c(5, 6, 7, 8)
row_3 <- c(9, 10, 11, 12)

# Row binding
matrix_rbind <- rbind(row_1, row_2, row_3)
cat("Matrix from row binding:\n")
```

Matrix from row binding:

```
print(matrix_rbind)
```

	[,1]	[,2]	[,3]	[,4]
row_1	1	2	3	4
row_2	5	6	7	8
row_3	9	10	11	12

```
# Column binding
col_matrix <- cbind(c(1, 5, 9), c(2, 6, 10), c(3, 7, 11), c(4, 8, 12))
cat("Matrix from column binding:\n")
```

Matrix from column binding:

```
print(col_matrix)
```

	[,1]	[,2]	[,3]	[,4]
[1,]	1	2	3	4
[2,]	5	6	7	8
[3,]	9	10	11	12

```
# Method 3: Adding dimensions to a vector
vector_to_matrix <- 1:12
dim(vector_to_matrix) <- c(3, 4)
cat("Vector converted to matrix:\n")
```

Vector converted to matrix:

```
print(vector_to_matrix)
```

	[,1]	[,2]	[,3]	[,4]
[1,]	1	4	7	10
[2,]	2	5	8	11
[3,]	3	6	9	12

18.1.2 Matrix Properties and Attributes

```
# Create a named matrix for demonstration
sales_matrix <- matrix(
  c(100, 150, 120, 200,
    110, 160, 130, 210,
    105, 155, 125, 195),
  nrow = 3,
  byrow = TRUE,
  dimnames = list(
    quarters = c("Q1", "Q2", "Q3"),
    products = c("Product_A", "Product_B", "Product_C", "Product_D")
  )
)

print(sales_matrix)
```

	products			
quarters	Product_A	Product_B	Product_C	Product_D
Q1	100	150	120	200
Q2	110	160	130	210
Q3	105	155	125	195

```
# Matrix properties
cat("Dimensions:", dim(sales_matrix), "\n")
```

Dimensions: 3 4

```
cat("Number of rows:", nrow(sales_matrix), "\n")
```

Number of rows: 3

```
cat("Number of columns:", ncol(sales_matrix), "\n")
```

Number of columns: 4

```
cat("Total elements:", length(sales_matrix), "\n")
```

Total elements: 12

```
# Dimension names  
cat("Row names:", rownames(sales_matrix), "\n")
```

Row names: Q1 Q2 Q3

```
cat("Column names:", colnames(sales_matrix), "\n")
```

Column names: Product_A Product_B Product_C Product_D

```
# Matrix type information  
cat("Class:", class(sales_matrix), "\n")
```

Class: matrix array

```
cat("Type:", typeof(sales_matrix), "\n")
```

Type: double

```
cat("Is matrix?", is.matrix(sales_matrix), "\n")
```

Is matrix? TRUE

```
cat("Is array?", is.array(sales_matrix), "\n") # Matrices are also arrays
```

Is array? TRUE

18.1.3 Matrix Indexing and Subsetting

```
# Single element access
cat("Element [2,3]:", sales_matrix[2, 3], "\n")
```

Element [2,3]: 130

```
cat("Q2 Product_C:", sales_matrix["Q2", "Product_C"], "\n")
```

Q2 Product_C: 130

```
# Row access (returns vector)
q1_sales <- sales_matrix[1, ]
cat("Q1 sales:", q1_sales, "\n")
```

Q1 sales: 100 150 120 200

```
cat("Class of row:", class(q1_sales), "\n") # Note: becomes vector
```

Class of row: numeric

```
# Column access
product_a_sales <- sales_matrix[, 1]
cat("Product A sales:", product_a_sales, "\n")
```

Product A sales: 100 110 105

```
# Multiple rows/columns (returns matrix)
multi_quarters <- sales_matrix[1:2, ]
cat("Q1-Q2 sales:\n")
```

Q1-Q2 sales:

```
print(multi_quarters)
```

```
      products
quarters Product_A Product_B Product_C Product_D
Q1         100       150       120       200
Q2         110       160       130       210
```

```
multi_products <- sales_matrix[, 2:4]
cat("Products B-D:\n")
```

Products B-D:

```
print(multi_products)
```

```
      products
quarters Product_B Product_C Product_D
Q1         150       120       200
Q2         160       130       210
Q3         155       125       195
```

```
# Submatrix
submatrix <- sales_matrix[1:2, 2:3]
cat("Submatrix Q1-Q2, Products B-C:\n")
```

Submatrix Q1-Q2, Products B-C:

```
print(submatrix)
```

```
      products
quarters Product_B Product_C
Q1         150       120
Q2         160       130
```

```
# Logical indexing
high_sales <- sales_matrix > 150
cat("High sales (>150):\n")
```

High sales (>150):

```
print(sales_matrix[high_sales]) # Returns as vector
```

```
[1] 160 155 200 210 195
```

18.1.4 Matrix Arithmetic Operations

```
# Create matrices for operations
A <- matrix(c(1, 2, 3, 4), nrow = 2, ncol = 2)
B <- matrix(c(5, 6, 7, 8), nrow = 2, ncol = 2)

cat("Matrix A:\n"); print(A)
```

Matrix A:

```
      [,1] [,2]
[1,]     1     3
[2,]     2     4
```

```
cat("Matrix B:\n"); print(B)
```

Matrix B:

```
      [,1] [,2]
[1,]     5     7
[2,]     6     8
```

```
# Element-wise operations
cat("A + B (element-wise addition):\n")
```

A + B (element-wise addition):

```
print(A + B)
```

```
      [,1] [,2]
[1,]     6    10
[2,]     8    12
```

```
cat("A * B (element-wise multiplication):\n")
```

A * B (element-wise multiplication):

```
print(A * B)
```

```
      [,1] [,2]  
[1,]     5  21  
[2,]    12  32
```

```
# Matrix multiplication (linear algebra)  
cat("A %% B (matrix multiplication):\n")
```

A %% B (matrix multiplication):

```
print(A %% B)
```

```
      [,1] [,2]  
[1,]    23  31  
[2,]    34  46
```

```
# Scalar operations  
cat("A * 3 (scalar multiplication):\n")
```

A * 3 (scalar multiplication):

```
print(A * 3)
```

```
      [,1] [,2]  
[1,]     3   9  
[2,]     6  12
```

```
# Matrix powers  
cat("A^2 (element-wise squaring):\n")
```

A^2 (element-wise squaring):


```
print(A^2)
```

```
      [,1] [,2]  
[1,]     1     9  
[2,]     4    16
```

18.1.5 Advanced Matrix Operations

```
# Create a square matrix for advanced operations  
square_matrix <- matrix(c(4, 7, 2, 6), nrow = 2, ncol = 2)  
cat("Square matrix:\n"); print(square_matrix)
```

Square matrix:

```
      [,1] [,2]  
[1,]     4     2  
[2,]     7     6
```

```
# Linear algebra operations  
cat("Transpose:\n"); print(t(square_matrix))
```

Transpose:

```
      [,1] [,2]  
[1,]     4     7  
[2,]     2     6
```

```
cat("Determinant:", det(square_matrix), "\n")
```

Determinant: 10

```
# Matrix inverse  
inverse_matrix <- solve(square_matrix)  
cat("Inverse matrix:\n"); print(inverse_matrix)
```

Inverse matrix:

```

      [,1] [,2]
[1,]  0.6 -0.2
[2,] -0.7  0.4

```

```

# Verify inverse:  A * A^(-1) = I
identity_check <- square_matrix %*% inverse_matrix
cat("A * A^(-1) (should be identity):\n")

```

A * A⁽⁻¹⁾ (should be identity):

```

print(round(identity_check, 10)) # Round to handle floating point errors

```

```

      [,1] [,2]
[1,]    1    0
[2,]    0    1

```

```

# Eigenvalues and eigenvectors
eigen_result <- eigen(square_matrix)
cat("Eigenvalues:", eigen_result$values, "\n")

```

Eigenvalues: 8.872983 1.127017

```

cat("Eigenvectors:\n"); print(eigen_result$vectors)

```

Eigenvectors:

```

      [,1]      [,2]
[1,] -0.3796908 -0.5713345
[2,] -0.9251135  0.8207173

```

```

# Matrix decomposition
qr_decomp <- qr(square_matrix)
cat("QR decomposition Q:\n"); print(qr.Q(qr_decomp))

```

QR decomposition Q:

```

      [,1]      [,2]
[1,] -0.4961389 -0.8682431
[2,] -0.8682431  0.4961389

```

```
cat("QR decomposition R:\n"); print(qr.R(qr_decomp))
```

QR decomposition R:

```
      [,1]      [,2]
[1,] -8.062258 -6.201737
[2,]  0.000000  1.240347
```

18.1.6 Matrix Apply Operations

```
# Create a larger matrix for demonstration
test_matrix <- matrix(rnorm(20, mean = 10, sd = 3), nrow = 4, ncol = 5)
rownames(test_matrix) <- paste("Row", 1:4)
colnames(test_matrix) <- paste("Col", 1:5)

cat("Test matrix:\n")
```

Test matrix:

```
print(round(test_matrix, 2))
```

```
      Col 1 Col 2 Col 3 Col 4 Col 5
Row 1 13.40 12.91 11.97 12.25  8.30
Row 2  8.45  8.97 11.70  6.17 11.90
Row 3 10.72 12.71  7.55  4.56  7.47
Row 4  5.57  6.71 14.28  9.06 13.02
```

```
# Apply functions across margins
# MARGIN = 1 for rows, MARGIN = 2 for columns
```

```
# Row operations
row_sums <- apply(test_matrix, 1, sum)
row_means <- apply(test_matrix, 1, mean)
row_ranges <- apply(test_matrix, 1, function(x) max(x) - min(x))

cat("Row sums:", round(row_sums, 2), "\n")
```

Row sums: 58.83 47.2 43.01 48.65

```
cat("Row means:", round(row_means, 2), "\n")
```

Row means: 11.77 9.44 8.6 9.73

```
cat("Row ranges:", round(row_ranges, 2), "\n")
```

Row ranges: 5.11 5.74 8.16 8.71

```
# Column operations
col_sums <- apply(test_matrix, 2, sum)
col_sds <- apply(test_matrix, 2, sd)

cat("Column sums:", round(col_sums, 2), "\n")
```

Column sums: 38.15 41.3 45.51 32.03 40.69

```
cat("Column standard deviations:", round(col_sds, 2), "\n")
```

Column standard deviations: 3.33 3.01 2.8 3.39 2.7

```
# Built-in functions (faster than apply for simple operations)
cat("rowSums() result:", round(rowSums(test_matrix), 2), "\n")
```

rowSums() result: 58.83 47.2 43.01 48.65

```
cat("colMeans() result:", round(colMeans(test_matrix), 2), "\n")
```

colMeans() result: 9.54 10.33 11.38 8.01 10.17

19 Data Frames: The Data Analysis Workhorse

19.1 Understanding Data Frames

Data frames are the most important data structure for data analysis. They're like matrices but allow different data types in different columns, making them perfect for real-world datasets.

19.1.1 Data Frame Creation

```
# Basic data frame creation
employee_df <- data.frame(
  employee_id = 1:6,
  first_name = c("Alice", "Bob", "Charlie", "Diana", "Eve", "Frank"),
  last_name = c("Johnson", "Smith", "Brown", "Davis", "Wilson", "Miller"),
  department = c("IT", "Sales", "IT", "HR", "Sales", "IT"),
  salary = c(75000, 65000, 80000, 70000, 68000, 82000),
  start_date = as.Date(c("2020-01-15", "2019-06-01", "2021-03-10",
                        "2020-08-20", "2019-11-05", "2022-02-01")),
  is_manager = c(FALSE, TRUE, FALSE, TRUE, FALSE, TRUE),
  performance_rating = factor(c("Excellent", "Good", "Excellent",
                                "Good", "Fair", "Excellent"),
                              levels = c("Poor", "Fair", "Good", "Excellent"),
                              ordered = TRUE),
  stringsAsFactors = FALSE # Keep strings as characters
)

print(employee_df)
```

	employee_id	first_name	last_name	department	salary	start_date	is_manager
1	1	Alice	Johnson	IT	75000	2020-01-15	FALSE
2	2	Bob	Smith	Sales	65000	2019-06-01	TRUE
3	3	Charlie	Brown	IT	80000	2021-03-10	FALSE
4	4	Diana	Davis	HR	70000	2020-08-20	TRUE
5	5	Eve	Wilson	Sales	68000	2019-11-05	FALSE

6	6	Frank	Miller	IT	82000	2022-02-01	TRUE
	performance_rating						
1	Excellent						
2	Good						
3	Excellent						
4	Good						
5	Fair						
6	Excellent						

19.1.2 Data Frame Properties and Inspection

```
# Basic properties
cat("Dimensions:", dim(employee_df), "\n")
```

Dimensions: 6 8

```
cat("Number of rows:", nrow(employee_df), "\n")
```

Number of rows: 6

```
cat("Number of columns:", ncol(employee_df), "\n")
```

Number of columns: 8

```
cat("Column names:", names(employee_df), "\n")
```

Column names: employee_id first_name last_name department salary start_date is_manager performance_rating

```
# Data structure overview
cat("Data frame structure:\n")
```

Data frame structure:

```
str(employee_df)
```

```
'data.frame': 6 obs. of 8 variables:
 $ employee_id      : int  1 2 3 4 5 6
 $ first_name       : chr  "Alice" "Bob" "Charlie" "Diana" ...
 $ last_name        : chr  "Johnson" "Smith" "Brown" "Davis" ...
 $ department       : chr  "IT" "Sales" "IT" "HR" ...
 $ salary           : num  75000 65000 80000 70000 68000 82000
 $ start_date       : Date, format: "2020-01-15" "2019-06-01" ...
 $ is_manager       : logi  FALSE TRUE FALSE TRUE FALSE TRUE
 $ performance_rating: Ord.factor w/ 4 levels "Poor"<"Fair"<...: 4 3 4 3 2 4
```

```
# Summary statistics
cat("Summary statistics:\n")
```

Summary statistics:

```
summary(employee_df)
```

employee_id	first_name	last_name	department
Min. :1.00	Length:6	Length:6	Length:6
1st Qu.:2.25	Class :character	Class :character	Class :character
Median :3.50	Mode :character	Mode :character	Mode :character
Mean :3.50			
3rd Qu.:4.75			
Max. :6.00			
salary	start_date	is_manager	performance_rating
Min. :65000	Min. :2019-06-01	Mode :logical	Poor :0
1st Qu.:68500	1st Qu.:2019-11-22	FALSE:3	Fair :1
Median :72500	Median :2020-05-03	TRUE :3	Good :2
Mean :73333	Mean :2020-07-14		Excellent:3
3rd Qu.:78750	3rd Qu.:2021-01-18		
Max. :82000	Max. :2022-02-01		

```
# Data types of each column
column_types <- sapply(employee_df, class)
cat("Column types:\n")
```

Column types:

```
print(column_types)
```

```
$employee_id  
[1] "integer"
```

```
$first_name  
[1] "character"
```

```
$last_name  
[1] "character"
```

```
$department  
[1] "character"
```

```
$salary  
[1] "numeric"
```

```
$start_date  
[1] "Date"
```

```
$is_manager  
[1] "logical"
```

```
$performance_rating  
[1] "ordered" "factor"
```

```
# Check for missing values  
missing_values <- colSums(is.na(employee_df))  
cat("Missing values per column:\n")
```

Missing values per column:

```
print(missing_values)
```

employee_id	first_name	last_name	department
0	0	0	0
salary	start_date	is_manager	performance_rating
0	0	0	0

19.1.3 Data Frame Indexing and Access

```
# Column access methods
cat("First names (using $):", employee_df$first_name, "\n")
```

First names (using \$): Alice Bob Charlie Diana Eve Frank

```
cat("First names (using [[]]):", employee_df[["first_name"]], "\n")
```

First names (using [[]]): Alice Bob Charlie Diana Eve Frank

```
cat("Class of extracted column:", class(employee_df$salary), "\n")
```

Class of extracted column: numeric

```
# Single bracket returns data frame
first_name_df <- employee_df["first_name"]
cat("Class of [column]:", class(first_name_df), "\n")
```

Class of [column]: data.frame

```
# Multiple column selection
name_salary <- employee_df[c("first_name", "last_name", "salary")]
cat("Multiple columns:\n")
```

Multiple columns:

```
print(head(name_salary, 3))
```

	first_name	last_name	salary
1	Alice	Johnson	75000
2	Bob	Smith	65000
3	Charlie	Brown	80000

```
# Row access
first_employee <- employee_df[1, ]
cat("First employee:\n")
```

First employee:

```
print(first_employee)
```

```
  employee_id first_name last_name department salary start_date is_manager
1           1      Alice   Johnson          IT  75000 2020-01-15      FALSE
  performance_rating
1           Excellent
```

```
# Specific cell access
alice_salary <- employee_df[1, "salary"]
cat("Alice's salary:", alice_salary, "\n")
```

Alice's salary: 75000

```
# Range selection
first_three <- employee_df[1:3, c("first_name", "department", "salary")]
cat("First three employees (selected columns):\n")
```

First three employees (selected columns):

```
print(first_three)
```

```
  first_name department salary
1      Alice          IT  75000
2        Bob        Sales  65000
3    Charlie          IT  80000
```

19.1.4 Data Frame Filtering and Subsetting

```

employee_df <- data.frame(
  employee_id = 1:6,
  first_name = c("Alice", "Bob", "Charlie", "Diana", "Eve", "Frank"),
  last_name = c("Johnson", "Smith", "Brown", "Davis", "Wilson", "Miller"),
  department = c("IT", "Sales", "IT", "HR", "Sales", "IT"),
  salary = c(75000, 65000, 80000, 70000, 68000, 82000),
  start_date = as.Date(c("2020-01-15", "2019-06-01", "2021-03-10",
                        "2020-08-20", "2019-11-05", "2022-02-01")),
  is_manager = c(FALSE, TRUE, FALSE, TRUE, FALSE, TRUE),
  performance_rating = factor(c("Excellent", "Good", "Excellent",
                                "Good", "Fair", "Excellent"),
                              levels = c("Poor", "Fair", "Good", "Excellent"),
                              ordered = TRUE),
  stringsAsFactors = FALSE
)
# Simple filtering
it_employees <- employee_df[employee_df$department == "IT", ]
cat("IT employees:\n")

```

IT employees:

```
print(it_employees[, c("first_name", "last_name", "salary")])
```

	first_name	last_name	salary
1	Alice	Johnson	75000
3	Charlie	Brown	80000
6	Frank	Miller	82000

```

# Multiple conditions with logical operators
high_earners_it <- employee_df[employee_df$department == "IT" &
                               employee_df$salary > 75000, ]
cat("High-earning IT employees:\n")

```

High-earning IT employees:

```
print(high_earners_it[, c("first_name", "salary")])
```

	first_name	salary
3	Charlie	80000
6	Frank	82000

```
# Using subset() function (more readable)
managers_subset <- subset(employee_df,
                           is_manager == TRUE,
                           select = c("first_name", "last_name", "department", "salary"))
cat("Managers (using subset):\n")
```

Managers (using subset):

```
print(managers_subset)
```

	first_name	last_name	department	salary
2	Bob	Smith	Sales	65000
4	Diana	Davis	HR	70000
6	Frank	Miller	IT	82000

```
# Complex filtering with %in%
target_departments <- c("IT", "Sales")
it_or_sales <- employee_df[employee_df$department %in% target_departments, ]
cat("IT or Sales employees:\n")
```

IT or Sales employees:

```
print(it_or_sales[, c("first_name", "department", "salary")])
```

	first_name	department	salary
1	Alice	IT	75000
2	Bob	Sales	65000
3	Charlie	IT	80000
5	Eve	Sales	68000
6	Frank	IT	82000

```
# Filtering with date conditions
recent_hires <- employee_df[employee_df$start_date > as.Date("2020-01-01"), ]
cat("Recent hires (after 2020-01-01):\n")
```

Recent hires (after 2020-01-01):

```
print(recent_hires[, c("first_name", "start_date")])
```

```

first_name start_date
1      Alice 2020-01-15
3    Charlie 2021-03-10
4      Diana 2020-08-20
6      Frank 2022-02-01

```

19.1.5 Data Frame Modification

```

# Create a working copy
df_work <- employee_df

# Add new columns
df_work$full_name <- paste(df_work$first_name, df_work$last_name)
df_work$years_employed <- as.numeric(Sys.Date() - df_work$start_date) / 365.25
df_work$salary_category <- ifelse(df_work$salary > 75000, "High", "Standard")

# Conditional column creation
df_work$bonus_eligible <- with(df_work,
  (years_employed > 2) & (performance_rating %in% c("Good", "Excellent"))
)

# Calculate bonus amount based on conditions
df_work$bonus <- ifelse(df_work$bonus_eligible,
  df_work$salary * 0.1, # 10% bonus
  0)

cat("Modified data frame with new columns:\n")

```

Modified data frame with new columns:

```
print(df_work[, c("full_name", "years_employed", "salary_category", "bonus")])
```

```

full_name years_employed salary_category bonus
1 Alice Johnson      5.908282      Standard  7500
2   Bob Smith       6.532512      Standard  6500
3 Charlie Brown     4.758385        High  8000

```

4	Diana Davis	5.311431	Standard	7000
5	Eve Wilson	6.102669	Standard	0
6	Frank Miller	3.860370	High	8200

```
# Modify existing values
df_work$salary[df_work$first_name == "Alice"] <- 78000 # Give Alice a raise
df_work$is_manager[df_work$first_name == "Charlie"] <- TRUE # Promote Charlie

# Add new rows
new_employee <- data.frame(
  employee_id = 7,
  first_name = "Grace",
  last_name = "Lee",
  department = "HR",
  salary = 72000,
  start_date = as.Date("2023-01-15"),
  is_manager = FALSE,
  performance_rating = factor("Good", levels = levels(df_work$performance_rating)),
  full_name = "Grace Lee",
  years_employed = 1.0,
  salary_category = "Standard",
  bonus_eligible = FALSE,
  bonus = 0
)

df_work <- rbind(df_work, new_employee)
cat("After adding new employee:", nrow(df_work), "total employees\n")
```

After adding new employee: 7 total employees

19.1.6 Advanced Data Frame Operations

```
# Using dplyr for modern data manipulation
library(dplyr)

# Modern data manipulation with pipes
summary_stats <- employee_df |>
  mutate(
    full_name = paste(first_name, last_name),
    years_employed = as.numeric(Sys.Date() - start_date) / 365.25,
```

```

    salary_k = salary / 1000 # Convert to thousands
  ) |>
  filter(department %in% c("IT", "Sales")) |>
  group_by(department, is_manager) |>
  summarise(
    count = n(),
    avg_salary = mean(salary),
    avg_years = mean(years_employed),
    .groups = "drop"
  ) |>
  arrange(department, desc(is_manager))

cat("Summary statistics by department and management status:\n")

```

Summary statistics by department and management status:

```
print(summary_stats)
```

```

# A tibble: 4 x 5
  department is_manager count avg_salary avg_years
  <chr>      <lgl>      <int>   <dbl>   <dbl>
1 IT        TRUE         1    82000    3.86
2 IT        FALSE        2    77500    5.33
3 Sales     TRUE         1    65000    6.53
4 Sales     FALSE        1    68000    6.10

```

```

# Advanced filtering with case_when
employee_df_enhanced <- employee_df |>
mutate(
  experience_level = case_when(
    as.numeric(Sys.Date() - start_date) / 365.25 < 2 ~ "Junior",
    as.numeric(Sys.Date() - start_date) / 365.25 < 4 ~ "Mid-level",
    TRUE ~ "Senior"
  ),
  compensation_tier = case_when(
    salary < 70000 ~ "Tier 1",
    salary < 80000 ~ "Tier 2",
    TRUE ~ "Tier 3"
  )
)

```

```
cat("Enhanced employee data with calculated fields:\n")
```

Enhanced employee data with calculated fields:

```
print(employee_df_enhanced |>
  select(first_name, department, experience_level, compensation_tier))
```

	first_name	department	experience_level	compensation_tier
1	Alice	IT	Senior	Tier 2
2	Bob	Sales	Senior	Tier 1
3	Charlie	IT	Senior	Tier 3
4	Diana	HR	Senior	Tier 2
5	Eve	Sales	Senior	Tier 1
6	Frank	IT	Mid-level	Tier 3

19.1.7 Data Frame Aggregation and Grouping

```
# Base R aggregation
dept_summary <- aggregate(salary ~ department, data = employee_df,
  FUN = function(x) c(mean = mean(x),
    median = median(x),
    count = length(x)))
cat("Department salary summary:\n")
```

Department salary summary:

```
print(dept_summary)
```

	department	salary.mean	salary.median	salary.count
1	HR	70000	70000	1
2	IT	79000	80000	3
3	Sales	66500	66500	2

```
# Multiple grouping variables
dept_mgr_summary <- aggregate(salary ~ department + is_manager,
  data = employee_df,
  FUN = mean)
cat("Average salary by department and management status:\n")
```


Average salary by department and management status:

```
print(dept_mgr_summary)
```

	department	is_manager	salary
1	IT	FALSE	77500
2	Sales	FALSE	68000
3	HR	TRUE	70000
4	IT	TRUE	82000
5	Sales	TRUE	65000

```
# Using by() function for more complex operations
performance_by_dept <- by(employee_df$performance_rating,
                           employee_df$department,
                           table)
cat("Performance ratings by department:\n")
```

Performance ratings by department:

```
print(performance_by_dept)
```

employee_df\$department: HR

Poor	Fair	Good	Excellent
0	0	1	0

employee_df\$department: IT

Poor	Fair	Good	Excellent
0	0	0	3

employee_df\$department: Sales

Poor	Fair	Good	Excellent
0	1	1	0

```
# Table operations for cross-tabulation
dept_performance_table <- table(employee_df$department,
                                 employee_df$performance_rating)
cat("Department vs Performance cross-table:\n")
```

Department vs Performance cross-table:

```
print(dept_performance_table)
```

	Poor	Fair	Good	Excellent
HR	0	0	1	0
IT	0	0	0	3
Sales	0	1	1	0

```
# Proportional tables
```

```
dept_performance_prop <- prop.table(dept_performance_table, margin = 1)  
cat("Performance distribution within each department:\n")
```

Performance distribution within each department:

```
print(round(dept_performance_prop, 3))
```

	Poor	Fair	Good	Excellent
HR	0.0	0.0	1.0	0.0
IT	0.0	0.0	0.0	1.0
Sales	0.0	0.5	0.5	0.0

20 Arrays: Multi-Dimensional Data

20.1 Understanding Arrays

Arrays extend matrices to more than two dimensions. They're useful for storing multi-dimensional scientific or business data.

20.1.1 Array Creation and Structure

```
# Create a 3-dimensional array (think: multiple matrices stacked)
# Dimensions: 4 quarters × 3 products × 2 years
sales_array <- array(
  data = c(100, 120, 110, 130,    # Q1-Q4 for Product A, Year 1
           150, 160, 140, 170,    # Q1-Q4 for Product B, Year 1
           200, 210, 190, 220,    # Q1-Q4 for Product C, Year 1
           110, 130, 120, 140,    # Q1-Q4 for Product A, Year 2
           160, 170, 150, 180,    # Q1-Q4 for Product B, Year 2
           210, 220, 200, 230),   # Q1-Q4 for Product C, Year 2
  dim = c(4, 3, 2), # 4 quarters, 3 products, 2 years
  dimnames = list(
    quarters = paste0("Q", 1:4),
    products = c("Product_A", "Product_B", "Product_C"),
    years = c("2022", "2023")
  )
)

print(sales_array)
```

```
, , years = 2022
```

```
      products
quarters Product_A Product_B Product_C
Q1         100       150       200
Q2         120       160       210
```

Q3	110	140	190
Q4	130	170	220

```
, , years = 2023
```

	products		
quarters	Product_A	Product_B	Product_C
Q1	110	160	210
Q2	130	170	220
Q3	120	150	200
Q4	140	180	230

```
# Array properties
cat("Array dimensions:", dim(sales_array), "\n")
```

Array dimensions: 4 3 2

```
cat("Dimension names:\n")
```

Dimension names:

```
print(dimnames(sales_array))
```

```
$quarters
[1] "Q1" "Q2" "Q3" "Q4"
```

```
$products
[1] "Product_A" "Product_B" "Product_C"
```

```
$years
[1] "2022" "2023"
```

```
cat("Total elements:", length(sales_array), "\n")
```

Total elements: 24

20.1.2 Array Indexing and Slicing

```
# Single element access
q2_productb_2022 <- sales_array["Q2", "Product_B", "2022"]
cat("Q2 Product B 2022 sales:", q2_productb_2022, "\n")
```

Q2 Product B 2022 sales: 160

```
# Slice operations
# All quarters and products for 2022
sales_2022 <- sales_array[, , "2022"]
cat("All 2022 sales:\n"); print(sales_2022)
```

All 2022 sales:

	products		
quarters	Product_A	Product_B	Product_C
Q1	100	150	200
Q2	120	160	210
Q3	110	140	190
Q4	130	170	220

```
# Product A sales across all quarters and years
product_a_sales <- sales_array[, "Product_A", ]
cat("Product A sales across time:\n"); print(product_a_sales)
```

Product A sales across time:

	years	
quarters	2022	2023
Q1	100	110
Q2	120	130
Q3	110	120
Q4	130	140

```
# Q1 sales for all products and years
q1_sales <- sales_array["Q1", , ]
cat("Q1 sales across products and years:\n"); print(q1_sales)
```

Q1 sales across products and years:

	years	
products	2022	2023
Product_A	100	110
Product_B	150	160
Product_C	200	210

```
# Multiple quarters for specific product and year
q1_q2_productc_2023 <- sales_array[c("Q1", "Q2"), "Product_C", "2023"]
cat("Q1-Q2 Product C 2023:", q1_q2_productc_2023, "\n")
```

Q1-Q2 Product C 2023: 210 220

20.1.3 Array Operations with Apply

```
# Apply operations across different dimensions
# MARGIN = 1: across quarters
# MARGIN = 2: across products
# MARGIN = 3: across years

# Total sales by quarter (sum across products and years)
quarterly_totals <- apply(sales_array, 1, sum)
cat("Quarterly totals:", quarterly_totals, "\n")
```

Quarterly totals: 930 1010 910 1070

```
# Total sales by product (sum across quarters and years)
product_totals <- apply(sales_array, 2, sum)
cat("Product totals:", product_totals, "\n")
```

Product totals: 960 1280 1680

```
# Total sales by year (sum across quarters and products)
yearly_totals <- apply(sales_array, 3, sum)
cat("Yearly totals:", yearly_totals, "\n")
```

Yearly totals: 1900 2020

```
# Average sales by quarter and product (across years)
avg_by_quarter_product <- apply(sales_array, c(1, 2), mean)
cat("Average sales by quarter and product (across years):\n")
```

Average sales by quarter and product (across years):

```
print(avg_by_quarter_product)
```

	products		
quarters	Product_A	Product_B	Product_C
Q1	105	155	205
Q2	125	165	215
Q3	115	145	195
Q4	135	175	225

```
# Growth rates between years for each quarter-product combination
growth_rates <- apply(sales_array, c(1, 2), function(x) (x[2] - x[1]) / x[1] * 100)
cat("Growth rates (2022 to 2023) by quarter and product:\n")
```

Growth rates (2022 to 2023) by quarter and product:

```
print(round(growth_rates, 1))
```

	products		
quarters	Product_A	Product_B	Product_C
Q1	10.0	6.7	5.0
Q2	8.3	6.2	4.8
Q3	9.1	7.1	5.3
Q4	7.7	5.9	4.5

21 Advanced Data Structures and Concepts

21.1 Environments: R's Scoping System

```
# Environments store object-name bindings
current_env <- environment()
global_env <- .GlobalEnv

# Create a new environment
my_env <- new.env()
my_env$x <- 10
my_env$y <- 20
my_env$calculate <- function(a, b) a + b

# List objects in environment
cat("Objects in my_env:", ls(my_env), "\n")
```

Objects in my_env: calculate x y

```
# Access environment objects
cat("x from my_env:", my_env$x, "\n")
```

x from my_env: 10

```
cat("Calculation result:", my_env$calculate(my_env$x, my_env$y), "\n")
```

Calculation result: 30

```
# Environment hierarchy
cat("Current environment:", environmentName(current_env), "\n")
```

Current environment: R_GlobalEnv


```
cat("Parent environment:", environmentName(parent.env(current_env)), "\n")
```

Parent environment: package:dplyr

```
# Search path for R objects
search_path <- search()
cat("R search path:\n")
```

R search path:

```
print(search_path)
```

```
[1] ".GlobalEnv"      "package:dplyr"      "package:purrrr"
[4] "package:forcats"  "package:stats"      "package:graphics"
[7] "package:grDevices" "package:utils"      "package:datasets"
[10] "package:methods"  "Autoloads"          "package:base"
```

21.2 Object-Oriented Programming: S3 Classes

```
# Create an S3 object (simple object-oriented programming)
create_bank_account <- function(owner, balance = 0) {
  account <- list(
    owner = owner,
    balance = balance,
    transactions = data.frame(
      date = as.Date(character(0)),
      type = character(0),
      amount = numeric(0),
      stringsAsFactors = FALSE
    )
  )
  class(account) <- "bank_account"
  return(account)
}

# Create methods for our class
print.bank_account <- function(x) {
```

```

    cat("Bank Account\n")
    cat("Owner:", x$owner, "\n")
    cat("Balance:  $", x$balance, "\n")
    cat("Transactions:", nrow(x$transactions), "\n")
  }

deposit.bank_account <- function(account, amount) {
  if (amount <= 0) {
    stop("Deposit amount must be positive")
  }
  account$balance <- account$balance + amount
  new_transaction <- data.frame(
    date = Sys.Date(),
    type = "deposit",
    amount = amount
  )
  account$transactions <- rbind(account$transactions, new_transaction)
  return(account)
}

withdraw.bank_account <- function(account, amount) {
  if (amount <= 0) {
    stop("Withdrawal amount must be positive")
  }
  if (amount > account$balance) {
    stop("Insufficient funds")
  }
  account$balance <- account$balance - amount
  new_transaction <- data.frame(
    date = Sys.Date(),
    type = "withdrawal",
    amount = -amount
  )
  account$transactions <- rbind(account$transactions, new_transaction)
  return(account)
}

# Use our S3 class
my_account <- create_bank_account("John Doe", 1000)
print(my_account)

```

Bank Account

Owner: John Doe
Balance: \$ 1000
Transactions: 0

```
my_account <- deposit.bank_account(my_account, 500)
my_account <- withdraw.bank_account(my_account, 200)
print(my_account)
```

Bank Account
Owner: John Doe
Balance: \$ 1300
Transactions: 2

22 Performance Optimization and Best Practices

22.1 Memory Efficiency and Performance

22.1.1 Pre-allocation vs Growing Objects

```
# Demonstrate the performance difference
n <- 10000

# BAD: Growing objects (very slow)
system.time({
  bad_vector <- numeric(0)
  for (i in 1:n) {
    bad_vector <- c(bad_vector, i^2)
  }
})
```

```
      user  system elapsed
0.090    0.015    0.105
```

```
# GOOD: Pre-allocation (much faster)
system.time({
  good_vector <- numeric(n)
  for (i in 1:n) {
    good_vector[i] <- i^2
  }
})
```

```
      user  system elapsed
0.003    0.000    0.002
```

```
# BEST: Vectorized operations (fastest)
system.time({
  best_vector <- (1:n)^2
})
```

```
      user  system elapsed
0.000   0.000   0.001
```

```
# Verify all methods give same result
cat("All methods equivalent:",
    identical(bad_vector, good_vector) && identical(good_vector, best_vector), "\n")
```

All methods equivalent: TRUE

22.1.2 Data Structure Selection for Performance

```
library(microbenchmark)

# Create test data
n_rows <- 1000
test_matrix <- matrix(rnorm(n_rows * 10), nrow = n_rows)
test_df <- as.data.frame(test_matrix)

# Compare performance for column operations
performance_test <- microbenchmark(
  matrix_colsums = colSums(test_matrix),
  df_apply = apply(test_df, 2, sum),
  df_sapply = sapply(test_df, sum),
  times = 100
)

print(performance_test)
```

Unit: microseconds

	expr	min	lq	mean	median	uq	max	neval
matrix_colsums		24.406	25.0715	26.9134	26.4695	27.5160	43.601	100
df_apply		167.262	172.3515	210.2912	186.9035	240.3530	337.079	100
df_sapply		36.007	37.8355	41.9073	40.8865	42.5845	71.233	100

```
# Memory usage comparison
cat("Matrix memory:", as.numeric(object.size(test_matrix)), "bytes\n")
```

Matrix memory: 80216 bytes

```
cat("Data frame memory:", as.numeric(object.size(test_df)), "bytes\n")
```

Data frame memory: 81904 bytes

22.1.3 Factors vs Characters for Memory

```
# Compare memory usage: factors vs characters for categorical data
n_observations <- 100000
categories <- c("Category_A", "Category_B", "Category_C", "Category_D")

# Create character vector
char_vector <- sample(categories, n_observations, replace = TRUE)

# Create factor
factor_vector <- factor(char_vector)

# Memory comparison
char_size <- as.numeric(object.size(char_vector))
factor_size <- as.numeric(object.size(factor_vector))

cat("Character vector memory:", char_size, "bytes\n")
```

Character vector memory: 800304 bytes

```
cat("Factor memory:", factor_size, "bytes\n")
```

Factor memory: 400720 bytes

```
cat("Factor saves:", round((char_size - factor_size) / char_size * 100, 1), "% memory\n")
```

Factor saves: 49.9 % memory

```
# Performance comparison for operations
cat("\nPerformance comparison for table operations:\n")
```

Performance comparison for table operations:

```
performance_categorical <- microbenchmark(
  character_table = table(char_vector),
  factor_table = table(factor_vector),
  times = 50
)
print(performance_categorical)
```

Unit: milliseconds

	expr	min	lq	mean	median	uq	max	neval
	character_table	3.531547	3.592912	4.112093	3.617718	3.739005	7.783007	50
	factor_table	1.373481	1.394762	1.585334	1.402436	1.419989	3.876461	50

22.2 Best Practices Summary

22.2.1 1. Data Structure Selection Guide

```
# Function to recommend data structure
recommend_structure <- function(data_description) {
  cat("Data Structure Recommendation Guide\n")
  cat("=====\n\n")

  recommendations <- list(
    "Homogeneous numeric data, mathematical operations" = "Matrix or Numeric Vector",
    "Heterogeneous tabular data" = "Data Frame or Tibble",
    "Categorical data with known levels" = "Factor",
    "Hierarchical or nested data" = "List",
    "Mixed-type collection of objects" = "List",
    "Multi-dimensional scientific data" = "Array",
    "Simple sequence of same-type values" = "Atomic Vector"
  )

  for (i in seq_along(recommendations)) {
```

```

    cat(".", names(recommendations)[i], "\n")
    cat("  →", recommendations[[i]], "\n\n")
  }
}

recommend_structure()

```

Data Structure Recommendation Guide

=====

- Homogeneous numeric data, mathematical operations
→ Matrix or Numeric Vector
- Heterogeneous tabular data
→ Data Frame or Tibble
- Categorical data with known levels
→ Factor
- Hierarchical or nested data
→ List
- Mixed-type collection of objects
→ List
- Multi-dimensional scientific data
→ Array
- Simple sequence of same-type values
→ Atomic Vector

22.2.2 2. Common Pitfalls and Solutions

```

# Common pitfall demonstrations and solutions

cat("PITFALL 1: Automatic type coercion\n")

```

PITFALL 1: Automatic type coercion


```
mixed_data <- c(1, 2, 3, "four")
cat("Result type:", typeof(mixed_data), "\n")
```

Result type: character

```
cat("Solution: Use lists for mixed types\n\n")
```

Solution: Use lists for mixed types

```
cat("PITFALL 2: Factor level surprises\n")
```

PITFALL 2: Factor level surprises

```
original_factor <- factor(c("low", "medium", "high"))
cat("Original levels:", levels(original_factor), "\n")
```

Original levels: high low medium

```
# Adding new level without updating factor
# new_factor <- c(original_factor, "very_high") # This would cause issues
cat("Solution: Update levels before adding new data\n\n")
```

Solution: Update levels before adding new data

```
cat("PITFALL 3: Ignoring missing values\n")
```

PITFALL 3: Ignoring missing values

```
data_with_na <- c(1, 2, NA, 4, 5)
cat("Mean without na. rm:", mean(data_with_na), "\n")
```

Mean without na. rm: NA

```
cat("Mean with na.rm=TRUE:", mean(data_with_na, na.rm = TRUE), "\n\n")
```

Mean with na.rm=TRUE: 3

```
cat("PITFALL 4: Matrix vs data frame confusion\n")
```

PITFALL 4: Matrix vs data frame confusion

```
test_matrix <- matrix(1:6, nrow = 2)
test_df <- data.frame(x = 1:2, y = 3:4, z = 5:6)
cat("Matrix column access returns:", class(test_matrix[, 1]), "\n")
```

Matrix column access returns: integer

```
cat("Data frame column access returns:", class(test_df[, 1]), "\n")
```

Data frame column access returns: integer

```
cat("Solution: Use drop=FALSE to maintain structure\n")
```

Solution: Use drop=FALSE to maintain structure

```
cat("Matrix with drop=FALSE:", class(test_matrix[, 1, drop = FALSE]), "\n")
```

Matrix with drop=FALSE: matrix array

22.2.3 3. Debugging and Validation Functions

```
# Utility functions for data structure debugging
debug_structure <- function(obj, name = deparse(substitute(obj))) {
  cat("=== Debug Info for", name, "===\n")
  cat("Class:", class(obj), "\n")
  cat("Type:", typeof(obj), "\n")
  cat("Length/Dimensions:",
      if(is.null(dim(obj))) length(obj) else paste(dim(obj), collapse = "x"), "\n")

  if (is.factor(obj)) {
    cat("Factor levels:", nlevels(obj), "\n")
    cat("Ordered:", is.ordered(obj), "\n")
  }
}
```

```

if (any(is.na(obj))) {
  cat("Missing values:", sum(is.na(obj)), "\n")
}

cat("Memory usage:", as.numeric(object.size(obj)), "bytes\n")
cat("=====\n\n")
}

# Validate data frame structure
validate_dataframe <- function(df, required_cols = NULL) {
  cat("Data Frame Validation\n")
  cat("=====\n")

  # Basic structure
  cat("Dimensions:", paste(dim(df), collapse = " x "), "\n")
  cat("Complete cases:", sum(complete.cases(df)), "/", nrow(df), "\n")

  # Check required columns
  if (!is.null(required_cols)) {
    missing_cols <- required_cols[!required_cols %in% names(df)]
    if (length(missing_cols) > 0) {
      cat("  Missing required columns:", paste(missing_cols, collapse = ", "), "\n")
    } else {
      cat("  All required columns present\n")
    }
  }

  # Data type summary
  cat("\nColumn types:\n")
  col_types <- sapply(df, function(x) paste(class(x), collapse = "/"))
  for (i in seq_along(col_types)) {
    cat("  ", names(col_types)[i], ":", col_types[i], "\n")
  }

  cat("=====\n\n")
}

# Example usage
debug_structure(employee_df)

```

=== Debug Info for employee_df ===

```
Class: data.frame
Type: list
Length/Dimensions: 6×8
Memory usage: 3752 bytes
=====
```

```
validate_dataframe(employee_df, required_cols = c("employee_id", "first_name", "salary"))
```

```
Data Frame Validation
```

```
=====
```

```
Dimensions: 6 × 8
```

```
Complete cases: 6 / 6
```

```
  All required columns present
```

```
Column types:
```

```
  employee_id : integer
```

```
  first_name  : character
```

```
  last_name   : character
```

```
  department  : character
```

```
  salary      : numeric
```

```
  start_date  : Date
```

```
  is_manager  : logical
```

```
  performance_rating : ordered/factor
```

```
=====
```

23 Summary and Quick Reference

23.1 Data Structure Comparison Table

Structure	Dimensions	Homogeneous	Best Use Cases	Memory Efficiency
Vector	1D	Yes	Mathematical operations, sequences	
Factor	1D	Yes	Categorical data, statistical modeling	
Matrix	2D	Yes	Linear algebra, image data	
Array	ND	Yes	Multi-dimensional scientific data	
List	1D	No	Complex nested structures	
Data Frame	2D	No	Data analysis, mixed-type tables	

23.2 Key Takeaways

23.2.1 For Beginners

- **Start with vectors and data frames** - they cover 80% of use cases
- **Understand indexing** - mastering \$, [], and [[]] is crucial
- **Use factors for categorical data** - they save memory and improve performance ###
For Intermediate Users
- **Leverage lists for complex data** - they allow nesting and mixed types
- **Master apply functions** - they enable efficient operations across data structures
- **Choose the right structure** - consider data type, operations, and memory needs ###
For Advanced Users
- **Optimize performance** - pre-allocate objects and prefer vectorized operations

- **Utilize S3 classes** - for custom object-oriented programming
- **Debug and validate** - use custom functions to ensure data integrity and structure

24 Manipulating Vectors, Data Frames, and Lists

```
# Load essential packages for pharmaceutical data analysis
library(haven)      # For reading SAS datasets (SDTM, ADaM)
library(dplyr)      # For data manipulation
library(tidyr)      # For data reshaping
library(stringr)    # For string manipulation
```

25 Chapter 1: Vector Operations in Pharmaceutical Context

Vectors are the fundamental building blocks of R. In pharmaceutical programming, vectors are used to store patient IDs, treatment codes, lab values, and categorical variables like adverse event terms.

25.1 1.1 Creating Vectors - The Foundation

```
# Creating vectors - essential for pharmaceutical data
# Patient IDs (sequential numbering)
patient_ids <- 1:10
patient_ids_desc <- 10:1
patient_ids_range <- -5:10

# Using c() function for explicit vector creation
dose_levels <- c(1, 5, 10, 25, 50, 100) # Dose escalation study
safety_population <- c(1:254)           # Safety population patient IDs

# Pharmaceutical example: Creating treatment codes
treatment_codes <- c("PLA", "LOW", "MED", "HIGH")
```

25.1.1 Understanding Vector Naming in Clinical Data

```
# Naming vectors is crucial for clinical data interpretation
dose_groups <- c(0, 5, 10, 25)
names(dose_groups) <- c("Placebo", "Low_Dose", "Medium_Dose", "High_Dose")
print(dose_groups)
```

Placebo	Low_Dose	Medium_Dose	High_Dose
0	5	10	25


```
# Clinical example: Lab normal ranges
lab_ranges <- c(70, 100, 140)
names(lab_ranges) <- c("Glucose_Low", "Glucose_Normal", "Glucose_High")
print(lab_ranges)
```

```
Glucose_Low Glucose_Normal  Glucose_High
          70           100           140
```

25.2 1.2 Vector Indexing - Accessing Patient Data

Vector indexing is critical when working with patient subsets, safety populations, or specific treatment arms.

```
# Sample adverse event severity scores
ae_severity <- c(1, 3, 5, 2, 4, 1, 3)

# Accessing specific patient data
ae_severity[2]           # Second patient's AE severity
```

```
[1] 3
```

```
ae_severity[c(1, 3, 5)]   # Patients 1, 3, and 5
```

```
[1] 1 5 4
```

```
ae_severity[-2]           # Exclude patient 2
```

```
[1] 1 5 2 4 1 3
```

```
ae_severity[-c(1, 3)]     # Exclude patients 1 and 3
```

```
[1] 3 2 4 1 3
```

```
# Important: Accessing non-existent indices
missing_patient <- ae_severity[10] # Returns NA
class(missing_patient)             # Still numeric class
```

```
[1] "numeric"
```

```
# Pharmaceutical application: Safety population subset
safety_ids <- 1:100
efficacy_subset <- safety_ids[1:85] # Efficacy population
```

25.3 1.3 Vector Modification - Updating Clinical Data

```
# Laboratory values that need updating
lab_values <- c(95, 87, 110, 92)

# Single value modification (data correction)
lab_values[2] <- 89 # Correcting second patient's value

# Multiple value updates (batch correction)
lab_values[c(1, 4)] <- c(96, 94)

# Extending vectors (new patient enrollment)
lab_values[8] <- lab_values[2] # Copying value
lab_values[9] <- 105           # New patient value

# Clinical scenario: Updating multiple positions
visit_scores <- c(10, 15, 20, 25)
visit_scores[c(2, 5)] <- c(16, 30) # Update visits 2 and 5
```

25.4 1.4 Arithmetic Operations - Statistical Calculations

```
# Biomarker values for statistical analysis
biomarker_values <- c(12.5, 15.3, 18.7, 14.2)

# Basic transformations common in clinical analysis
biomarker_values + 2.5 # Adding baseline adjustment
```

```
[1] 15.0 17.8 21.2 16.7
```

```
biomarker_values - 10          # Subtracting control value
```

```
[1] 2.5 5.3 8.7 4.2
```

```
biomarker_values * 1.2        # Scaling factor
```

```
[1] 15.00 18.36 22.44 17.04
```

```
biomarker_values / 2          # Half-life calculation
```

```
[1] 6.25 7.65 9.35 7.10
```

```
# Advanced arithmetic operations
```

```
biomarker_values %% 2         # Integer division
```

```
[1] 6 7 9 7
```

```
biomarker_values %% 3         # Modulo operation
```

```
[1] 0.5 0.3 0.7 2.2
```

```
# Essential statistical functions for clinical data
```

```
min(biomarker_values)        # Minimum value
```

```
[1] 12.5
```

```
max(biomarker_values)        # Maximum value
```

```
[1] 18.7
```

```
median(biomarker_values)     # Median (robust to outliers)
```

```
[1] 14.75
```

```
mean(biomarker_values)          # Mean value
```

```
[1] 15.175
```

```
range(biomarker_values)        # Range (min, max)
```

```
[1] 12.5 18.7
```

```
var(biomarker_values)          # Variance
```

```
[1] 6.849167
```

```
sd(biomarker_values)           # Standard deviation
```

```
[1] 2.617091
```

```
quantile(biomarker_values)     # Quartiles
```

```
      0%      25%      50%      75%     100%  
12.500 13.775 14.750 16.150 18.700
```

```
IQR(biomarker_values)          # Interquartile range
```

```
[1] 2.375
```

```
# Custom quantiles for clinical cutoffs  
quantile(biomarker_values, probs = c(0.25, 0.5, 0.75, 0.95))
```

```
      25%      50%      75%      95%  
13.775 14.750 16.150 18.190
```

25.5 1.5 The WHICH Function - Clinical Data Subsetting

The `which()` function is essential for identifying patients meeting specific clinical criteria.

```
# Patient vital signs data
systolic_bp <- c(120, 145, 135, 160, 110, 180, 125, 155)

# Identifying hypertensive patients (>140 mmHg)
hypertensive_logical <- systolic_bp > 140
hypertensive_positions <- which(systolic_bp > 140) # Returns positions
hypertensive_values <- systolic_bp[which(systolic_bp > 140)] # Returns values
hypertensive_values_alt <- systolic_bp[systolic_bp > 140] # Alternative method

# Finding extreme values (safety signals)
min_bp <- min(systolic_bp)
min_position <- which(systolic_bp == min_bp) # Position of minimum
min_value <- systolic_bp[which(systolic_bp == min_bp)] # Actual minimum value

# Using which.min() and which.max() for efficiency
which.min(systolic_bp) # Position only
```

```
[1] 5
```

```
which.max(systolic_bp) # Position only
```

```
[1] 6
```

```
# Complex clinical criteria (combination conditions)
# Patients with moderate hypertension (140-160 mmHg)
moderate_hypertension <- which(systolic_bp > 140 & systolic_bp <= 160)
moderate_bp_values <- systolic_bp[moderate_hypertension]

# Patients with either low (<110) or high (>170) BP
extreme_bp_positions <- which(systolic_bp < 110 | systolic_bp > 170)
extreme_bp_values <- systolic_bp[extreme_bp_positions]
```

25.6 1.6 The REP Function - Creating Repetitive Clinical Data

The `rep()` function is valuable for creating balanced study designs and simulating clinical data.

```

# Creating balanced treatment allocation
treatment_sequence <- rep(1:4, times = 25) # 25 patients per treatment (4 treatments)
placebo_only <- rep("Placebo", times = 50) # 50 placebo patients

# Alternating treatment pattern
alternating_trt <- rep(c("Active", "Placebo"), times = 10) # 10 cycles

# Creating patient identifiers with repetition
patient_labels <- rep("Patient", times = 100)
center_codes <- rep(c("Site_001", "Site_002", "Site_003"), times = c(30, 40, 50))

# Complex repetition patterns for study design
dose_levels <- rep(1:4, c(10, 15, 20, 25)) # Unequal allocation
treatment_blocks <- rep(1:3, each = 20) # 20 patients per block

# Multi-level repetition (nested study design)
nested_design <- rep(1:4, each = 5, times = 6) # 4 treatments, 5 reps, 6 cycles

```

25.7 1.7 The SEQ Function - Creating Clinical Sequences

The `seq()` function generates sequences critical for visit schedules, dose escalations, and time series analysis.

```

# Visit schedule creation (days from baseline)
visit_days <- seq(from = 0, to = 365, by = 28) # Monthly visits for 1 year
dose_escalation <- seq(from = 1, to = 100, by = 10) # Dose escalation steps

# Time-based sequences
hourly_timepoints <- seq(from = 0, to = 24, by = 2) # PK sampling every 2 hours
weekly_visits <- seq(from = 7, to = 84, by = 7) # Weekly visits for 12 weeks

# Creating equal intervals
pk_timepoints <- seq(from = 0, to = 24, length = 13) # 13 timepoints over 24 hours
biomarker_schedule <- seq(from = 1, by = 14, length = 8) # Bi-weekly for 16 weeks

# Combining sequences for complex study designs
baseline_visits <- seq(from = -14, to = 0, by = 7) # Screening period
treatment_visits <- seq(from = 1, to = 168, by = 14) # Treatment period
followup_visits <- seq(from = 182, to = 365, by = 28) # Follow-up period

complete_schedule <- c(baseline_visits, treatment_visits, followup_visits)

```

25.8 1.8 Sequence Helper Functions

```
# Working with patient vectors
patient_vitals <- c(120, 135, 142, 128, 156, 118, 145, 139)

# seq_len() - creates sequence from 1 to n
patient_count <- length(patient_vitals)
patient_indices <- seq_len(patient_count) # 1, 2, 3, ..., 8

# seq_along() - creates sequence matching vector length
visit_numbers <- seq_along(patient_vitals) # Same as seq_len(length(x))
```

25.9 1.9 Missing Data Handling - Critical in Clinical Studies

```
# Clinical lab values with missing data (common in real studies)
hemoglobin_values <- c(12.5, NA, 13.2, 11.8, NA, 14.1)

# Detecting missing values
missing_pattern <- is.na(hemoglobin_values) # Logical vector
complete_values <- hemoglobin_values[!is.na(hemoglobin_values)] # Complete cases only
complete_data <- na.omit(hemoglobin_values) # Alternative method

# Important: Never use == with NA
# hemoglobin_values == NA # WRONG - always returns NA
# hemoglobin_values[hemoglobin_values == NA] # WRONG

# Correct approach for missing data analysis
has_missing <- any(is.na(hemoglobin_values))
missing_count <- sum(is.na(hemoglobin_values))
complete_count <- sum(!is.na(hemoglobin_values))
```

25.10 1.10 Vector Element Membership - Clinical Criteria Checking

```
# Safety population and efficacy population patient IDs
safety_population <- 1:200
efficacy_population <- 1:150
per_protocol_population <- c(1:50, 55:120, 125:148)

# Checking patient membership in populations
patient_in_safety <- 175 %in% safety_population      # TRUE
patient_in_efficacy <- 175 %in% efficacy_population  # FALSE

# Population overlap analysis
safety_in_efficacy <- safety_population %in% efficacy_population
efficacy_in_pp <- efficacy_population %in% per_protocol_population

# Alternative using is.element()
element_check1 <- is.element(safety_population, efficacy_population)
element_check2 <- is.element(efficacy_population, per_protocol_population)

# Clinical application: Treatment compliance checking
compliant_patients <- c(1, 5, 8, 12, 15, 18, 22, 25, 30)
all_patients <- 1:50
compliance_status <- all_patients %in% compliant_patients
```

25.11 1.11 String Operations - Essential for Clinical Data

```
# Basic string output
print("Clinical Study Report")
```

```
[1] "Clinical Study Report"
```

```
# String concatenation for patient identifiers
patient_prefix <- "STUDY001-"
patient_numbers <- sprintf("%03d", 1:10) # Zero-padded numbers
patient_ids <- paste(patient_prefix, patient_numbers, sep = "")

# Creating visit labels
visit_labels <- paste("Visit", 1:10, sep = "_")
```



```

timepoint_labels <- paste("Day", c(1, 7, 14, 28, 56), sep = "_")

# Handling vectors of different lengths
site_codes <- c("001", "002", "003")
patient_ranges <- 1:30
full_patient_ids <- paste(site_codes, patient_ranges, sep = "-")

# Collapsing for summary output
treatment_arms <- c("Placebo", "Low_Dose", "High_Dose")
treatment_summary <- paste(treatment_arms, collapse = ", ")

# paste0() - efficient concatenation
adverse_event_ids <- paste0("AE", sprintf("%04d", 1:100))
lab_test_codes <- paste0("LAB_", c("HEM", "CHEM", "URN"), "_", 1:5)

# Using cat() for formatted output
cat("Study Status:", "Ongoing", "\n")

```

Study Status: Ongoing

```
cat("Enrolled Patients:", 156, "/", 200, "\n")
```

Enrolled Patients: 156 / 200

25.12 1.12 Advanced String Manipulation for Clinical Data

```

# Character counting for data validation
max_adverse_event_length <- max(nchar(c("Headache", "Nausea", "Fatigue", "Dizziness")))

# Case conversion for standardization
raw_adverse_events <- c("headache", "NAUSEA", "Fatigue", "dizziness")
standardized_upper <- toupper(raw_adverse_events)
standardized_lower <- tolower(raw_adverse_events)

# Using casefold() for flexible case conversion
ae_upper <- casefold(raw_adverse_events, upper = TRUE)
ae_lower <- casefold(raw_adverse_events, upper = FALSE)

```

```

# Character translation for data cleaning
lab_values_dirty <- "0. 5mg/dL" # 0 instead of 0
lab_values_clean <- chartr(old = "0", new = "0", x = lab_values_dirty)

# Sorting clinical terms
medical_terms <- c("Zyrtec", "Aspirin", "Benadryl", "Claritin")
terms_ascending <- sort(medical_terms, decreasing = FALSE)
terms_descending <- sort(medical_terms, decreasing = TRUE)

# Substring extraction for coding
study_id <- "PROTO-2024-ONCOLOGY-001"
protocol_year <- substr(study_id, start = 7, stop = 10)
therapeutic_area <- substr(study_id, start = 12, stop = 19)

```

25.13 1.13 Regular Expressions for Clinical Data Cleaning

```

# Load country data for site analysis
library(countrycode)
countries <- as.vector(countrycode::codelist$country.name.en)

# Countries starting with "A" (common site locations)
countries_a <- grep(pattern = "^A", x = countries, value = TRUE)
countries_a_positions <- grep(pattern = "^A", x = countries)

# Countries ending with "y"
countries_ending_y <- countries[grepl(pattern = "y$", x = countries)]

# Multi-word country names (important for site coding)
multiword_countries <- countries[grepl(pattern = "\\w\\s\\w", x = countries)]

# Countries ending with "e" OR "i"
countries_e_or_i <- countries[grepl(pattern = "e|i$", x = countries)]

# Countries containing "gin" (pattern matching)
countries_with_gin <- countries[grepl(pattern = "gin", x = countries)]

# Metacharacter handling in clinical data
financial_data <- c("cost $1000", "budget", "expense $500")

```

```

# Escaping special characters
dollar_entries <- financial_data[grepl(pattern = "\\$", x = financial_data)]

# Finding decimal numbers in lab results
lab_results <- c("12.5 mg/dL", "normal", "15.7 mg/dL")
decimal_results <- lab_results[grepl(pattern = "\\.", x = lab_results)]

# Anchoring patterns
medication_codes <- c("MED123", "ADMIN456", "MED789")
med_codes_only <- medication_codes[grepl("^MED", medication_codes)]
admin_codes_only <- medication_codes[grepl("456$", medication_codes)]

# Character classes for validation
study_codes <- c("ST123", "STUDY", "ST456")
numeric_codes <- study_codes[grepl("\\d", study_codes)]           # Contains digits
alpha_codes <- study_codes[grepl("\\D", study_codes)]             # Contains non-digits

# Vowel detection in adverse event terms
ae_terms <- c("headache", "nausea", "rash")
vowel_containing <- ae_terms[grepl("[aeiou]", ae_terms)]
numeric_containing <- ae_terms[grepl("[0-9]", ae_terms)]

```

25.14 1.14 String Replacement - Data Standardization

```

# Clinical data standardization
report_text <- "Patient received Drug A, and Patient showed improvement with Drug A."

# Single replacement vs. global replacement
single_replace <- sub(pattern = "Drug A", replacement = "Investigational Product", x = report_text)
global_replace <- gsub(pattern = "Drug A", replacement = "Investigational Product", x = report_text)

# Removing whitespace for data cleaning
messy_data <- "Patient ID: 12345 Status: Active"
clean_data <- gsub(pattern = "\\s+", replacement = " ", x = messy_data) # Multiple spaces to single
no_spaces <- gsub(pattern = "\\s", replacement = "", x = messy_data)    # Remove all spaces

# String splitting for data parsing
patient_info <- "LastName,FirstName,DOB,Gender"
info_components <- strsplit(x = patient_info, split = ",")

```

```
# Parsing structured clinical data
phone_numbers <- c("555-123-4567", "555-987-6543")
phone_parts <- strsplit(x = phone_numbers, split = "-")
```

26 Chapter 2: Data Frame Manipulation with dplyr - Clinical Data Analysis

Data frames are the primary data structure for clinical datasets (SDTM, ADaM). The dplyr package provides a grammar of data manipulation essential for clinical programming.

26.1 2.1 Creating Data Frames - Study Data Structure

```
# Basic clinical data frame
patient_demographics <- data.frame(
  USUBJID = paste0("STUDY001-", sprintf("%03d", 1:5)),
  AGE = c(45, 67, 23, 55, 34),
  SEX = c("M", "F", "M", "F", "M"),
  RACE = c("WHITE", "BLACK", "ASIAN", "WHITE", "OTHER"),
  ARM = c("Placebo", "Treatment", "Placebo", "Treatment", "Treatment"),
  RANDDT = as.Date(c("2024-01-15", "2024-01-16", "2024-01-17", "2024-01-18", "2024-01-19"))
)

# More complex data frame with vectors
visit_dates <- seq(as.Date("2024-01-01"), length = 10, by = "weeks")
visit_numbers <- 1:10
study_status <- rep(c("Ongoing", "Completed"), each = 5)

visit_schedule <- data.frame(
  VISITNUM = visit_numbers,
  VISITDT = visit_dates,
  STATUS = study_status,
  stringsAsFactors = FALSE # Important for character data
)

# Converting matrix to data frame (rare but useful)
lab_matrix <- matrix(data = rnorm(50, mean = 100, sd = 15), nrow = 10, ncol = 5)
rownames(lab_matrix) <- paste("Patient", 1:10, sep = "_")
```

```
colnames(lab_matrix) <- paste("Lab", LETTERS[1:5], sep = "_")
lab_dataframe <- as.data.frame(lab_matrix)
```

```
# Data frame inspection
```

```
dim(patient_demographics)      # Dimensions:  rows x columns
```

```
[1] 5 6
```

```
nrow(patient_demographics)     # Number of patients
```

```
[1] 5
```

```
ncol(patient_demographics)     # Number of variables
```

```
[1] 6
```

```
class(patient_demographics)    # Object class
```

```
[1] "data.frame"
```

```
str(patient_demographics)      # Structure summary
```

```
'data.frame':  5 obs. of  6 variables:
 $ USUBJID: chr  "STUDY001-001" "STUDY001-002" "STUDY001-003" "STUDY001-004" ...
 $ AGE    : num  45 67 23 55 34
 $ SEX    : chr  "M" "F" "M" "F" ...
 $ RACE   : chr  "WHITE" "BLACK" "ASIAN" "WHITE" ...
 $ ARM    : chr  "Placebo" "Treatment" "Placebo" "Treatment" ...
 $ RANDDT : Date, format: "2024-01-15" "2024-01-16" ...
```

26.2 2.2 Accessing Data Frame Elements - Patient Data Extraction

```
# Single column extraction (returns data frame)
age_df <- patient_demographics["AGE"]
class(age_df) # Still a data frame
```

```
[1] "data.frame"
```

```
# Single column extraction (returns vector)
age_vector <- patient_demographics[["AGE"]]
age_vector_alt <- patient_demographics$AGE
class(age_vector) # Vector
```

```
[1] "numeric"
```

```
# Multiple column extraction
demographics_subset <- patient_demographics[c("USUBJID", "AGE", "SEX")]

# Row and column slicing (critical for patient subsets)
# First patient's age
first_patient_age <- patient_demographics[1, "AGE"]

# First 3 patients, last 3 variables
subset_data <- patient_demographics[1:3, 4:6]

# All male patients (logical indexing)
male_patients <- patient_demographics[patient_demographics$SEX == "M", ]

# Treatment arm patients only
treatment_patients <- patient_demographics[patient_demographics$ARM == "Treatment", ]
```

26.3 2.3 Modifying Data Frames - Clinical Data Updates

```
# Create working copy
clinical_data <- patient_demographics

# Adding new columns (common in clinical programming)
clinical_data$AGEGR1 <- ifelse(clinical_data$AGE < 65, "<65", ">=65")
clinical_data$STUDY_DAY <- as.numeric(Sys.Date() - clinical_data$RANDDT)

# Adding multiple patients (enrollment updates)
new_patient <- data.frame(
  USUBJID = "STUDY001-006",
  AGE = 42,
  SEX = "F",
```

```

RACE = "HISPANIC",
ARM = "Placebo",
RANDDT = as.Date("2024-01-20"),
AGEGR1 = "<65",
STUDY_DAY = 1
)

# Proper way to add rows (rbind)
clinical_data <- rbind(clinical_data, new_patient)

# Adding site information (column-wise merge)
site_info <- data.frame(
  SITEID = c("001", "002", "001", "002", "001", "003"),
  COUNTRY = c("USA", "USA", "USA", "USA", "USA", "CAN")
)

clinical_data <- cbind(clinical_data, site_info)

# Removing columns (data cleanup)
clinical_data$STUDY_DAY <- NULL

# Removing rows (patient withdrawal)
# Remove last patient (index 6)
clinical_data <- clinical_data[-6, ]

```

26.4 2.4 Data Frame Summary and Subsetting

```

# Comprehensive data summary
summary(clinical_data)

```

USUBJID	AGE	SEX	RACE
Length:5	Min. :23.0	Length:5	Length:5
Class :character	1st Qu.:34.0	Class :character	Class :character
Mode :character	Median :45.0	Mode :character	Mode :character
	Mean :44.8		
	3rd Qu.:55.0		
	Max. :67.0		
ARM	RANDDT	AGEGR1	SITEID
Length:5	Min. :2024-01-15	Length:5	Length:5


```

Class :character    1st Qu.:2024-01-16    Class :character    Class :character
Mode  :character    Median :2024-01-17    Mode  :character    Mode  :character
                        Mean   :2024-01-17
                        3rd Qu.:2024-01-18
                        Max.   :2024-01-19

```

```

COUNTRY
Length:5
Class :character
Mode  :character

```

```

# Subsetting with conditions (safety population)
male_patients <- subset(clinical_data, SEX == "M")
elderly_female <- subset(clinical_data, SEX == "F" & AGE >= 65)
treatment_elderly <- subset(clinical_data, ARM == "Treatment" & AGE >= 65)

# Using which() for row indices
male_indices <- which(clinical_data$SEX == "M")
male_patients_alt <- clinical_data[male_indices, ]

# Statistical calculations
n_managers <- sum(clinical_data$ARM == "Treatment") # Treatment group size
mean_age <- mean(clinical_data$AGE)
age_by_treatment <- aggregate(AGE ~ ARM, data = clinical_data, FUN = mean)

# Creating composite variables
clinical_data$PATIENT_LABEL <- paste(clinical_data$USUBJID, clinical_data$ARM, sep = "_")

# Apply functions for calculations
numeric_columns <- c("AGE")
apply(clinical_data[numeric_columns], 2, function(x) c(mean = mean(x), sd = sd(x)))

```

```

AGE
mean 44.80000
sd   17.23949

```

27 Chapter 3: Reading SAS Datasets - SDTM and ADaM

```
# Reading SDTM datasets (Study Data Tabulation Model)
# Commented out as files may not exist in all environments
# dm <- haven::read_sas("data/sdtm/dm.sas7bdat") # Demographics
# ae <- haven::read_sas("data/sdtm/ae.sas7bdat") # Adverse Events
# vs <- haven::read_sas("data/sdtm/vs.sas7bdat") # Vital Signs
# lb <- haven::read_sas("data/sdtm/lb.sas7bdat") # Laboratory

# Reading ADaM datasets (Analysis Data Model)
# adsl <- haven::read_sas("data/adam/adsl.sas7bdat") # Subject Level Analysis Dataset
# adae <- haven::read_sas("data/adam/adae.sas7bdat") # Adverse Events Analysis Dataset
# adlb <- haven::read_sas("data/adam/adlb.sas7bdat") # Laboratory Analysis Dataset

# For demonstration, create sample datasets
dm <- data.frame(
  USUBJID = paste0("STUDY001-", sprintf("%03d", 1:100)),
  ARM = rep(c("Placebo", "Treatment A", "Treatment B"), length.out = 100),
  AGE = sample(18:80, 100, replace = TRUE),
  SEX = sample(c("M", "F"), 100, replace = TRUE),
  RACE = sample(c("WHITE", "BLACK", "ASIAN", "OTHER"), 100, replace = TRUE),
  RFSTDTC = sample(seq(as.Date("2024-01-01"), as.Date("2024-06-01"), by = "day"), 100),
  stringsAsFactors = FALSE
)

ae <- data.frame(
  USUBJID = sample(dm$USUBJID, 200, replace = TRUE),
  AETERM = sample(c("Headache", "Nausea", "Fatigue", "Dizziness", "Rash"), 200, replace = TRUE),
  AESEV = sample(c("MILD", "MODERATE", "SEVERE"), 200, replace = TRUE),
  AEOUT = sample(c("RECOVERED", "ONGOING", "NOT RECOVERED"), 200, replace = TRUE),
  stringsAsFactors = FALSE
)
```

28 Chapter 4: Advanced dplyr Operations - Clinical Data Manipulation

28.1 4.1 Column Selection - Choosing Clinical Variables

The `select()` function is fundamental for creating analysis datasets from raw SDTM data.

```
# Basic column selection for demographics analysis
dm_analysis <- dm %>%
  dplyr::select(USUBJID, ARM, AGE, SEX)

# Range selection (consecutive columns)
dm_basic <- dm %>%
  dplyr::select(USUBJID: ARM) # From USUBJID to ARM

# Numeric position selection
dm_first_three <- dm %>%
  dplyr::select(1:3)

# Helper functions for clinical programming
treatment_vars <- dm %>%
  dplyr::select(starts_with("ARM")) # Treatment-related variables

date_vars <- dm %>%
  dplyr::select(contains("DT")) # Date variables

baseline_vars <- dm %>%
  dplyr::select(ends_with("BL")) # Baseline variables

# Combining selections
key_demographics <- dm %>%
  dplyr::select(USUBJID, AGE, SEX, starts_with("ARM"))

# Alternative syntax (less preferred in pipelines)
dm_subset <- dplyr::select(dm, USUBJID, ARM)
```

28.2 4.2 Dropping Columns - Data Cleanup

```
# Dropping unnecessary columns for analysis
# Create sample ADL with many columns
adsl <- dm %>%
  dplyr::mutate(
    STUDYID = "STUDY001",
    TRT01A = ARM,
    TRT01P = ARM,
    TRT01AN = case_when(
      ARM == "Placebo" ~ 0,
      ARM == "Treatment A" ~ 1,
      ARM == "Treatment B" ~ 2
    ),
    SAFFL = "Y",
    ITTFL = "Y",
    RANDDT = RFSTDTC
  )

# Drop individual columns
adsl_clean <- adsl %>%
  dplyr::select(-c(STUDYID, RFSTDTC))

# Drop column ranges
adsl_minimal <- adsl %>%
  dplyr::select(-c(TRT01A: TRT01AN))

# Drop by position
adsl_no_first_col <- adsl %>%
  dplyr::select(-1)

# Drop multiple non-consecutive columns
adsl_focused <- adsl %>%
  dplyr::select(-c(1, 3, 5:7))

# Drop using helper functions
adsl_no_treatment <- adsl %>%
  dplyr::select(-starts_with("TRT"))

adsl_no_dates <- adsl %>%
  dplyr::select(-ends_with("DT"))
```

```
adsl_no_flags <- adsl %>%
  dplyr::select(-contains("FL"))
```

28.3 4.3 Filtering Rows - Patient Population Selection

Row filtering is critical for defining analysis populations (Safety, ITT, Per-Protocol).

```
# Safety population (patients who received at least one dose)
safety_population <- adsl %>%
  dplyr::filter(SAFFL == "Y")

# Treatment group analysis
placebo_patients <- adsl %>%
  dplyr::select(USUBJID, TRT01A, SEX, AGE) %>%
  dplyr::filter(TRT01A == "Placebo")

# Complex filtering (multiple conditions with AND)
elderly_male_treatment <- adsl %>%
  dplyr::select(USUBJID, TRT01A, SEX, AGE) %>%
  dplyr::filter(TRT01A == "Treatment A" & SEX == "M" & AGE >= 65)

# OR conditions (either condition can be true)
special_population <- adsl %>%
  dplyr::select(USUBJID, TRT01A, SEX, AGE) %>%
  dplyr::filter(TRT01A == "Placebo" & (SEX == "M" | AGE >= 70))

# Age range filtering
adult_population <- adsl %>%
  dplyr::select(USUBJID, TRT01A, SEX, AGE) %>%
  dplyr::filter(AGE >= 18 & AGE <= 65)

# Multiple value selection using %in%
active_treatments <- adsl %>%
  dplyr::select(USUBJID, TRT01A, SEX, AGE) %>%
  dplyr::filter(TRT01A %in% c("Treatment A", "Treatment B"))

# Exclusion filtering using ! (NOT operator)
non_placebo <- adsl %>%
  dplyr::select(USUBJID, TRT01A, SEX, AGE) %>%
  dplyr::filter(! TRT01A == "Placebo")
```

```
# Excluding multiple values
excluded_ages <- adsl %>%
  dplyr::select(USUBJID, TRT01A, SEX, AGE) %>%
  dplyr::filter(! AGE %in% c(70, 75, 80))

# Combined inclusion and exclusion
analysis_population <- adsl %>%
  dplyr::select(USUBJID, TRT01A, SEX, AGE) %>%
  dplyr::filter(TRT01A %in% c("Treatment A", "Treatment B") &
    ! SEX == "F" &
    AGE >= 18)
```

28.4 4.4 Variable Renaming - Standardization

```
# Single variable rename
adsl_renamed <- adsl %>%
  dplyr::rename(SUBJECT_ID = USUBJID)

# Multiple variable renames
adsl_standard <- adsl %>%
  dplyr::rename(
    SUBJECT_ID = USUBJID,
    TREATMENT = TRT01P,
    PLANNED_TREATMENT = TRT01A
  )

# Converting to lowercase (common requirement)
adsl_lower <- adsl
names(adsl_lower) <- tolower(names(adsl_lower))
```

28.5 4.5 Sorting Data - Clinical Data Ordering

```
# Single variable sorting
adsl_by_age <- adsl %>%
  dplyr::select(USUBJID, SEX, AGE, TRT01A) %>%
  dplyr::arrange(AGE)
```

```

# Multiple variable sorting (treatment, then age)
adsl_trt_age <- adsl %>%
  dplyr::select(USUBJID, SEX, AGE, TRT01A) %>%
  dplyr::arrange(TRT01A, AGE)

# Descending order
adsl_age_desc <- adsl %>%
  dplyr::select(USUBJID, SEX, AGE, TRT01A) %>%
  dplyr::arrange(desc(AGE))

# Mixed ascending and descending
adsl_mixed_sort <- adsl %>%
  dplyr::select(USUBJID, SEX, AGE, TRT01A) %>%
  dplyr::arrange(AGE, desc(TRT01A))

```

28.6 4.6 Creating New Variables - Derived Variables

Variable creation is essential for analysis datasets, creating flags, categorizations, and derived endpoints.

```

# Simple variable creation
adsl_enhanced <- adsl %>%
  dplyr::mutate(STUDY_NAME = "Phase III Efficacy Study")

# Conditional variable creation using if_else()
adsl_derived <- adsl %>%
  dplyr::mutate(
    AGEGR1 = dplyr::if_else(AGE < 65, "<65", ">=65"),           # Age group
    SAFFL_DERIVED = dplyr::if_else(! is.na(RANDDT), "Y", "N"), # Safety flag
    SEX_FULL = dplyr::if_else(SEX == "M", "Male", "Female")    # Full gender
  ) %>%
  dplyr::select(USUBJID, AGE, AGEGR1, SEX, SEX_FULL, RANDDT, SAFFL_DERIVED)

# Complex conditional logic with case_when()
adsl_complex <- adsl %>%
  dplyr::mutate(
    AGE_GROUP = dplyr::case_when(
      AGE < 18 ~ "Pediatric",
      AGE >= 18 & AGE < 65 ~ "Adult",
      AGE >= 65 & AGE < 75 ~ "Elderly",
    )
  )

```

```

    AGE >= 75 ~ "Very Elderly",
    TRUE ~ "Unknown" # Default case for missing values
  ),
  RISK_CATEGORY = dplyr::case_when(
    AGE >= 65 & SEX == "F" ~ "High Risk",
    AGE >= 70 ~ "High Risk",
    AGE < 30 ~ "Low Risk",
    TRUE ~ "Standard Risk"
  )
) %>%
dplyr::select(USUBJID, AGE, SEX, AGE_GROUP, RISK_CATEGORY)

```

28.7 4.7 Data Binding - Combining Datasets

```

# Create sample datasets for binding examples
dataset_a <- data.frame(
  USUBJID = c("001", "002", "003"),
  VISIT = c("V1", "V1", "V1"),
  LBTEST = c("Hemoglobin", "Hemoglobin", "Hemoglobin"),
  LBSTRESN = c(12.5, 13.2, 11.8)
)

dataset_b <- data.frame(
  USUBJID = c("004", "005", "006"),
  VISIT = c("V1", "V1", "V1"),
  LBTEST = c("Hemoglobin", "Hemoglobin", "Hemoglobin"),
  LBSTRESN = c(14.1, 12.9, 13.5)
)

# Row binding (adding more patients/observations)
combined_lab <- dplyr::bind_rows(dataset_a, dataset_b)

# Row binding with source identification
combined_with_source <- dplyr::bind_rows(
  "Batch_1" = dataset_a,
  "Batch_2" = dataset_b,
  .id = "DATA_SOURCE"
)

```



```

# Binding with repeated datasets (useful for simulation)
repeated_data <- dplyr::bind_rows(dataset_a, dataset_b, dataset_a, .id = "ITERATION")

# Creating total rows for summary tables
adsl_with_total <- dplyr::bind_rows(
  adsl %>%
    dplyr::select(USUBJID, TRT01P, TRT01AN) %>%
    dplyr::slice_head(n = 10), # First 10 for example

  adsl %>%
    dplyr::slice_head(n = 10) %>%
    dplyr::mutate(TRT01P = "Total", TRT01AN = 99) %>%
    dplyr::select(USUBJID, TRT01P, TRT01AN)
)

```

28.8 4.8 Recoding Variables - Data Standardization

```

# Creating lookup table for recoding
sex_mapping <- c("M" = "Male", "F" = "Female")

# Using recode() function
adsl_recoded <- adsl %>%
  dplyr::mutate(SEX_LABEL = dplyr::recode(SEX, !!!sex_mapping)) %>%
  dplyr::select(USUBJID, SEX, SEX_LABEL)

# Direct recoding without lookup table
adsl_direct_recode <- adsl %>%
  dplyr::mutate(SEX_FULL = dplyr::recode(SEX,
                                         "M" = "Male",
                                         "F" = "Female")) %>%
  dplyr::select(USUBJID, SEX, SEX_FULL)

# Alternative using if_else (often more readable)
adsl_alternative <- adsl %>%
  dplyr::mutate(SEX_DESC = dplyr::if_else(SEX == "M", "Male", "Female")) %>%
  dplyr::select(USUBJID, SEX, SEX_DESC)

```

28.9 4.9 Factor Creation - Categorical Data Management

```
# Creating factors with specified levels (important for analysis and reporting)
adsl_factors <- adsl %>%
  dplyr::mutate(
    AGEGR1_FACTOR = factor(
      dplyr::if_else(AGE < 65, "<65", ">=65"),
      levels = c("<65", ">=65")
    ),
    SEX_FACTOR = factor(SEX, levels = c("M", "F")),
    TRT01A_FACTOR = factor(TRT01A, levels = c("Placebo", "Treatment A", "Treatment B"))
  ) %>%
  dplyr::select(USUBJID, AGE, AGEGR1_FACTOR, SEX_FACTOR, TRT01A_FACTOR)

# Verify factor levels
levels(adsl_factors$AGEGR1_FACTOR)
```

```
[1] "<65" ">=65"
```

```
levels(adsl_factors$SEX_FACTOR)
```

```
[1] "M" "F"
```

```
levels(adsl_factors$TRT01A_FACTOR)
```

```
[1] "Placebo"      "Treatment A" "Treatment B"
```

29 Chapter 5: Advanced Data Operations

29.1 5.1 Table Joins - Merging Clinical Datasets

Joins are fundamental for combining SDTM domains or linking analysis datasets.

```
# Create realistic clinical datasets
patients_demo <- data.frame(
  USUBJID = paste0("STUDY001-", sprintf("%03d", 1:6)),
  AGE = c(45, 67, 23, 55, 34, 42),
  SEX = c("M", "F", "M", "F", "M", "F"),
  ARM = c("Placebo", "Treatment", "Placebo", "Treatment", "Treatment", "Placebo")
)

lab_results <- data.frame(
  USUBJID = paste0("STUDY001-", sprintf("%03d", c(2, 3, 4, 5, 7, 8))),
  LBTEST = rep(c("Hemoglobin", "Creatinine"), each = 3),
  LBSTRESN = c(12.5, 13.2, 11.8, 1.1, 0.9, 1.3),
  VISITNUM = c(1, 1, 1, 1, 1, 1)
)

print("Demographics Data:")
```

```
[1] "Demographics Data:"
```

```
print(patients_demo)
```

	USUBJID	AGE	SEX	ARM
1	STUDY001-001	45	M	Placebo
2	STUDY001-002	67	F	Treatment
3	STUDY001-003	23	M	Placebo
4	STUDY001-004	55	F	Treatment
5	STUDY001-005	34	M	Treatment
6	STUDY001-006	42	F	Placebo

```
print("Laboratory Results:")
```

```
[1] "Laboratory Results:"
```

```
print(lab_results)
```

	USUBJID	LBTEST	LBSTRESN	VISITNUM
1	STUDY001-002	Hemoglobin	12.5	1
2	STUDY001-003	Hemoglobin	13.2	1
3	STUDY001-004	Hemoglobin	11.8	1
4	STUDY001-005	Creatinine	1.1	1
5	STUDY001-007	Creatinine	0.9	1
6	STUDY001-008	Creatinine	1.3	1

29.1.1 Inner Join - Only Patients with Lab Results

```
# Inner join: Only patients with both demographics and lab data
inner_result <- dplyr::inner_join(patients_demo, lab_results, by = "USUBJID")
print("Inner Join Result (patients with lab data):")
```

```
[1] "Inner Join Result (patients with lab data):"
```

```
print(inner_result)
```

	USUBJID	AGE	SEX	ARM	LBTEST	LBSTRESN	VISITNUM
1	STUDY001-002	67	F	Treatment	Hemoglobin	12.5	1
2	STUDY001-003	23	M	Placebo	Hemoglobin	13.2	1
3	STUDY001-004	55	F	Treatment	Hemoglobin	11.8	1
4	STUDY001-005	34	M	Treatment	Creatinine	1.1	1

29.1.2 Left Join - All Patients, Lab Data Where Available

```
# Left join: All patients from demographics, lab data where available
left_result <- dplyr::left_join(patients_demo, lab_results, by = "USUBJID")
print("Left Join Result (all patients):")
```

```
[1] "Left Join Result (all patients):"
```

```
print(left_result)
```

	USUBJID	AGE	SEX	ARM	LBTEST	LBSTRESN	VISITNUM
1	STUDY001-001	45	M	Placebo	<NA>	NA	NA
2	STUDY001-002	67	F	Treatment	Hemoglobin	12.5	1
3	STUDY001-003	23	M	Placebo	Hemoglobin	13.2	1
4	STUDY001-004	55	F	Treatment	Hemoglobin	11.8	1
5	STUDY001-005	34	M	Treatment	Creatinine	1.1	1
6	STUDY001-006	42	F	Placebo	<NA>	NA	NA

29.1.3 Right Join - All Lab Results, Demographics Where Available

```
# Right join: All lab results, demographics where available
right_result <- dplyr::right_join(patients_demo, lab_results, by = "USUBJID")
print("Right Join Result (all lab results):")
```

```
[1] "Right Join Result (all lab results):"
```

```
print(right_result)
```

	USUBJID	AGE	SEX	ARM	LBTEST	LBSTRESN	VISITNUM
1	STUDY001-002	67	F	Treatment	Hemoglobin	12.5	1
2	STUDY001-003	23	M	Placebo	Hemoglobin	13.2	1
3	STUDY001-004	55	F	Treatment	Hemoglobin	11.8	1
4	STUDY001-005	34	M	Treatment	Creatinine	1.1	1
5	STUDY001-007	NA	<NA>	<NA>	Creatinine	0.9	1
6	STUDY001-008	NA	<NA>	<NA>	Creatinine	1.3	1

29.1.4 Full Join - All Records from Both Datasets

```
# Full join: All records from both datasets
full_result <- dplyr::full_join(patients_demo, lab_results, by = "USUBJID")
print("Full Join Result (all records):")
```

```
[1] "Full Join Result (all records):"
```

```
print(full_result)
```

	USUBJID	AGE	SEX	ARM	LBTEST	LBSTRESN	VISITNUM
1	STUDY001-001	45	M	Placebo	<NA>	NA	NA
2	STUDY001-002	67	F	Treatment	Hemoglobin	12.5	1
3	STUDY001-003	23	M	Placebo	Hemoglobin	13.2	1
4	STUDY001-004	55	F	Treatment	Hemoglobin	11.8	1
5	STUDY001-005	34	M	Treatment	Creatinine	1.1	1
6	STUDY001-006	42	F	Placebo	<NA>	NA	NA
7	STUDY001-007	NA	<NA>	<NA>	Creatinine	0.9	1
8	STUDY001-008	NA	<NA>	<NA>	Creatinine	1.3	1

29.1.5 Semi Join - Patients Who Have Lab Results (No Lab Columns)

```
# Semi join: Patients who have lab results (demographics only)
semi_result <- dplyr::semi_join(patients_demo, lab_results, by = "USUBJID")
print("Semi Join Result (patients with labs, demographics only):")
```

```
[1] "Semi Join Result (patients with labs, demographics only):"
```

```
print(semi_result)
```

	USUBJID	AGE	SEX	ARM
1	STUDY001-002	67	F	Treatment
2	STUDY001-003	23	M	Placebo
3	STUDY001-004	55	F	Treatment
4	STUDY001-005	34	M	Treatment

29.1.6 Anti Join - Patients Without Lab Results

```
# Anti join: Patients without lab results
anti_result <- dplyr::anti_join(patients_demo, lab_results, by = "USUBJID")
print("Anti Join Result (patients without labs):")
```

```
[1] "Anti Join Result (patients without labs):"
```

```
print(anti_result)
```

	USUBJID	AGE	SEX	ARM
1	STUDY001-001	45	M	Placebo
2	STUDY001-006	42	F	Placebo

```
# Reverse anti join: Lab results without demographics
anti_reverse <- dplyr::anti_join(lab_results, patients_demo, by = "USUBJID")
print("Reverse Anti Join (labs without demographics):")
```

```
[1] "Reverse Anti Join (labs without demographics):"
```

```
print(anti_reverse)
```

	USUBJID	LBTEST	LBSTRESN	VISITNUM
1	STUDY001-007	Creatinine	0.9	1
2	STUDY001-008	Creatinine	1.3	1

29.2 5.2 Data Reshaping with tidyr - Wide vs. Long Format

Clinical data often needs reshaping between wide and long formats for analysis and reporting.

```
# Create wide-format visit data (common in clinical databases)
visit_data_wide <- data.frame(
  USUBJID = paste0("STUDY001-", sprintf("%03d", 1:5)),
  BASELINE = c(95, 87, 110, 92, 88),
  WEEK_4 = c(92, 85, 108, 89, 85),
  WEEK_8 = c(89, 83, 105, 87, 83),
  WEEK_12 = c(87, 81, 103, 85, 81)
)

print("Wide Format Data (one row per patient):")
```

```
[1] "Wide Format Data (one row per patient):"
```

```
print(visit_data_wide)
```

	USUBJID	BASELINE	WEEK_4	WEEK_8	WEEK_12
1	STUDY001-001	95	92	89	87
2	STUDY001-002	87	85	83	81
3	STUDY001-003	110	108	105	103
4	STUDY001-004	92	89	87	85
5	STUDY001-005	88	85	83	81

```
# Convert to long format using pivot_longer()
visit_data_long <- visit_data_wide %>%
  tidyr::pivot_longer(
    cols = c(BASELINE, WEEK_4, WEEK_8, WEEK_12),
    names_to = "VISIT",
    values_to = "LAB_VALUE"
  )

print("Long Format Data (one row per patient-visit):")
```

```
[1] "Long Format Data (one row per patient-visit):"
```

```
print(visit_data_long)
```

```
# A tibble: 20 x 3
  USUBJID      VISIT  LAB_VALUE
  <chr>      <chr>    <dbl>
1 STUDY001-001 BASELINE      95
2 STUDY001-001 WEEK_4        92
3 STUDY001-001 WEEK_8        89
4 STUDY001-001 WEEK_12       87
5 STUDY001-002 BASELINE      87
6 STUDY001-002 WEEK_4        85
7 STUDY001-002 WEEK_8        83
8 STUDY001-002 WEEK_12       81
9 STUDY001-003 BASELINE     110
10 STUDY001-003 WEEK_4       108
11 STUDY001-003 WEEK_8       105
12 STUDY001-003 WEEK_12      103
13 STUDY001-004 BASELINE      92
14 STUDY001-004 WEEK_4        89
15 STUDY001-004 WEEK_8        87
16 STUDY001-004 WEEK_12       85
17 STUDY001-005 BASELINE      88
```



```
18 STUDY001-005 WEEK_4      85
19 STUDY001-005 WEEK_8      83
20 STUDY001-005 WEEK_12     81
```

```
# Convert back to wide format using pivot_wider()
visit_data_reconstructed <- visit_data_long %>%
  tidyr::pivot_wider(
    id_cols = USUBJID,
    names_from = VISIT,
    values_from = LAB_VALUE,
    names_prefix = "VISIT_" # Optional prefix
  )

print("Reconstructed Wide Format:")
```

```
[1] "Reconstructed Wide Format:"
```

```
print(visit_data_reconstructed)
```

```
# A tibble: 5 x 5
  USUBJID      VISIT_BASELINE VISIT_WEEK_4 VISIT_WEEK_8 VISIT_WEEK_12
  <chr>          <dbl>      <dbl>      <dbl>      <dbl>
1 STUDY001-001      95        92        89        87
2 STUDY001-002      87        85        83        81
3 STUDY001-003     110       108       105       103
4 STUDY001-004      92        89        87        85
5 STUDY001-005      88        85        83        81
```

```
# Complex reshaping example with multiple measures
complex_data <- data.frame(
  USUBJID = rep(paste0("STUDY001-", sprintf("%03d", 1:3)), each = 2),
  VISIT = rep(c("BASELINE", "WEEK_4"), 3),
  HEMOGLOBIN = c(12.5, 12.8, 13.1, 13.3, 11.8, 12.1),
  CREATININE = c(1.1, 1.0, 0.9, 0.9, 1.2, 1.1)
)

# Reshape to have one column per visit-test combination
complex_wide <- complex_data %>%
  tidyr::pivot_wider(
    id_cols = USUBJID,
    names_from = VISIT,
```

```

    values_from = c(HEMOGLOBIN, CREATININE),
    names_sep = "_"
  )

print("Complex Wide Format:")

```

```
[1] "Complex Wide Format:"
```

```
print(complex_wide)
```

```

# A tibble: 3 x 5
  USUBJID      HEMOGLOBIN_BASELINE HEMOGLOBIN_WEEK_4 CREATININE_BASELINE
  <chr>                <dbl>                <dbl>                <dbl>
1 STUDY001-001          12.5                12.8                1.1
2 STUDY001-002          13.1                13.3                0.9
3 STUDY001-003          11.8                12.1                1.2
# i 1 more variable: CREATININE_WEEK_4 <dbl>

```

29.3 5.3 Counting and Summarizing - Clinical Summary Tables

```

# Simple counting for demographic tables
treatment_counts <- adsl %>%
  dplyr::count(TRT01A) %>%
  dplyr::rename(Treatment = TRT01A, N = n)

print("Treatment Allocation:")

```

```
[1] "Treatment Allocation:"
```

```
print(treatment_counts)
```

```

  Treatment    N
1   Placebo  34
2 Treatment A  33
3 Treatment B  33

```

```
# Cross-tabulation (treatment by sex)
treatment_sex_counts <- adsl %>%
  dplyr::count(TRT01A, SEX) %>%
  dplyr::arrange(TRT01A, SEX)
```

```
print("Treatment by Sex:")
```

```
[1] "Treatment by Sex:"
```

```
print(treatment_sex_counts)
```

	TRT01A	SEX	n
1	Placebo	F	14
2	Placebo	M	20
3	Treatment A	F	16
4	Treatment A	M	17
5	Treatment B	F	14
6	Treatment B	M	19

```
# Filtering before counting (safety population)
```

```
safety_counts <- adsl %>%
  dplyr::filter(SAFFL == "Y") %>%
  dplyr::count(TRT01A) %>%
  dplyr::rename(Treatment = TRT01A, Safety_N = n)
```

```
# Complex counting with adverse events
```

```
# Note: Using simulated AE data since file may not exist
```

```
adae_sim <- data.frame(
  USUBJID = sample(adsl$USUBJID, 200, replace = TRUE),
  AEBODSYS = sample(c("Nervous System", "Gastrointestinal", "Respiratory"), 200, replace = TRUE),
  AEDECOD = sample(c("Headache", "Nausea", "Cough", "Fatigue"), 200, replace = TRUE),
  TRTA = sample(c("Placebo", "Treatment A", "Treatment B"), 200, replace = TRUE),
  SAFFL = "Y"
)
```

```
# Remove duplicates (one AE per patient-term combination)
```

```
adae_unique <- adae_sim %>%
  dplyr::distinct(USUBJID, AEBODSYS, AEDECOD, TRTA, SAFFL)
```

```
# Count AEs by treatment
```

```

ae_summary <- adae_unique %>%
  dplyr::filter(SAFFL == "Y") %>%
  dplyr::group_by(TRTA) %>%
  dplyr::count(AEBODSYS, AEDECOD) %>%
  dplyr::ungroup()

# Add denominators for percentages
bign <- adsl %>%
  dplyr::filter(SAFFL == "Y") %>%
  dplyr::count(TRT01A) %>%
  dplyr::rename(TRTA = TRT01A, total_n = n)

# Calculate percentages
ae_with_percent <- ae_summary %>%
  dplyr::left_join(bign, by = "TRTA") %>%
  dplyr::mutate(
    percent = round(n / total_n * 100, 1),
    display = paste0(n, " (", percent, "%)")
  )

# Reshape for reporting
ae_table <- ae_with_percent %>%
  dplyr::select(AEBODSYS, AEDECOD, TRTA, display) %>%
  tidyr::pivot_wider(
    id_cols = c(AEBODSYS, AEDECOD),
    names_from = TRTA,
    values_from = display,
    values_fill = "0 (0.0%)"
  )

print("Adverse Events by Treatment (first 10 rows):")

```

```
[1] "Adverse Events by Treatment (first 10 rows):"
```

```
print(head(ae_table, 10))
```

```

# A tibble: 10 x 5
  AEBODSYS      AEDECOD Placebo   `Treatment A` `Treatment B`
  <chr>         <chr>   <chr>      <chr>        <chr>
1 Gastrointestinal Cough    6 (17.6%)  6 (18.2%)    5 (15.2%)
2 Gastrointestinal Fatigue   5 (14.7%)  7 (21.2%)    3 (9.1%)

```

3	Gastrointestinal	Headache	3 (8.8%)	1 (3%)	6 (18.2%)
4	Gastrointestinal	Nausea	7 (20.6%)	3 (9.1%)	7 (21.2%)
5	Nervous System	Cough	6 (17.6%)	3 (9.1%)	12 (36.4%)
6	Nervous System	Fatigue	12 (35.3%)	2 (6.1%)	6 (18.2%)
7	Nervous System	Headache	4 (11.8%)	7 (21.2%)	3 (9.1%)
8	Nervous System	Nausea	6 (17.6%)	8 (24.2%)	4 (12.1%)
9	Respiratory	Cough	2 (5.9%)	7 (21.2%)	6 (18.2%)
10	Respiratory	Fatigue	7 (20.6%)	5 (15.2%)	3 (9.1%)

29.4 5.4 Distinct Values and Unique Records

```
# Get unique treatment combinations
unique_treatments <- adsl %>%
  dplyr::distinct(TRT01A, TRT01AN) %>%
  dplyr::arrange(TRT01AN)

print("Unique Treatment Codes:")
```

```
[1] "Unique Treatment Codes:"
```

```
print(unique_treatments)
```

	TRT01A	TRT01AN
1	Placebo	0
2	Treatment A	1
3	Treatment B	2

```
# Get unique patient-visit combinations (useful for verification)
unique_patients <- adsl %>%
  dplyr::distinct(USUBJID) %>%
  dplyr::summarise(unique_patients = n())

print("Number of Unique Patients:")
```

```
[1] "Number of Unique Patients:"
```

```
print(unique_patients)
```

```
unique_patients  
1              100
```

```
# Remove duplicate records while keeping specific columns  
adae_distinct <- adae_sim %>%  
  dplyr::distinct(USUBJID, AEDECOD, .keep_all = TRUE) # One AE term per patient  
print(paste("Original AE records:", nrow(adae_sim)))
```

```
[1] "Original AE records: 200"
```

```
print(paste("After removing duplicates:", nrow(adae_distinct)))
```

```
[1] "After removing duplicates: 151"
```

30 Chapter 6: Best Practices and Advanced Techniques

30.1 6.1 Efficient Pipeline Construction

```
# Example of an efficient clinical programming pipeline
analysis_ready_data <- adsl %>%
  # Step 1: Filter to analysis population
  dplyr::filter(SAFFL == "Y", ! is.na(TRT01A)) %>%

  # Step 2: Create derived variables
  dplyr::mutate(
    AGEGR1 = dplyr::case_when(
      AGE < 45 ~ "<45",
      AGE >= 45 & AGE < 65 ~ "45-64",
      AGE >= 65 ~ ">=65",
      TRUE ~ "Missing"
    ),
    AGEGR1 = factor(AGEGR1, levels = c("<45", "45-64", ">=65")),
    SEX_LABEL = dplyr::if_else(SEX == "M", "Male", "Female")
  ) %>%

  # Step 3: Select final variables
  dplyr::select(USUBJID, TRT01A, TRT01AN, AGE, AGEGR1, SEX, SEX_LABEL) %>%

  # Step 4: Sort for consistent output
  dplyr::arrange(TRT01AN, USUBJID)

print("Analysis Ready Dataset Structure:")
```

```
[1] "Analysis Ready Dataset Structure:"
```

```
str(analysis_ready_data)
```

```
'data.frame': 100 obs. of 7 variables:
 $ USUBJID : chr "STUDY001-001" "STUDY001-004" "STUDY001-007" "STUDY001-010" ...
 $ TRT01A : chr "Placebo" "Placebo" "Placebo" "Placebo" ...
 $ TRT01AN : num 0 0 0 0 0 0 0 0 0 0 ...
 $ AGE : int 52 78 75 50 34 64 47 29 40 44 ...
 $ AGEGR1 : Factor w/ 3 levels "<45","45-64",...: 2 3 3 2 1 2 2 1 1 1 ...
 $ SEX : chr "M" "F" "F" "F" ...
 $ SEX_LABEL: chr "Male" "Female" "Female" "Female" ...
```

```
print(head(analysis_ready_data, 10))
```

	USUBJID	TRT01A	TRT01AN	AGE	AGEGR1	SEX	SEX_LABEL
1	STUDY001-001	Placebo	0	52	45-64	M	Male
2	STUDY001-004	Placebo	0	78	>=65	F	Female
3	STUDY001-007	Placebo	0	75	>=65	F	Female
4	STUDY001-010	Placebo	0	50	45-64	F	Female
5	STUDY001-013	Placebo	0	34	<45	F	Female
6	STUDY001-016	Placebo	0	64	45-64	M	Male
7	STUDY001-019	Placebo	0	47	45-64	M	Male
8	STUDY001-022	Placebo	0	29	<45	M	Male
9	STUDY001-025	Placebo	0	40	<45	M	Male
10	STUDY001-028	Placebo	0	44	<45	M	Male

30.2 6.2 Error Handling and Data Validation

```
# Data validation pipeline
validation_summary <- adsl %>%
  dplyr::summarise(
    total_patients = n(),
    missing_age = sum(is.na(AGE)),
    missing_sex = sum(is.na(SEX)),
    missing_treatment = sum(is.na(TRT01A)),
    age_range = paste(min(AGE, na.rm = TRUE), "to", max(AGE, na.rm = TRUE)),
    unique_treatments = n_distinct(TRT01A, na.rm = TRUE),
    .groups = "drop"
```



```
)

print("Data Validation Summary:")
```

```
[1] "Data Validation Summary:"
```

```
print(validation_summary)
```

```
total_patients missing_age missing_sex missing_treatment age_range
1             100           0           0                 0 19 to 80
unique_treatments
1                3
```

```
# Check for data integrity issues
integrity_checks <- adsl %>%
  dplyr::mutate(
    age_flag = dplyr::case_when(
      AGE < 0 | AGE > 120 ~ "Age out of range",
      is.na(AGE) ~ "Missing age",
      TRUE ~ "OK"
    ),
    sex_flag = dplyr::case_when(
      ! SEX %in% c("M", "F") ~ "Invalid sex code",
      is.na(SEX) ~ "Missing sex",
      TRUE ~ "OK"
    )
  ) %>%
  dplyr::filter(age_flag != "OK" | sex_flag != "OK") %>%
  dplyr::select(USUBJID, AGE, age_flag, SEX, sex_flag)

print("Data Integrity Issues:")
```

```
[1] "Data Integrity Issues:"
```

```
print(integrity_checks)
```

```
[1] USUBJID AGE      age_flag SEX      sex_flag
<0 rows> (or 0-length row.names)
```

This comprehensive guide provides pharmaceutical programmers with the essential R skills for clinical data manipulation, from basic vector operations to advanced dplyr techniques used in SDTM and ADaM dataset creation and analysis.

31 String Functions

32 Learning Objectives

By the end of this chapter, you will be able to:

- Master fundamental string manipulation functions in R
- Apply string functions to pharmaceutical data cleaning and processing
- Use regular expressions for complex pattern matching in drug names and codes
- Handle medication names, dosage forms, and clinical terminology
- Clean and standardize pharmaceutical datasets using string operations

33 Introduction

String manipulation is crucial in pharmaceutical data analysis. You'll frequently work with:

- Drug names and generic/brand name mapping
- Dosage forms and strengths
- Medical terminology and coding systems (ICD, MedDRA, ATC codes)
- Patient identifiers and study codes
- Adverse event descriptions
- Clinical trial data with text fields

34 Setup

```
# Load required packages
library(stringr)
library(dplyr)
```

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

filter, lag

The following objects are masked from 'package:base':

intersect, setdiff, setequal, union

```
library(purrr)
library(tools)

# Suppress warnings for cleaner output
options(warn = -1)
```

35 Basic String Functions

35.1 1. String Length and Basic Information

```
# Basic string functions with pharmaceutical examples
drug_names <- c("Aspirin", "Ibuprofen", "Acetaminophen", "Warfarin", "Metformin")

# Get string lengths
cat("String lengths:\n")
```

String lengths:

```
print(nchar(drug_names))
```

```
[1]  7  9 13  8  9
```

```
# Check if strings are empty
medical_terms <- c("Hypertension", "", "Diabetes", NA, "Pneumonia")
cat("\nEmpty string check:\n")
```

Empty string check:

```
print(nzchar(medical_terms)) # Returns FALSE for empty strings
```

```
[1] TRUE FALSE TRUE TRUE TRUE
```

```
# Handle NA values properly
cat("\nNA value check:\n")
```

NA value check:

```
print(is.na(medical_terms))
```

```
[1] FALSE FALSE FALSE  TRUE FALSE
```

35.2 2. Case Conversion

```
# Case conversion for drug names  
brand_names <- c("Tylenol", "Advil", "Lipitor", "Nexium")  
  
cat("Original names:", paste(brand_names, collapse = ", "), "\n")
```

Original names: Tylenol, Advil, Lipitor, Nexium

```
cat("Uppercase:", paste(toupper(brand_names), collapse = ", "), "\n")
```

Uppercase: TYLENOL, ADVIL, LIPITOR, NEXIUM

```
cat("Lowercase:", paste(tolower(brand_names), collapse = ", "), "\n")
```

Lowercase: tylenol, advil, lipitor, nexium

```
cat("Title Case:", paste(toTitleCase(tolower(brand_names)), collapse = ", "), "\n")
```

Title Case: Tylenol, Advil, Lipitor, Nexium

35.3 3. String Concatenation

```
# Combining drug information  
drug_name <- c("Aspirin", "Ibuprofen", "Acetaminophen")  
strength <- c("81mg", "200mg", "500mg")  
form <- c("Tablet", "Capsule", "Tablet")  
  
# Basic concatenation  
cat("Basic concatenation:\n")
```


Basic concatenation:

```
print(paste(drug_name, strength, form))
```

```
[1] "Aspirin 81mg Tablet"      "Ibuprofen 200mg Capsule"  
[3] "Acetaminophen 500mg Tablet"
```

```
# Concatenation with custom separator  
cat("\nWith custom separator:\n")
```

With custom separator:

```
print(paste(drug_name, strength, form, sep = " | "))
```

```
[1] "Aspirin | 81mg | Tablet"      "Ibuprofen | 200mg | Capsule"  
[3] "Acetaminophen | 500mg | Tablet"
```

```
# Concatenation without separator  
cat("\nWithout separator:\n")
```

Without separator:

```
print(paste0(drug_name, "_", strength))
```

```
[1] "Aspirin_81mg"      "Ibuprofen_200mg"      "Acetaminophen_500mg"
```

36 Substring Operations

36.1 1. Extracting Substrings

```
# Working with NDC (National Drug Code) numbers
ndc_codes <- c("12345-678-90", "98765-432-10", "11111-222-33")

cat("NDC Codes:", paste(ndc_codes, collapse = ", "), "\n")
```

NDC Codes: 12345-678-90, 98765-432-10, 11111-222-33

```
# Extract manufacturer code (first part)
cat("Manufacturer codes (base R):", paste(substr(ndc_codes, 1, 5), collapse = ", "), "\n")
```

Manufacturer codes (base R): 12345, 98765, 11111

```
# Extract product code (middle part)
cat("Product codes:", paste(substr(ndc_codes, 7, 9), collapse = ", "), "\n")
```

Product codes: 678, 432, 222

```
# Using stringr for more intuitive operations
cat("First 5 chars (stringr):", paste(str_sub(ndc_codes, 1, 5), collapse = ", "), "\n")
```

First 5 chars (stringr): 12345, 98765, 11111

```
cat("Last 2 chars:", paste(str_sub(ndc_codes, -2, -1), collapse = ", "), "\n")
```

Last 2 chars: 90, 10, 33

36.2 2. Finding and Replacing Text

```
# Standardizing drug names
messy_drug_names <- c("aspirin 81 mg", "IBUPROFEN-200MG", "Acetaminophen/500mg")

cat("Original messy names:\n")
```

Original messy names:

```
print(messy_drug_names)
```

```
[1] "aspirin 81 mg"      "IBUPROFEN-200MG"    "Acetaminophen/500mg"
```

```
# Replace hyphens and slashes with spaces
cleaned_names <- str_replace_all(messy_drug_names, "[-/]", " ")
cat("\nAfter replacing separators:\n")
```

After replacing separators:

```
print(cleaned_names)
```

```
[1] "aspirin 81 mg"      "IBUPROFEN 200MG"    "Acetaminophen 500mg"
```

```
# Remove extra whitespace
cat("\nAfter removing extra whitespace:\n")
```

After removing extra whitespace:

```
print(str_squish(cleaned_names))
```

```
[1] "aspirin 81 mg"      "IBUPROFEN 200MG"    "Acetaminophen 500mg"
```

```
# Find positions of patterns
drug_text <- "Patient took Aspirin 81mg twice daily"
cat("\nPosition of 'Aspirin' in text:\n")
```

Position of 'Aspirin' in text:

```
print(str_locate(drug_text, "Aspirin"))
```

```
      start end
[1,]    14  20
```

```
cat("\nPositions of all numbers:\n")
```

Positions of all numbers:

```
print(str_locate_all(drug_text, "[0-9]+"))
```

```
[[1]]
      start end
[1,]    22  23
```

37 Pattern Matching and Regular Expressions

37.1 1. Basic Pattern Detection

```
# Detect patterns in medication descriptions
medications <- c(
  "Aspirin 81mg tablet once daily",
  "Metformin 500mg twice daily",
  "Insulin injection as needed",
  "Lisinopril 10mg daily",
  "Warfarin 5mg as directed"
)

cat("Medications containing 'mg':\n")
```

Medications containing 'mg':

```
print(medications[str_detect(medications, "mg")])
```

```
[1] "Aspirin 81mg tablet once daily" "Metformin 500mg twice daily"
[3] "Lisinopril 10mg daily"          "Warfarin 5mg as directed"
```

```
cat("\nMedications starting with 'Aspirin':\n")
```

Medications starting with 'Aspirin':

```
print(medications[str_detect(medications, "^Aspirin")])
```

```
[1] "Aspirin 81mg tablet once daily"
```

```
cat("\nMedications ending with 'daily':\n")
```

Medications ending with 'daily':

```
print(medications[str_detect(medications, "daily$")])
```

```
[1] "Aspirin 81mg tablet once daily" "Metformin 500mg twice daily"  
[3] "Lisinopril 10mg daily"
```

37.2 2. Advanced Regular Expressions

```
# Extract dosage information  
dosage_pattern <- "[0-9]+\\.?[0-9]*mg"  
extracted_dosages <- str_extract(medications, dosage_pattern)  
  
cat("Extracted dosages:\n")
```

Extracted dosages:

```
for(i in seq_along(medications)) {  
  cat(sprintf("%s -> %s\n", medications[i],  
              ifelse(is.na(extracted_dosages[i]), "No dosage found", extracted_dosages[i])))  
}
```

```
Aspirin 81mg tablet once daily -> 81mg  
Metformin 500mg twice daily -> 500mg  
Insulin injection as needed -> No dosage found  
Lisinopril 10mg daily -> 10mg  
Warfarin 5mg as directed -> 5mg
```

```
# Complex pattern: Extract drug name and dosage  
drug_dosage_pattern <- "^[A-Za-z+)]\\s+([0-9]+\\.?[0-9]*mg)"  
matches <- str_match(medications, drug_dosage_pattern)  
  
cat("\nDrug name and dosage extraction:\n")
```

Drug name and dosage extraction:

```
for(i in seq_along(medications)) {  
  if(! is.na(matches[i, 1])) {  
    cat(sprintf("Drug: %s, Dosage:  %s\n", matches[i, 2], matches[i, 3]))  
  } else {  
    cat(sprintf("No match for:  %s\n", medications[i]))  
  }  
}
```

Drug: Aspirin, Dosage: 81mg
Drug: Metformin, Dosage: 500mg
No match for: Insulin injection as needed
Drug: Lisinopril, Dosage: 10mg
Drug: Warfarin, Dosage: 5mg

38 Working with coded terms

38.1 1. ICD-10 Code Processing

```
# Working with ICD-10 diagnostic codes
icd_codes <- c("E11.9", "I10", "J44.1", "N18.6", "M79.3")
icd_descriptions <- c(
  "Type 2 diabetes mellitus without complications",
  "Essential hypertension",
  "Chronic obstructive pulmonary disease with acute exacerbation",
  "End stage renal disease",
  "Panniculitis, unspecified"
)

# Create a lookup function
create_icd_lookup <- function(codes, descriptions) {
  lookup_table <- setNames(descriptions, codes)
  return(lookup_table)
}

icd_lookup <- create_icd_lookup(icd_codes, icd_descriptions)

# Function to get description from code
get_icd_description <- function(code, lookup_table) {
  description <- lookup_table[code]
  ifelse(is.na(description), "Code not found", description)
}

# Test the function
patient_codes <- c("E11.9", "I10", "Z99.9") # Last one doesn't exist
cat("ICD-10 Code Lookup Results:\n")
```

ICD-10 Code Lookup Results:


```
for(i in seq_along(patient_codes)) {
  cat(sprintf("%s: %s\n", patient_codes[i], get_icd_description(patient_codes[i], icd_lookup)))
}
```

E11.9: Type 2 diabetes mellitus without complications

I10: Essential hypertension

Z99.9: Code not found

38.2 2. ATC Code Analysis

```
# Anatomical Therapeutic Chemical (ATC) codes
atc_codes <- c("A02BC01", "C09AA02", "A10BA02", "B01AA03", "N02BA01")
drug_names_atc <- c("Omeprazole", "Enalapril", "Metformin", "Warfarin", "Aspirin")

# Extract anatomical group (first character)
anatomical_groups <- str_sub(atc_codes, 1, 1)

# Create descriptive labels
anatomical_labels <- c(
  "A" = "Alimentary tract and metabolism",
  "B" = "Blood and blood forming organs",
  "C" = "Cardiovascular system",
  "D" = "Dermatologicals",
  "G" = "Genito urinary system and sex hormones",
  "H" = "Systemic hormonal preparations",
  "J" = "Antiinfectives for systemic use",
  "L" = "Antineoplastic and immunomodulating agents",
  "M" = "Musculo-skeletal system",
  "N" = "Nervous system",
  "P" = "Antiparasitic products",
  "R" = "Respiratory system",
  "S" = "Sensory organs",
  "V" = "Various"
)

# Map codes to descriptions
atc_mapping <- data.frame(
  Drug = drug_names_atc,
  ATC_Code = atc_codes,
```

```

    Anatomical_Group = anatomical_labels[anatomical_groups],
    stringsAsFactors = FALSE
)

cat("ATC Code Analysis:\n")

```

ATC Code Analysis:

```
print(atc_mapping)
```

	Drug	ATC_Code	Anatomical_Group
1	Omeprazole	A02BC01	Alimentary tract and metabolism
2	Enalapril	C09AA02	Cardiovascular system
3	Metformin	A10BA02	Alimentary tract and metabolism
4	Warfarin	B01AA03	Blood and blood forming organs
5	Aspirin	N02BA01	Nervous system

39 Data Cleaning Applications

39.1 1. Standardizing Drug Names

```
# Real-world messy drug data
messy_drug_data <- c(
  "ASPIRIN 81 MG TABLET",
  "aspirin 81mg tab",
  "Aspirin, 81 mg, tablet",
  "ASA 81MG TABS",
  "aspirin_81_mg_tablet",
  "Aspirin (81mg) - Tablet"
)

# Function to standardize drug names
standardize_drug_name <- function(drug_names) {
  # Convert to lowercase
  clean_names <- tolower(drug_names)

  # Replace various separators with spaces
  clean_names <- str_replace_all(clean_names, "[_,\\-()-]+", " ")

  # Standardize abbreviations
  clean_names <- str_replace_all(clean_names, "\\btab[s]?\\b", "tablet")
  clean_names <- str_replace_all(clean_names, "\\basa\\b", "aspirin")

  # Remove extra whitespace
  clean_names <- str_squish(clean_names)

  # Standardize mg format
  clean_names <- str_replace_all(clean_names, "([0-9]+)\\s*mg", "\\1mg")

  return(clean_names)
}
```

```
standardized_names <- standardize_drug_name(messy_drug_data)

cat("Drug Name Standardization Results:\n")
```

Drug Name Standardization Results:

```
for(i in seq_along(messy_drug_data)) {
  cat(sprintf("Original: %s\n", messy_drug_data[i]))
  cat(sprintf("Standardized: %s\n\n", standardized_names[i]))
}
```

Original: ASPIRIN 81 MG TABLET
Standardized: aspirin 81mg tablet

Original: aspirin 81mg tab
Standardized: aspirin 81mg tablet

Original: Aspirin, 81 mg, tablet
Standardized: aspirin 81mg tablet

Original: ASA 81MG TABS
Standardized: aspirin 81mg tablet

Original: aspirin_81_mg_tablet
Standardized: aspirin 81mg tablet

Original: Aspirin (81mg) - Tablet
Standardized: aspirin 81mg tablet

39.2 2. Parsing Dosage Information

```
# Extract structured information from prescription strings
prescriptions <- c(
  "Metformin 500mg twice daily with meals",
  "Lisinopril 10mg once daily in morning",
  "Warfarin 5mg daily as directed by INR",
  "Aspirin 81mg once daily for cardioprotection",
  "Insulin 10 units subcutaneous before meals"
```

```

)

# Function to parse prescription information
parse_prescription <- function(prescription_text) {
  # Extract drug name (assumes it's the first word)
  drug_name <- str_extract(prescription_text, "^[A-Za-z]+")

  # Extract dosage with units
  dosage <- str_extract(prescription_text, "[0-9]+\\.?[0-9]*\\s?(mg|units|mcg|g)")

  # Extract frequency information
  frequency <- case_when(
    str_detect(prescription_text, "twice|bid") ~ "Twice daily",
    str_detect(prescription_text, "once|daily") ~ "Once daily",
    str_detect(prescription_text, "three times|tid") ~ "Three times daily",
    str_detect(prescription_text, "four times|qid") ~ "Four times daily",
    TRUE ~ "As directed"
  )

  # Extract special instructions
  special_instructions <- case_when(
    str_detect(prescription_text, "with meals") ~ "With meals",
    str_detect(prescription_text, "before meals") ~ "Before meals",
    str_detect(prescription_text, "morning") ~ "In morning",
    str_detect(prescription_text, "bedtime") ~ "At bedtime",
    TRUE ~ "No special instructions"
  )

  return(data.frame(
    Drug = drug_name,
    Dosage = dosage,
    Frequency = frequency,
    Instructions = special_instructions,
    stringsAsFactors = FALSE
  ))
}

# Parse all prescriptions
prescription_data <- map_dfr(prescriptions, parse_prescription)

cat("Parsed Prescription Information:\n")

```

Parsed Prescription Information:

```
print(prescription_data)
```

	Drug	Dosage	Frequency	Instructions
1	Metformin	500mg	Twice daily	With meals
2	Lisinopril	10mg	Once daily	In morning
3	Warfarin	5mg	Once daily	No special instructions
4	Aspirin	81mg	Once daily	No special instructions
5	Insulin	10 units	As directed	Before meals

40 Advanced String Operations

40.1 1. Working with Patient Identifiers

```
# Function to generate patient IDs
generate_patient_id <- function(site_code, patient_number) {
  # Format: SITE001-P-0001
  formatted_number <- str_pad(patient_number, 4, pad = "0")
  patient_id <- paste0(site_code, "-P-", formatted_number)
  return(patient_id)
}

# Generate some patient IDs
site_codes <- c("NYC", "LAX", "CHI")
patient_numbers <- 1:3

# Create patient IDs for demonstration
sample_ids <- c()
for(site in site_codes) {
  for(num in patient_numbers) {
    sample_ids <- c(sample_ids, generate_patient_id(site, num))
  }
}

cat("Generated Patient IDs:\n")
```

Generated Patient IDs:

```
print(sample_ids[1:6]) # Show first 6
```

```
[1] "NYC-P-0001" "NYC-P-0002" "NYC-P-0003" "LAX-P-0001" "LAX-P-0002"
[6] "LAX-P-0003"
```

```
# Function to validate patient ID format
validate_patient_id <- function(patient_id) {
  pattern <- "[A-Z]{3}-P-[0-9]{4}$"
  return(str_detect(patient_id, pattern))
}

# Test validation
test_ids <- c("NYC-P-0001", "INVALID-ID", "LAX-P-99", "CHI-P-0123")
cat("\nValidation Results:\n")
```

Validation Results:

```
for(i in seq_along(test_ids)) {
  cat(sprintf("%s: %s\n", test_ids[i],
              ifelse(validate_patient_id(test_ids[i]), "Valid", "Invalid"))))
}
```

```
NYC-P-0001: Valid
INVALID-ID: Invalid
LAX-P-99: Invalid
CHI-P-0123: Valid
```

40.2 2. Processing Adverse Event Reports

```
# Working with MedDRA terms
adverse_events <- c(
  "Nausea and vomiting",
  "Severe headache",
  "Skin rash and itching",
  "Diarrhea, mild",
  "Fatigue and weakness",
  "Allergic reaction - hives"
)

# Function to standardize adverse event terms
standardize_ae_terms <- function(ae_terms) {
  # Convert to title case
  clean_terms <- str_to_title(ae_terms)
```



```

# Standardize common words
clean_terms <- str_replace_all(clean_terms, "\\bAnd\\b", "and")
clean_terms <- str_replace_all(clean_terms, "\\bOf\\b", "of")

# Remove severity indicators for consistency
severity_pattern <- "(\\b(mild|moderate|severe|serious)\\s*,? \\s*|\\s*-\\s*)"
clean_terms <- str_remove_all(clean_terms, regex(severity_pattern, ignore_case = TRUE))

# Remove extra whitespace
clean_terms <- str_squish(clean_terms)

return(clean_terms)
}

standardized_ae <- standardize_ae_terms(adverse_events)

cat("Adverse Event Standardization:\n")

```

Adverse Event Standardization:

```

for(i in seq_along(adverse_events)) {
  cat(sprintf("Original: %s\n", adverse_events[i]))
  cat(sprintf("Standardized: %s\n\n", standardized_ae[i]))
}

```

Original: Nausea and vomiting
Standardized: Nausea and Vomiting

Original: Severe headache
Standardized: Headache

Original: Skin rash and itching
Standardized: Skin Rash and Itching

Original: Diarrhea, mild
Standardized: Diarrhea, Mild

Original: Fatigue and weakness
Standardized: Fatigue and Weakness

Original: Allergic reaction - hives

40.3 3. Clinical Trial Data Processing Example

```
library(purrr)
# Comprehensive example: Processing clinical trial medication data
clinical_trial_data <- data.frame(
  subject_id = c("001-001", "001-002", "001-003"),
  visit = c("Baseline", "Week 4", "Week 8"),
  concomitant_medications = c(
    "Metformin 500mg BID, Lisinopril 10mg QD",
    "Metformin 500mg BID, Lisinopril 10mg QD, Aspirin 81mg QD",
    "Metformin 1000mg BID, Lisinopril 20mg QD"
  ),
  stringsAsFactors = FALSE
)

# Function to parse medication strings into structured data
parse_medications <- function(med_string, subject_id, visit) {
  # Split by comma to get individual medications
  individual_meds <- str_split(med_string, ",\\s*")[[1]]

  # Extract information for each medication
  med_data <- map_dfr(individual_meds, function(med) {
    # Extract drug name (first word(s) before dosage)
    # Fixed: Removed invalid lookahead syntax
    drug_name <- str_extract(med, "[A-Za-z\\s]+(? =\\s+[0-9])")
    drug_name <- str_trim(drug_name)

    # Extract dosage
    dosage <- str_extract(med, "[0-9]+\\.?[0-9]*\\s*(mg|mcg|g|units)")

    # Extract frequency
    frequency <- case_when(
      str_detect(med, "BID|twice") ~ "Twice daily",
      str_detect(med, "QD|once|daily") ~ "Once daily",
      str_detect(med, "TID|three") ~ "Three times daily",
      str_detect(med, "QID|four") ~ "Four times daily",
      TRUE ~ "As directed"
    )
  })
}
```

```

    return(data.frame(
      drug_name = drug_name,
      dosage = dosage,
      frequency = frequency,
      stringsAsFactors = FALSE
    ))
  })
})

# Add subject and visit information
med_data$subject_id <- subject_id
med_data$visit <- visit

return(med_data)
}

# Alternative function that's more robust (doesn't rely on lookahead)
parse_medications_robust <- function(med_string, subject_id, visit) {
  # Split by comma to get individual medications
  individual_meds <- str_split(med_string, ",\\s*")[[1]]

  # Extract information for each medication
  med_data <- map_dfr(individual_meds, function(med) {
    # Method 1: Use word boundaries to extract drug name
    # Extract everything before the first number
    drug_name <- str_extract(med, "^([0-9]+)")
    drug_name <- str_trim(drug_name)

    # Method 2: Alternative approach using str_match
    # match <- str_match(med, "^([A-Za-z\\s]+)\\s+([0-9]+\\.?[0-9]*\\s*(mg|mcg|g|units))")
    # drug_name <- str_trim(match[, 2])
    # dosage <- match[, 3]

    # Extract dosage
    dosage <- str_extract(med, "[0-9]+\\.?[0-9]*\\s*(mg|mcg|g|units)")

    # Extract frequency
    frequency <- case_when(
      str_detect(med, "BID|twice") ~ "Twice daily",
      str_detect(med, "QD|once|daily") ~ "Once daily",
      str_detect(med, "TID|three") ~ "Three times daily",
      str_detect(med, "QID|four") ~ "Four times daily",
      TRUE ~ "As directed"
    )
  })
}

```

```

    )

    return(data.frame(
      drug_name = drug_name,
      dosage = dosage,
      frequency = frequency,
      stringsAsFactors = FALSE
    ))
  })
})

# Add subject and visit information
med_data$subject_id <- subject_id
med_data$visit <- visit

return(med_data)
}

# Process all medication data using the robust function
processed_meds <- pmap_dfr(
  list(
    clinical_trial_data$concomitant_medications,
    clinical_trial_data$subject_id,
    clinical_trial_data$visit
  ),
  parse_medications_robust
)

# Display the structured data
cat("Processed Clinical Trial Medication Data:\n")

```

Processed Clinical Trial Medication Data:

```
print(processed_meds[, c("subject_id", "visit", "drug_name", "dosage", "frequency")])
```

	subject_id	visit	drug_name	dosage	frequency
1	001-001	Baseline	Metformin	500mg	Twice daily
2	001-001	Baseline	Lisinopril	10mg	Once daily
3	001-002	Week 4	Metformin	500mg	Twice daily
4	001-002	Week 4	Lisinopril	10mg	Once daily
5	001-002	Week 4	Aspirin	81mg	Once daily
6	001-003	Week 8	Metformin	1000mg	Twice daily
7	001-003	Week 8	Lisinopril	20mg	Once daily

41 Performance Tips and Best Practices

41.1 1. Efficient String Operations

```
# Create a moderately sized dataset for demonstration
large_drug_list <- rep(c("Aspirin", "Ibuprofen", "Acetaminophen"), 1000)

# Vectorized operations are more efficient
cat("Efficient approach - vectorized operation:\n")
```

Efficient approach - vectorized operation:

```
start_time <- Sys.time()
drug_doses_vectorized <- paste(large_drug_list, "100mg")
end_time <- Sys.time()
cat(sprintf("Time taken: %s seconds\n", round(end_time - start_time, 4)))
```

Time taken: 0.0011 seconds

```
# Avoid loops for simple operations
cat("\nLess efficient approach - loop-based:\n")
```

Less efficient approach - loop-based:

```
start_time <- Sys.time()
drug_doses_loop <- character(length(large_drug_list))
for (i in seq_along(large_drug_list)) {
  drug_doses_loop[i] <- paste(large_drug_list[i], "100mg")
}
end_time <- Sys.time()
cat(sprintf("Time taken: %s seconds\n", round(end_time - start_time, 4)))
```

Time taken: 0.0096 seconds

```
cat("\nFirst 5 results are identical:")
```

First 5 results are identical:

```
cat(sprintf("Vectorized: %s\n", paste(drug_doses_vectorized[1:5], collapse = ", ")))
```

Vectorized: Aspirin 100mg, Ibuprofen 100mg, Acetaminophen 100mg, Aspirin 100mg, Ibuprofen 100mg

```
cat(sprintf("Loop-based: %s\n", paste(drug_doses_loop[1:5], collapse = ", ")))
```

Loop-based: Aspirin 100mg, Ibuprofen 100mg, Acetaminophen 100mg, Aspirin 100mg, Ibuprofen 100mg

41.2 2. Error Handling in String Operations

```
# Robust function for extracting dosage information
extract_dosage_safe <- function(medication_string) {
  tryCatch({
    # Handle NULL or NA inputs
    if (is.null(medication_string) || is.na(medication_string)) {
      return(NA_character_)
    }

    # Convert to character if not already
    med_str <- as.character(medication_string)

    # Extract dosage pattern
    dosage_pattern <- "[0-9]+\\.?[0-9]*\\s*(mg|mcg|g|units)"
    dosage <- str_extract(med_str, regex(dosage_pattern, ignore_case = TRUE))

    # Return standardized format
    if (! is.na(dosage)) {
      return(str_to_lower(str_replace(dosage, "\\s+", "")))
    } else {
      return("No dosage found")
    }
  })
}
```

```

    }, error = function(e) {
      warning(paste("Error processing medication string:", medication_string, "-", e$message))
      return("Error in processing")
    })
  }

# Test with various inputs
test_medications <- c(
  "Aspirin 81mg",
  "Invalid format",
  NA,
  "Metformin 500 MG twice daily",
  ""
)

cat("Safe Dosage Extraction Results:\n")

```

Safe Dosage Extraction Results:

```

for(i in seq_along(test_medications)) {
  result <- extract_dosage_safe(test_medications[i])
  cat(sprintf("Input: '%s' -> Output: '%s'\n",
              ifelse(is.na(test_medications[i]), "NA", test_medications[i]),
              result))
}

```

```

Input: 'Aspirin 81mg' -> Output: '81mg'
Input: 'Invalid format' -> Output: 'No dosage found'
Input: 'NA' -> Output: 'NA'
Input: 'Metformin 500 MG twice daily' -> Output: '500mg'
Input: '' -> Output: 'No dosage found'

```

42 Summary and Key Takeaways

42.1 Essential String Functions for Pharmaceutical Analysis

The key functions you should master include:

1. **Basic Operations:** `nchar()`, `toupper()`, `tolower()`, `paste()`, `paste0()`
2. **Substring Operations:** `substr()`, `str_sub()`, `str_extract()`
3. **Pattern Matching:** `str_detect()`, `str_locate()`, `str_match()`
4. **Text Replacement:** `str_replace()`, `str_replace_all()`, `str_remove()`
5. **Text Cleaning:** `str_squish()`, `str_trim()`

42.2 Best Practices

```
# Create a summary of best practices
best_practices <- data.frame(
  Practice = c(
    "Standardization",
    "Error Handling",
    "Vectorization",
    "Regular Expressions",
    "Documentation"
  ),
  Description = c(
    "Always clean and standardize data before analysis",
    "Use tryCatch() for robust string processing",
    "Use vectorized operations for better performance",
    "Learn regex patterns for complex text matching",
    "Document your string processing logic clearly"
  ),
  Example = c(
    "toupper(), str_squish()",
    "tryCatch(), is.na() checks",
    "str_replace_all() vs loops",
  )
)
```



```

    "str_extract() with patterns",
    "Comments and function names"
  ),
  stringsAsFactors = FALSE
)

cat("String Processing Best Practices:\n")

```

String Processing Best Practices:

```
print(best_practices)
```

	Practice	Description
1	Standardization	Always clean and standardize data before analysis
2	Error Handling	Use tryCatch() for robust string processing
3	Vectorization	Use vectorized operations for better performance
4	Regular Expressions	Learn regex patterns for complex text matching
5	Documentation	Document your string processing logic clearly

Example

- 1 toupper(), str_squish()
- 2 tryCatch(), is.na() checks
- 3 str_replace_all() vs loops
- 4 str_extract() with patterns
- 5 Comments and function names

42.3 Common Pharmaceutical Applications

- **Drug name standardization and mapping:** Cleaning messy drug names from different sources
- **Dosage extraction and validation:** Parsing prescription information
- **Medical coding:** Working with ICD-10, ATC, MedDRA codes
- **Patient identifier processing:** Generating and validating study IDs
- **Adverse event text mining:** Standardizing AE terms and descriptions
- **Clinical trial data parsing:** Processing complex medication strings

42.4 Next Steps

1. Practice these techniques with your own pharmaceutical datasets

2. Explore more advanced text mining packages (tm, quanteda, tidytext)
3. Learn about natural language processing for clinical notes
4. Study regulatory requirements for data standardization (CDISC standards)
5. Implement automated data quality checks using string validation functions

This comprehensive guide provides the foundation for effective string manipulation in pharmaceutical R programming. Remember to adapt these techniques to your specific use cases and always validate your results with domain experts.

43 Comparing stringr and base R string functions

We'll begin with a lookup table between the most important stringr functions and their base R equivalents.

```
library(stringr)
data_stringr_base_diff <- tibble::tribble(
  ~stringr,
  "str_detect(string, pattern)",
  "str_dup(string, times)",
  "str_extract(string, pattern)",
  "str_extract_all(string, pattern)",
  "str_length(string)",
  "str_locate(string, pattern)",
  "str_locate_all(string, pattern)",
  "str_match(string, pattern)",
  "str_order(string)",
  "str_replace(string, pattern, replacement)",
  "str_replace_all(string, pattern, replacement)",
  "str_sort(string)",
  "str_split(string, pattern)",
  "str_sub(string, start, end)",
  "str_subset(string, pattern)",
  "str_to_lower(string)",
  "str_to_title(string)",
  "str_to_upper(string)",
  "str_trim(string)",
  "str_which(string, pattern)",
  "str_wrap(string)",
  ~base_r,
  "grepl(pattern, x)",
  "strrep(x, times)",
  "regmatches(x, m = regexpr(pattern, text))",
  "regmatches(x, m = gregexpr(pattern, text))",
  "nchar(x)",
  "regexpr(pattern, text)",
  "gregexpr(pattern, text)",
  "regmatches(x, m = regexec(pattern, text))",
  "order(...)",
  "sub(pattern, replacement, x)",
  "gsub(pattern, replacement, x)",
  "sort(x)",
  "strsplit(x, split)",
  "substr(x, start, stop)",
  "grep(pattern, x, value = TRUE)",
  "tolower(x)",
  "tools::toTitleCase(text)",
  "toupper(x)",
  "trimws(x)",
  "grep(pattern, x)",
  "strwrap(x)"
)

# create MD table, arranged alphabetically by stringr fn name
data_stringr_base_diff |>
  dplyr::mutate(dplyr::across(
    .cols = everything(),
```

stringr	base R
<code>str_detect(string, pattern)</code>	<code>grepl(pattern, x)</code>
<code>str_dup(string, times)</code>	<code>strrep(x, times)</code>
<code>str_extract(string, pattern)</code>	<code>regmatches(x, m = regexpr(pattern, text))</code>
<code>str_extract_all(string, pattern)</code>	<code>regmatches(x, m = gregexpr(pattern, text))</code>
<code>str_length(string)</code>	<code>nchar(x)</code>
<code>str_locate(string, pattern)</code>	<code>regexpr(pattern, text)</code>
<code>str_locate_all(string, pattern)</code>	<code>gregexpr(pattern, text)</code>
<code>str_match(string, pattern)</code>	<code>regmatches(x, m = regexec(pattern, text))</code>
<code>str_order(string)</code>	<code>order(...)</code>
<code>str_replace(string, pattern, replacement)</code>	<code>sub(pattern, replacement, x)</code>
<code>str_replace_all(string, pattern, replacement)</code>	<code>gsub(pattern, replacement, x)</code>
<code>str_sort(string)</code>	<code>sort(x)</code>
<code>str_split(string, pattern)</code>	<code>strsplit(x, split)</code>
<code>str_sub(string, start, end)</code>	<code>substr(x, start, stop)</code>
<code>str_subset(string, pattern)</code>	<code>grep(pattern, x, value = TRUE)</code>
<code>str_to_lower(string)</code>	<code>tolower(x)</code>
<code>str_to_title(string)</code>	<code>tools::toTitleCase(text)</code>
<code>str_to_upper(string)</code>	<code>toupper(x)</code>
<code>str_trim(string)</code>	<code>trimws(x)</code>
<code>str_which(string, pattern)</code>	<code>grep(pattern, x)</code>
<code>str_wrap(string)</code>	<code>strwrap(x)</code>

```

    .fns = ~ paste0("`", .x, "`")
  ) |>
  dplyr::arrange(stringr) |>
  dplyr::rename(`base R` = base_r) |>
  gt::gt() |>
  gt::fmt_markdown(columns = everything()) |>
  gt::tab_options(column_labels.font.weight = "bold")

```

Overall the main differences between base R and stringr are:

1. stringr functions start with `str_` prefix; base R string functions have no consistent naming scheme.
2. The order of inputs is usually different between base R and stringr. In base R, the `pattern` to match usually comes first; in stringr, the `string` to manipulate always comes first. This makes stringr easier to use in pipes, and with `lapply()` or `purrr::map()`.

3. Functions in `stringr` tend to do less, where many of the string processing functions in base R have multiple purposes.
4. The output and input of `stringr` functions has been carefully designed. For example, the output of `str_locate()` can be fed directly into `str_sub()`; the same is not true of `regexpr()` and `substr()`.
5. Base functions use arguments (like `perl`, `fixed`, and `ignore.case`) to control how the pattern is interpreted. To avoid dependence between arguments, `stringr` instead uses helper functions (like `fixed()`, `regex()`, and `coll()`).

Next we'll walk through each of the functions, noting the similarities and important differences. These examples are adapted from the `stringr` documentation and here they are contrasted with the analogous base R operations.

```
library(dplyr)
ae <- haven::read_sas("./data/sdtm/ae.sas7bdat")
ae1 <- ae |>
  select (USUBJID, AEDECOD, AEBODSYS)

#find function in sas, str_detect

ae2 <- ae1 |>
  filter(str_detect(AEDECOD, "HER"))

ae2 <- ae1 |>
  filter(str_detect(AEDECOD, "^HIATUS ")) # start of the string

ae2 <- ae1 |>
  filter(str_detect(AEDECOD, "HERNIA$")) # end of the string

#substr in sas

ae2 <- ae1 |>
  mutate(newvar1=str_sub(AEDECOD, 1, 6),
         newvar2=str_sub(AEDECOD, 2, 6),
         newvar3=str_sub(AEDECOD, -3),
         newvar4=str_length(AEDECOD))

#cat function in sas , Str_c in r

ae2 <- ae1 |>
  mutate(newvar1=str_c(AEDECOD, AEBODSYS, sep="/"),
         newvar2=paste(AEDECOD, AEBODSYS, sep="/"))
```

```

# scan function in sas, word in r

ae2 <- ae1 |>
  mutate(newvar1=word(AEDECOD,1),
         newvar2=word(AEDECOD,2))

# upper & lower case

ae2 <- ae1 |>
  mutate(newvar1=str_to_lower(AEDECOD),
         newvar2=str_to_upper(AEDECOD),
         newvar3=str_to_title(AEDECOD),
         newvar4=str_to_sentence(AEDECOD))

#str_trim

a <- "  this is    my String  "
b <- str_trim(a)
c <- str_replace_all(a," "," ")
d <- str_squish(a)

```

44 date and time functions

44.0.1 Date and Time Base Functions in R

Date and time manipulation is a common task in data analysis. R provides several base functions to work with dates and times. Here are some of the most commonly used base R functions for handling date and time data:

```
# Get the current date
current_date <- Sys.Date()
print(current_date)
```

```
[1] "2025-12-12"
```

```
# Get the current date and time
current_datetime <- Sys.time()
print(current_datetime)
```

```
[1] "2025-12-12 19:14:02 UTC"
```

44.0.2 Creating Date and Time Objects

44.0.3 Year Formats

Code	Meaning	Example (<code>format(dt, ...)</code>)
%Y	4-digit year	"2025"
%y	2-digit year (00–99)	"25"
%C	Century (year / 100, 00–99)	"20" (for 2025; OS-dep.)

44.0.4 Month Formats

Code	Meaning	Example
%m	Month number (01–12)	"11"
%b	Abbrev. month name (locale)	"Nov"
%B	Full month name (locale)	"November"
%h	Same as %b (abbrev. month name)	"Nov"

44.0.5 Day Formats

Code	Meaning	Example
%d	Day of month (01–31)	"14"
%e	Day of month (1–31, padded with space)	"14" (or " 7" for 7th; OS-dep.)
%j	Day of year (001–366)	"318"

44.0.6 Weekday Formats

Code	Meaning	Example
%a	Abbrev. weekday name (locale)	"Fri"
%A	Full weekday name (locale)	"Friday"
%w	Weekday number (0–6, 0 = Sunday)	"5"
%u	ISO weekday number (1–7, 1 = Monday) (OS-dep.)	"5" (Friday)

44.0.7 Time Formats

Code	Meaning	Example
%H	Hour (00–23)	"14"
%I	Hour (01–12)	"02"
%M	Minute (00–59)	"30"
%S	Second (00–61, allows for leap)	"05"
%p	AM/PM indicator (locale)	"PM"

###Locale-dependent “full date/time” codes

###These depend on your system locale (language + region):

Code	Meaning	Example
%c	Date and time representation	"Fri Nov 14 16:05:30 2025"
%x	Date representation	"11/14/25" (varies by locale)
%X	Time representation	"16:05:30"

```
# Create a Date object
dt <- as.Date("2025-11-14")
# Create a POSIXct object (date-time)
dt <- as.POSIXct("2025-11-14 16:05:30")
# Format examples
format(dt, "%Y-%m-%d") # e.g. "2025-11-14"
```

```
[1] "2025-11-14"
```

```
format(dt, "%m/%d/%y") # e.g. "11/14/25"
```

```
[1] "11/14/25"
```

```
format(dt, "%B %d, %Y") # e.g. "November 14, 2025"
```

```
[1] "November 14, 2025"
```

```
format(dt, "%c") # e.g. "Fri Nov 14 16:05:30 2025"
```

```
[1] "Fri Nov 14 16:05:30 2025"
```

```
format(dt, "%x") # e.g. "11/14/25" (depends on locale)
```

```
[1] "11/14/25"
```

```
format(dt, "%X") # e.g. "16:05:30"
```

```
[1] "16:05:30"
```

SAS informat	R parsing
ddmmyy10.	as.Date(x, "%d/%m/%Y")
ddmmyyd10.	as.Date(x, "%d-%m-%Y")
ddmmyyp10.	as.Date(x, "%d.%m.%Y")
mmddyy10.	as.Date(x, "%m/%d/%Y")
date9.	as.Date(x, "%d%b%Y")
julian7.	as.Date(strptime(x, "%Y%j"))
time8.	as.POSIXct(x, "%H:%M:%S")
time10.	as.POSIXct(x, "%I:%M:%S%p") (if am/pm)
datetime18.	as.POSIXct(x, "%d%b%Y:%H:%M:%S")
datetime20.	as.POSIXct(x, "%d%b%Y:%I:%M:%S%p")

SAS format	Meaning	R equivalent
worddate18.	Month date, year (e.g., February 23, 2003)	format(d, "%B %d, %Y")
weekdate24.	Weekday, month date, year	format(d, "%A, %B %d, %Y")
year.	numeric year	as.integer(format(d, "%Y"))
month.	month number (1–12)	as.integer(format(d, "%m"))
day.	day of month	as.integer(format(d, "%d"))
weekday.	day of week (1–7)	as.integer(format(d, "%u")) (Mon = 1)

```
# Convert a character string to a Date object
date_string <- "2023-10-01"
date_object <- as.Date(date_string)
print(date_object)
```

```
[1] "2023-10-01"
```

```
# Convert a character string to a POSIXct object (date-time)
datetime_string <- "2023-10-01 12:34:56"
datetime_object <- as.POSIXct(datetime_string)
print(datetime_object)
```

```
[1] "2023-10-01 12:34:56 UTC"
```

```
# Extract components from a Date object
year <- format(date_object, "%Y")
month <- format(date_object, "%m")
day <- format(date_object, "%d")
print(paste("Year:", year, "Month:", month, "Day:", day))
```

```
[1] "Year: 2023 Month: 10 Day: 01"
```

```
# Extract components from a POSIXct object
hour <- format(datetime_object, "%H")
minute <- format(datetime_object, "%M")
second <- format(datetime_object, "%S")
print(paste("Hour:", hour, "Minute:", minute, "Second:", second))
```

```
[1] "Hour: 12 Minute: 34 Second: 56"
```

```
# Perform date arithmetic
tomorrow <- current_date + 1
yesterday <- current_date - 1
print(paste("Tomorrow:", tomorrow, "Yesterday:", yesterday))
```

```
[1] "Tomorrow: 2025-12-13 Yesterday: 2025-12-11"
```

```
# Calculate the difference between two dates
date_diff <- as.numeric(difftime(current_date, date_object, units = "days"))
print(paste("Difference in days:", date_diff))
```

```
[1] "Difference in days: 803"
```

```
# Format a Date object to a specific string format
formatted_date <- format(current_date, "%B %d, %Y")
print(formatted_date)
```

```
[1] "December 12, 2025"
```

```
# Format a POSIXct object to a specific string format
formatted_datetime <- format(current_datetime, "%Y-%m-%d %H:%M:%S")
print(formatted_datetime)
```

```
[1] "2025-12-12 19:14:02"
```

These functions allow you to create, manipulate, and format date and time data in R. For more advanced date and time handling, you might consider using the **lubridate** package, which provides a more user-friendly interface for working with dates and times.

A practical, CDISC-friendly guide to working with **dates/times** and implementing **partial date imputation** in R using **base R** and **lubridate**.

44.1 1. Date & Time Basics in R

44.1.1 1.1 Core classes

- **Date**: calendar dates without time.
- **POSIXct**: seconds since 1970-01-01 (compact, good for storage/arithmetic).
- **POSIXlt**: list of components (year, mon, mday, hour, min, sec) — convenient to extract parts.

44.1.2 1.2 Converting numerics to Date (origins matter)

Different systems use different **origins** for numeric dates: - Excel: 1900-01-01 (beware 1900 leap-year bug in older Excels) - SAS: 1960-01-01 - R: 1970-01-01

```
num_dates <- c(0, 159, 464, 15674)

as.Date(num_dates, origin = "1960-01-01") # SAS origin
```

```
[1] "1960-01-01" "1960-06-08" "1961-04-09" "2002-11-30"
```

```
as.Date(num_dates, origin = "1970-01-01") # R origin
```

```
[1] "1970-01-01" "1970-06-09" "1971-04-10" "2012-11-30"
```

```
as.Date(num_dates, origin = "1900-01-01") # Excel origin
```

```
[1] "1900-01-01" "1900-06-09" "1901-04-10" "1942-12-01"
```

44.1.3 1.3 Custom date formats

```
x1 <- c("01/05/1965", "08/16/1975")
as.Date(x1, "%m/%d/%Y")

x2 <- c("05-01-1965", "16-08-1975")
as.Date(x2, "%d-%m-%Y")

x3 <- c("01MAY1965", "16AUG1975")
as.Date(x3, "%d%b%Y")

# Unknown format? Try multiple patterns
x4 <- c("01MAY1965MORNING", "16AUGUST1975")
as.Date(x4, tryFormats = c("%d%b%Y", "%d%B%Y"))
```

44.1.4 1.4 Datetime strings (ISO 8601 & time zones)

```
x <- c("2015-10-19T10:15", "2018-12-19T23:05")
# Keep the literal 'T' in the format string
as.POSIXct(x, format = "%Y-%m-%dT%H:%M", tz = "UTC")

# Extract parts
xt <- as.POSIXlt(x, format = "%Y-%m-%dT%H:%M")
format(xt, "%Y-%m-%d") # date part
format(xt, "%H:%M")   # time part
```

44.2 2. Lubridate: Fast, Friendly Parsing & Arithmetic

Install & load: `install.packages("lubridate"); library(lubridate)`

44.2.1 2.1 Intuitive parsers

- `ymd()`, `mdy()`, `dmy()` for dates
- `ymd_hms()`, `ymd_hm()`, `ymd_h()` for date-times

```
ymd("20210529")
dmy("29/05/2021")
ymd_hm("2019/01/19T17:15") # delimiter agnostic
```

44.2.2 2.2 Accessors and rounding

```
x <- ymd_hms("2009-08-03 12:01:59")
year(x); month(x); mday(x); wday(x); hour(x); minute(x); second(x)

floor_date(x, "month") # 2009-08-01
ceiling_date(x, "month") # 2009-09-01
round_date(x, "month") # 2009-08-01
rollback(x) # last day of previous month
```

44.2.3 2.3 Timespans

- **Durations:** exact seconds, e.g., `ddays(2)`
- **Periods:** human units, e.g., `months(1)`
- **Intervals:** start-end pair, e.g., `interval(start, end)`

```
start <- ymd("2015-10-19")
end <- ymd("2018-12-19")
int <- interval(start, end)
as.duration(int) # exact seconds
as.period(int) # years/months/days
```

44.2.4 3 ADaM-Style Partial Date Imputation (Start & End)

Imputation rules vary by study; always follow the SAP. A common pattern:

44.2.5 Start date (AESTDTC)

- YYYY-MM-DD → use as is
- YYYY-MM → impute **day** = **01** (first of month)
- YYYY → impute **month** = **01**, **day** = **01** (01-Jan)

44.2.6 End date (AEENDTC)

- YYYY-MM-DD → use as is
- YYYY-MM → impute **last day of that month**
- YYYY → impute **31-Dec**

44.2.7 Censoring by death / last alive date

Additionally, bound AEENDT by death / last alive date (when present):

- CENSOR = coalesce(DTHDT, LASTALVDT)
- AEENDT = min(imputed_end, CENSOR)

44.2.8 Imputation flags

Add flags:

- AESTDTF, AEENDTF:
 - "D" → day imputed
 - "M,D" → month & day imputed
- Add "C" if we had to **censor the end date** at DTHDT / LASTALVDT.

```
library(dplyr, warn.conflicts = FALSE)
library(stringr, warn.conflicts = FALSE)
library(lubridate, warn.conflicts = FALSE)

df <- data.frame(
  USUBJID   = sprintf("01-001-%03d", 1:6),
  AESTDTC   = c("2015-10-19", "2015-10", "2015", "2019-02", "2020-02", "2018-12-31"),
  AEENDTC   = c("2018-12-19", "2018-10", "2017-05-29", "2019-02", "2020-02", "2020"),
  RANDDT    = ymd(c("2015-10-01", "2015-10-01", "2015-01-10", "2019-01-15", "2020-02-10", "2020-02-10")),
  DTHDT     = ymd(c(NA, NA, NA, NA, "2020-08-03", NA)),
  LASTALVDT = ymd(c(NA, NA, NA, NA, NA, "2020-12-15"))
)
```

```

df_imp <- df |>
mutate(
  # Length of partial date strings
  AESTDTC_LEN = str_length(AESTDTC),
  AEENDTC_LEN = str_length(AEENDTC),
  ## --- AE start date imputation (minimum: first possible date) ---
  AESTDT = case_when(
    is.na(AESTDTC) ~ as.Date(NA),
    AESTDTC_LEN == 10 ~ ymd(AESTDTC, quiet = TRUE),
    AESTDTC_LEN == 7 ~ ymd(str_c(AESTDTC, "-01"), quiet = TRUE),
    AESTDTC_LEN == 4 ~ ymd(str_c(AESTDTC, "-01-01"), quiet = TRUE),
    TRUE ~ as.Date(NA)
  ),
  AESTDTF = case_when(
    is.na(AESTDTC) ~ NA_character_,
    AESTDTC_LEN == 10 ~ "",
    AESTDTC_LEN == 7 ~ "D",
    AESTDTC_LEN == 4 ~ "M,D",
    TRUE ~ "UNK"
  ),
  ## --- AE end date imputation (maximum: latest possible date) ---
  AEENDT_RAW = case_when(
    is.na(AEENDTC) ~ as.Date(NA),
    AEENDTC_LEN == 10 ~ ymd(AEENDTC, quiet = TRUE),
    AEENDTC_LEN == 7 ~ ymd(str_c(AEENDTC, "-01"), quiet = TRUE),
    AEENDTC_LEN == 4 ~ ymd(str_c(AEENDTC, "-12-31"), quiet = TRUE),
    TRUE ~ as.Date(NA)
  ),
  AEENDT_RAW = case_when(
    is.na(AEENDTC) ~ NA_character_,
    AEENDTC_LEN == 10 ~ "",
    AEENDTC_LEN == 7 ~ "D",
    AEENDTC_LEN == 4 ~ "M,D",
    TRUE ~ "UNK"
  )
) %>%
mutate(
  AEENDT_RAW = if_else(
    AEENDTC_LEN == 7 & !is.na(AEENDT_RAW),
    ceiling_date(AEENDT_RAW, "month") - days(1),
    AEENDT_RAW
  ),

```



```

CENSOR = coalesce(DTHDT, LASTALVDT),
AEENDT = case_when(
  !is.na(CENSOR) & !is.na(AEENDT_RAW) & AEENDT_RAW > CENSOR ~ CENSOR,
  TRUE ~ AEENDT_RAW
),
AEENDTF = case_when(
  is.na(AEENDT) ~ AEENDTF_RAW,
  !is.na(CENSOR) & !is.na(AEENDT_RAW) & AEENDT_RAW > CENSOR & AEENDTF_RAW == "" ~ "C",
  !is.na(CENSOR) & !is.na(AEENDT_RAW) & AEENDT_RAW > CENSOR & AEENDTF_RAW != "" ~ str_c(
  TRUE ~ AEENDTF_RAW
)
)

```

44.3 4. Durations & Analysis Variables

44.3.1 4.1 AE duration (AEDUR) in days

45 Using difftime (base) or as.numeric on Date differences

```
AEDUR <- as.integer(out$AEENDT - out$ASTDT)
```

45.0.1 6.2 Handy lubridate parsers (subset)

- Dates: `ymd()`, `mdy()`, `dmy()`, `yq()`
 - Datetimes: `ymd_hms()`, `ymd_hm()`, `ymd_h()`
 - Times: `hms()`, `hm()`, `ms()`
 - Accessors: `year()`, `month()`, `mday()`, `wday()`, `hour()`, `minute()`, `second()`, `date()`
 - Rounding: `floor_date()`, `ceiling_date()`, `round_date()`, `rollback()`
-

46 Reading SAS Datasets and Working with XPT Files

47 Reading SAS Datasets and Working with XPT Files

47.1 Introduction

When working with SAS datasets in R, you'll often need to read various SAS file formats, write XPT (transport) files for regulatory submissions, and manage dataset and variable attributes. This chapter covers the essential tools and techniques for these tasks.

47.2 Required Packages

```
library(haven)      # For reading/writing SAS files
library(dplyr)      # For data manipulation
library(labelled)   # For working with labels
library(sjlabelled) # Additional label functions
library(foreign)    # Alternative for XPT files
```

47.3 Reading SAS Datasets

47.3.1 Reading SAS7BDAT Files

The `haven` package is the most reliable way to read modern SAS datasets:

```
# Read a SAS dataset
data <- read_sas("path/to/dataset.sas7bdat")

# Read with specific encoding
data <- read_sas("path/to/dataset.sas7bdat", encoding = "latin1")

# Read and specify column types
data <- read_sas("path/to/dataset.sas7bdat",
```

```
col_types = cols(
  subjid = col_character(),
  age = col_double(),
  visit = col_factor()
))
```

47.3.2 Reading XPT (Transport) Files

XPT files are commonly used in pharmaceutical submissions:

```
# Using haven (recommended)
data <- read_xpt("path/to/dataset.xpt")

# Using foreign package
data <- read.xport("path/to/dataset.xpt")

# Using SASxport package for more control
data <- read.xport("path/to/dataset.xpt", verbose = TRUE)
```

47.3.3 Handling Different SAS Formats

```
# Read dataset with format catalog
data <- read_sas("dataset.sas7bdat",
  catalog = "formats.sas7bcat")

# Read older SAS transport files
data <- read_xpt("old_dataset.xpt")

# Handle missing values properly
data <- read_sas("dataset.sas7bdat",
  user_na = c(".", "", " "))
```

47.4 Working with Labels and Attributes

47.4.1 Understanding SAS Labels in R

When reading SAS datasets, labels are preserved as attributes:

```

# Check if data has labels
has_labels(data)

# View all variable labels
var_label(data)

# View label for specific variable
var_label(data$age)

# View dataset label
attr(data, "label")

```

47.4.2 Adding and Modifying Labels

```

# Add variable labels
data <- data %>%
  set_variable_labels(
    subjid = "Subject Identifier",
    age = "Age at Baseline (years)",
    sex = "Gender",
    visit = "Study Visit",
    weight = "Body Weight (kg)",
    height = "Height (cm)"
  )

# Add dataset label
attr(data, "label") <- "Demographics Dataset"

# Alternative using labelled package
data$age <- labelled(data$age, label = "Age at Baseline (years)")

# Add value labels (for categorical variables)
data$sex <- labelled(data$sex,
  labels = c("Male" = "M", "Female" = "F"),
  label = "Gender")

```

47.4.3 Working with Formats

```
# Apply format to a variable
data$visit <- factor(data$visit,
                    levels = c(1, 2, 3, 4),
                    labels = c("Screening", "Baseline", "Week 12", "Follow-up"))

# Create custom format function
format_visit <- function(x) {
  case_when(
    x == 1 ~ "Screening",
    x == 2 ~ "Baseline",
    x == 3 ~ "Week 12",
    x == 4 ~ "Follow-up",
    TRUE ~ as.character(x)
  )
}

data$visit_formatted <- format_visit(data$visit)
```

47.5 Writing XPT Files

47.5.1 Basic XPT File Creation

```
# Write XPT file using haven
write_xpt(data, "output_dataset.xpt")

# Write with specific options
write_xpt(data, "output_dataset.xpt",
          version = 8, # SAS version
          name = "DEMOG") # Dataset name in XPT

# Write multiple datasets to separate XPT files
datasets <- list(
  demog = demographics,
  vital = vital_signs,
  lab = laboratory
)
```

```
lwalk(datasets, ~write_xpt(.x, paste0(.y, ".xpt")))
```

47.5.2 Advanced XPT Writing with SASxport

```
# Prepare data for XPT with proper formatting
data_for_xpt <- data %>%
  # Ensure character variables are properly sized
  mutate(
    subjid = str_pad(subjid, width = 10, side = "right"),
    sex = str_pad(sex, width = 1, side = "right")
  ) %>%
  # Convert dates to SAS date format
  mutate(
    randdt = as.numeric(as.Date(randdt) - as.Date("1960-01-01"))
  )

# Write with SASxport package for more control
write_xport(list(DEMOG = data_for_xpt),
  file = "demog.xpt",
  verbose = TRUE,
  sasVer = "7.00",
  osType = "WIN_PRO")
```

47.5.3 Handling Variable Names and Lengths

```
# Ensure variable names comply with SAS naming rules
prepare_for_xpt <- function(data) {
  data %>%
    # Rename variables to meet SAS constraints
    rename_with(~str_to_upper(.)) %>% # Uppercase
    rename_with(~str_sub(., 1, 8)) %>% # Max 8 characters
    rename_with(~str_replace_all(., "[^A-Z0-9_]", "")) %>% # Valid characters only
    # Ensure character variables are not too long
    mutate(across(where(is.character), ~str_trunc(., width = 200)))
}

data_xpt_ready <- prepare_for_xpt(data)
write_xpt(data_xpt_ready, "dataset.xpt")
```


47.6 Adding Dataset Attributes

47.6.1 Dataset-Level Attributes

```
# Add comprehensive dataset attributes
add_dataset_attributes <- function(data,
                                   label = NULL,
                                   creation_date = Sys.Date(),
                                   creator = Sys.info()["user"],
                                   description = NULL) {

  # Add standard attributes
  attr(data, "label") <- label
  attr(data, "creation.date") <- as.character(creation_date)
  attr(data, "created.by") <- creator
  attr(data, "description") <- description

  # Add CDISC-style attributes for regulatory compliance
  attr(data, "SAS.dataset.label") <- label
  attr(data, "SAS.dataset.name") <- deparse(substitute(data))

  return(data)
}

# Apply dataset attributes
demographics <- add_dataset_attributes(
  demographics,
  label = "Subject Demographics",
  description = "Baseline demographic and background characteristics",
  creator = "Clinical Data Management"
)
```

47.6.2 Variable-Level Attributes

```
# Function to add comprehensive variable attributes
add_variable_attributes <- function(data, var_specs) {

  for (var_name in names(var_specs)) {
    if (var_name %in% names(data)) {
```

```

    spec <- var_specs[[var_name]]

    # Add label
    if (!is.null(spec$label)) {
      attr(data[[var_name]], "label") <- spec$label
    }

    # Add format
    if (!is.null(spec$format)) {
      attr(data[[var_name]], "format.sas") <- spec$format
    }

    # Add derivation comment
    if (!is.null(spec$derivation)) {
      attr(data[[var_name]], "derivation") <- spec$derivation
    }

    # Add value labels for categorical variables
    if (!is.null(spec$values)) {
      data[[var_name]] <- labelled(data[[var_name]],
                                   labels = spec$values,
                                   label = spec$label)
    }
  }
}

return(data)
}

# Define variable specifications
demog_specs <- list(
  SUBJID = list(
    label = "Subject Identifier",
    format = "$10."
  ),
  AGE = list(
    label = "Age at Randomization",
    format = "3.",
    derivation = "Calculated from birth date and randomization date"
  ),
  SEX = list(
    label = "Gender",

```

```

    format = "$1.",
    values = c("Male" = "M", "Female" = "F")
  ),
  RACE = list(
    label = "Race",
    format = "$20.",
    values = c(
      "White" = "WHITE",
      "Black or African American" = "BLACK",
      "Asian" = "ASIAN",
      "Other" = "OTHER"
    )
  )
)

# Apply variable attributes
demographics <- add_variable_attributes(demographics, demog_specs)

```

47.7 Quality Checks and Validation

47.7.1 Validating XPT Files

```

# Function to validate XPT file compliance
validate_xpt_compliance <- function(data) {

  issues <- list()

  # Check variable names
  var_names <- names(data)
  invalid_names <- var_names[!str_detect(var_names, "^[A-Z][A-Z0-9_]{0,7}$")]
  if (length(invalid_names) > 0) {
    issues$invalid_names <- invalid_names
  }

  # Check character variable lengths
  char_vars <- data %>% select(where(is.character))
  long_vars <- char_vars %>%
    summarise(across(everything(), ~max(nchar(.x), na.rm = TRUE))) %>%
    pivot_longer(everything(), names_to = "variable", values_to = "max_length") %>%

```

```

    filter(max_length > 200)

    if (nrow(long_vars) > 0) {
      issues$long_character_vars <- long_vars
    }

    # Check for missing labels
    unlabeled_vars <- var_names[sapply(data, function(x) is.null(attr(x, "label")))]
    if (length(unlabeled_vars) > 0) {
      issues$unlabeled_vars <- unlabeled_vars
    }

    return(issues)
  }

# Validate before writing
validation_issues <- validate_xpt_compliance(demographics)
if (length(validation_issues) == 0) {
  cat("Dataset passes XPT compliance checks\n")
  write_xpt(demographics, "demographics.xpt")
} else {
  cat("Validation issues found:\n")
  print(validation_issues)
}

```

47.7.2 Creating a Data Dictionary

```

# Function to create a data dictionary
create_data_dictionary <- function(data) {

  dict <- data.frame(
    Variable = names(data),
    Label = sapply(data, function(x) attr(x, "label") %||% ""),
    Type = sapply(data, class),
    Length = sapply(data, function(x) {
      if (is.character(x)) max(nchar(x), na.rm = TRUE) else NA
    }),
    Format = sapply(data, function(x) attr(x, "format.sas") %||% ""),
    stringsAsFactors = FALSE
  )
}

```

```

# Add value labels for categorical variables
dict$Values <- sapply(names(data), function(var) {
  x <- data[[var]]
  if (is.labelled(x)) {
    labels <- attr(x, "labels")
    if (!is.null(labels)) {
      paste0(names(labels), "=", labels, collapse = "; ")
    } else ""
  } else ""
})

return(dict)
}

# Create and export data dictionary
data_dict <- create_data_dictionary(demographics)
write.csv(data_dict, "demographics_dictionary.csv", row.names = FALSE)

```

47.8 Best Practices

47.8.1 Recommended Workflow

1. **Read SAS data** with appropriate encoding and column types
2. **Validate data integrity** after reading
3. **Add comprehensive labels** for variables and datasets
4. **Apply proper formatting** for categorical variables
5. **Validate XPT compliance** before writing
6. **Create documentation** (data dictionary)
7. **Write XPT files** with proper attributes

47.8.2 Performance Tips

```

# For large datasets, read in chunks
read_large_sas <- function(file_path, chunk_size = 10000) {
  # Use data.table for faster processing
  library(data.table)

  # Read metadata first

```

```

meta <- read_sas(file_path, n_max = 1)

# Process in chunks
results <- list()
skip <- 0

repeat {
  chunk <- read_sas(file_path, skip = skip, n_max = chunk_size)
  if (nrow(chunk) == 0) break

  # Process chunk here
  results[[length(results) + 1]] <- chunk
  skip <- skip + chunk_size
}

# Combine results
do.call(rbind, results)
}

# Use fst format for intermediate storage (faster than SAS)
library(fst)
write_fst(data, "intermediate_data.fst")
data <- read_fst("intermediate_data.fst")

```

47.9 Summary

This chapter covered:

- Reading various SAS file formats (SAS7BDAT, XPT)
- Working with variable and dataset labels
- Adding comprehensive attributes for regulatory compliance
- Writing XPT files with proper formatting
- Validation and quality checks
- Best practices for performance and documentation

These techniques form the foundation for working with SAS datasets in R, particularly in regulated environments where proper documentation and file formats are critical.

48 Base R Functions & Apply Family

49 Common Utilities

```
x <- 1:10  
sum(x); mean(x); sd(x); var(x); quantile(x)
```

```
[1] 55
```

```
[1] 5.5
```

```
[1] 3.02765
```

```
[1] 9.166667
```

0%	25%	50%	75%	100%
1.00	3.25	5.50	7.75	10.00

```
seq(0, 1, by=0.1); rep(5, times=3)
```

```
[1] 0.0 0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9 1.0
```

```
[1] 5 5 5
```


50 Apply Family

```
m <- matrix(1:9, nrow=3)
apply(m, 1, mean) # row means
```

```
[1] 4 5 6
```

```
apply(m, 2, mean) # col means
```

```
[1] 2 5 8
```

```
lst <- list(a=1:3, b=10:12)
sapply(lst, mean) # simplifies result
```

```
  a  b
2 11
```

```
mapply(sum, 1:3, 10:12)
```

```
[1] 11 13 15
```

51 Subsetting Essentials

```
df <- data.frame(id=1:3, val=c(10,20,30))  
df[1, "val"]
```

```
[1] 10
```

```
df[df$val > 10, ]
```

```
  id val  
2  2  20  
3  3  30
```

52 Exercises

1. Use `apply` to get the max per column of a numeric matrix.
2. Write a base R snippet to compute IQR for each column of `mtcars`.
3. Compare `lapply` vs `sapply` in behavior on a list with mixed types.

53 Custom Functions & Validation

54 Introduction to Functions in R

In R, a **function** is a reusable block of code that performs a specific task. Functions are fundamental building blocks that help you:

- Avoid repeating the same code multiple times (DRY principle)
- Organize code into logical, manageable pieces
- Make your code easier to read, test, and maintain
- Create modular, scalable programs
- Enable code sharing and collaboration

R has thousands of **built-in functions** (e.g., `sum()`, `mean()`, `lm()`, `ggplot()`), and you can also define your **own functions** to solve specific problems.

```
# Example: using built-in functions
numbers <- c(1, 2, 3, 4, 5, NA)
sum(numbers, na.rm = TRUE)
```

```
[1] 15
```

```
mean(numbers, na.rm = TRUE)
```

```
[1] 3
```

```
length(numbers)
```

```
[1] 6
```

55 Function Fundamentals

55.1 Anatomy of a Function

Every function in R has three essential components:

1. **The `formals()`:** The list of arguments that control how you call the function
2. **The `body()`:** The code inside the function that does the actual work
3. **The `environment()`:** The data structure that determines how the function finds values

```
# Create a simple function
my_function <- function(x, y = 2) {
  # This is the body
  result <- x * y + 1
  return(result)
}

# Examine the components
formals(my_function)    # Shows the arguments
```

```
$x
```

```
$y
[1] 2
```

```
body(my_function)      # Shows the code
```

```
{
  result <- x * y + 1
  return(result)
}
```

```
environment(my_function) # Shows the environment
```

```
<environment: R_GlobalEnv>
```

55.2 Basic Function Syntax

The general syntax for creating a function is:

```
function_name <- function(arg1, arg2 = default_value, ...) {  
  # Function body  
  # Your code here  
  return(result) # Optional explicit return  
}
```

55.2.1 Simple Examples

```
# 1. Function with no arguments  
say_hello <- function() {  
  return("Hello, World!")  
}  
  
say_hello()
```

```
[1] "Hello, World!"
```

```
# 2. Function with one argument  
square <- function(x) {  
  return(x^2)  
}  
  
square(4)
```

```
[1] 16
```

```
square(c(1, 2, 3, 4))
```

```
[1] 1 4 9 16
```

```
# 3. Function with multiple arguments
add_numbers <- function(a, b) {
  sum_result <- a + b
  return(sum_result)
}

add_numbers(3, 5)
```

```
[1] 8
```

```
add_numbers(c(1,2), c(3,4))
```

```
[1] 4 6
```

55.3 Function Arguments and Parameters

55.3.1 Default Arguments

```
# Function with default arguments
greet <- function(name = "Guest", greeting = "Hello") {
  message <- paste(greeting, name, "!")
  return(message)
}

greet()                                # Uses all defaults
```

```
[1] "Hello Guest !"
```

```
greet("Alice")                        # Uses default greeting
```

```
[1] "Hello Alice !"
```

```
greet("Bob", "Hi")                   # Overrides both defaults
```

```
[1] "Hi Bob !"
```



```
greet(greeting = "Hey", name = "Charlie") # Named arguments
```

```
[1] "Hey Charlie !"
```

55.3.2 The ... (Dots) Parameter

The ... allows functions to accept any number of arguments:

```
# Function using dots to pass arguments to other functions
my_summary <- function(x, ...) {
  cat("Length:", length(x), "\n")
  cat("Mean:", mean(x, ...), "\n")
  cat("SD:", sd(x, ...), "\n")
}

data_with_na <- c(1, 2, 3, NA, 5)
my_summary(data_with_na, na.rm = TRUE)
```

```
Length: 5
Mean: 2.75
SD: 1.707825
```

```
# Function that combines multiple vectors
combine_all <- function(...) {
  all_args <- list(...)
  combined <- unlist(all_args)
  return(combined)
}

combine_all(1:3, 4:6, c(7, 8))
```

```
[1] 1 2 3 4 5 6 7 8
```

55.4 Return Values

55.4.1 Explicit vs Implicit Returns

```
# Explicit return
explicit_return <- function(x) {
  result <- x * 2
  return(result) # Explicitly return the value
}

# Implicit return (last expression is returned)
implicit_return <- function(x) {
  result <- x * 2
  result # Last expression is automatically returned
}

explicit_return(5)
```

```
[1] 10
```

```
implicit_return(5)
```

```
[1] 10
```

```
# Multiple possible returns
check_number <- function(x) {
  if (x > 0) {
    return("Positive")
  } else if (x < 0) {
    return("Negative")
  } else {
    return("Zero")
  }
}

check_number(5)
```

```
[1] "Positive"
```

```
check_number(-3)
```

```
[1] "Negative"
```

```
check_number(0)
```

```
[1] "Zero"
```

55.4.2 Returning Multiple Values

```
# Return multiple values using a list
calculate_stats <- function(x) {
  stats_list <- list(
    mean = mean(x, na.rm = TRUE),
    median = median(x, na.rm = TRUE),
    sd = sd(x, na.rm = TRUE),
    min = min(x, na.rm = TRUE),
    max = max(x, na.rm = TRUE),
    n = length(x[!is.na(x)])
  )
  return(stats_list)
}

sample_data <- c(1, 2, 3, 4, 5, NA, 7, 8, 9, 10)
results <- calculate_stats(sample_data)
print(results)
```

```
$mean
[1] 5.444444
```

```
$median
[1] 5
```

```
$sd
[1] 3.205897
```

```
$min
[1] 1
```

```
$max
[1] 10
```

```
$n
[1] 9
```

```
# Access individual elements  
results$mean
```

```
[1] 5.444444
```

```
results$sd
```

```
[1] 3.205897
```

56 Intermediate Function Concepts

56.1 Function Scope and Environments

Functions create their own local environment, separate from the global environment:

```
# Global variable
global_var <- 100

scope_demo <- function() {
  local_var <- 50      # Local to this function
  global_var <- 200    # This creates a NEW local variable

  cat("Inside function - local_var:", local_var, "\n")
  cat("Inside function - global_var:", global_var, "\n")

  return(local_var + global_var)
}

result <- scope_demo()
```

```
Inside function - local_var: 50
Inside function - global_var: 200
```

```
cat("Outside function - global_var:", global_var, "\n")
```

```
Outside function - global_var: 100
```

```
# cat("Outside function - local_var:", local_var, "\n") # This would cause an error
```

56.1.1 Lexical Scoping

R uses lexical scoping - functions look for variables in the environment where they were defined:

```
# Demonstrate lexical scoping
x <- 10

make_function <- function(multiplier) {
  # This function "captures" the multiplier value
  multiply_by <- function(y) {
    return(y * multiplier * x) # Uses multiplier from parent and x from global
  }
  return(multiply_by)
}

multiply_by_5 <- make_function(5)
multiply_by_3 <- make_function(3)

multiply_by_5(4) # 4 * 5 * 10 = 200
```

```
[1] 200
```

```
multiply_by_3(4) # 4 * 3 * 10 = 120
```

```
[1] 120
```

56.2 Input Validation and Error Handling

56.2.1 Basic Input Validation

```
# Simple validation with stopifnot
safe_sqrt <- function(x) {
  stopifnot("Input must be numeric" = is.numeric(x))
  stopifnot("Input must be non-negative" = all(x >= 0, na.rm = TRUE))

  return(sqrt(x))
}

safe_sqrt(c(4, 9, 16))
```

```
[1] 2 3 4
```

```
# safe_sqrt(-4) # Would throw an error

# More detailed validation
validate_and_process <- function(data, column_name) {
  # Check if data is a data frame
  if (!is.data.frame(data)) {
    stop("Input 'data' must be a data frame")
  }

  # Check if column exists
  if (!column_name %in% names(data)) {
    stop(paste("Column", column_name, "not found in data"))
  }

  # Check if column is numeric
  if (!is.numeric(data[[column_name]])) {
    stop(paste("Column", column_name, "must be numeric"))
  }

  # Process the data
  result <- mean(data[[column_name]], na.rm = TRUE)
  return(result)
}

# Test data
test_data <- data.frame(
  age = c(25, 30, 35, NA, 45),
  name = c("Alice", "Bob", "Charlie", "Diana", "Eve")
)

validate_and_process(test_data, "age")
```

```
[1] 33.75
```

```
# validate_and_process(test_data, "name") # Would throw an error
# validate_and_process(test_data, "height") # Would throw an error
```

56.2.2 Advanced Error Handling with tryCatch

```
# Robust function with error handling
robust_operation <- function(x, operation = "mean") {
  result <- tryCatch({
    switch(operation,
      "mean" = mean(x, na.rm = TRUE),
      "median" = median(x, na.rm = TRUE),
      "sd" = sd(x, na.rm = TRUE),
      stop("Unsupported operation")
    )
  },
  warning = function(w) {
    cat("Warning occurred:", conditionMessage(w), "\n")
    return(NA)
  },
  error = function(e) {
    cat("Error occurred:", conditionMessage(e), "\n")
    return(NA)
  },
  finally = {
    cat("Operation attempted for", operation, "\n")
  })

  return(result)
}

# Test the function
test_data <- c(1, 2, 3, NA, 5)
robust_operation(test_data, "mean")
```

Operation attempted for mean

[1] 2.75

```
robust_operation(test_data, "median")
```

Operation attempted for median

[1] 2.5


```
robust_operation(test_data, "invalid_op") # Handles error gracefully
```

```
Error occurred: Unsupported operation  
Operation attempted for invalid_op
```

```
[1] NA
```

57 Advanced Function Concepts

57.1 Higher-Order Functions

Functions that take other functions as arguments or return functions:

```
# Function that takes another function as argument
apply_operation <- function(x, operation_func) {
  return(operation_func(x))
}
```

```
# Test with different functions
numbers <- c(1, 2, 3, 4, 5)
apply_operation(numbers, sum)
```

```
[1] 15
```

```
apply_operation(numbers, mean)
```

```
[1] 3
```

```
apply_operation(numbers, function(x) max(x) - min(x))
```

```
[1] 4
```

```
# Function that returns a function
make_multiplier <- function(n) {
  function(x) {
    return(x * n)
  }
}
```

```
double <- make_multiplier(2)
```

```
triple <- make_multiplier(3)
```

```
double(10) # 20
```

```
[1] 20
```

```
triple(10) # 30
```

```
[1] 30
```

57.2 Anonymous Functions and Lambda Functions

```
# Traditional function definition
```

```
square_traditional <- function(x) x^2
```

```
# Anonymous function (lambda)
```

```
numbers <- 1:5
```

```
# Using anonymous function with lapply
```

```
squared_numbers <- lapply(numbers, function(x) x^2)
```

```
print(unlist(squared_numbers))
```

```
[1] 1 4 9 16 25
```

```
# Using anonymous function with sapply
```

```
cubed_numbers <- sapply(numbers, function(x) x^3)
```

```
print(cubed_numbers)
```

```
[1] 1 8 27 64 125
```

```
# More complex anonymous function
```

```
complex_calc <- function(data, func) {
```

```
  sapply(data, func)
```

```
}
```

```
complex_calc(1:5, function(x) (x^2 + 2*x + 1))
```

```
[1] 4 9 16 25 36
```

57.3 Working with Data Frames

57.3.1 Single Input, Single Output Functions

```
# Function to calculate BMI
calculate_bmi <- function(weight_kg, height_m) {
  # Input validation
  stopifnot(is.numeric(weight_kg), is.numeric(height_m))
  stopifnot(all(weight_kg > 0, na.rm = TRUE))
  stopifnot(all(height_m > 0, na.rm = TRUE))

  bmi <- weight_kg / (height_m^2)
  return(round(bmi, 2))
}

# Test the function
weights <- c(70, 80, 65, 75)
heights <- c(1.75, 1.80, 1.65, 1.70)
bmis <- calculate_bmi(weights, heights)
print(bmis)
```

```
[1] 22.86 24.69 23.88 25.95
```

```
# Create a data frame function
process_patient_data <- function(patient_df) {
  # Validate input
  required_cols <- c("weight", "height")
  if (!all(required_cols %in% names(patient_df))) {
    stop("Data frame must contain 'weight' and 'height' columns")
  }

  # Calculate BMI and add to data frame
  patient_df$bmi <- calculate_bmi(patient_df$weight, patient_df$height)

  # Add BMI category
  patient_df$bmi_category <- cut(patient_df$bmi,
                                breaks = c(0, 18.5, 25, 30, Inf),
                                labels = c("Underweight", "Normal", "Overweight", "Obese"),
                                include.lowest = TRUE)
```

```

    return(patient_df)
}

# Test data
patient_data <- data.frame(
  id = 1:4,
  weight = c(70, 80, 65, 75),
  height = c(1.75, 1.80, 1.65, 1.70),
  age = c(25, 30, 35, 40)
)

processed_data <- process_patient_data(patient_data)
print(processed_data)

```

	id	weight	height	age	bmi	bmi_category
1	1	70	1.75	25	22.86	Normal
2	2	80	1.80	30	24.69	Normal
3	3	65	1.65	35	23.88	Normal
4	4	75	1.70	40	25.95	Overweight

57.3.2 Multiple Input, Multiple Output Functions

```

# Comprehensive statistical analysis function
comprehensive_analysis <- function(data, numeric_vars, group_var = NULL) {
  # Input validation
  if (!is.data.frame(data)) stop("Data must be a data frame")
  if (!all(numeric_vars %in% names(data))) stop("Some numeric variables not found")

  results <- list()

  # Basic descriptive statistics
  descriptive_stats <- data.frame(
    variable = numeric_vars,
    n = sapply(numeric_vars, function(var) sum(!is.na(data[[var]]))),
    mean = sapply(numeric_vars, function(var) round(mean(data[[var]]), na.rm = TRUE), 3)),
    median = sapply(numeric_vars, function(var) round(median(data[[var]]), na.rm = TRUE), 3)),
    sd = sapply(numeric_vars, function(var) round(sd(data[[var]]), na.rm = TRUE), 3)),
    min = sapply(numeric_vars, function(var) round(min(data[[var]]), na.rm = TRUE), 3)),
    max = sapply(numeric_vars, function(var) round(max(data[[var]]), na.rm = TRUE), 3)),
    stringsAsFactors = FALSE
  )
}

```

```

)

results$descriptive <- descriptive_stats

# Correlation matrix
if (length(numeric_vars) > 1) {
  cor_data <- data[, numeric_vars, drop = FALSE]
  cor_data <- cor_data[complete.cases(cor_data), ]
  results$correlation <- round(cor(cor_data), 3)
}

# Group analysis if group variable provided
if (!is.null(group_var) && group_var %in% names(data)) {
  group_stats <- list()
  groups <- unique(data[[group_var]])
  groups <- groups[!is.na(groups)]

  for (group in groups) {
    group_data <- data[data[[group_var]] == group & !is.na(data[[group_var]]), ]

    group_descriptive <- data.frame(
      variable = numeric_vars,
      group = rep(group, length(numeric_vars)),
      n = sapply(numeric_vars, function(var) sum(!is.na(group_data[[var]]))),
      mean = sapply(numeric_vars, function(var) round(mean(group_data[[var]]), na.rm = TRUE)),
      median = sapply(numeric_vars, function(var) round(median(group_data[[var]]), na.rm = TRUE)),
      sd = sapply(numeric_vars, function(var) round(sd(group_data[[var]]), na.rm = TRUE)),
      stringsAsFactors = FALSE
    )

    group_stats[[as.character(group)]] <- group_descriptive
  }

  results$group_analysis <- do.call(rbind, group_stats)
}

# Data quality report
quality_report <- data.frame(
  variable = names(data),
  type = sapply(data, class),
  missing_count = sapply(data, function(x) sum(is.na(x))),
  missing_percent = sapply(data, function(x) round(100 * sum(is.na(x)) / length(x), 2)),

```

```

    stringsAsFactors = FALSE
  )

  results$data_quality <- quality_report

  return(results)
}

# Test with sample data
sample_data <- data.frame(
  id = 1:100,
  age = rnorm(100, 40, 10),
  weight = rnorm(100, 70, 15),
  height = rnorm(100, 1.7, 0.1),
  group = sample(c("A", "B", "C"), 100, replace = TRUE),
  stringsAsFactors = FALSE
)

# Add some missing values
sample_data$age[c(5, 15, 25)] <- NA
sample_data$weight[c(10, 20)] <- NA

analysis_results <- comprehensive_analysis(
  data = sample_data,
  numeric_vars = c("age", "weight", "height"),
  group_var = "group"
)

# View results
print("Descriptive Statistics:")

```

```
[1] "Descriptive Statistics:"
```

```
print(analysis_results$descriptive)
```

	variable	n	mean	median	sd	min	max
age	age	97	38.982	38.734	9.269	17.102	64.736
weight	weight	98	68.333	70.019	14.639	31.171	97.827
height	height	100	1.698	1.703	0.105	1.395	1.944

```
print("\nCorrelation Matrix:")
```

```
[1] "\nCorrelation Matrix:"
```

```
print(analysis_results$correlation)
```

```
      age weight height
age    1.000 -0.085  0.052
weight -0.085  1.000  0.140
height  0.052  0.140  1.000
```

```
print("\nGroup Analysis (first few rows):")
```

```
[1] "\nGroup Analysis (first few rows):"
```

```
print(head(analysis_results$group_analysis))
```

	variable	group	n	mean	median	sd
C.age	age	C	31	37.256	36.767	8.109
C.weight	weight	C	33	68.755	70.322	14.117
C.height	height	C	33	1.719	1.738	0.093
B.age	age	B	25	40.050	38.154	10.361
B.weight	weight	B	26	67.939	67.886	16.860
B.height	height	B	26	1.689	1.672	0.105

```
print("\nData Quality Report:")
```

```
[1] "\nData Quality Report:"
```

```
print(analysis_results$data_quality)
```

	variable	type	missing_count	missing_percent
id	id	integer	0	0
age	age	numeric	3	3
weight	weight	numeric	2	2
height	height	numeric	0	0
group	group	character	0	0

58 Tidy Evaluation and Advanced dplyr Usage

58.1 Understanding `{{}}`, `!!`, and `!!!`

These operators are part of tidy evaluation, used extensively in the tidyverse:

```
library(dplyr)
```

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

`filter`, `lag`

The following objects are masked from 'package:base':

`intersect`, `setdiff`, `setequal`, `union`

```
library(rlang)

# Using {{}} for direct variable capture
summarize_variable <- function(data, group_var, summary_var) {
  data %>%
    group_by({{ group_var }}) %>%
    summarise(
      n = n(),
      mean = mean({{ summary_var }}, na.rm = TRUE),
      median = median({{ summary_var }}, na.rm = TRUE),
      sd = sd({{ summary_var }}, na.rm = TRUE),
      .groups = 'drop'
    )
}
```

```

# Using !! for unquoting single variables
summarize_variable_unquote <- function(data, group_var, summary_var) {
  group_sym <- sym(group_var)
  summary_sym <- sym(summary_var)

  data %>%
    group_by(!! group_sym) %>%
    summarise(
      n = n(),
      mean = mean(!! summary_sym, na.rm = TRUE),
      median = median(!!summary_sym, na.rm = TRUE),
      sd = sd(!!summary_sym, na.rm = TRUE),
      .groups = 'drop'
    )
}

# Using !!! for splicing multiple variables
select_multiple_vars <- function(data, ...) {
  vars <- list(...)
  data %>% select(!!! vars)
}

# Test data
test_df <- data.frame(
  group = rep(c("A", "B"), each = 50),
  value1 = rnorm(100, 10, 2),
  value2 = rnorm(100, 20, 3),
  id = 1:100
)

# Test the functions
result1 <- summarize_variable(test_df, group, value1)
print(result1)

```

```

# A tibble: 2 x 5
  group      n mean median    sd
  <chr> <int> <dbl>  <dbl> <dbl>
1 A       50  10.1   10.3  2.13
2 B       50  10.5   10.6  1.86

```

```
result2 <- summarize_variable_unquote(test_df, "group", "value1")
print(result2)
```

```
# A tibble: 2 x 5
  group      n mean median    sd
  <chr> <int> <dbl> <dbl> <dbl>
1 A      50  10.1   10.3  2.13
2 B      50  10.5   10.6  1.86
```

```
selected_data <- select_multiple_vars(test_df, "id", "group", "value1")
print(head(selected_data))
```

```
   id group  value1
1  1     A  7.898372
2  2     A  6.915630
3  3     A  9.916303
4  4     A  7.161049
5  5     A 10.027807
6  6     A 10.202519
```

58.2 Advanced Data Processing Function

```
library(tidyr)
library(rlang)
library(dplyr)
# Comprehensive data processing function using tidy evaluation
process_clinical_data <- function(data, group_vars, analysis_vars,
                                   id_var = "USUBJID", stats = c("n", "mean", "median", "sd"))

  # Input validation
  stopifnot(is.data.frame(data))
  stopifnot(all(c(group_vars, analysis_vars, id_var) %in% names(data)))

  # Convert character vectors to symbols
  group_syms <- syms(group_vars)
  analysis_syms <- syms(analysis_vars)
  id_sym <- sym(id_var)
```

```

results <- list()

# Process each analysis variable
for (var in analysis_vars) {
  var_sym <- sym(var)

  # Create summary statistics
  summary_stats <- data %>%
    group_by(!!! group_syms) %>%
    summarise(
      n = n_distinct(!!id_sym),
      mean = round(mean(!!var_sym, na.rm = TRUE), 2),
      median = round(median(!!var_sym, na.rm = TRUE), 2),
      sd = round(sd(!! var_sym, na.rm = TRUE), 3),
      min = round(min(!!var_sym, na.rm = TRUE), 2),
      max = round(max(!!var_sym, na.rm = TRUE), 2),
      q25 = round(quantile(!! var_sym, 0.25, na.rm = TRUE), 2),
      q75 = round(quantile(!!var_sym, 0.75, na.rm = TRUE), 2),
      .groups = 'drop'
    )

  # Select only requested statistics
  summary_stats <- summary_stats %>%
    select(all_of(c(group_vars, stats)))

  # Reshape to long format then wide format for presentation
  if (length(group_vars) == 1) {
    transposed_data <- summary_stats %>%
      pivot_longer(
        cols = all_of(stats),
        names_to = "statistic",
        values_to = "value"
      ) %>%
      mutate(value = as.character(value)) %>%
      pivot_wider(
        names_from = all_of(group_vars[1]),
        values_from = value
      ) %>%
      mutate(variable = var) %>%
      select(variable, statistic, everything())

    results[[var]] <- transposed_data
  }
}

```

```

    } else {
      results[[var]] <- summary_stats
    }
  }

  return(results)
}

# Create sample clinical data
set.seed(123)
clinical_data <- data.frame(
  USUBJID = paste0("SUBJ-", sprintf("%03d", 1:150)),
  TRT01A = sample(c("Placebo", "Treatment A", "Treatment B"), 150, replace = TRUE),
  AGE = round(rnorm(150, 65, 12)),
  WEIGHTBL = round(rnorm(150, 75, 15), 1),
  HEIGHTBL = round(rnorm(150, 170, 10), 1),
  SEX = sample(c("M", "F"), 150, replace = TRUE),
  stringsAsFactors = FALSE
)

# Process the data
clinical_results <- process_clinical_data(
  data = clinical_data,
  group_vars = "TRT01A",
  analysis_vars = c("AGE", "WEIGHTBL"),
  id_var = "USUBJID",
  stats = c("n", "mean", "median", "sd")
)

print("Age Analysis:")

```

```
[1] "Age Analysis:"
```

```
print(clinical_results$AGE)
```

```
# A tibble: 4 x 5
  variable statistic Placebo `Treatment A` `Treatment B`
  <chr>      <chr>      <chr>    <chr>          <chr>
1 AGE        n          42      54            54
2 AGE        mean       65.19   63.8          67.07
3 AGE        median      63.5    63.5          69
```

4	AGE	sd	11.502	12.057	13.806
---	-----	----	--------	--------	--------

```
print("\nWeight Analysis:")
```

```
[1] "\nWeight Analysis:"
```

```
print(clinical_results$WEIGHTBL)
```

```
# A tibble: 4 x 5
  variable statistic Placebo `Treatment A` `Treatment B`
  <chr>      <chr>      <chr>    <chr>         <chr>
1 WEIGHTBL n          42      54          54
2 WEIGHTBL mean      69.25   76.05      76.26
3 WEIGHTBL median    71.15   75.95      77.6
4 WEIGHTBL sd        14.849  13.408     17.145
```

59 Advanced Topics

59.1 Function Factories and Closures

```
# Function factory that creates specialized functions
create_validator <- function(min_val = -Inf, max_val = Inf, allow_na = TRUE) {
  function(x, var_name = "variable") {
    # Check if NA values are allowed
    if (! allow_na && any(is.na(x))) {
      stop(paste(var_name, "contains NA values, which are not allowed"))
    }

    # Check range
    x_clean <- x[!is.na(x)]
    if (any(x_clean < min_val | x_clean > max_val)) {
      stop(paste(var_name, "contains values outside the allowed range [",
                  min_val, ",", max_val, "]"))
    }

    return(TRUE)
  }
}

# Create specialized validators
age_validator <- create_validator(min_val = 0, max_val = 120, allow_na = FALSE)
weight_validator <- create_validator(min_val = 0, max_val = 500, allow_na = TRUE)
percentage_validator <- create_validator(min_val = 0, max_val = 100, allow_na = FALSE)

# Test the validators
test_ages <- c(25, 35, 45, 55, 65)
age_validator(test_ages, "Age")
```

```
[1] TRUE
```

```
test_weights <- c(50, 75, 90, NA, 120)
weight_validator(test_weights, "Weight")
```

```
[1] TRUE
```

```
# This would throw an error:
# bad_ages <- c(25, 35, -5, 200)
# age_validator(bad_ages, "Age")
```

59.2 Memoization for Performance

```
# Function with memoization for expensive calculations
create_memoized_function <- function(func) {
  cache <- new.env(parent = emptyenv())

  function(...) {
    # Create a key from the arguments
    key <- paste(list(...), collapse = "_")

    # Check if result is in cache
    if (exists(key, cache)) {
      cat("Cache hit!\n")
      return(get(key, cache))
    }

    # Calculate result and store in cache
    cat("Computing result...\n")
    result <- func(...)
    assign(key, result, cache)
    return(result)
  }
}

# Expensive calculation function
expensive_calculation <- function(n) {
  Sys.sleep(1) # Simulate expensive operation
  return(sum(1:n))
}
```



```
# Create memoized version
fast_calculation <- create_memoized_function(expensive_calculation)

# Test memoization
system.time(result1 <- fast_calculation(1000)) # First call - slow
```

Computing result...

```
      user  system elapsed
0.000    0.000    1.001
```

```
system.time(result2 <- fast_calculation(1000)) # Second call - fast (cached)
```

Cache hit!

```
      user  system elapsed
0         0         0
```

```
system.time(result3 <- fast_calculation(2000)) # New argument - slow again
```

Computing result...

```
      user  system elapsed
0.002    0.000    1.002
```

59.3 Method Dispatch and S3 Classes

```
# Create a custom class and methods
create_summary_stats <- function(x, ...) {
  UseMethod("create_summary_stats")
}

# Method for numeric vectors
create_summary_stats.numeric <- function(x, ...) {
  result <- list(
    data = x,
```

```

    mean = mean(x, na.rm = TRUE),
    median = median(x, na.rm = TRUE),
    sd = sd(x, na.rm = TRUE),
    n = length(x),
    missing = sum(is.na(x))
  )
  class(result) <- "numeric_summary"
  return(result)
}

# Method for data frames
create_summary_stats.data.frame <- function(x, ...) {
  numeric_vars <- sapply(x, is.numeric)
  result <- list(
    data = x,
    numeric_summaries = lapply(x[numeric_vars], create_summary_stats.numeric),
    dimensions = dim(x),
    variables = names(x)
  )
  class(result) <- "dataframe_summary"
  return(result)
}

# Print methods
print.numeric_summary <- function(x, ...) {
  cat("Numeric Summary\n")
  cat("=====\n")
  cat("n:", x$n, "\n")
  cat("Missing:", x$missing, "\n")
  cat("Mean:", round(x$mean, 3), "\n")
  cat("Median:", round(x$median, 3), "\n")
  cat("SD:", round(x$sd, 3), "\n")
}

print.dataframe_summary <- function(x, ...) {
  cat("Data Frame Summary\n")
  cat("=====\n")
  cat("Dimensions:", x$dimensions[1], "rows x", x$dimensions[2], "columns\n")
  cat("Variables:", paste(x$variables, collapse = ", "), "\n")
  cat("Numeric variables:", length(x$numeric_summaries), "\n")
}

```

```
# Test the methods
numeric_data <- rnorm(100, 50, 10)
df_data <- data.frame(
  x = rnorm(50),
  y = rnorm(50, 10, 2),
  group = sample(letters[1:3], 50, replace = TRUE)
)

summary1 <- create_summary_stats(numeric_data)
summary2 <- create_summary_stats(df_data)

print(summary1)
```

```
Numeric Summary
=====
n: 100
Missing: 0
Mean: 49.429
Median: 49.686
SD: 9.467
```

```
print(summary2)
```

```
Data Frame Summary
=====
Dimensions: 50 rows x 3 columns
Variables: x, y, group
Numeric variables: 2
```

60 Testing and Documentation

60.1 Unit Testing with testthat

```
# Install required packages (run once)
if (!require(testthat)) install.packages("testthat")
if (!require(devtools)) install.packages("devtools")

# Set up testing structure
usethis::use_testthat()
```

Example test file (tests/testthat/test-statistical-functions.R):

```
# Test file content
library(testthat)

test_that("calculate_bmi works correctly", {
  # Test normal case
  expect_equal(calculate_bmi(70, 1.75), 22.86)

  # Test vector input
  weights <- c(70, 80)
  heights <- c(1.75, 1.8)
  expected <- c(22.86, 24.69)
  expect_equal(calculate_bmi(weights, heights), expected, tolerance = 0.01)

  # Test error conditions
  expect_error(calculate_bmi(-70, 1.75)) # Negative weight
  expect_error(calculate_bmi(70, -1.75)) # Negative height
  expect_error(calculate_bmi("70", 1.75)) # Non-numeric input
})

test_that("comprehensive_analysis handles edge cases", {
  # Test with minimal data
  minimal_data <- data.frame(x = c(1, 2, 3), y = c(4, 5, 6))
```

```

result <- comprehensive_analysis(minimal_data, c("x", "y"))

expect_true(is.list(result))
expect_true("descriptive" %in% names(result))
expect_true("correlation" %in% names(result))
expect_equal(nrow(result$descriptive), 2)

# Test with missing data
na_data <- data.frame(x = c(1, NA, 3), y = c(4, 5, NA))
result_na <- comprehensive_analysis(na_data, c("x", "y"))

expect_true(any(result_na$descriptive$n < 3)) # Should handle missing values
})

```

60.2 Documentation with roxygen2

```

#' Calculate Body Mass Index (BMI)
#'
#' This function calculates BMI using the standard formula: weight (kg) / height (m)^2
#'
#' @param weight_kg Numeric vector of weights in kilograms. Must be positive.
#' @param height_m Numeric vector of heights in meters. Must be positive.
#'
#' @return Numeric vector of BMI values, rounded to 2 decimal places
#'
#' @details
#' BMI categories:
#' - Underweight: < 18.5
#' - Normal weight: 18.5-24.9
#' - Overweight: 25-29.9
#' - Obese: 30
#'
#' @examples
#' # Single values
#' calculate_bmi(70, 1.75)
#'
#' # Multiple values
#' weights <- c(60, 70, 80, 90)
#' heights <- c(1.6, 1.7, 1.8, 1.9)

```

```

#' calculate_bmi(weights, heights)
#'
#' @seealso
#' \url{https://www.cdc.gov/healthyweight/assessing/bmi/} for BMI information
#'
#' @export
calculate_bmi <- function(weight_kg, height_m) {
  # Input validation
  stopifnot("Weight must be numeric" = is.numeric(weight_kg))
  stopifnot("Height must be numeric" = is.numeric(height_m))
  stopifnot("Weight must be positive" = all(weight_kg > 0, na.rm = TRUE))
  stopifnot("Height must be positive" = all(height_m > 0, na.rm = TRUE))

  # Calculate BMI
  bmi <- weight_kg / (height_m^2)
  return(round(bmi, 2))
}

#' Comprehensive Statistical Analysis
#'
#' Performs comprehensive statistical analysis on numeric variables in a dataset,
#' including descriptive statistics, correlations, and optional group comparisons.
#'
#' @param data A data frame containing the variables to analyze
#' @param numeric_vars Character vector of numeric variable names to analyze
#' @param group_var Optional character string specifying a grouping variable for
#'   stratified analysis
#'
#' @return A list containing:
#'   \item{descriptive}{Data frame with descriptive statistics for each variable}
#'   \item{correlation}{Correlation matrix (if multiple numeric variables)}
#'   \item{group_analysis}{Stratified analysis by group (if group_var specified)}
#'   \item{data_quality}{Data quality report with missing value information}
#'
#' @examples
#' # Basic analysis
#' data(mtcars)
#' results <- comprehensive_analysis(mtcars, c("mpg", "hp", "wt"))
#'
#' # Analysis with grouping
#' results_grouped <- comprehensive_analysis(mtcars, c("mpg", "hp"), "cyl")
#'

```

```

#' @export
comprehensive_analysis <- function(data, numeric_vars, group_var = NULL) {
  # [Function body as defined earlier]
  # Input validation
  if (!is.data.frame(data)) stop("Data must be a data frame")
  if (!all(numeric_vars %in% names(data))) stop("Some numeric variables not found")

  results <- list()

  # Basic descriptive statistics
  descriptive_stats <- data.frame(
    variable = numeric_vars,
    n = sapply(numeric_vars, function(var) sum(!is.na(data[[var]]))),
    mean = sapply(numeric_vars, function(var) round(mean(data[[var]], na.rm = TRUE), 3)),
    median = sapply(numeric_vars, function(var) round(median(data[[var]], na.rm = TRUE), 3)),
    sd = sapply(numeric_vars, function(var) round(sd(data[[var]], na.rm = TRUE), 3)),
    min = sapply(numeric_vars, function(var) round(min(data[[var]], na.rm = TRUE), 3)),
    max = sapply(numeric_vars, function(var) round(max(data[[var]], na.rm = TRUE), 3)),
    stringsAsFactors = FALSE
  )

  results$descriptive <- descriptive_stats

  # Correlation matrix
  if (length(numeric_vars) > 1) {
    cor_data <- data[, numeric_vars, drop = FALSE]
    cor_data <- cor_data[complete.cases(cor_data), ]
    results$correlation <- round(cor(cor_data), 3)
  }

  # Group analysis if group variable provided
  if (!is.null(group_var) && group_var %in% names(data)) {
    group_stats <- list()
    groups <- unique(data[[group_var]])
    groups <- groups[!is.na(groups)]

    for (group in groups) {
      group_data <- data[data[[group_var]] == group & !is.na(data[[group_var]]), ]

      group_descriptive <- data.frame(
        variable = numeric_vars,
        group = rep(group, length(numeric_vars)),

```

```

    n = sapply(numeric_vars, function(var) sum(!is.na(group_data[[var]]))),
    mean = sapply(numeric_vars, function(var) round(mean(group_data[[var]]), na.rm = TRUE), 2),
    median = sapply(numeric_vars, function(var) round(median(group_data[[var]]), na.rm = TRUE), 2),
    sd = sapply(numeric_vars, function(var) round(sd(group_data[[var]]), na.rm = TRUE), 2),
    stringsAsFactors = FALSE
  )

  group_stats[[as.character(group)]] <- group_descriptive
}

results$group_analysis <- do.call(rbind, group_stats)
}

# Data quality report
quality_report <- data.frame(
  variable = names(data),
  type = sapply(data, class),
  missing_count = sapply(data, function(x) sum(is.na(x))),
  missing_percent = sapply(data, function(x) round(100 * sum(is.na(x)) / length(x), 2)),
  stringsAsFactors = FALSE
)

results$data_quality <- quality_report

return(results)
}

```


61 Performance and Optimization

61.1 Vectorization vs Loops

```
# Demonstrate vectorization benefits
set.seed(123)
large_vector <- rnorm(1000000)

# Slow approach - using a loop
loop_square <- function(x) {
  result <- numeric(length(x))
  for (i in seq_along(x)) {
    result[i] <- x[i]^2
  }
  return(result)
}

# Fast approach - vectorized
vectorized_square <- function(x) {
  return(x^2)
}

# Benchmark the functions
library(microbenchmark)

benchmark_results <- microbenchmark(
  loop = loop_square(large_vector[1:1000]),
  vectorized = vectorized_square(large_vector[1:1000]),
  times = 100
)

print(benchmark_results)
```

Unit: microseconds

expr	min	lq	mean	median	uq	max	neval
------	-----	----	------	--------	----	-----	-------

loop	58.198	59.406	93.45963	60.7080	61.665	3295.877	100
vectorized	4.008	4.433	4.84468	4.6735	4.950	9.047	100

61.2 Memory Management

```
# Efficient data processing function
efficient_data_processor <- function(data, chunk_size = 1000) {
  n_rows <- nrow(data)
  n_chunks <- ceiling(n_rows / chunk_size)

  results <- vector("list", n_chunks)

  for (i in seq_len(n_chunks)) {
    start_row <- (i - 1) * chunk_size + 1
    end_row <- min(i * chunk_size, n_rows)

    chunk <- data[start_row:end_row, , drop = FALSE]

    # Process chunk (example: calculate row means for numeric columns)
    numeric_cols <- sapply(chunk, is.numeric)
    if (any(numeric_cols)) {
      chunk_means <- rowMeans(chunk[, numeric_cols, drop = FALSE], na.rm = TRUE)
      results[[i]] <- data.frame(
        chunk = i,
        row_start = start_row,
        row_end = end_row,
        mean_value = mean(chunk_means, na.rm = TRUE)
      )
    }
  }

  return(do.call(rbind, results))
}

# Test with large dataset
large_data <- data.frame(
  id = 1:5000,
  x1 = rnorm(5000),
  x2 = rnorm(5000, 10, 2),
  x3 = rnorm(5000, -5, 3),
```

```

  group = sample(letters[1:5], 5000, replace = TRUE)
)

chunk_results <- efficient_data_processor(large_data, chunk_size = 1000)
print(chunk_results)

```

	chunk	row_start	row_end	mean_value
1	1	1	1000	126.3689
2	2	1001	2000	376.3112
3	3	2001	3000	626.4316
4	4	3001	4000	876.3716
5	5	4001	5000	1126.3299

62 Best Practices and Common Patterns

62.1 Function Design Principles

```
# Good function design example
#' Process Survey Responses
#'
#' A well-designed function that follows best practices:
#' - Single responsibility
#' - Clear naming
#' - Input validation
#' - Meaningful return values
#' - Error handling
process_survey_responses <- function(responses,
                                     scale_min = 1,
                                     scale_max = 5,
                                     remove_incomplete = TRUE,
                                     return_summary = FALSE) {

  # Input validation with clear error messages
  if (!is.data.frame(responses)) {
    stop("Argument 'responses' must be a data. frame")
  }

  if (!is.numeric(scale_min) || !is.numeric(scale_max) || scale_min >= scale_max) {
    stop("scale_min and scale_max must be numeric with scale_min < scale_max")
  }

  # Required columns check
  required_cols <- c("participant_id", "response_value")
  missing_cols <- setdiff(required_cols, names(responses))
  if (length(missing_cols) > 0) {
    stop("Missing required columns: ", paste(missing_cols, collapse = ", "))
  }
}
```

```

# Data processing
processed_data <- responses

# Remove out-of-range responses
valid_range <- processed_data$response_value >= scale_min &
  processed_data$response_value <= scale_max
processed_data <- processed_data[valid_range & !is.na(valid_range), ]

# Remove incomplete responses if requested
if (remove_incomplete) {
  complete_cases <- complete.cases(processed_data)
  processed_data <- processed_data[complete_cases, ]
}

# Add standardized score (0-1 scale)
processed_data$standardized_score <-
  (processed_data$response_value - scale_min) / (scale_max - scale_min)

# Return summary or full data based on parameter
if (return_summary) {
  summary_stats <- list(
    n_responses = nrow(processed_data),
    mean_response = mean(processed_data$response_value),
    mean_standardized = mean(processed_data$standardized_score),
    response_distribution = table(processed_data$response_value)
  )
  return(summary_stats)
} else {
  return(processed_data)
}
}

# Example usage
sample_responses <- data.frame(
  participant_id = paste0("P", 1:100),
  response_value = sample(1:5, 100, replace = TRUE, prob = c(0.1, 0.2, 0.4, 0.2, 0.1)),
  age_group = sample(c("18-30", "31-50", "51+"), 100, replace = TRUE)
)

processed_responses <- process_survey_responses(sample_responses)
summary_only <- process_survey_responses(sample_responses, return_summary = TRUE)

```

```
print("Processed data (first 10 rows):")
```

```
[1] "Processed data (first 10 rows):"
```

```
print(head(processed_responses, 10))
```

	participant_id	response_value	age_group	standardized_score
1	P1	1	31-50	0.00
2	P2	2	51+	0.25
3	P3	3	51+	0.50
4	P4	4	51+	0.75
5	P5	1	51+	0.00
6	P6	3	51+	0.50
7	P7	1	51+	0.00
8	P8	5	31-50	1.00
9	P9	1	51+	0.00
10	P10	4	31-50	0.75

```
print("\nSummary statistics:")
```

```
[1] "\nSummary statistics:"
```

```
print(summary_only)
```

```
$n_responses
```

```
[1] 100
```

```
$mean_response
```

```
[1] 2.93
```

```
$mean_standardized
```

```
[1] 0.4825
```

```
$response_distribution
```

```
1 2 3 4 5  
13 25 31 18 13
```

62.2 Error Handling Patterns

```
# Comprehensive error handling example
robust_file_processor <- function(file_path, expected_columns = NULL) {

  # Create a custom error class for better error handling
  create_processing_error <- function(message, type = "general") {
    err <- structure(
      list(message = message, type = type, call = sys.call(-1)),
      class = c("processing_error", "error", "condition")
    )
    return(err)
  }

  tryCatch({
    # Check if file exists
    if (!file.exists(file_path)) {
      stop(create_processing_error(
        paste("File does not exist:", file_path),
        "file_not_found"
      ))
    }

    # Try to read the file
    file_ext <- tools::file_ext(file_path)
    data <- switch(tolower(file_ext),
      "csv" = read.csv(file_path, stringsAsFactors = FALSE),
      "txt" = read.table(file_path, header = TRUE, stringsAsFactors = FALSE),
      stop(create_processing_error(
        paste("Unsupported file extension:", file_ext),
        "unsupported_format"
      ))
    )

    # Validate columns if specified
    if (!is.null(expected_columns)) {
      missing_cols <- setdiff(expected_columns, names(data))
      if (length(missing_cols) > 0) {
        stop(create_processing_error(
          paste("Missing expected columns:", paste(missing_cols, collapse = ", ")),
          "missing_columns"
        )
      )
    }
  })
}
```

```

    ))
  }
}

# Return successful result
return(list(
  success = TRUE,
  data = data,
  message = paste("Successfully processed", nrow(data), "rows from", basename(file_path))
))

}, processing_error = function(e) {
  return(list(
    success = FALSE,
    data = NULL,
    error_type = e$type,
    message = e$message
  ))
}, error = function(e) {
  return(list(
    success = FALSE,
    data = NULL,
    error_type = "unexpected",
    message = paste("Unexpected error:", e$message)
  ))
})
}

# Test the error handling (create a temporary file for demonstration)
temp_file <- tempfile(fileext = ".csv")
write.csv(data.frame(x = 1:5, y = letters[1:5]), temp_file, row.names = FALSE)

# Successful case
result_success <- robust_file_processor(temp_file, c("x", "y"))
print("Success case:")

```

```
[1] "Success case:"
```

```
print(result_success$message)
```

```
[1] "Successfully processed 5 rows from file29f04597b720.csv"
```



```
# Error case - missing columns
result_error <- robust_file_processor(temp_file, c("x", "y", "z"))
print("\nError case:")
```

```
[1] "\nError case:"
```

```
print(paste("Error type:", result_error$error_type))
```

```
[1] "Error type: missing_columns"
```

```
print(paste("Message:", result_error$message))
```

```
[1] "Message: Missing expected columns: z"
```

```
# Clean up
unlink(temp_file)
```

63 Practical Exercises

63.1 Exercise 1: Data Validation Function

Create a comprehensive data validation function:

```
#' Comprehensive Data Validation
#'
#' Validates a data frame against specified rules and returns a detailed report
validate_dataset <- function(data, rules = NULL) {
  validation_results <- list(
    is_valid = TRUE,
    errors = character(0),
    warnings = character(0),
    summary = list()
  )

  # Basic validation
  if (!is.data.frame(data)) {
    validation_results$is_valid <- FALSE
    validation_results$errors <- c(validation_results$errors, "Input is not a data frame")
    return(validation_results)
  }

  validation_results$summary$n_rows <- nrow(data)
  validation_results$summary$n_cols <- ncol(data)
  validation_results$summary$column_names <- names(data)

  # Check for empty dataset
  if (nrow(data) == 0) {
    validation_results$warnings <- c(validation_results$warnings, "Dataset is empty")
  }

  # Rule-based validation
  if (!is.null(rules)) {
    for (rule_name in names(rules)) {
```

```

rule <- rules[[rule_name]]

if (rule$type == "required_columns") {
  missing_cols <- setdiff(rule$columns, names(data))
  if (length(missing_cols) > 0) {
    validation_results$is_valid <- FALSE
    validation_results$errors <- c(
      validation_results$errors,
      paste("Missing required columns:", paste(missing_cols, collapse = ", "))
    )
  }
}

if (rule$type == "data_types") {
  for (col in names(rule$expected_types)) {
    if (col %in% names(data)) {
      expected_type <- rule$expected_types[[col]]
      actual_type <- class(data[[col]])[1]

      if (actual_type != expected_type) {
        validation_results$warnings <- c(
          validation_results$warnings,
          paste("Column", col, "expected type", expected_type,
            "but found", actual_type)
        )
      }
    }
  }
}

if (rule$type == "value_ranges") {
  for (col in names(rule$ranges)) {
    if (col %in% names(data) && is.numeric(data[[col]])) {
      range_rule <- rule$ranges[[col]]
      out_of_range <- data[[col]] < range_rule$min | data[[col]] > range_rule$max
      out_of_range <- out_of_range[!is.na(out_of_range)]

      if (any(out_of_range)) {
        validation_results$warnings <- c(
          validation_results$warnings,
          paste("Column", col, "has", sum(out_of_range),
            "values outside range [", range_rule$min, ",", range_rule$max, "]")
        )
      }
    }
  }
}

```

```

    }
  }
}

# Check for missing values
missing_summary <- sapply(data, function(x) sum(is.na(x)))
validation_results$summary$missing_values <- missing_summary[missing_summary > 0]

return(validation_results)
}

# Define validation rules
validation_rules <- list(
  required_cols = list(
    type = "required_columns",
    columns = c("id", "age", "treatment")
  ),
  data_types = list(
    type = "data_types",
    expected_types = list(
      id = "character",
      age = "numeric",
      treatment = "character"
    )
  ),
  ranges = list(
    type = "value_ranges",
    ranges = list(
      age = list(min = 18, max = 100)
    )
  )
)

# Test data
test_data_valid <- data.frame(
  id = paste0("P", 1:10),
  age = sample(25:75, 10),
  treatment = sample(c("A", "B"), 10, replace = TRUE),

```

```

    stringsAsFactors = FALSE
  )

test_data_invalid <- data.frame(
  id = paste0("P", 1:10),
  age = c(sample(25:75, 8), 15, 105), # Some ages out of range
  weight = rnorm(10, 70, 10),         # Missing treatment column
  stringsAsFactors = FALSE
)

# Run validations
validation_valid <- validate_dataset(test_data_valid, validation_rules)
validation_invalid <- validate_dataset(test_data_invalid, validation_rules)

print("Valid dataset validation:")

```

```
[1] "Valid dataset validation:"
```

```
print(paste("Is valid:", validation_valid$is_valid))
```

```
[1] "Is valid: TRUE"
```

```
print("Warnings:")
```

```
[1] "Warnings:"
```

```
print(validation_valid$warnings)
```

```
[1] "Column age expected type numeric but found integer"
```

```
print("\nInvalid dataset validation:")
```

```
[1] "\nInvalid dataset validation:"
```

```
print(paste("Is valid:", validation_invalid$is_valid))
```

```
[1] "Is valid: FALSE"
```

```
print("Errors:")
```

```
[1] "Errors:"
```

```
print(validation_invalid$errors)
```

```
[1] "Missing required columns: treatment"
```

```
print("Warnings:")
```

```
[1] "Warnings:"
```

```
print(validation_invalid$warnings)
```

```
[1] "Column age has 2 values outside range [ 18 , 100 ]"
```

63.2 Exercise 2: Advanced Statistical Function

```
#' Advanced Statistical Analysis with Bootstrap Confidence Intervals
advanced_statistics <- function(data, variable, group = NULL, conf_level = 0.95, n_bootstrap) {

  # Input validation
  stopifnot(is.data.frame(data))
  stopifnot(variable %in% names(data))
  stopifnot(is.numeric(data[[variable]]))
  stopifnot(conf_level > 0 & conf_level < 1)

  # Bootstrap function for confidence intervals
  bootstrap_mean <- function(x, n_boot = n_bootstrap) {
    boot_means <- replicate(n_boot, {
      sample_data <- sample(x, length(x), replace = TRUE)
      mean(sample_data, na.rm = TRUE)
    })
    return(boot_means)
  }
}
```

```

# Calculate confidence interval
calc_ci <- function(x, conf_level) {
  alpha <- 1 - conf_level
  boot_samples <- bootstrap_mean(x[!is.na(x)])
  ci_lower <- quantile(boot_samples, alpha/2)
  ci_upper <- quantile(boot_samples, 1 - alpha/2)
  return(c(lower = ci_lower, upper = ci_upper))
}

# Analysis function
analyze_group <- function(data_subset, group_name = "Overall") {
  values <- data_subset[[variable]]
  clean_values <- values[!is.na(values)]

  if (length(clean_values) < 2) {
    return(data.frame(
      group = group_name,
      n = length(clean_values),
      mean = NA,
      median = NA,
      sd = NA,
      se = NA,
      ci_lower = NA,
      ci_upper = NA,
      skewness = NA,
      kurtosis = NA
    ))
  }

  # Basic statistics
  mean_val <- mean(clean_values)
  median_val <- median(clean_values)
  sd_val <- sd(clean_values)
  se_val <- sd_val / sqrt(length(clean_values))

  # Confidence interval
  ci <- calc_ci(clean_values, conf_level)

  # Higher moments
  skewness <- function(x) {
    x_centered <- x - mean(x)
    n <- length(x)

```

```

    skew <- (sum(x_centered^3) / n) / (sum(x_centered^2) / n)^(3/2)
    return(skew)
}

kurtosis <- function(x) {
  x_centered <- x - mean(x)
  n <- length(x)
  kurt <- (sum(x_centered^4) / n) / (sum(x_centered^2) / n)^2 - 3
  return(kurt)
}

return(data.frame(
  group = group_name,
  n = length(clean_values),
  mean = round(mean_val, 3),
  median = round(median_val, 3),
  sd = round(sd_val, 3),
  se = round(se_val, 3),
  ci_lower = round(ci[1], 3),
  ci_upper = round(ci[2], 3),
  skewness = round(skewness(clean_values), 3),
  kurtosis = round(kurtosis(clean_values), 3)
))
}

# Perform analysis
if (is.null(group)) {
  results <- analyze_group(data, "Overall")
} else {
  if (!group %in% names(data)) {
    stop(paste("Group variable", group, "not found in data"))
  }

  groups <- unique(data[[group]])
  groups <- groups[!is.na(groups)]

  group_results <- lapply(groups, function(g) {
    subset_data <- data[data[[group]] == g & !is.na(data[[group]]), ]
    analyze_group(subset_data, as.character(g))
  })

  results <- do.call(rbind, group_results)
}

```



```

}

return(results)
}

# Test the advanced statistics function
set.seed(123)
advanced_test_data <- data.frame(
  id = 1:200,
  value = c(rnorm(100, 50, 10), rnorm(100, 55, 12)),
  group = rep(c("Control", "Treatment"), each = 100),
  stringsAsFactors = FALSE
)

# Add some missing values
advanced_test_data$value[sample(1:200, 10)] <- NA

# Run analysis
overall_analysis <- advanced_statistics(advanced_test_data, "value")
grouped_analysis <- advanced_statistics(advanced_test_data, "value", "group")

print("Overall Analysis:")

```

```
[1] "Overall Analysis:"
```

```
print(overall_analysis)
```

```

      group    n  mean median    sd    se ci_lower ci_upper skewness
lower.2.5% Overall 190 52.511 51.724 10.607 0.769   51.027   54.019    0.527
      kurtosis
lower.2.5%    0.783

```

```
print("\nGrouped Analysis:")
```

```
[1] "\nGrouped Analysis:"
```

```
print(grouped_analysis)
```

	group	n	mean	median	sd	se	ci_lower	ci_upper	skewness
lower.2.5%	Control	93	50.870	50.530	9.302	0.965	48.922	52.725	0.079
lower.2.5%1	Treatment	97	54.084	52.634	11.553	1.173	51.593	56.446	0.621
	kurtosis								
lower.2.5%									-0.199
lower.2.5%1									0.685

64 Summary and Best Practices

64.1 Key Takeaways

1. **Start Simple:** Begin with basic functions and gradually add complexity
2. **Validate Inputs:** Always check that inputs are what you expect
3. **Handle Errors Gracefully:** Use tryCatch and informative error messages
4. **Document Your Functions:** Use roxygen2 comments for clear documentation
5. **Test Your Functions:** Write unit tests to ensure reliability
6. **Follow Naming Conventions:** Use clear, descriptive function and variable names
7. **Keep Functions Focused:** Each function should do one thing well
8. **Consider Performance:** Use vectorization and efficient algorithms
9. **Plan for Reusability:** Write functions that can be easily used in different contexts
10. **Version Control:** Track changes to your functions over time

64.2 Function Writing Checklist

Before finalizing a function, ask yourself:

- ☐ Does the function have a clear, single purpose?
- ☐ Are the inputs validated appropriately?
- ☐ Are error messages helpful and informative?
- ☐ Is the function documented with examples?
- ☐ Have edge cases been considered and tested?
- ☐ Is the function efficient for its intended use?
- ☐ Can the function be easily tested?
- ☐ Is the code readable and well-commented?
- ☐ Does the function follow naming conventions?
- ☐ Are there any dependencies that should be explicit?

By following these principles and practices, you'll be able to create robust, reusable, and maintainable functions that will serve you well in your R programming journey.

65 R Package Development

A practical, end-to-end cookbook for building robust **R packages** with **devtools**, **roxygen2**, and **testthat**. Written for day-to-day use, from scaffolding to checks and tests.

65.1 1. Prerequisites

- R (4.1 recommended so you have the base pipe |>)
- RTools (Windows) / Xcode command line tools (macOS) for compilation
- Recommended packages:
 - `install.packages(c("devtools", "roxygen2", "usethis", "testthat", "pkgdown"))`

```
# Load helpers
library(devtools)
```

Loading required package: usethis

```
library(usethis)
```

65.2 2. Create the Package Skeleton

65.2.1 2.1 Create a new package directory

```
# Creates simutils with DESCRIPTION, NAMESPACE placeholder, R/ etc.
#usethis::create_package("simutils")
# Or from an existing project folder
# usethis::create_package(getwd())
```

65.2.2 2.2 Initialize Git and a README

```
usethis::use_git()
usethis::use_readme_rmd() # README.Rmd + badge placeholders
```

65.2.3 2.3 The minimal structure (what you get)

```
simutils/
DESCRIPTION
NAMESPACE
R/
man/
```

Tip: Keep all your R code inside R/. Avoid subfolders there. Group related functions in sensible files (e.g., `na.R`, `sample.R`).

65.3 3. DESCRIPTION: Package Metadata

Run helpers and then edit as needed.

```
usethis::use_description(fields = list(
  Title = "Simulation Utilities",
  Description = "Convenience functions for sampling and NA counting.",
  `Authors@R` = 'person(given = "Your", family = "Name", role = c("aut","cre"), email = "you@your.org")',
  License = "MIT + file LICENSE"
))

usethis::use_mit_license("Your Name")
```

Key fields you usually set:

- Package, Title, Version, Description
- Authors@R (with aut, cre roles)
- License (e.g., MIT)
- Depends, Imports, Suggests (use helpers below)

65.4 4. NAMESPACE: Exports and Imports (auto-generated)

You generally **do not** edit **NAMESPACE** by hand. It's generated from your roxygen tags in code.

- `@export` marks a function for users.
- `@import`, `@importFrom` declare your dependencies at function level.

You'll generate it with `devtools::document()`.

65.5 5. Write Functions + roxygen2 Headers

Create your first R file:

```
# File: R/na.R
#' Count NAs in a vector
#'
#' Count number of missing values (NA) in a numeric/logical/character vector.
#'
#' @param x A vector.
#' @return Integer count of NAs in `x`.
#' @examples
#' sum_na(c(1, NA, 3))
#' sum_na(airquality$Ozone)
#' @export
sum_na <- function(x) {
  sum(is.na(x))
}
```

Add a second exported function that works on data frames/matrices:

```
# File: R/na_counter.R
#' Column-wise NA counts
#'
#' Returns the number of NAs per column for a data frame or matrix.
#'
#' @param x A data frame or matrix.
#' @return A named integer vector of NA counts per column.
#' @examples
#' na_counter(airquality)
#' @export
```

```
na_counter <- function(x) {
  if (!is.data.frame(x) && !is.matrix(x)) stop("x must be a data.frame or matrix")
  colSums(is.na(x))
}
```

A simple sampler utility with a documented argument:

```
# File: R/sample_from_data.R
#' Sample rows from a data frame
#'
#' Draw a row-wise sample with optional replacement.
#'
#' @param data A data frame.
#' @param size Number of rows to sample.
#' @param replace Logical; sample with replacement?
#' @return A data frame containing the sampled rows.
#' @examples
#' sample_from_data(airquality, 10)
#' sample_from_data(airquality, 10, replace = TRUE)
#' @export
sample_from_data <- function(data, size, replace = FALSE) {
  stopifnot(is.data.frame(data))
  idx <- sample.int(nrow(data), size, replace = replace)
  data[idx, , drop = FALSE]
}
```

65.6 6. Generate Documentation

```
# Parses roxygen headers -> creates/updates NAMESPACE and man/*.Rd
devtools::document()
```

This will populate `man/` with help files and refresh `NAMESPACE`.

65.7 7. Add Data, Vignettes, Tests (Optional but Recommended)

65.7.1 7.1 Package data

```
# Create example data
sim_dat <- data.frame(
  ID = 1:10,
  Value = sample(1:11, 10),
  Apples = sample(c(TRUE, FALSE), 10, replace = TRUE)
)

usethis::use_data(sim_dat, overwrite = TRUE)
```

65.7.2 7.2 Vignettes

```
usethis::use_vignette("getting-started")
# Edit vignettes/getting-started.Rmd, then:
devtools::build_vignettes()
```

65.7.3 7.3 Tests with testthat

```
usethis::use_testthat()
# Creates tests/testthat.R and tests/testthat/ directory

usethis::use_test("na_counter")
# -> tests/testthat/test-na_counter.R (edit and add tests)
```

Example tests:

```
# File: tests/testthat/test-na_counter.R

library(testthat)

test_that("na_counter counts NAs correctly for data.frame and matrix", {
  test_matrix <- matrix(c(NA, 1, 4, NA, 5, 6), nrow = 2)
  air_expected <- c(Ozone = 37, Solar.R = 7, Wind = 0, Temp = 0, Month = 0, Day = 0)
  mat_expected <- c(V1 = 1, V2 = 1, V3 = 0)
```



```

expect_equal(na_counter(airquality), air_expected)
expect_equal(na_counter(test_matrix), mat_expected)
})

# Non-exported functions can be called with ::: if truly needed
# expect_equal(simutils:::sum_na(airquality$Ozone), 37)

```

Common expectations:

- `expect_identical()`, `expect_equal(tolerance = ...)`, `expect_equivalent()`
- `expect_error()`, `expect_warning()`
- `expect_output()` / `expect_output_file()` for console text

Run the test suite:

```
devtools::test()
```

65.8 8. Manage Dependencies

Use helpers to declare dependencies in DESCRIPTION (rather than editing by hand):

```

usethis::use_package("dplyr")           # add to Imports
usethis::use_package("ggplot2", type = "Suggests")

```

Guidelines:

- **Imports:** packages needed at runtime for your functions.
- **Suggests:** optional packages used in examples/tests/vignettes.
- **Depends:** rarely needed; if used, it attaches packages for users (discouraged).

65.9 9. Check Your Package Thoroughly

```

# Full check like CRAN does; read 00check.log for details
rc <- devtools::check()
rc

```

Typical findings & quick fixes:

- **Missing fields** (e.g., `License`) → add via `use_mit_license()` or set in `DESCRIPTION`.
- **Undocumented args** → ensure every parameter appears in `@param` and usage matches.
- **Broken examples** → run your examples; fix typos (e.g., `airquality`, not `airuality`).
- **Missing system tools** for building PDF manual/vignettes → install LaTeX if you need PDFs.
- **Failing tests** → run `devtools::test()`; update your code or expectations.

65.10 10. Build & Install Locally

```
# Build source tarball
devtools::build()

# Install the package from source
devtools::install()

# Try it out
#library(simutils)
na_counter(airquality)
```

65.11 11. Continuous Integration (optional)

You can wire up CI so checks run automatically on each commit. (Any provider works; pick what your org supports.)

```
# Example: set up a CI service config (adjust per provider)
# usethis::use_github_action_check_standard()
# or legacy: usethis::use_travis()
```

65.12 12. Package Website (optional)

```
usethis::use_pkgdown()
pkgdown::build_site()
```

65.13 13. Release Checklist

- ☐ R CMD check passes locally with **0 errors, 0 warnings, 0 notes**
 - ☐ All exported functions documented with examples
 - ☐ Vignettes render without errors
 - ☐ Tests pass on your platforms (and CI)
 - ☐ NEWS.md updated; version bumped in DESCRIPTION
 - ☐ License added and included
-

65.13.1 Appendix A — Common Files (at a glance)

- DESCRIPTION: metadata & dependencies
- NAMESPACE: auto-generated exports/imports
- R/: function source files
- man/: help files (.Rd) generated by roxygen2
- tests/: unit tests (testthat)
- vignettes/: long-form docs
- data/ & R/sysdata.rda: package data

65.13.2 Appendix B — Troubleshooting Snippets

```
# 1) Missing License in check
usethis::use_mit_license("Your Name")

# 2) Undocumented argument in check
# -> Make sure @param includes it and re-run document()

devtools::document(); devtools::check()

# 3) Examples fail in check (typo)
# -> Fix the example code, rerun check

# 4) pdflatex not found during check
# -> Install a LaTeX distribution or run check without manual building

# 5) Dependency not available
# -> Ensure it's in Imports/Suggests and installed in your environment
```

Turn this file into your team's template. Duplicate it, search/replace `simutils`, and you're ready to scaffold a new package in minutes.

66 Git in RStudio (Setup & Auth)

66.1 One-Time Setup

- Install Git and ensure `git --version` works.
- In R:

```
usethis::use_git_config(user.name = "Your Name", user.email = "you@example.com")
```

66.2 Initialize Git for the Current Project

```
usethis::use_git()
```

66.3 Connect to GitHub

- Create a GitHub account.
- In R:

```
usethis::create_github_token()
gitcreds::gitcreds_set() # paste token when prompted
usethis::use_github(protocol = "https")
```

Or set up SSH keys via RStudio (Tools > Global Options > Git/SVN).

66.4 Typical Workflow

1. Stage changes (Git pane in RStudio).
2. Commit with a clear message.
3. Push to origin (GitHub).

66.5 Remove Git from a Project (macOS/RStudio)

- In Finder/Terminal, delete the hidden `.git` folder in the project root (careful!).
- Or from Terminal at project root:

```
rm -rf .git
```

- Reopen project in RStudio; Git pane will disappear.

Exercises - Create a new repo for this Quarto course and push it. - Branch, make a change, open a Pull Request on GitHub.

67 Creating ADaM: ADSL from SDTM-like Inputs

```
####----
# Program Name : adsl
# Protocol Name:
# Author      :
# Creation Date: `<DDMMYYYY>` by <initials>
# Purpose      : Create ADSL dataset from SDTM datasets
# Modification : <DDMMYYYY> by <initials>: <summary of changes>
####----

library(tidyverse)
library(lubridate)
library(xportr)
library(haven)
library(stringr)

##-----
## 0. Helper functions
##-----

# Function to group together missing strings and NAs
str_missing <- function(x) {
  is.na(x) | str_trim(as.character(x)) == ""
}

# Function to check dataframe metadata
contents <- function(dat) {

  row_contents <- function(nm, dat) {
    var <- dat[[nm]]
    data.frame(
      Variable = nm,
      Class    = class(var)[1],

```

```

    Label    = ifelse(is.null(attr(var, "label")), "", attr(var, "label")),
    Format    = ifelse(is.null(attr(var, "SASformat")), "", attr(var, "SASformat")),
    stringsAsFactors = FALSE
  )
}

purrr::map_dfr(names(dat), row_contents, dat = dat)
}

##-----
## 1. Load SDTM from PHUSE Test Data Factory
##-----

sdtm <- "D:/R2SAS/r4sas/data/sdtm"

dm <- read_sas(paste0(sdtm, "/dm.sas7bdat"))
ex <- read_sas(paste0(sdtm, "/ex.sas7bdat"))
vs <- read_sas(paste0(sdtm, "/vs.sas7bdat"))
ds <- read_sas(paste0(sdtm, "/ds.sas7bdat"))

##-----
## 2. Derive DM-based variables (demo)
##-----

demo <- dm |>
  # Keep only necessary columns
  select(
    STUDYID, USUBJID, SUBJID, SITEID,
    AGE, AGEU, SEX, RACE, ETHNIC,
    DTHDTC, ARM, ACTARM
  ) |>
  mutate(
    # Age groups
    AGEGR1 = case_when(
      AGE < 65 ~ "<65",
      AGE >= 65 & AGE <= 70 ~ "65-70",
      AGE > 70 ~ ">70",
      TRUE ~ NA_character_
    ),
    AGEGR1N = case_when(
      AGEGR1 == "<65" ~ 1,
      AGEGR1 == "65-70" ~ 2,

```



```

    AGEGR1 == ">70" ~ 3,
    TRUE ~ NA_real_
  ),

  # Race numeric
  RACEN = case_when(
    RACE == "WHITE" ~ 1,
    RACE == "BLACK OR AFRICAN AMERICAN" ~ 2,
    RACE == "ASIAN" ~ 3,
    RACE == "AMERICAN INDIAN OR ALASKA NATIVE" ~ 4,
    RACE == "NATIVE HAWAIIAN OR OTHER PACIFIC ISLANDER" ~ 5,
    RACE == "UNKNOWN" ~ 6,
    TRUE ~ NA_real_
  ),

  # Planned treatment
  TRT01P = ARM,
  TRT01PN = case_when(
    TRT01P == "Placebo" ~ 0,
    TRT01P == "Xanomeline Low Dose" ~ 6,
    TRT01P == "Xanomeline High Dose" ~ 9,
    TRUE ~ NA_real_
  ),

  # Actual treatment
  TRT01A = ACTARM,
  TRT01AN = case_when(
    TRT01A == "Placebo" ~ 0,
    TRT01A == "Xanomeline Low Dose" ~ 6,
    TRT01A == "Xanomeline High Dose" ~ 9,
    TRUE ~ NA_real_
  ),

  # ITT flag: Y if ARM not missing
  ITTFL = if_else(!str_missing(ARM), "Y", "N"),

  # Death date
  DTHDT = as_date(DTHDTC)
) |>
select(-DTHDTC)

```

##-----

```

## 3. EX-based variables: treatment dates, duration, dose
##-----

# Treatment dates
treat <- ex |>
  select(USUBJID, EXSTDTC, EXENDTC) |>
  mutate(
    across(ends_with("DTC"), as_date)
  ) |>
  group_by(USUBJID) |>
  summarise(
    TRTSDT = suppressWarnings(min(EXSTDTC, na.rm = TRUE)),
    TRTEDT = suppressWarnings(max(EXENDTC, na.rm = TRUE)),
    .groups = "drop"
  ) |>
  mutate(
    # Handle Inf from min/max when all values are NA
    TRTSDT = replace(TRTSDT, is.infinite(TRTSDT), NA_Date_),
    TRTEDT = replace(TRTEDT, is.infinite(TRTEDT), NA_Date_),
    # Treatment duration (days)
    TRTDURD = as.numeric(TRTEDT - TRTSDT + (TRTEDT >= TRTSDT))
  )

# Dosing information
dose <- ex |>
  group_by(USUBJID) |>
  summarise(
    DOSE01A = sum(EXDOSE, na.rm = TRUE),
    .groups = "drop"
  ) |>
  mutate(
    DOSE01U = "mg"
  ) |>
  select(USUBJID, DOSE01A, DOSE01U)

# Merge DM- and EX-based info
demo1 <- reduce(
  list(demo, treat, dose),
  .f = left_join,
  by = "USUBJID"
) |>
  mutate(

```

```

# Safety flag: Y if ITTFL = Y and TRTSDT not missing
SAFFL = if_else(ITTFL == "Y" & !str_missing(TRTSDT), "Y", "N")
)

##-----
## 4. DS-based variables: EOS, discontinuation, screen failure
##-----

demo2 <- demo1 |>
  left_join(
    ds |>
      filter(DSCAT == "DISPOSITION EVENT") |>
      select(USUBJID, DSCAT, DSDECOD, DSTERM),
    by = "USUBJID"
  ) |>
  mutate(
    # End of study status
    EOSSTT = case_when(
      str_missing(DSDECOD) ~ "ONGOING",
      DSDECOD == "COMPLETED" ~ "COMPLETED",
      TRUE ~ "DISCONTINUED"
    ),
    # Discontinuation reason and text
    DCSREAS = if_else(
      DSDECOD != "COMPLETED" & !str_missing(DSDECOD),
      DSDECOD,
      "",
      missing = ""
    ),
    DCSREASP = if_else(
      DSDECOD != "COMPLETED" & !str_missing(DSTERM),
      DSTERM,
      "",
      missing = ""
    ),
    # Screen failure flag
    SCRNFL = if_else(
      DSDECOD == "SCREEN FAILURE",
      "Y",
      "N",
      missing = "N"
    )
  )

```

```

) |>
select(-c(DSCAT, DSDECOD, DSTERM))

##-----
## 5. VS-based variables: weight, height, BMI
##-----

# Baseline weight (visit 3)
weight <- vs |>
  filter(VSTESTCD == "WEIGHT", VISITNUM == 3) |>
  select(USUBJID, WEIGHTBL = VSSTRESN)

# Baseline height (visit 1)
height <- vs |>
  filter(VSTESTCD == "HEIGHT", VISITNUM == 1) |>
  select(USUBJID, HEIGHTBL = VSSTRESN)

demo3 <- reduce(
  list(demo2, weight, height),
  .f = left_join,
  by = "USUBJID"
) |>
  mutate(
    BMIBL = (WEIGHTBL / HEIGHTBL / HEIGHTBL) * 10000,
    BMIGR1 = case_when(
      BMIBL < 25 ~ "<25",
      BMIBL >= 25 ~ ">=25",
      TRUE ~ NA_character_
    )
  )

##-----
## 6. Metadata (dataset + variable level) and final ADSL
##-----

# Dataset-level metadata
dlm <- tribble(
  ~dataset, ~label,
  "adsl", "Subject-Level Analysis Dataset"
)

# Variable-level metadata

```

```

vlm <- tribble(
  ~dataset, ~variable, ~label, ~type, ~format,
  "adsl", "STUDYID", "Study Identifier", "character", NA_character_,
  "adsl", "USUBJID", "Unique Subject Identifier", "character", NA_character_,
  "adsl", "SUBJID", "Subject Identifier for the Study", "character", NA_character_,
  "adsl", "SITEID", "Study Site Identifier", "character", NA_character_,
  "adsl", "AGE", "Age", "numeric", NA_character_,
  "adsl", "AGEU", "Age Units", "character", NA_character_,
  "adsl", "AGEGR1", "Pooled Age Group 1", "character", NA_character_,
  "adsl", "AGEGR1N", "Pooled Age Group 1 (N)", "numeric", NA_character_,
  "adsl", "SEX", "Sex", "character", NA_character_,
  "adsl", "RACE", "Race", "character", NA_character_,
  "adsl", "RACEN", "Race (N)", "numeric", NA_character_,
  "adsl", "ETHNIC", "Ethnicity", "character", NA_character_,
  "adsl", "SAFFL", "Safety Population Flag", "character", NA_character_,
  "adsl", "ITTFL", "Intent-To-Treat Population Flag", "character", NA_character_,
  "adsl", "SCRNFL", "Screen Failure Population Flag", "character", NA_character_,
  "adsl", "ARM", "Description of Planned Arm", "character", NA_character_,
  "adsl", "ACTARM", "Description of Actual Arm", "character", NA_character_,
  "adsl", "TRT01P", "Planned Treatment for Period 01", "character", NA_character_,
  "adsl", "TRT01PN", "Planned Treatment for Period 01 (N)", "numeric", NA_character_,
  "adsl", "TRT01A", "Actual Treatment for Period 01", "character", NA_character_,
  "adsl", "TRT01AN", "Actual Treatment for Period 01 (N)", "numeric", NA_character_,
  "adsl", "DOSE01A", "Actual Treatment Dose for Period 01", "numeric", NA_character_,
  "adsl", "DOSE01U", "Units for Dose for Period 01", "character", NA_character_,
  "adsl", "TRTSDT", "Date of First Exposure to Treatment", "Date", "DATE9.",
  "adsl", "TRTEDT", "Date of Last Exposure to Treatment", "Date", "DATE9.",
  "adsl", "TRTDURD", "Total Treatment Duration (Days)", "numeric", NA_character_,
  "adsl", "EOSSTT", "End of Study Status", "character", NA_character_,
  "adsl", "DCSREAS", "Reason for Discontinuation from Study", "character", NA_character_,
  "adsl", "DCSREASP", "Reason Spec for Discont from Study", "character", NA_character_,
  "adsl", "DTHDT", "Date of Death", "Date", "DATE9.",
  "adsl", "WEIGHTBL", "Weight (kg) at Baseline", "numeric", NA_character_,
  "adsl", "HEIGHTBL", "Height (cm) at Baseline", "numeric", NA_character_,
  "adsl", "BMIBL", "Body Mass Index (kg/m2) at Baseline", "numeric", NA_character_,
  "adsl", "BMIGR1", "Pooled BMI Group 1", "character", NA_character_
)

adsl <- demo3 |>
  # Order variables
  select(
    STUDYID, USUBJID, SUBJID, SITEID,

```

```

    AGE, AGEU, AGEGR1, AGEGR1N, SEX, RACE, RACEN, ETHNIC,
    SAFFL, ITTFL, SCRNFL,
    ARM, ACTARM,
    TRT01P, TRT01PN, TRT01A, TRT01AN,
    DOSE01A, DOSE01U,
    TRTSDT, TRTEDT, TRTDURD,
    EOSSTT, DCSREAS, DCSREASP,
    DTHDT,
    WEIGHTBL, HEIGHTBL, BMIBL, BMIGR1
  ) |>
  arrange(USUBJID) |>
  # Apply labels / types / formats
  xportr_df_label(dlm, domain = "adsl") |>
  xportr_label(vlm, domain = "adsl") |>
  xportr_type(vlm, domain = "adsl") |>
  xportr_format(vlm, domain = "adsl") |>
  # Replace character NA with ""
  mutate(across(where(is.character), ~ replace(., is.na(.), "")))

##-----
## 7. QC: label + metadata + export
##-----

# Check dataframe label
attr(adsl, "label")

# Check metadata
adsl |>
  contents()

# Output to specified path
xportr_write(adsl, "./adsl.xpt")

```

Note: Real ADSL creation must follow **ADaM IG** (derive flags, dates, imputations, populations). This example is educational only.

68 ADVS program

```
# ADVS Dataset Creation Program
# Analysis Dataset for Vital Signs
# Following CDISC ADaM Standards

# Load required libraries
library(dplyr)
library(lubridate)
library(haven)

# Function to create ADVS dataset
create_advs <- function(vs_data, adsl_data) {

  # Step 1: Start with VS domain and merge with ADSL
  advs <- vs_data %>%
    # Filter for vital signs records
    filter(! is.na(vsstresn) | !is.na(vsstresc)) %>%
    # Merge with ADSL to get treatment information
    left_join(adsl_data, by = c("studyid" = "STUDYID", "usubjid" = "USUBJID"))

  # Step 2: Create basic variables
  advs <- advs %>%
    mutate(
      # Study and subject identifiers
      STUDYID = studyid,
      USUBJID = usubjid,

      # Treatment variables from ADSL
      TRTP = TRT01P,
      TRTPN = TRT01PN,
      TRTA = TRT01A,
      TRTAN = TRT01AN,

      # Parameter information
      PARAM = case_when(
```

```

    vstestcd == "SYSBP" ~ "Systolic Blood Pressure (mmHg)",
    vstestcd == "DIABP" ~ "Diastolic Blood Pressure (mmHg)",
    vstestcd == "PULSE" ~ "Pulse Rate (beats/min)",
    vstestcd == "RESP" ~ "Respiratory Rate (breaths/min)",
    vstestcd == "TEMP" ~ "Body Temperature (C)",
    vstestcd == "WEIGHT" ~ "Weight (kg)",
    vstestcd == "HEIGHT" ~ "Height (cm)",
    TRUE ~ vstest
  ),
  PARAMCD = vstestcd,
  PARAMN = case_when(
    vstestcd == "SYSBP" ~ 1,
    vstestcd == "DIABP" ~ 2,
    vstestcd == "PULSE" ~ 3,
    vstestcd == "RESP" ~ 4,
    vstestcd == "TEMP" ~ 5,
    vstestcd == "WEIGHT" ~ 6,
    vstestcd == "HEIGHT" ~ 7,
    TRUE ~ as.numeric(NA)
  ),

  # Analysis value
  AVAL = as.numeric(vsstresn),

  # Date/time variables
  ADT = as.Date(substr(vsdtc, 1, 10)),
  ADY = as.numeric(ADT - as.Date(TRTSDT) + (ADT >= as.Date(TRTSDT))),

  # Visit information
  VISIT = visit,
  ATPT = vstpt,
  ATPTN = vstptnum
) %>%
# Filter to keep only records with non-missing AVAL
filter(! is.na(AVAL))

# Step 3: Create analysis visit windows (this would typically come from a separate mapping)
# For demonstration, creating basic visit windows
advs <- advs %>%
  mutate(
    AVISIT = case_when(
      grepl("SCREENING|SCREEN", toupper(VISIT)) ~ "Screening",

```



```

    grepl("BASELINE|DAY 1", toupper(VISIT)) ~ "Baseline",
    grepl("WEEK 2|DAY 14", toupper(VISIT)) ~ "Week 2",
    grepl("WEEK 4|DAY 28", toupper(VISIT)) ~ "Week 4",
    grepl("WEEK 8|DAY 56", toupper(VISIT)) ~ "Week 8",
    grepl("WEEK 12|DAY 84", toupper(VISIT)) ~ "Week 12",
    grepl("FOLLOW", toupper(VISIT)) ~ "Follow-up",
    TRUE ~ VISIT
  ),
  AVISITN = case_when(
    AVISIT == "Screening" ~ -1,
    AVISIT == "Baseline" ~ 0,
    AVISIT == "Week 2" ~ 2,
    AVISIT == "Week 4" ~ 4,
    AVISIT == "Week 8" ~ 8,
    AVISIT == "Week 12" ~ 12,
    AVISIT == "Follow-up" ~ 99,
    TRUE ~ as.numeric(NA)
  ),

  # Analysis window variables (example values)
  AWTARGET = AVISITN * 7, # Target day
  AWU = "DAYS",
  AWLO = AWTARGET - 3, # 3-day window
  AWHI = AWTARGET + 3,
  AWTDIFF = abs(ADY - AWTARGET)
)

# Step 4: Determine baseline records
advs <- advs %>%
  group_by(USUBJID, PARAMCD) %>%
  mutate(
    # Baseline flag - last assessment on or before treatment start date
    baseline_candidate = case_when(
      ADT <= as.Date(TRTSDT) ~ ADT,
      TRUE ~ as.Date(NA)
    )
  ) %>%
  group_by(USUBJID, PARAMCD) %>%
  mutate(
    max_baseline_date = max(baseline_candidate, na.rm = TRUE),
    ABLFL = case_when(
      ! is.na(baseline_candidate) & baseline_candidate == max_baseline_date ~ "Y",

```

```

      TRUE ~ ""
    )
  ) %>%
  ungroup()

# Step 5: Create BASE, CHG, and PCHG
advs <- advs %>%
  group_by(USUBJID, PARAMCD) %>%
  mutate(
    BASE = case_when(
      any(ABLFL == "Y") ~ AVAL[ABLFL == "Y"][1],
      TRUE ~ as.numeric(NA)
    ),
    CHG = case_when(
      ! is.na(BASE) & ADT > as.Date(TRTSDT) ~ AVAL - BASE,
      TRUE ~ as.numeric(NA)
    ),
    PCHG = case_when(
      !is.na(BASE) & BASE != 0 & ADT > as.Date(TRTSDT) ~ ((AVAL - BASE) / BASE) * 100,
      TRUE ~ as.numeric(NA)
    )
  ) %>%
  ungroup()

# Step 6: Create analysis flags
advs <- advs %>%
  mutate(
    # ANL01FL - Analysis Flag 01 (include all valid records)
    ANL01FL = "Y",

    # DTYPE - Derivation Type (empty for observed values)
    DTYPE = ""
  ) %>%
  group_by(USUBJID, PARAMCD, AVISIT) %>%
  mutate(
    # ANL02FL - One record per visit per parameter
    visit_rank = rank(AWTDIFF, ties.method = "first"),
    ANL02FL = case_when(
      ANL01FL == "Y" & visit_rank == 1 ~ "Y",
      TRUE ~ ""
    )
  ) %>%

```

```
ungroup()
```

```
# Step 7: Create criterion variables for blood pressure parameters
```

```
advs <- advs %>%
```

```
mutate(
```

```
  # Criterion 1: Low BP
```

```
  CRIT1 = case_when(
```

```
    PARAMCD == "SYSBP" ~ "SYSBP <= 90 mmHg",
```

```
    PARAMCD == "DIABP" ~ "DIABP <= 60 mmHg",
```

```
    TRUE ~ ""
```

```
  ),
```

```
  CRIT1FL = case_when(
```

```
    PARAMCD == "SYSBP" & AVAL <= 90 ~ "Y",
```

```
    PARAMCD == "DIABP" & AVAL <= 60 ~ "Y",
```

```
    PARAMCD %in% c("SYSBP", "DIABP") ~ "",
```

```
    TRUE ~ ""
```

```
  ),
```

```
  # Criterion 2: High BP
```

```
  CRIT2 = case_when(
```

```
    PARAMCD == "SYSBP" ~ "SYSBP >= 140 mmHg",
```

```
    PARAMCD == "DIABP" ~ "DIABP >= 90 mmHg",
```

```
    TRUE ~ ""
```

```
  ),
```

```
  CRIT2FL = case_when(
```

```
    PARAMCD == "SYSBP" & AVAL >= 140 ~ "Y",
```

```
    PARAMCD == "DIABP" & AVAL >= 90 ~ "Y",
```

```
    PARAMCD %in% c("SYSBP", "DIABP") ~ "",
```

```
    TRUE ~ ""
```

```
  ),
```

```
  # Criterion 3: >20% increase from baseline
```

```
  CRIT3 = case_when(
```

```
    PARAMCD == "SYSBP" ~ "SYSBP change from baseline > 20% increase",
```

```
    PARAMCD == "DIABP" ~ "DIABP change from baseline > 20% increase",
```

```
    TRUE ~ ""
```

```
  ),
```

```
  CRIT3FL = case_when(
```

```
    PARAMCD == "SYSBP" & ! is.na(PCHG) & PCHG > 20 ~ "Y",
```

```
    PARAMCD == "DIABP" & ! is.na(PCHG) & PCHG > 20 ~ "Y",
```

```
    PARAMCD %in% c("SYSBP", "DIABP") & ! is.na(PCHG) ~ "",
```

```
    TRUE ~ ""
```

```

    ),

    # Criterion 4: >20% decrease from baseline
    CRIT4 = case_when(
      PARAMCD == "SYSBP" ~ "SYSBP change from baseline > 20% decrease",
      PARAMCD == "DIABP" ~ "DIABP change from baseline > 20% decrease",
      TRUE ~ ""
    ),
    CRIT4FL = case_when(
      PARAMCD == "SYSBP" & !is.na(PCHG) & PCHG < -20 ~ "Y",
      PARAMCD == "DIABP" & !is.na(PCHG) & PCHG < -20 ~ "Y",
      PARAMCD %in% c("SYSBP", "DIABP") & !is.na(PCHG) ~ "",
      TRUE ~ ""
    )
  )
)

# Step 8: Select final variables in correct order
advs_final <- advs %>%
  select(
    STUDYID, USUBJID, TRTP, TRTPN, TRTA, TRTAN,
    PARAM, PARAMCD, PARAMN, AVAL, BASE, CHG, PCHG,
    DTYPE, ADT, ADY, AVISIT, AVISITN, AWTARGET, AWTDIFF,
    AWLO, AWHI, AWU, VISIT, ATPT, ATPTN,
    ABLFL, ANLO1FL, ANLO2FL,
    CRIT1, CRIT1FL, CRIT2, CRIT2FL, CRIT3, CRIT3FL, CRIT4, CRIT4FL,
    # Include core ADSL variables
    AGE, AGEU, AGEGR1, AGEGR1N, SEX, RACE, RACEN, SAFFL, ITTFL
  ) %>%
  arrange(STUDYID, USUBJID, PARAMN, ADT, ATPTN)

  return(advs_final)
}

# Example usage:
# Load your data
# vs <- read_sas("path/to/vs.sas7bdat")
# adsl <- read_sas("path/to/adsl.sas7bdat")

# Create ADVS dataset
# advs <- create_advs(vs, adsl)

# Write to file

```

```
# write_sas(advs, "path/to/advs.sas7bdat")

# Print summary
# cat("ADVS dataset created with", nrow(advs), "records\n")
# cat("Parameters included:", paste(unique(advs$PARAMCD), collapse = ", "), "\n")
# cat("Subjects included:", length(unique(advs$USUBJID)), "\n")
```

Exercises 1. Add analysis populations (e.g., SAFFL, FASFL) based on simple rules. 2. Derive AGEGR1 as <65 / 65 and use ordered factor. 3. Add a treatment end date TRTEDT and compute treatment duration.

69 TLFs: Table, Figure, Listing

69.1 Create demographic table from ADSL

```
library(r2rtf)
library(dplyr)
library(tidyr)
library(haven)

adsl <- haven::read_sas("data/adam/adsl.sas7bdat")

bs_count <- function(data, grp, var,
                      var_label = var,
                      decimal = 1,
                      total = TRUE,
                      blank_row = FALSE) {
  data <- data |>
    dplyr::rename(grp = !!grp, var = !!var)

  coding <- levels(factor(data$grp))

  data <- data |>
    dplyr::mutate(grp = as.numeric(factor(grp)))

  res <- with(data, table(var, grp)) |>
    as.data.frame() |>
    dplyr::mutate(grp = as.numeric(grp))

  if (total) {
    res_tot <- with(data, table(var)) |>
      as.data.frame() |>
      dplyr::mutate(grp = 9999)
    res <- dplyr::bind_rows(res, res_tot)
  }
}
```

```

res <- res |>
  rename(n = Freq)

res <- res |>
  mutate(pct = formatC(n / sum(n) * 100, digits = decimal, format = "f", flag = "0"),
    pct1= paste0(n, "(", pct, "%)")

res$n <- as.character(res$n)
res <- res |>
  dplyr::select(-n,-pct)
res <- res |>
  pivot_longer(cols = c(pct1), names_to = "key", values_to = "value") |>
  unite(keys, grp, key) |>
  pivot_wider(names_from = keys, values_from = value) |>
  mutate(var_label = var_label) |>
  mutate(var = as.character(var))

names(res) <- gsub("_n", "", names(res), fixed = TRUE)
attr(res, "coding") <- coding
if (blank_row) {
  res <- bind_rows(tibble(var_label = var_label), res)
}
res
}

bs_continuous <- function(data, grp, var,
                          var_label = var,
                          decimal = 1,
                          total = TRUE,
                          blank_row = FALSE) {

  data <- data |>
    rename(grp = !!grp, var = !!var)
  coding <- levels(factor(data$grp))
  data <- data |> mutate(grp = as.numeric(factor(grp)))

  res <- data |>
    select(grp, var) |>
    na.omit() |>
    group_by(grp) |>
    summarise(
      n= n(),
      Mean = formatC(mean(var), digits = decimal, format = "f", flag = "0"),

```

```

    SD = formatC(sd(var), digits = decimal, format = "f", flag = "0"),
    Median = formatC(median(var), digits = decimal, format = "f", flag = "0"),
    Range = paste(range(var), collapse = " to ")
  )

  if (total) {
    res_tot <- data |>
      select(grp, var) |>
      na.omit() |>
      summarise(
        n = n(),
        Mean = formatC(mean(var), digits = decimal, format = "f", flag = "0"),
        SD = formatC(sd(var), digits = decimal, format = "f", flag = "0"),
        Median = formatC(median(var), digits = decimal, format = "f", flag = "0"),
        Range = paste(range(var), collapse = " to ")
      ) |>
      mutate(grp = 9999)
    res <- bind_rows(res, res_tot)
  }

  res$"n" <- as.character(res$"n")

  res <- res |>
    pivot_longer(cols = -grp, names_to = "key", values_to = "value") |>
    mutate(key = factor(key, levels = c("n", "Mean", "SD", "Median", "Range"))) |>
    pivot_wider(names_from = grp, values_from = value) %>%
    mutate(var_label = var_label) |>
    mutate(key = as.character(key)) |>
    rename(var = key)

  if (blank_row) {
    res <- bind_rows(tibble(var_label = var_label), res)
  }

  res
}

# The code above define two utility function for baseline characteristic tables.

# Analysis Set

```



```

ana <- adsl |>
  subset(ITTFL == "Y")

ana <- ana |>
  mutate(
    RACE = factor(
      RACE,
      c("WHITE", "BLACK OR AFRICAN AMERICAN", "AMERICAN INDIAN OR ALASKA NATIVE"),
      c("White", "Black", "Other")
    ),
    SEX = factor(
      SEX,
      c("F", "M"),
      c("Female", "Male")
    ),
    AGEGR1 = factor(
      AGEGR1,
      levels = c("<65", "65-80", ">80")
    )
  )

age_1 <- bs_continuous(ana, "TRT01AN", "AGE", "Age (Years)", blank_row = TRUE)
names(age_1) <- c("var_label", "var", "1_pct1", "2_pct1", "3_pct1", "9999_pct1")
# Build Data for r2rtf
bs_tb <- bind_rows(
  bs_count(ana, "TRT01AN", "SEX", "Gender", blank_row = TRUE),
  bs_count(ana, "TRT01AN", "AGEGR1", "Age (Years)", blank_row = TRUE),
  age_1,
  bs_count(ana, "TRT01AN", "RACE", "Race", blank_row = TRUE)
) |>
  mutate(
    var_label = factor(var_label, levels = c("Gender", "Age (Years)", "Race"))
  )
bs_tb <- bs_tb |>
  dplyr::mutate(lab1 = if_else(is.na(var), var_label, paste0(" ", var))) |>
  dplyr::select(lab1, `1_pct1`, `2_pct1`, `3_pct1`, `9999_pct1`)

bs_tb[is.na(bs_tb)] <- "" # convert NA to blank string.

# reporting
bs_rtf <- bs_tb |>

```

```

rtf_page(width = 9.5,
         border_first = "single",
         border_last = "single") |>
rtf_title("Demographic and Anthropometric Characteristics", "ITT Subjects") |>
rtf_colheader(" | Placebo | Drug Low Dose | Drug High Dose | Total ",
              col_rel_width = c(3, rep(2, 4)),
              border_top = rep("",5),
              border_right = rep("",5),
              border_left =rep("",5)
) |>
rtf_colheader(" | n (%) | n (%) | n (%) | n (%) ",
              col_rel_width =c(3, rep(2, 4)),
              border_top = rep("",5),
              border_right = rep("",5),
              border_left =rep("",5)
) |>
rtf_body(
  #page_by = "var_label",
  col_rel_width = c(3, rep(2, 4)),
  text_justification = c("l", rep("c", 4)),
  #text_format = c(rep("", 5), "b"),
  border_top = rep("",5),
  border_right = rep("",5),
  border_left =rep("",5)
) |>
rtf_footnote("This is a footnote",
             border_left = "",
             border_right = "",
             border_bottom = "")

# Output .rtf file
bs_rtf %>%
  rtf_encode() %>%
  write_rtf("bs_example.rtf")

```

Demographic and Anthropometric Characteristics
ITT Subjects

	Placebo n (%)	Drug Low Dose n (%)	Drug High Dose n (%)	Total n (%)
Gender				
Female	53(10.4%)	50(9.8%)	40(7.9%)	143(28.1%)
Male	33(6.5%)	34(6.7%)	44(8.7%)	111(21.9%)
Age (Years)				
<65	14(2.8%)	8(1.6%)	11(2.2%)	33(6.5%)
65-80	42(8.3%)	47(9.3%)	55(10.8%)	144(28.3%)
>80	30(5.9%)	29(5.7%)	18(3.5%)	77(15.2%)
Age (Years)				
n	86	84	84	254
Mean	75.2	75.7	74.4	75.1
SD	8.6	8.3	7.9	8.2
Median	76.0	77.5	76.0	77.0
Range	52 to 89	51 to 88	56 to 88	51 to 89
Race				
White	78(15.4%)	78(15.4%)	74(14.6%)	230(45.3%)
Black	8(1.6%)	6(1.2%)	9(1.8%)	23(4.5%)
Other	0(0.0%)	0(0.0%)	1(0.2%)	1(0.2%)
This is a footnote				

69.2 Create adverse event table from ADAE

```
library(r2rtf)
library(dplyr)
library(tidyr)

####Read SAS Data####
#keep the variable
adsl <- haven::read_sas("./data/adam/adsl.sas7bdat")
adae <- haven::read_sas("./data/adam/adae.sas7bdat")

adae <- adae |>
  dplyr::mutate(AEDECOD=stringr::str_to_title(AEDECOD))

ae_t1 <- adae %>%
  group_by(TRTA) %>%
  mutate(n_subj = n_distinct(USUBJID)) %>%
  group_by(TRTA, AEDECOD) %>%
  summarise(
    n_ae = n_distinct(USUBJID),
    pct = round(n_ae / unique(n_subj) * 100, 2)
  ) %>%
  ungroup() %>%
  dplyr::mutate(pct = paste0(n_ae,"(",pct, "%)")) %>%
  dplyr::filter(n_ae > 5) %>%
  # only show AE terms with at least 5 subjects in one treatment group.
  pivot_longer(cols = c( pct), names_to = "var", values_to = "value") %>%
  unite(temp, TRTA, var) %>%
  pivot_wider(names_from = temp, values_from = value, values_fill = "0(0.00%)") |>
  select(-n_ae)

ae_tbl <- ae_t1 %>%
  rtf_page(orientation = "landscape",
    border_first = "single",
    border_last = "single") %>%
  rtf_title(
    "Analysis of Subjects With Specific Adverse Events",
    c(
      "(Incidence > 5 Subjects in One or More Treatment Groups)",
```

```

      "ASaT"
    )
  ) %>%
  rtf_colheader(" | Placebo | Drug High Dose | Drug Low Dose",
    col_rel_width = c(4, rep(2, 3)),
    border_top = rep("",4),
    border_right = rep("",4),
    border_left =rep("",4)
  ) %>%
  rtf_colheader(" | n (%) | n (%) | n (%)",
    col_rel_width = c(4, rep(2, 3)),
    border_top = rep("",4),
    border_right = rep("",4),
    border_left =rep("",4)

  ) %>%
  rtf_body(
    col_rel_width = c(4, rep(2, 3)),
    text_justification = c("l", rep("c", 3)),
    border_top = rep("",4),
    border_right = rep("",4),
    border_left =rep("",4)
  ) %>%
  rtf_footnote(c("{^\\dagger}This is footnote 1", "This is footnote 2"),
    border_left = "",
    border_right = "",
    border_bottom = "")

# Output .rtf file
ae_tbl %>%
  rtf_encode(page_title = "all",
    page_footnote = "all") %>%
  write_rtf("ae_example.rtf")

```

Analysis of Subjects With Specific Adverse Events
(Incidence > 5 Subjects in One or More Treatment Groups)
ASaT

	Placebo n (%)	Drug High Dose n (%)	Drug Low Dose n (%)
Application Site Pruritus	6(8.7%)	0(0.00%)	0(0.00%)
Diarrhoea	9(13.04%)	0(0.00%)	0(0.00%)
Erythema	9(13.04%)	0(0.00%)	0(0.00%)
Headache	7(10.14%)	0(0.00%)	0(0.00%)
Pruritus	8(11.59%)	0(0.00%)	0(0.00%)
Upper Respiratory Tract Infection	6(8.7%)	0(0.00%)	0(0.00%)
Application Site Dermatitis	0(0.00%)	7(8.86%)	0(0.00%)
Application Site Erythema	0(0.00%)	15(18.99%)	0(0.00%)
Application Site Irritation	0(0.00%)	9(11.39%)	9(11.69%)
Application Site Pruritus	0(0.00%)	22(27.85%)	22(28.57%)
Application Site Vesicles	0(0.00%)	6(7.59%)	0(0.00%)
Dizziness	0(0.00%)	12(15.19%)	0(0.00%)
Erythema	0(0.00%)	14(17.72%)	0(0.00%)
Headache	0(0.00%)	6(7.59%)	0(0.00%)
Hyperhidrosis	0(0.00%)	8(10.13%)	0(0.00%)
Nasopharyngitis	0(0.00%)	6(7.59%)	0(0.00%)
Nausea	0(0.00%)	6(7.59%)	0(0.00%)

*This is footnote 1
This is footnote 2

Exercises 1. Format Table 1 to N ($\text{mean} \pm \text{SD}$) for age. 2. Add risk table to the KM plot (use an extension like survminer outside of this minimal example). 3. Create a listing that includes population flags once you derive them.

70 ggplot2 in R with CDISC ADaM Examples

71 Introduction

This tutorial is a **step-by-step guide to ggplot2** in R, designed to take you from **beginner to advanced**, using **CDISC ADaM-style datasets** throughout.

- ADSL: Subject-level data
- ADLB: Laboratory data
- ADVS: Vital signs (optional example)
- ADTTE: Time-to-event data (e.g. PFS)

These are **simulated examples** with ADaM-like structures and variable names. In a real project you would replace them with your actual ADSL/ADLB/ADTTE datasets.

We will cover:

- Basic **Grammar of Graphics** concepts
 - Building plots layer by layer with `ggplot()`
 - Common geoms: points, lines, bars, histograms, boxplots
 - Aesthetics: color, shape, size, linetype
 - Faceting, scales, themes, coordinates
 - Combining `dplyr` + `ggplot2` in a tidy ADaM workflow
 - Advanced: annotations, secondary axes, simple KM plot from ADTTE
 - Saving plots for reports/presentations
-

72 Creating Simulated ADaM-Style Data

In this section we create small ADaM-like datasets entirely in R so the tutorial is self-contained.

72.1 ADSL: Subject-Level Data

```
ADSL <- haven::read_sas("./data/adam/adsl.sas7bdat")  
  
ADSL <- ADSL |>  
  dplyr::mutate(TRTA=ARM)
```

72.2 ADLB: Laboratory Data (e.g. ALT)

We simulate longitudinal ALT values (PARAMCD = "ALT") at multiple visits:

```
ADLB <- haven::read_sas("./data/adam/adlbhy.sas7bdat") |>  
  dplyr::filter(PARAMCD == "ALT")
```

72.3 ADTTE: Time-to-Event Data (e.g. PFS)

We simulate a simple ADTTE:

- PARAMCD = "PFS"
- AVAL: time (months)
- CNSR: censor flag (0 = event, 1 = censored)

```

ADTTE <- ADSL %>%
  select(STUDYID, USUBJID, ARM, TRTA, SEX, AGE) %>%
  mutate(
    PARAMCD = "PFS",
    PARAM    = "Progression-Free Survival (Months)",
    # Arm-specific exponential rates
    rate = case_when(
      ARM == "Placebo" ~ 0.12,
      ARM == "Dose 1"  ~ 0.09,
      TRUE             ~ 0.07
    ),
    AVAL = rexp(n(), rate = rate),
    AVAL = pmin(AVAL, 36), # administrative censoring at 36 months
    CNSR = if_else(AVAL >= 36, 1, 0)
  ) %>%
  select(-rate)

ADTTE %>% head()

```

```

# A tibble: 6 x 10
  STUDYID    USUBJID    ARM    TRTA  SEX    AGE PARAMCD PARAM    AVAL  CNSR
  <chr>      <chr>      <chr>  <chr> <chr> <dbl> <chr>   <chr>  <dbl> <dbl>
1 CDISCPIL0T01 01-701-1015 Placebo Plac~ F      63 PFS     Prog~  7.03     0
2 CDISCPIL0T01 01-701-1023 Placebo Plac~ M      64 PFS     Prog~  4.81     0
3 CDISCPIL0T01 01-701-1028 Xanomel~ Xano~ M      71 PFS     Prog~ 19.0     0
4 CDISCPIL0T01 01-701-1033 Xanomel~ Xano~ M      74 PFS     Prog~  0.451    0
5 CDISCPIL0T01 01-701-1034 Xanomel~ Xano~ F      77 PFS     Prog~  0.803    0
6 CDISCPIL0T01 01-701-1047 Placebo Plac~ F      85 PFS     Prog~  2.64     0

```

With these datasets we can now explore ggplot2 concepts **in an ADaM context**.

73 Grammar of Graphics with ADaM

The basic idea of ggplot2 (Grammar of Graphics):

- **Data:** ADSL / ADLB / ADTTE
- **Aesthetics** (`aes`): how variables map to x, y, color, etc.
- **Geoms:** geometric objects (points, lines, bars, etc.)
- **Scales, facets, coordinates, theme:** control look and structure

The general pattern:

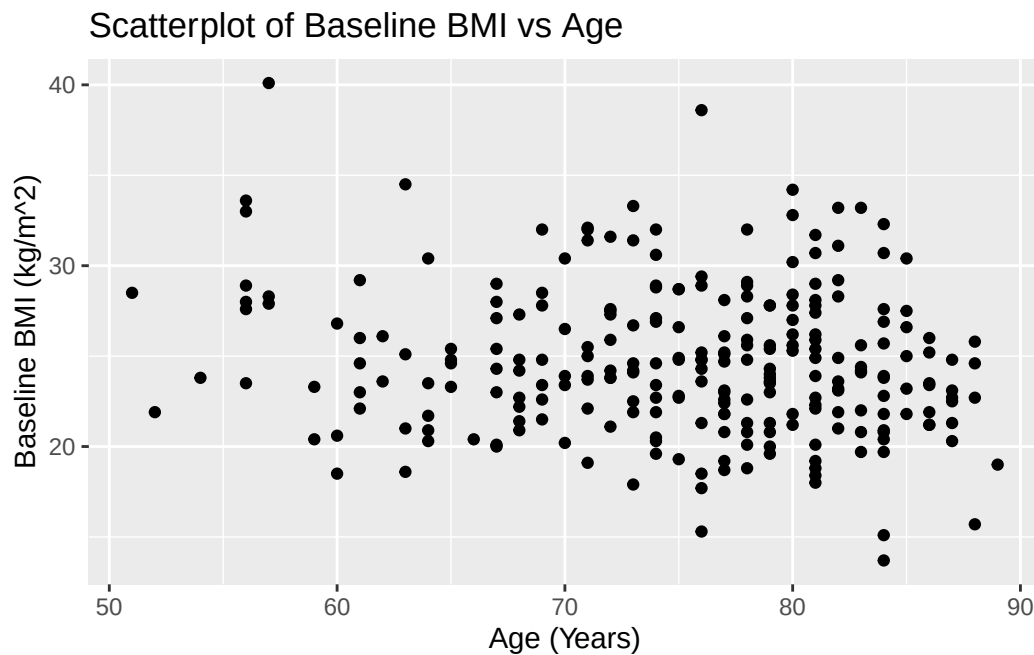
```
ADSL <- haven::read_sas("./data/adam/adsl.sas7bdat")
ggplot(data = ADSL, aes(x = AGE, y = BMIBL)) +
  geom_point()
```

We will start with subject-level plots from **ADSL**, then move to longitudinal labs from **ADLB**, and finally to **ADTTE**.

74 Basic Plots from ADSL (Beginner Level)

74.1 Scatterplot: AGE vs BMIBL

```
ggplot(ADSL, aes(x = AGE, y = BMIBL)) +  
  geom_point() +  
  labs(  
    title = "Scatterplot of Baseline BMI vs Age",  
    x = "Age (Years)",  
    y = "Baseline BMI (kg/m^2)"  
  )
```



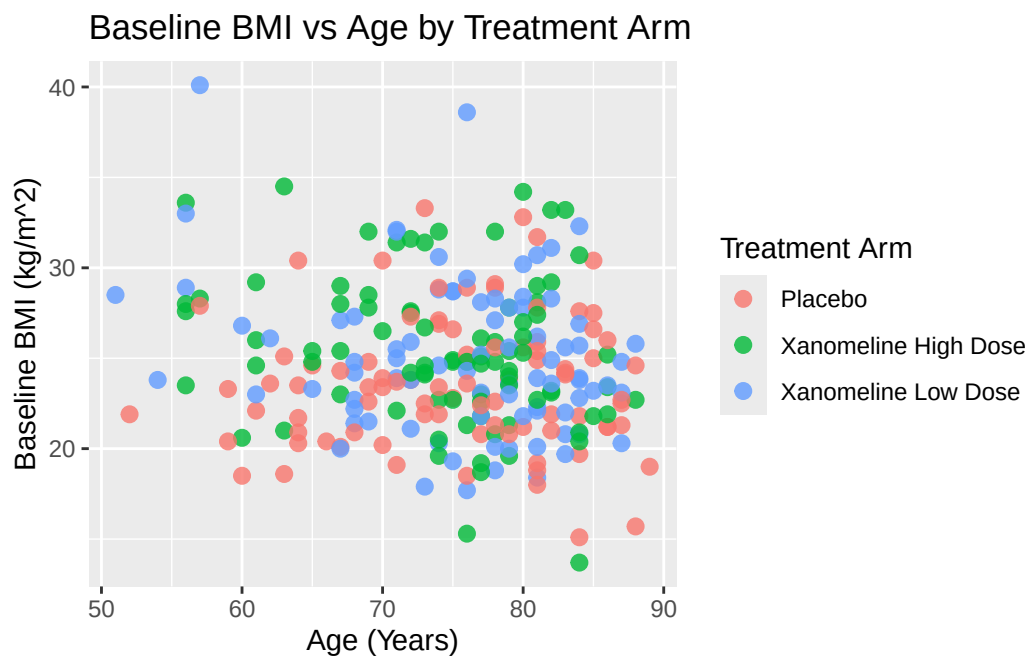
74.1.1 Mapping vs Setting Aesthetics

- **Mapping** (inside `aes()`): color/shape/size depend on data (e.g. ARM).

- **Setting** (outside `aes()`): fixed values.

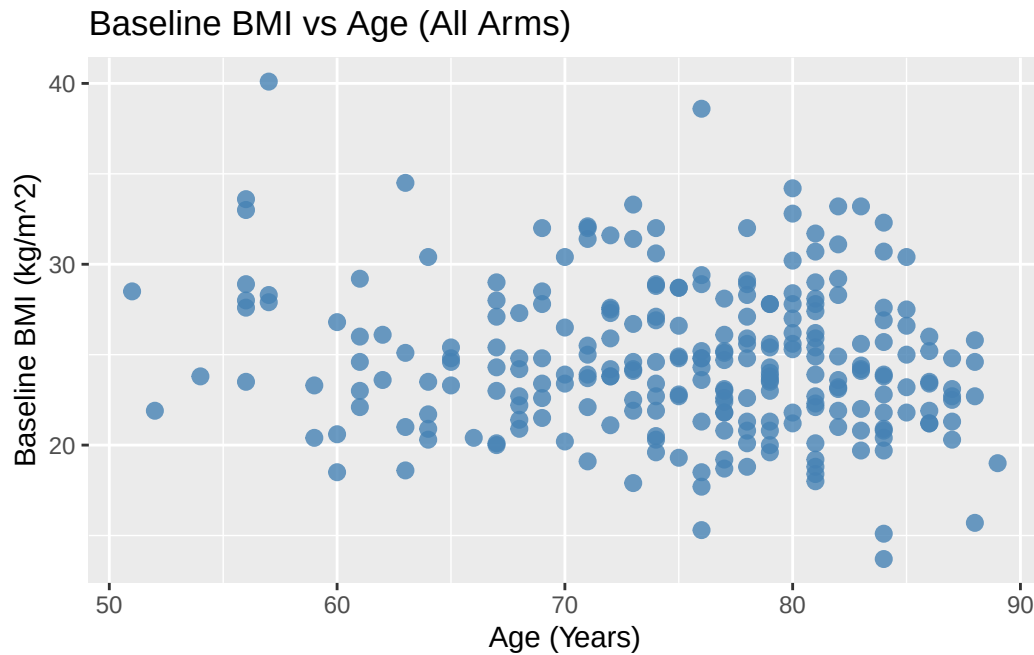
Map ARM to color:

```
ggplot(ADSL, aes(x = AGE, y = BMIBL, color = TRT01A)) +
  geom_point(size = 2.5, alpha = 0.8) +
  labs(
    title = "Baseline BMI vs Age by Treatment Arm",
    color = "Treatment Arm",
    x = "Age (Years)",
    y = "Baseline BMI (kg/m^2)"
  )
```



Set color manually (not based on data):

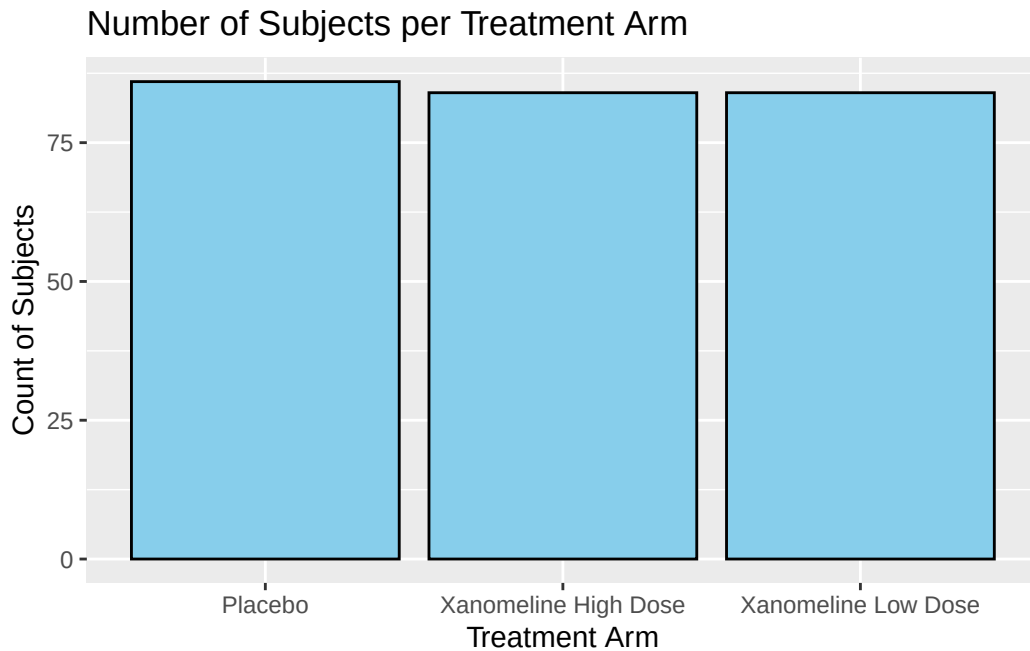
```
ggplot(ADSL, aes(x = AGE, y = BMIBL)) +
  geom_point(color = "steelblue", size = 2.5, alpha = 0.8) +
  labs(
    title = "Baseline BMI vs Age (All Arms)",
    x = "Age (Years)",
    y = "Baseline BMI (kg/m^2)"
  )
```



74.2 Bar Plot: Subject Counts per Arm

Using ADSL, we can show how many subjects per arm:

```
ggplot(ADSL, aes(x = TRT01A)) +  
  geom_bar(fill = "skyblue", color = "black") +  
  labs(  
    title = "Number of Subjects per Treatment Arm",  
    x = "Treatment Arm",  
    y = "Count of Subjects"  
  )
```



If you pre-compute counts (e.g. for TLG), use `geom_col()`:

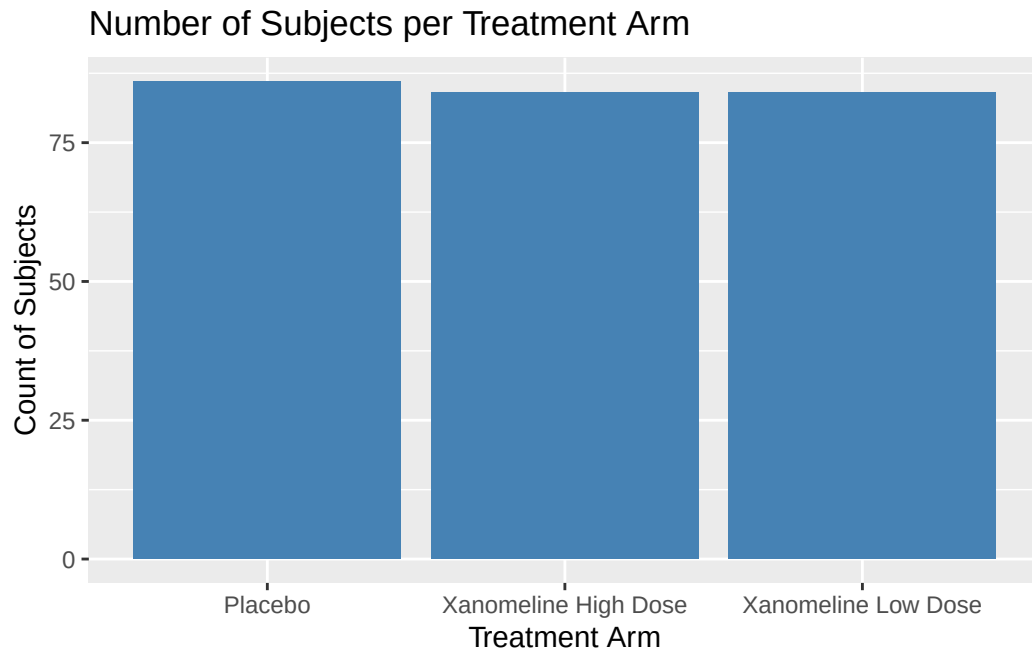
```
adsl_counts <- ADSL %>%  
  count(TRT01A, name = "n")
```

```
adsl_counts
```

```
# A tibble: 3 x 2
```

TRT01A	n
<chr>	<int>
1 Placebo	86
2 Xanomeline High Dose	84
3 Xanomeline Low Dose	84

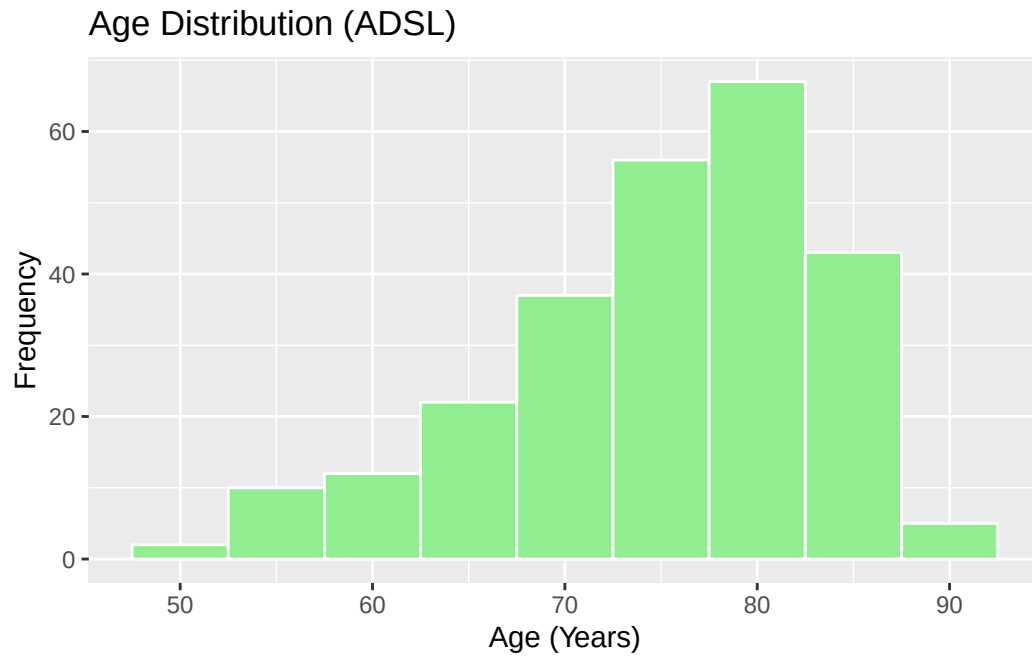
```
ggplot(adsl_counts, aes(x = TRT01A, y = n)) +  
  geom_col(fill = "steelblue") +  
  labs(  
    title = "Number of Subjects per Treatment Arm",  
    x = "Treatment Arm",  
    y = "Count of Subjects"  
  )
```



74.3 Histogram and Density: Age Distribution

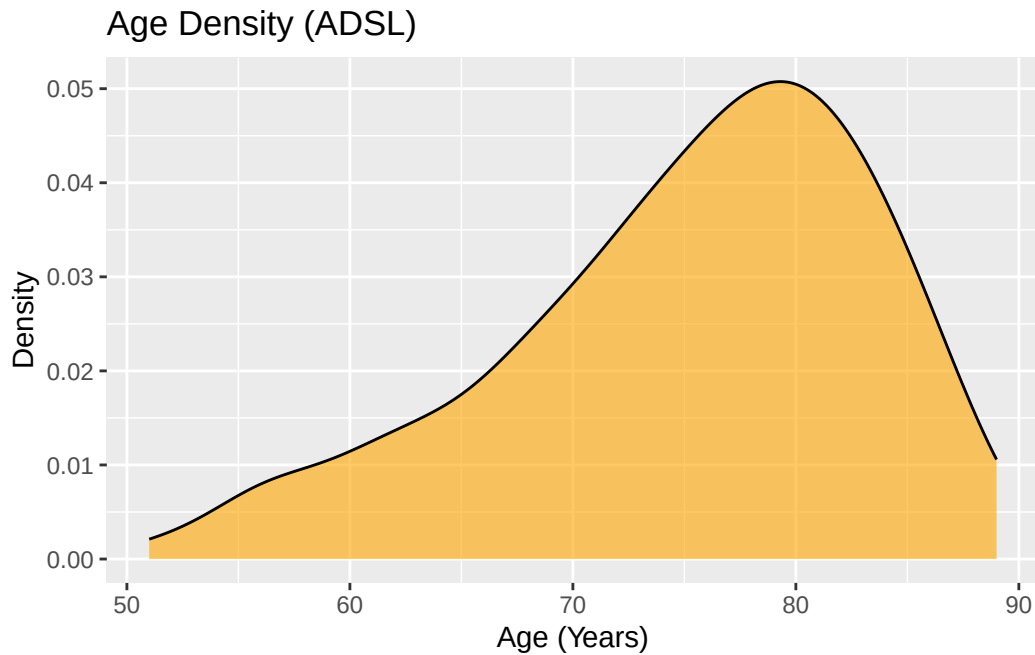
Histogram of AGE:

```
ggplot(ADSL, aes(x = AGE)) +  
  geom_histogram(binwidth = 5, fill = "lightgreen", color = "white") +  
  labs(  
    title = "Age Distribution (ADSL)",  
    x = "Age (Years)",  
    y = "Frequency"  
  )
```

Density plot:

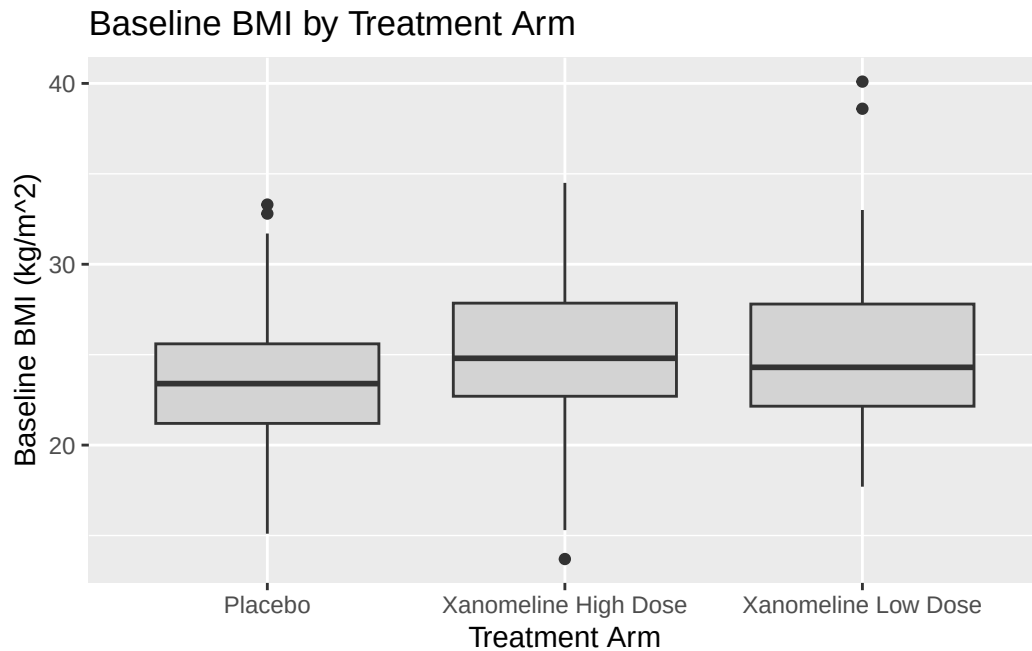
```
ggplot(ADSL, aes(x = AGE)) +  
  geom_density(fill = "orange", alpha = 0.6) +  
  labs(  
    title = "Age Density (ADSL)",  
    x = "Age (Years)",  
    y = "Density"  
  )
```



74.4 Boxplot: BMIBL by ARM

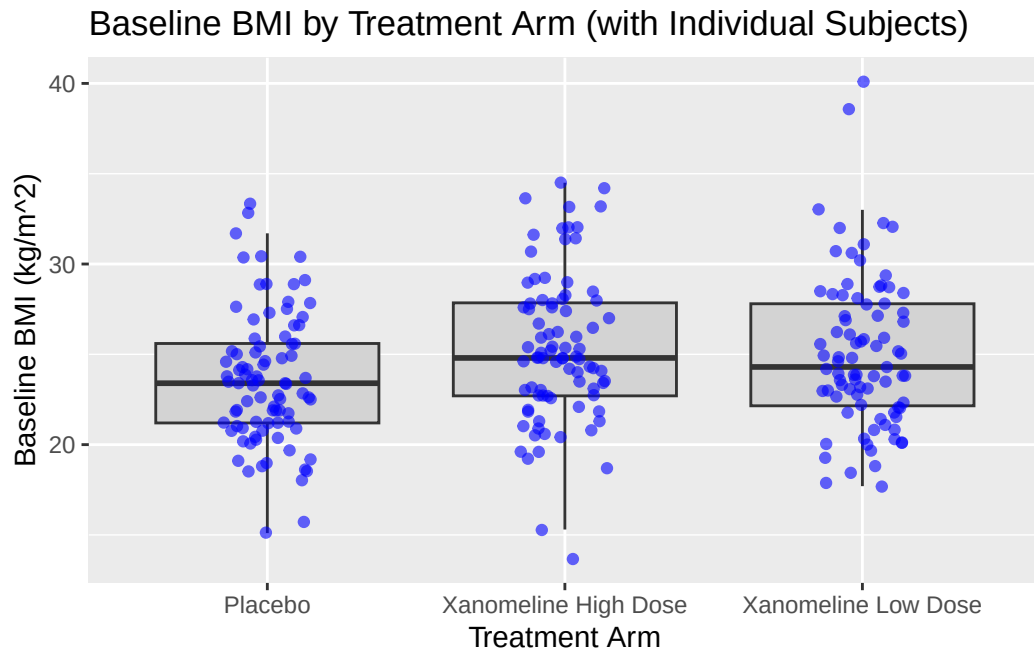
Boxplots are useful in CSR or listings to compare distributions across arms.

```
ggplot(ADSL, aes(x = TRT01A, y = BMIBL)) +  
  geom_boxplot(fill = "lightgray") +  
  labs(  
    title = "Baseline BMI by Treatment Arm",  
    x = "Treatment Arm",  
    y = "Baseline BMI (kg/m^2)"  
  )
```



You can combine boxplot with jittered points:

```
ggplot(ADSL, aes(x = TRT01A, y = BMIBL)) +  
  geom_boxplot(fill = "lightgray", outlier.shape = NA) +  
  geom_jitter(width = 0.15, alpha = 0.6, color = "blue") +  
  labs(  
    title = "Baseline BMI by Treatment Arm (with Individual Subjects)",  
    x = "Treatment Arm",  
    y = "Baseline BMI (kg/m^2)"  
  )
```



75 Longitudinal Plots from ADLB (Intermediate)

Now we use **ADLB** to illustrate time course plots, which are common in clinical reports.

75.1 Line Plot: Mean ALT Over Time by ARM

First summarise ADLB:

```
adlb_alt_summary <- ADLB %>%
  group_by(TRTA, AVISITN, AVISIT) %>%
  summarise(
    mean_ALT = mean(AVAL, na.rm=TRUE),
    sd_ALT    = sd(AVAL, na.rm=TRUE),
    n         = n(),
    se_ALT    = sd_ALT / sqrt(n),
    .groups = "drop"
  )

adlb_alt_summary$AVISIT <- factor(adlb_alt_summary$AVISIT, levels=unique(adlb_alt_summary$AVISIT))
adlb_alt_summary
```

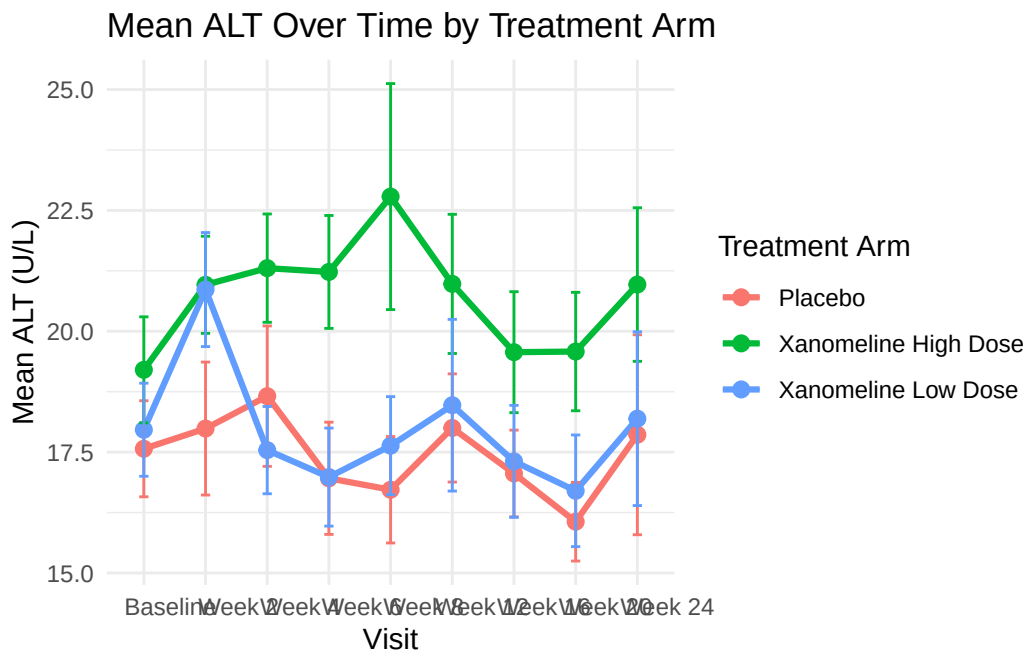
A tibble: 27 x 7

TRTA	AVISITN	AVISIT	mean_ALT	sd_ALT	n	se_ALT
<chr>	<dbl>	<fct>	<dbl>	<dbl>	<int>	<dbl>
1 Placebo	0	"Baseline"	17.6	9.22	86	0.994
2 Placebo	2	"Week 2"	18.0	12.5	83	1.38
3 Placebo	4	"Week 4"	18.7	12.9	79	1.45
4 Placebo	6	"Week 6"	17.0	9.92	73	1.16
5 Placebo	8	"Week 8"	16.7	9.34	72	1.10
6 Placebo	12	"Week 12"	18	9.16	67	1.12
7 Placebo	16	"Week 16"	17.1	7.39	68	0.897
8 Placebo	20	"Week 20"	16.1	6.56	65	0.814
9 Placebo	24	"Week 24"	17.9	15.6	57	2.07

```
10 Xanomeline High Dose      0 "      Baseline"      19.2  10.0      84  1.10
# i 17 more rows
```

Line plot with error bars:

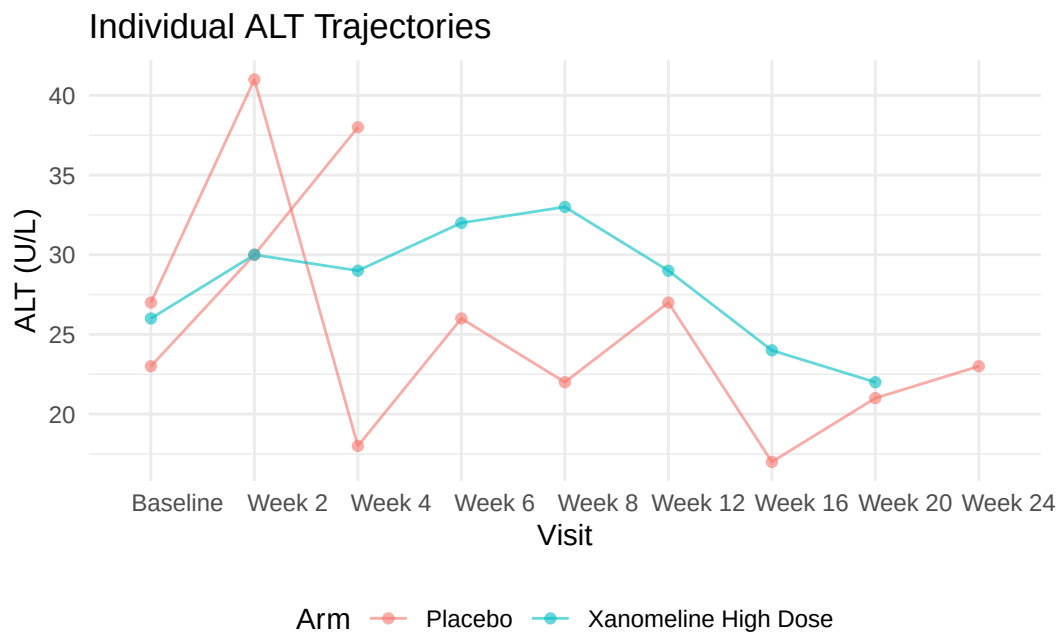
```
ggplot(adlb_alt_summary,
       aes(x = AVISIT, y = mean_ALT, group = TRTA, color = TRTA)) +
  geom_line(linewidth = 1.1) +
  geom_point(size = 2.5) +
  geom_errorbar(
    aes(ymin = mean_ALT - se_ALT, ymax = mean_ALT + se_ALT),
    width = 0.15
  ) +
  labs(
    title = "Mean ALT Over Time by Treatment Arm",
    x = "Visit",
    y = "Mean ALT (U/L)",
    color = "Treatment Arm"
  ) +
  theme_minimal()
```



75.2 Individual Profiles (Spaghetti Plot)

To see variability, we can plot individual patients (subset to avoid overcrowding):

```
ADLB_sub <- ADLB |>
  head(20)
ADLB_sub$AVISIT <- factor(ADLB_sub$AVISIT, levels=unique(ADLB_sub$AVISIT[order(ADLB_sub$AVISIT)]))
ggplot(ADLB_sub,
  aes(x = AVISIT, y = AVAL, group = USUBJID, color = TRTA)) +
  geom_line(alpha = 0.6) +
  geom_point(alpha = 0.6, size = 1.5) +
  labs(
    title = "Individual ALT Trajectories ",
    x = "Visit",
    y = "ALT (U/L)",
    color = "Arm"
  ) +
  theme_minimal() +
  theme(legend.position = "bottom")
```



76 Faceting with ADaM Data

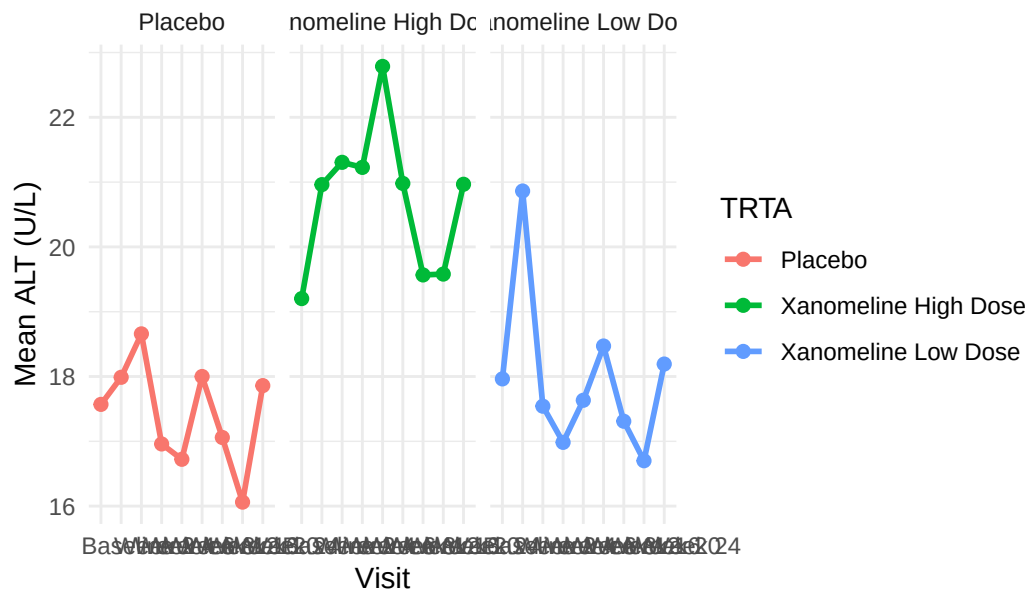
Faceting is useful when splitting results by subgroups, lab parameters, or regions.

Here we only have PARAMCD = "ALT", but we can pretend we have multiple parameters.

76.1 Example: Facet ALT by Sex

```
ggplot(adlb_alt_summary,
       aes(x = AVISIT, y = mean_ALT, group = TRTA, color = TRTA)) +
  geom_line(linewidth = 1) +
  geom_point(size = 2) +
  facet_wrap(~ TRTA) +
  labs(
    title = "Mean ALT Over Time - Faceted by Treatment TRTA",
    x = "Visit",
    y = "Mean ALT (U/L)",
    color = "TRTA"
  ) +
  theme_minimal()
```


Mean ALT Over Time – Faceted by Treatment TRTA

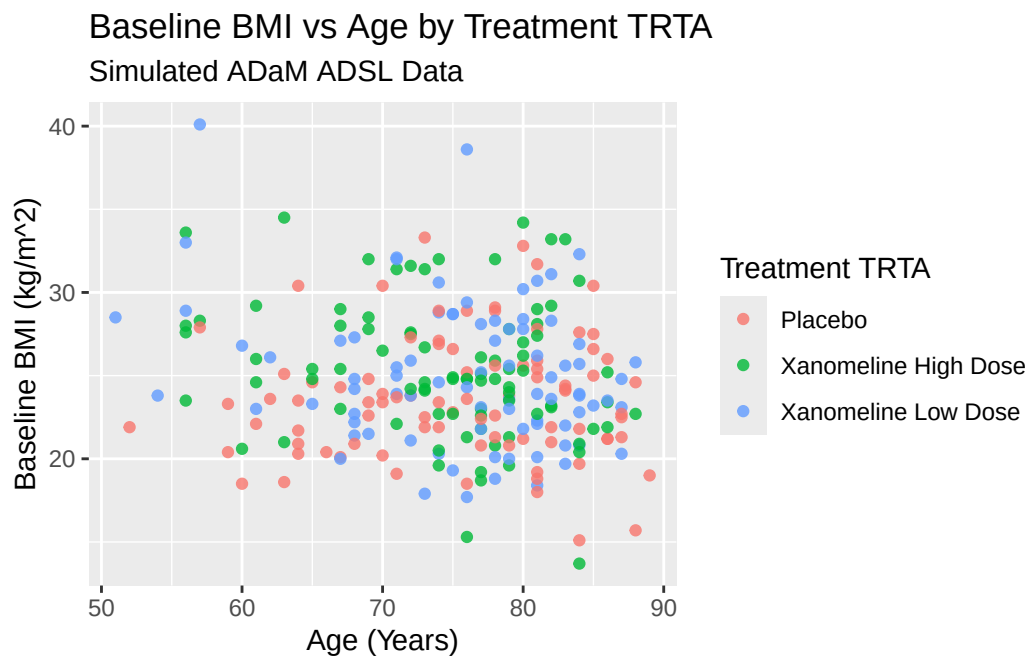


You can similarly facet by **SEX**, **RACE**, or any other ADSL variable once merged into ADLB.

77 Scales, Labels, and Themes in ADaM Context

77.1 Changing Axis Labels and Titles

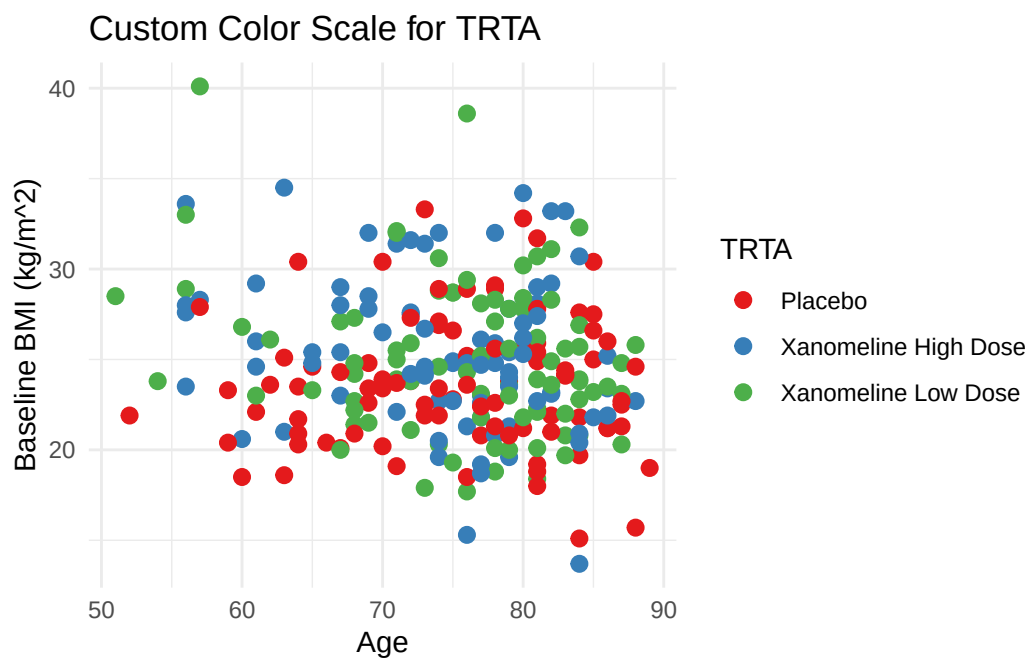
```
ggplot(ADSL, aes(x = AGE, y = BMIBL, color = TRT01A)) +  
  geom_point(alpha = 0.8) +  
  labs(  
    title = "Baseline BMI vs Age by Treatment TRTA",  
    subtitle = "Simulated ADaM ADSL Data",  
    x = "Age (Years)",  
    y = "Baseline BMI (kg/m^2)",  
    color = "Treatment TRTA"  
  )
```



77.2 Controlling Color Scales

For discrete TRTA:

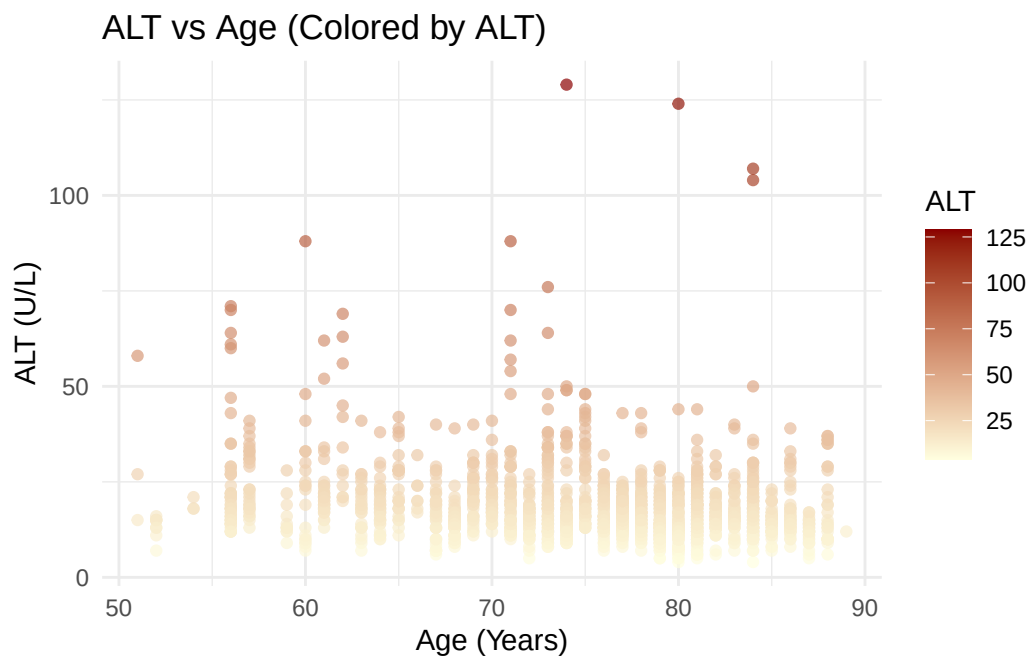
```
ggplot(ADSL, aes(x = AGE, y = BMIBL, color = TRT01A)) +  
  geom_point(size = 2.5) +  
  scale_color_brewer(palette = "Set1") +  
  labs(  
    title = "Custom Color Scale for TRTA",  
    color = "TRTA"  
  ) +  
  theme_minimal()
```



For continuous values (e.g. ALT):

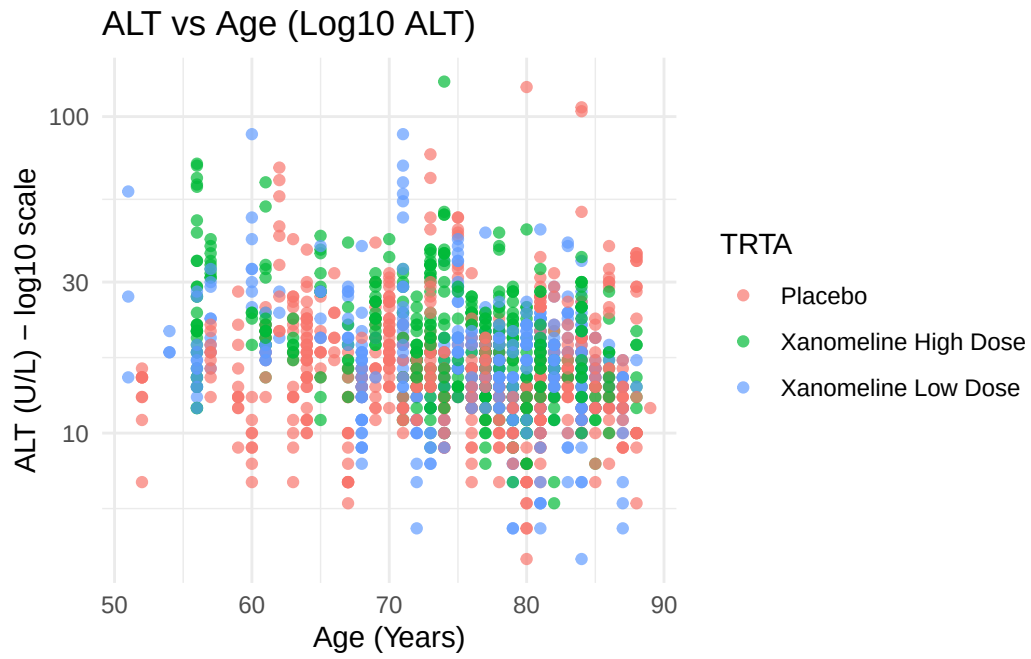
```
ggplot(ADLB, aes(x = AGE, y = AVAL, color = AVAL)) +  
  geom_point(alpha = 0.7) +  
  scale_color_gradient(low = "lightyellow", high = "darkred") +  
  labs(  
    title = "ALT vs Age (Colored by ALT)",  
    x = "Age (Years)",  
    y = "ALT (U/L)",  
    color = "ALT"  
  )
```

```
) +  
theme_minimal()
```



77.3 Transforming Scales (e.g., log10 ALT)

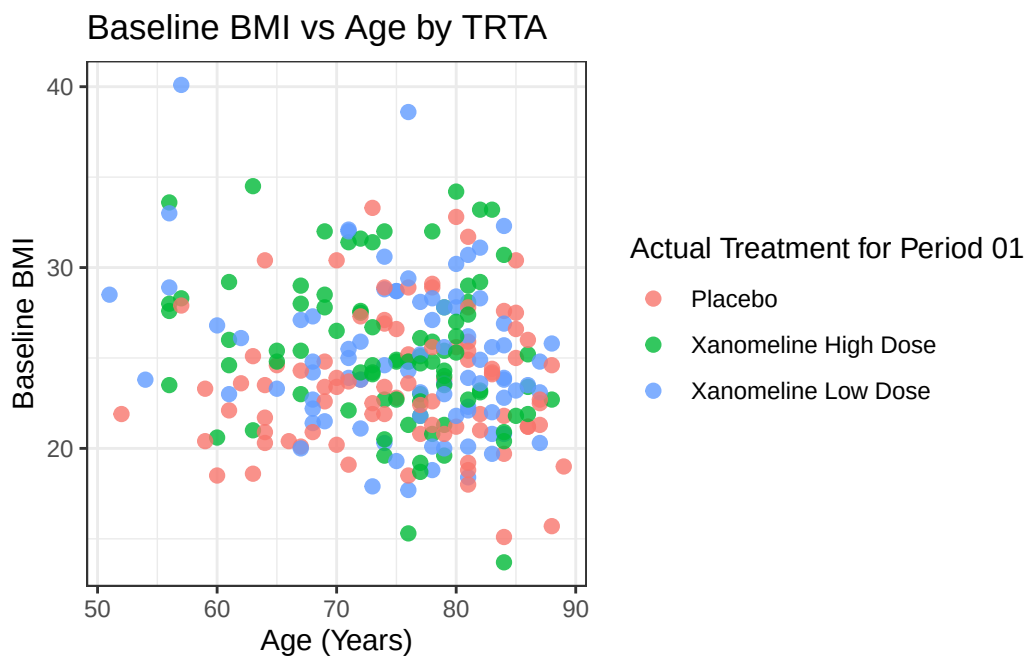
```
ggplot(ADLB, aes(x = AGE, y = AVAL, color = TRTA)) +  
  geom_point(alpha = 0.7) +  
  scale_y_log10() +  
  labs(  
    title = "ALT vs Age (Log10 ALT)",  
    x = "Age (Years)",  
    y = "ALT (U/L) - log10 scale",  
    color = "TRTA"  
  ) +  
  theme_minimal()
```



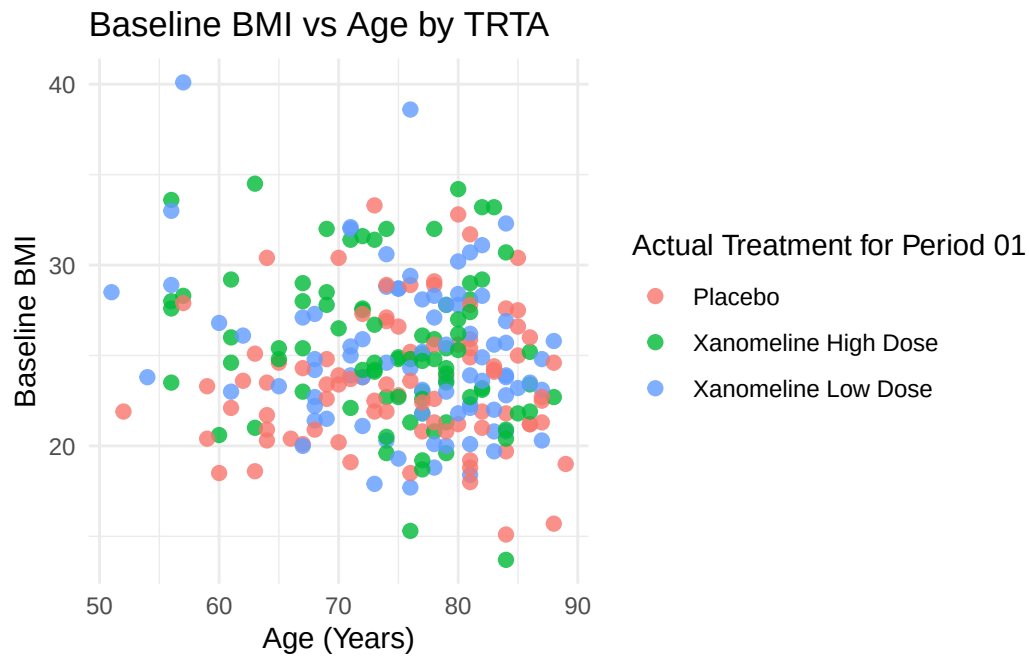
78 Themes and Publication-Style Graphics

78.1 Using Built-In Themes

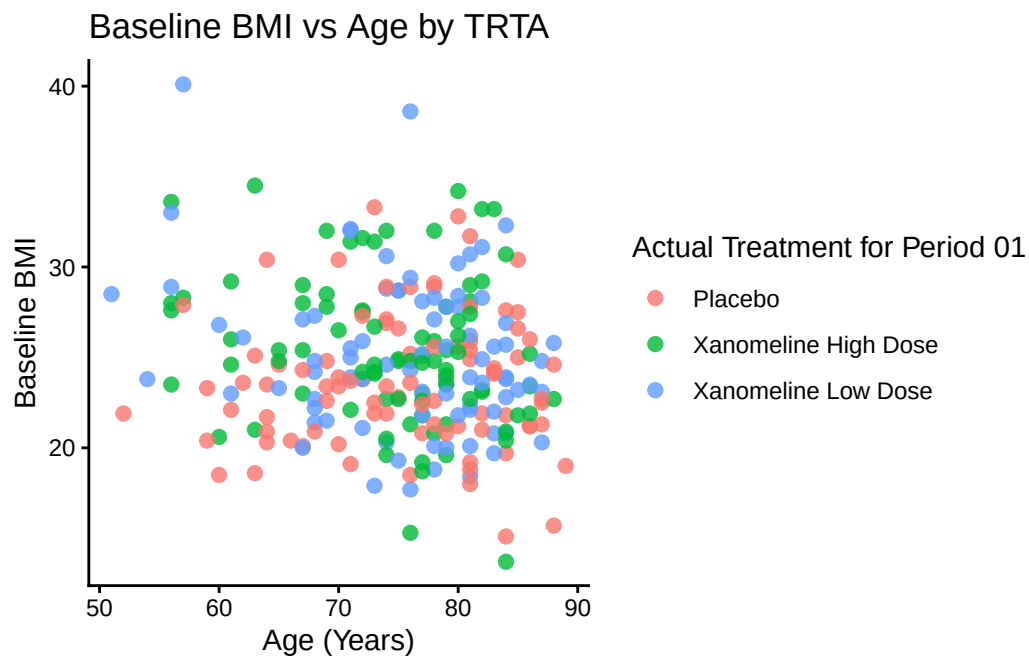
```
p_base <- ggplot(ADSL, aes(x = AGE, y = BMIBL, color = TRT01A)) +  
  geom_point(size = 2.2, alpha = 0.8) +  
  labs(  
    title = "Baseline BMI vs Age by TRTA",  
    x = "Age (Years)",  
    y = "Baseline BMI"  
  )  
  
p_base + theme_bw()
```



```
p_base + theme_minimal()
```

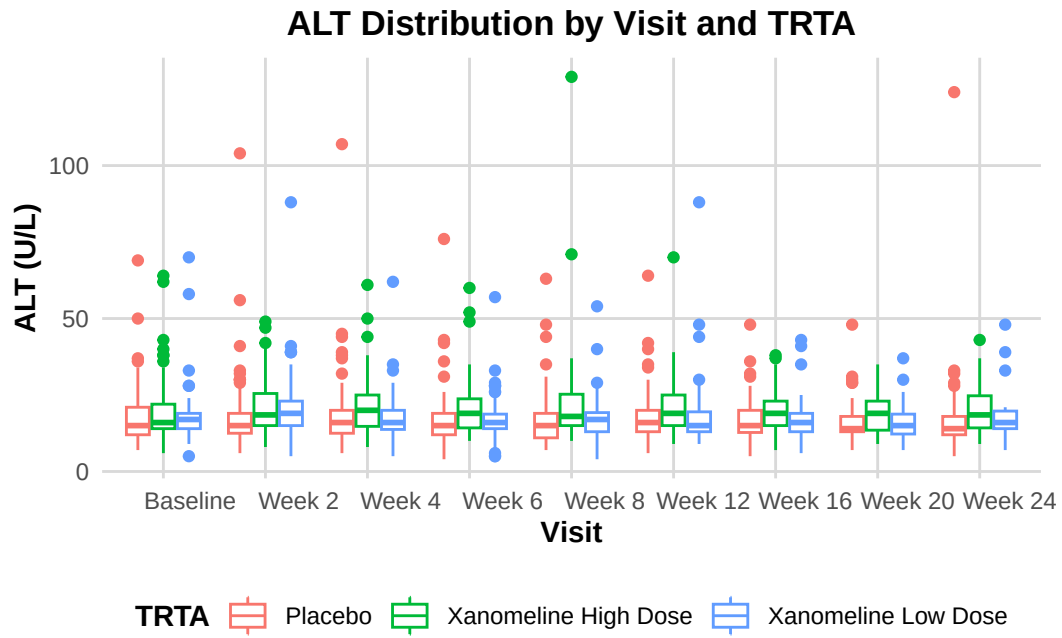


```
p_base + theme_classic()
```



78.2 Custom Theme for Clinical Reports

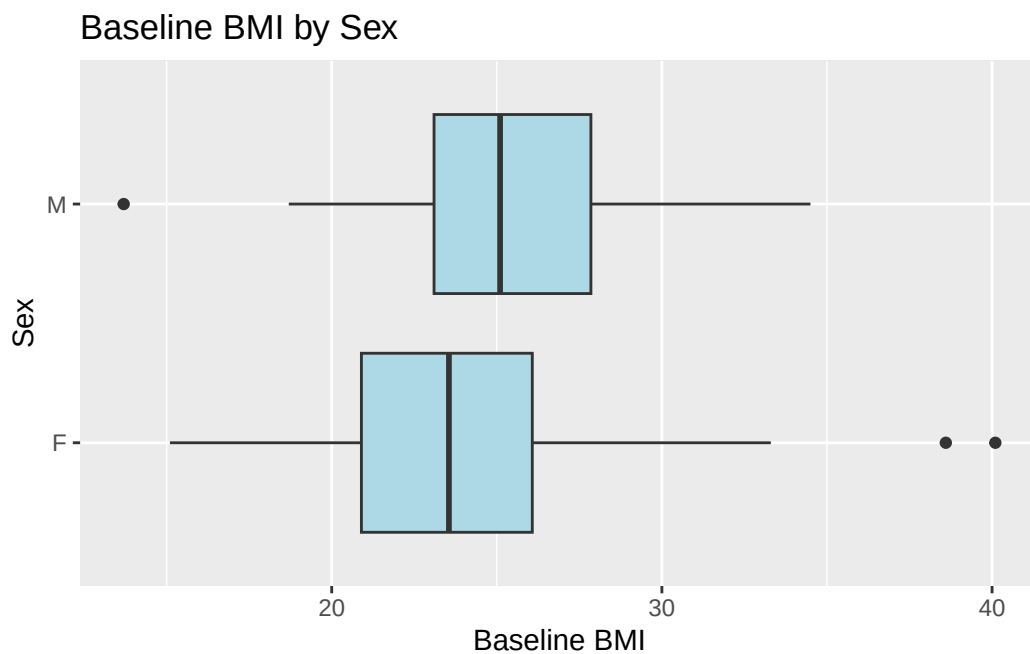
```
theme_adam_pub <- function() {  
  theme_minimal(base_size = 11) +  
    theme(  
      plot.title = element_text(face = "bold", hjust = 0.5, size = 13),  
      axis.title = element_text(face = "bold"),  
      panel.grid.major = element_line(color = "grey85"),  
      panel.grid.minor = element_blank(),  
      legend.position = "bottom",  
      legend.title = element_text(face = "bold")  
    )  
}  
  
ADLB$AVISIT <- factor(ADLB$AVISIT, levels=unique(ADLB$AVISIT[order(ADLB$AVISITN)]))  
ggplot(ADLB, aes(x = AVISIT, y = AVAL, color = TRTA)) +  
  geom_boxplot() +  
  labs(  
    title = "ALT Distribution by Visit and TRTA",  
    x = "Visit",  
    y = "ALT (U/L)",  
    color = "TRTA"  
  ) +  
  theme_adam_pub()
```

79 Coordinates and Positions

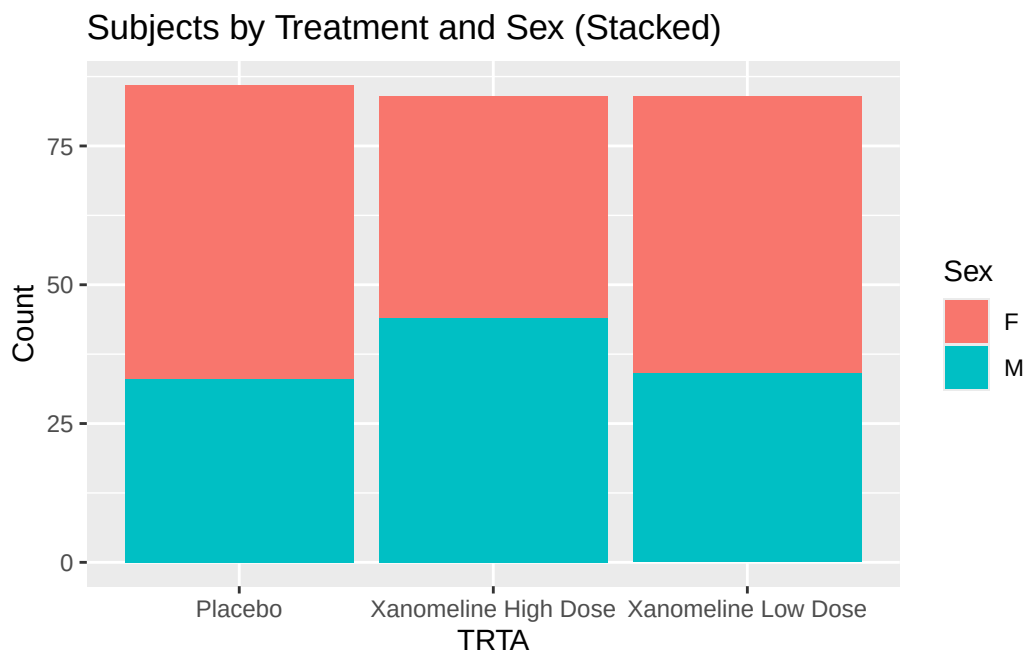
79.1 Flipping Coordinates: Boxplot of BMI by Sex

```
ggplot(ADSL, aes(x = SEX, y = BMIBL)) +  
  geom_boxplot(fill = "lightblue") +  
  coord_flip() +  
  labs(  
    title = "Baseline BMI by Sex",  
    x = "Sex",  
    y = "Baseline BMI"  
  )
```

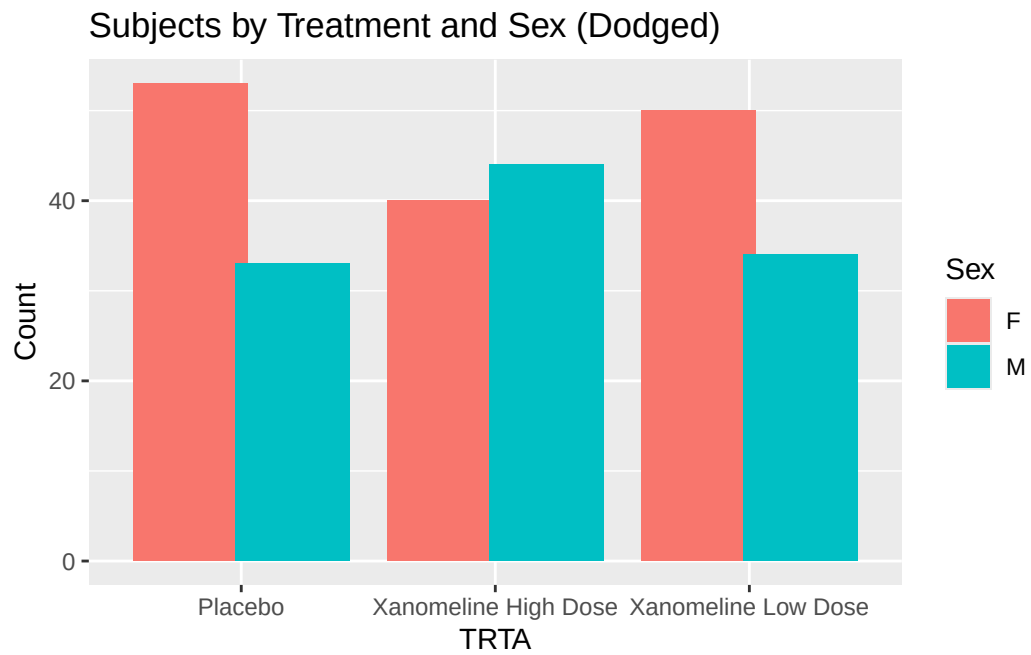


79.2 Stacked vs Dodged Bar Plots: TRTA by SEX

```
ggplot(ADSL, aes(x = TRTA, fill = SEX)) +  
  geom_bar(position = "stack") +  
  labs(  
    title = "Subjects by Treatment and Sex (Stacked)",  
    x = "TRTA",  
    y = "Count",  
    fill = "Sex"  
  )
```



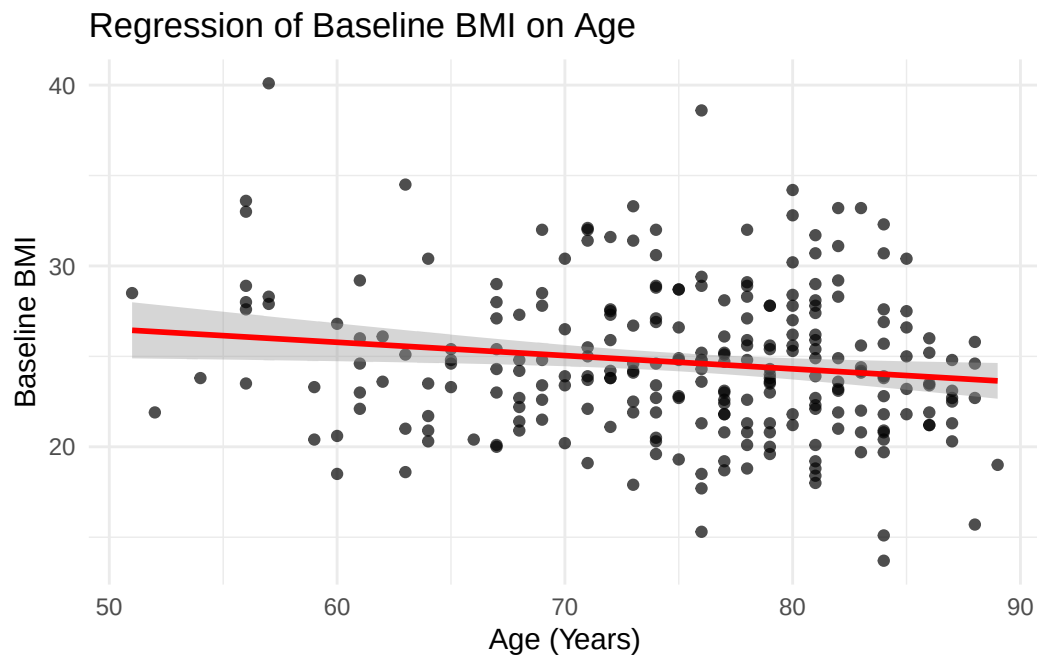
```
ggplot(ADSL, aes(x = TRTA, fill = SEX)) +  
  geom_bar(position = position_dodge(width = 0.8)) +  
  labs(  
    title = "Subjects by Treatment and Sex (Dodged)",  
    x = "TRTA",  
    y = "Count",  
    fill = "Sex"  
  )
```



80 Adding Statistical Layers

80.1 Linear Regression: BMIBL vs AGE

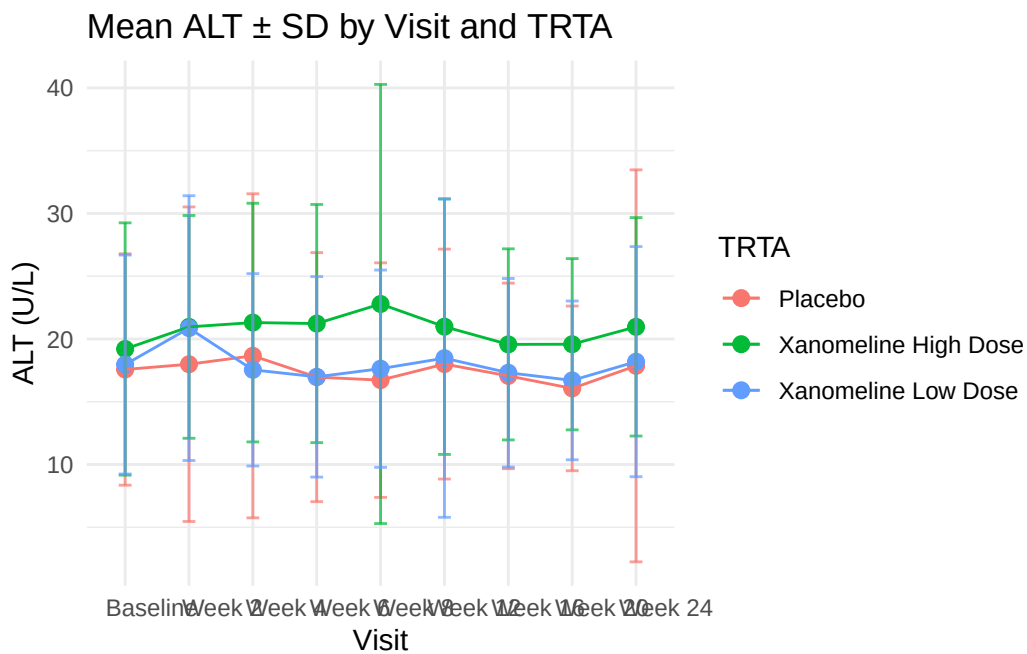
```
ggplot(ADSL, aes(x = AGE, y = BMIBL)) +  
  geom_point(alpha = 0.7) +  
  stat_smooth(method = "lm", se = TRUE, color = "red") +  
  labs(  
    title = "Regression of Baseline BMI on Age",  
    x = "Age (Years)",  
    y = "Baseline BMI"  
  ) +  
  theme_minimal()
```



80.2 Summaries: Mean ALT $\pm$ SD per Visit and TRTA

We already computed `adlb_alt_summary`. Plot mean $\pm$ SD:

```
ggplot(adlb_alt_summary,
       aes(x = AVISIT, y = mean_ALT, color = TRTA, group = TRTA)) +
  geom_point(size = 2.5) +
  geom_line() +
  geom_errorbar(
    aes(ymin = mean_ALT - sd_ALT, ymax = mean_ALT + sd_ALT),
    width = 0.2, alpha = 0.7
  ) +
  labs(
    title = "Mean ALT  $\pm$  SD by Visit and TRTA",
    x = "Visit",
    y = "ALT (U/L)",
    color = "TRTA"
  ) +
  theme_minimal()
```



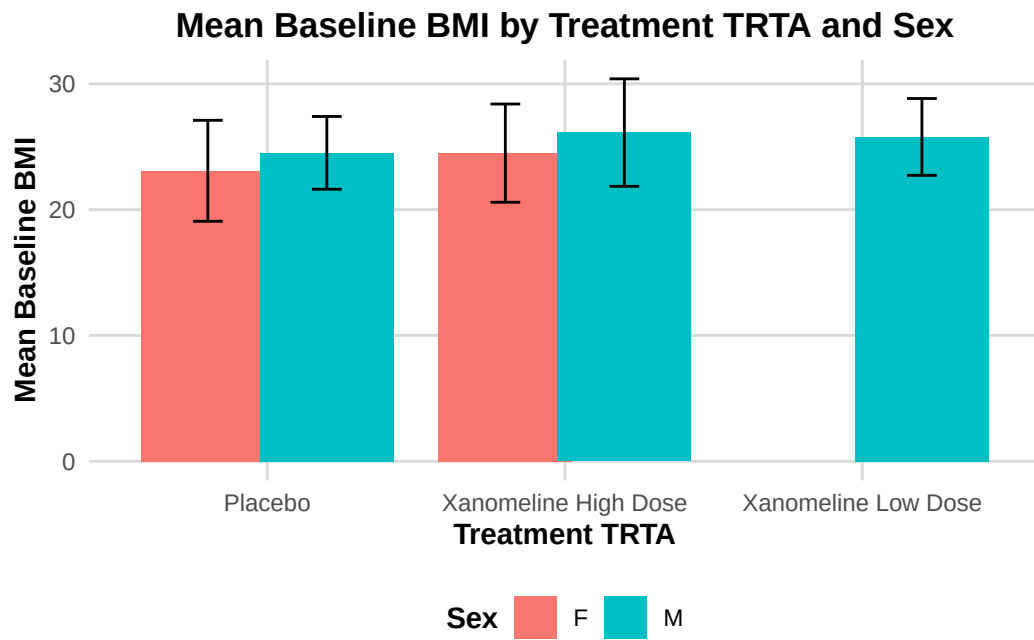
81 Tidy Workflow: dplyr + ggplot2 with ADaM

A very common pattern in clinical reporting:

1. Start from ADaM dataset
2. Filter / summarise with `dplyr`
3. Plot with `ggplot2`

Example: Mean BMI by TRTA and SEX:

```
ADSL %>%
  group_by(TRTA, SEX) %>%
  summarise(
    mean_BMIBL = mean(BMIBL),
    sd_BMIBL   = sd(BMIBL),
    n          = n(),
    .groups = "drop"
  ) %>%
  ggplot(aes(x = TRTA, y = mean_BMIBL, fill = SEX)) +
  geom_col(position = position_dodge(width = 0.8)) +
  geom_errorbar(
    aes(ymin = mean_BMIBL - sd_BMIBL, ymax = mean_BMIBL + sd_BMIBL),
    position = position_dodge(width = 0.8),
    width = 0.2
  ) +
  labs(
    title = "Mean Baseline BMI by Treatment TRTA and Sex",
    x = "Treatment TRTA",
    y = "Mean Baseline BMI",
    fill = "Sex"
  ) +
  theme_adam_pub()
```

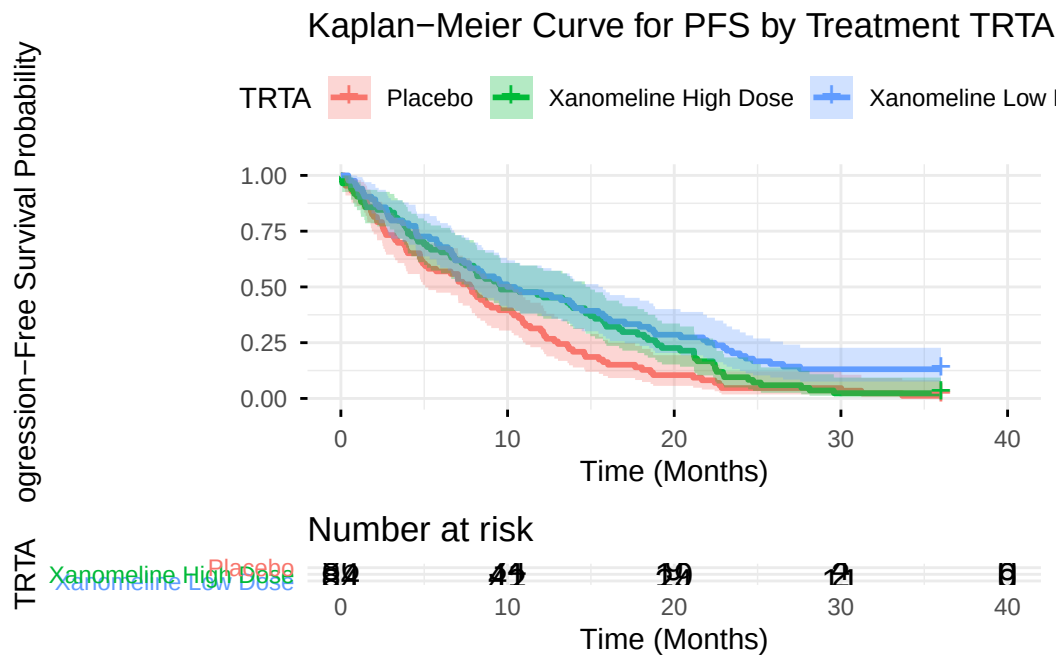


82 Advanced: Simple Kaplan–Meier Plot from ADTTE

In ADaM, **ADTTE** contains time-to-event information (e.g. PFS, OS).

Here we show a simple KM plot using `survminer::ggsurvplot()` which uses `ggplot2` under the hood.

```
adtte_pfs <- ADTTE %>%  
  filter(PARAMCD == "PFS")  
  
fit_pfs <- survfit(Surv(AVAL, 1 - CNSR) ~ TRTA, data = adtte_pfs)  
  
km_plot <- ggsurvplot(  
  fit_pfs,  
  data = adtte_pfs,  
  conf.int = TRUE,  
  risk.table = TRUE,  
  risk.table.height = 0.25,  
  ggtheme = theme_minimal(),  
  legend.title = "TRTA",  
  legend.labs = levels(factor(adtte_pfs$TRTA)),  
  title = "Kaplan-Meier Curve for PFS by Treatment TRTA",  
  xlab = "Time (Months)",  
  ylab = "Progression-Free Survival Probability"  
)  
  
km_plot
```



For a **pure ggplot2** approach, you could extract the `survfit` summary into a data frame and use `geom_step()` manually, but `ggsurvplot()` is convenient and still fully compatible with ggplot theming.

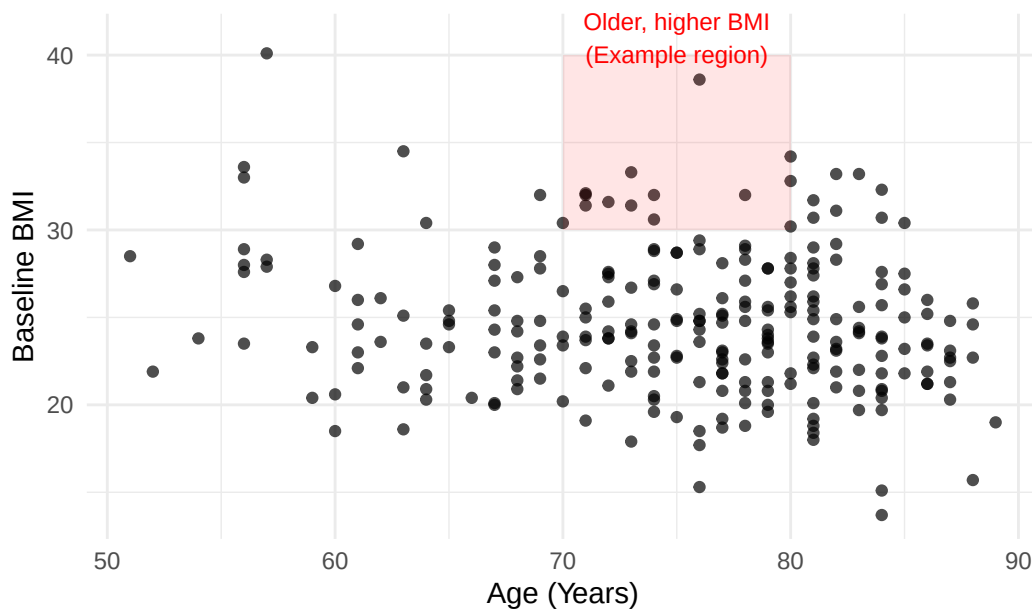
83 Advanced: Annotations and Secondary Axes

83.1 Annotations on an ADaM Plot

Example: highlight older patients with high BMI:

```
ggplot(ADSL, aes(x = AGE, y = BMIBL)) +  
  geom_point(alpha = 0.7) +  
  annotate(  
    "rect", xmin = 70, xmax = 80,  
    ymin = 30, ymax = 40,  
    alpha = 0.1, fill = "red"  
  ) +  
  annotate(  
    "text", x = 75, y = 41,  
    label = "Older, higher BMI  
(Example region)",  
    size = 3, color = "red"  
  ) +  
  labs(  
    title = "Annotations on ADSL Scatterplot",  
    x = "Age (Years)",  
    y = "Baseline BMI"  
  ) +  
  theme_minimal()
```

Annotations on ADSL Scatterplot

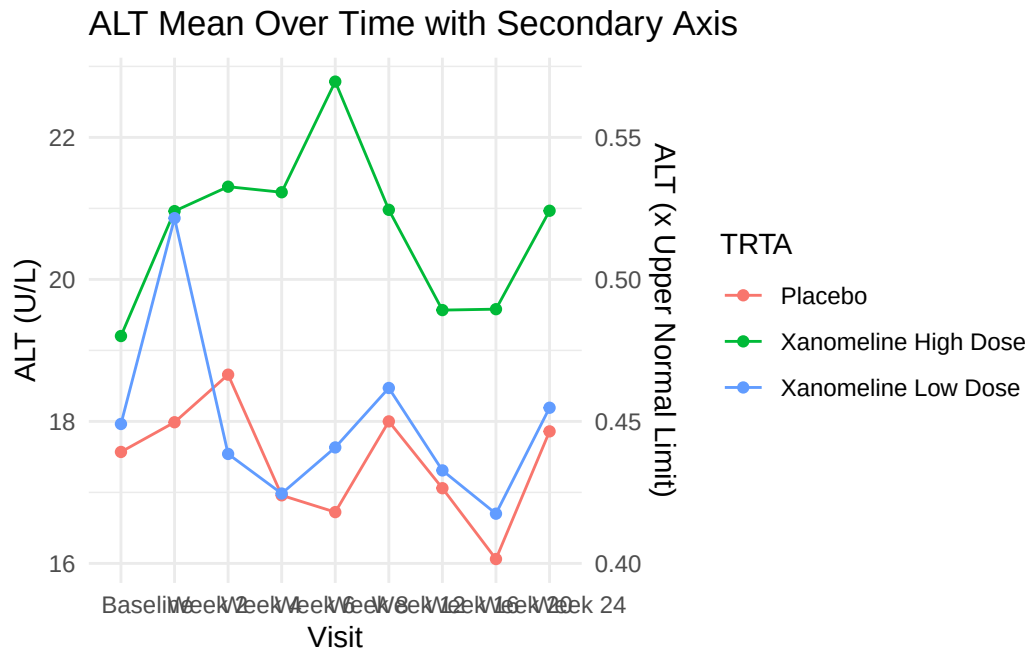


83.2 Secondary Axis (Use with Care)

ggplot2 allows secondary axes only if they are a simple transformation of the primary axis.

Example (purely illustrative) converting ALT to approximate multiple:

```
ggplot(adlb_alt_summary, aes(x = AVISIT, y = mean_ALT, group = TRTA, color = TRTA)) +  
  geom_line() +  
  geom_point() +  
  scale_y_continuous(  
    name = "ALT (U/L)",  
    sec.axis = sec_axis(~ . / 40, name = "ALT (x Upper Normal Limit)")  
  ) +  
  labs(  
    title = "ALT Mean Over Time with Secondary Axis",  
    x = "Visit",  
    color = "TRTA"  
  ) +  
  theme_minimal()
```

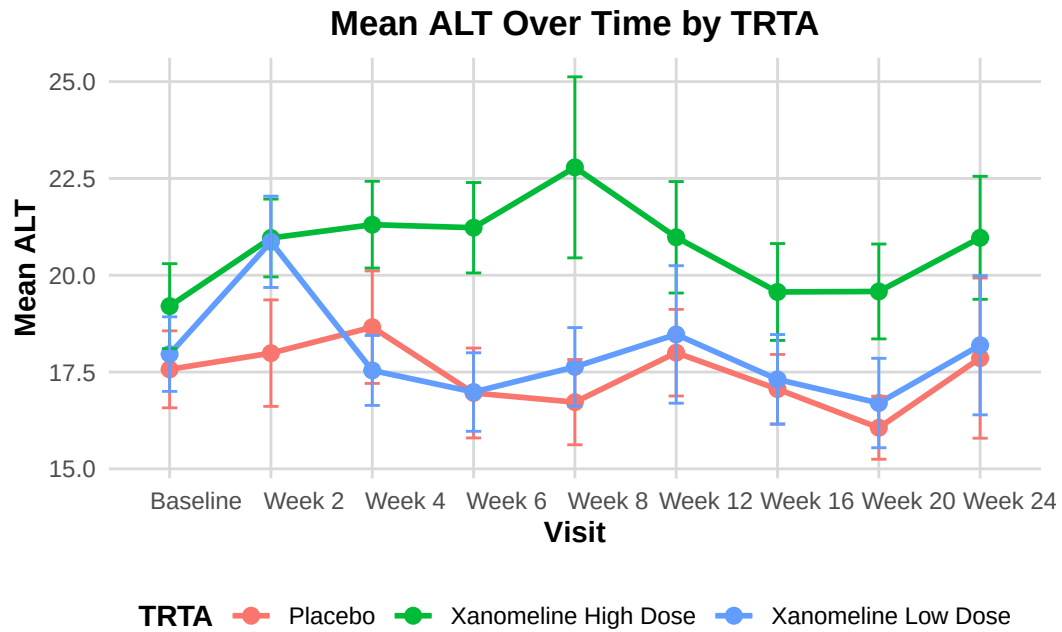


In production, ensure the secondary axis has a clinically correct and meaningful transformation.

84 Writing Reusable Plot Functions for ADaM

You can encapsulate standard plots (e.g., lab time course by TRTA) into functions.

```
plot_lab_timecourse <- function(adlb_df, paramcd = "ALT") {  
  df <- adlb_df %>%  
    filter(PARAMCD == paramcd) %>%  
    group_by(TRTA, AVISIT) %>%  
    summarise(  
      mean_aval = mean(AVAL),  
      se_aval   = sd(AVAL) / sqrt(n()),  
      .groups = "drop"  
    )  
  
  ggplot(df, aes(x = AVISIT, y = mean_aval, color = TRTA, group = TRTA)) +  
    geom_line(linewidth = 1) +  
    geom_point(size = 2.5) +  
    geom_errorbar(  
      aes(ymin = mean_aval - se_aval, ymax = mean_aval + se_aval),  
      width = 0.15  
    ) +  
    labs(  
      title = paste("Mean", paramcd, "Over Time by TRTA"),  
      x = "Visit",  
      y = paste("Mean", paramcd),  
      color = "TRTA"  
    ) +  
    theme_adam_pub()  
}  
  
plot_lab_timecourse(ADLB, paramcd = "ALT")
```



This approach is very powerful for building **standardized plotting utilities** for your ADaM pipeline.

85 Saving Plots for Reports

Use `ggsave()` to save plots for use in RTF, Word, or PowerPoint.

```
p <- ggplot(ADSL, aes(x = AGE, y = BMIBL, color = TRTA)) +  
  geom_point() +  
  labs(  
    title = "Baseline BMI vs Age by TRTA",  
    x = "Age (Years)",  
    y = "Baseline BMI"  
  ) +  
  theme_adam_pub()  
  
# Save as PNG  
ggsave("bmi_vs_age_by_TRTA.png", plot = p, width = 6, height = 4, dpi = 300)  
  
# Save as PDF  
ggsave("bmi_vs_age_by_TRTA.pdf", plot = p, width = 6, height = 4)
```


86 Introduction

This document walks through **six common oncology graphs** step by step, using the exact code you provided:

1. Bar graph of **sum of lesion diameters** plus individual lesions
2. **Waterfall plot** of best percentage change from baseline
3. **Swimmer plot** for duration of response
4. **Spider plot** (tumor trajectories over time)
5. **Kaplan–Meier** survival curve
6. **Forest plot** for subgroup hazard ratios

For each figure, we will:

- Build a simple **example dataset** in R (similar structure to what you'd get from ADaM like ADTR/ADTTE).
- Explain the main **ggplot2 layers** and options.
- Show the **final code** to generate the plot.

In a real clinical project, you would replace the toy data here with your ADaM datasets, but the plotting logic stays the same.

87 Figure 1 – Bar Graph with Individual Lesion Lines

This plot shows, for a single subject:

- Bars: **Sum of Lesion Diameters (SUMLD)** at each visit
- Lines/points: each **individual lesion** (tumor) over time

Clinically, this helps you see how the total tumor burden and individual lesions evolve across visits.

87.1 1.1 Create bar-graph data

We first build a small dataset with:

- `visit`: visit label
- `SUMLD`: sum of lesion diameters
- `trstresn_tumor1/2/3`: individual lesion diameters

```
bar_data <- data.frame(  
  visit = c("Baseline", "Week 4", "Week 8", "Week 12", "Week 16"),  
  SUMLD = c(100, 85, 70, 65, 60), # Sum of Lesion Diameters  
  trstresn_tumor1 = c(50, 45, 35, 30, 28),  
  trstresn_tumor2 = c(30, 25, 20, 20, 18),  
  trstresn_tumor3 = c(20, 15, 15, 15, 14)  
)
```

`bar_data`

	visit	SUMLD	trstresn_tumor1	trstresn_tumor2	trstresn_tumor3
1	Baseline	100	50	30	20
2	Week 4	85	45	25	15
3	Week 8	70	35	20	15
4	Week 12	65	30	20	15
5	Week 16	60	28	18	14

In real ADaM:

- `visit` could map to `AVISIT`,
- `SUMLD` would be a derived variable (sum of target lesions),
- `trstresn_tumorX` are separate lesions (often separate rows in ADTR; here we keep them as columns for simplicity).

87.2 1.2 Reshape to long format for lines

To draw multiple lesions as separate lines, we reshape the wide columns (`trstresn_tumor1/2/3`) into a long format using `pivot_longer()`:

```
bar_data_long <- bar_data %>%
  pivot_longer(
    cols = starts_with("trstresn"),
    names_to = "tuloc",
    values_to = "trstresn"
  )

bar_data_long
```

```
# A tibble: 15 x 4
  visit    SUMLD tuloc      trstresn
  <chr>    <dbl> <chr>      <dbl>
1 Baseline  100 trstresn_tumor1    50
2 Baseline  100 trstresn_tumor2    30
3 Baseline  100 trstresn_tumor3    20
4 Week 4    85 trstresn_tumor1    45
5 Week 4    85 trstresn_tumor2    25
6 Week 4    85 trstresn_tumor3    15
7 Week 8    70 trstresn_tumor1    35
8 Week 8    70 trstresn_tumor2    20
9 Week 8    70 trstresn_tumor3    15
10 Week 12   65 trstresn_tumor1    30
11 Week 12   65 trstresn_tumor2    20
12 Week 12   65 trstresn_tumor3    15
13 Week 16   60 trstresn_tumor1    28
14 Week 16   60 trstresn_tumor2    18
15 Week 16   60 trstresn_tumor3    14
```

- `tuloc` acts like **tumor location / lesion identifier**.

- `trstresn` holds the lesion diameter for each visit-lesion combination.

87.3 1.3 Build the bar + line plot

Now we combine:

- `geom_col()` for SUMLD (bars)
- `geom_line()` and `geom_point()` for individual lesions
- `scale_y_continuous()` with a **secondary axis** label (note: both axes use same numeric scale, only labels differ)

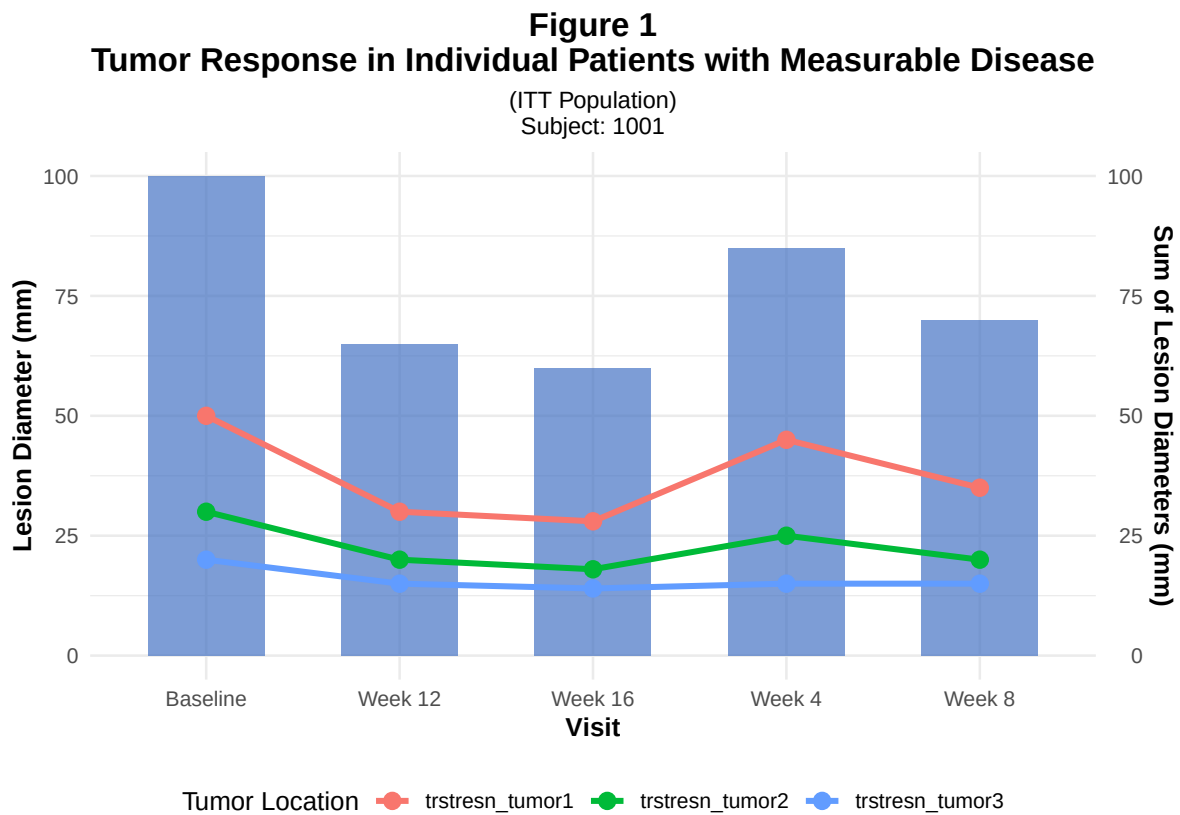
```
p1 <- ggplot() +
  # Bars = Sum of lesion diameters per visit
  geom_col(
    data = bar_data,
    aes(x = visit, y = SUMLD),
    fill = "#4472C4",
    width = 0.6,
    alpha = 0.7
  ) +
  # Lines = individual lesions
  geom_line(
    data = bar_data_long,
    aes(x = visit, y = trstresn,
        group = tuloc, color = tuloc),
    size = 1.2
  ) +
  # Points = individual lesion measurements
  geom_point(
    data = bar_data_long,
    aes(x = visit, y = trstresn,
        color = tuloc),
    size = 3
  ) +
  # Primary and secondary y-axes (same numeric scale)
  scale_y_continuous(
    name = "Lesion Diameter (mm)",
    sec.axis = sec_axis(~., name = "Sum of Lesion Diameters (mm)")
  ) +
  labs(
    title = "Figure 1"
```

```

Tumor Response in Individual Patients with Measurable Disease",
  subtitle = "(ITT Population)
Subject: 1001",
  x = "Visit",
  color = "Tumor Location"
) +
theme_minimal() +
theme(
  plot.title = element_text(hjust = 0.5, face = "bold", size = 14),
  plot.subtitle = element_text(hjust = 0.5, size = 10),
  axis.title = element_text(face = "bold", size = 11),
  legend.position = "bottom"
)

```

p1



Key points:

- We used an **empty** `ggplot()` and passed data separately to each layer.
 - This is useful when we have different data frames for bars (`bar_data`) and lines (`bar_data_long`).
-

88 Figure 2 – Waterfall Plot of Best Percentage Change from Baseline

Waterfall plots show each patient's **best % change from baseline** in tumor size. They're very common in oncology to visualize response categories like CR/PR/SD/PD.

88.1 2.1 Create waterfall data

We simulate:

- **patientid**: patient identifier
- **pchg**: best % change from baseline in sum of diameters
- **response**: categorical RECIST-like response (CR/PR/SD/PD)

```
waterfall_data <- data.frame(  
  patientid = paste0("PT", sprintf("%03d", 1:30)),  
  pchg = c(-65, -55, -48, -42, -38, -35, -30, -28, -25, -22,  
           -18, -15, -12, -10, -8, -5, -2, 5, 8, 12,  
           15, 20, 25, 30, 35, 40, 45, 50, 55, 60),  
  response = c(rep("CR", 2), rep("PR", 13), rep("SD", 10), rep("PD", 5))  
)  
  
waterfall_data
```

	patientid	pchg	response
1	PT001	-65	CR
2	PT002	-55	CR
3	PT003	-48	PR
4	PT004	-42	PR
5	PT005	-38	PR
6	PT006	-35	PR
7	PT007	-30	PR
8	PT008	-28	PR
9	PT009	-25	PR

10	PT010	-22	PR
11	PT011	-18	PR
12	PT012	-15	PR
13	PT013	-12	PR
14	PT014	-10	PR
15	PT015	-8	PR
16	PT016	-5	SD
17	PT017	-2	SD
18	PT018	5	SD
19	PT019	8	SD
20	PT020	12	SD
21	PT021	15	SD
22	PT022	20	SD
23	PT023	25	SD
24	PT024	30	SD
25	PT025	35	SD
26	PT026	40	PD
27	PT027	45	PD
28	PT028	50	PD
29	PT029	55	PD
30	PT030	60	PD

88.2 2.2 Sort patients by change

To get the typical waterfall shape (bars sorted from best to worst response), we sort by `pchg` and then set factor levels:

```
waterfall_data <- waterfall_data %>%
  arrange(pchg) %>%
  mutate(patientid = factor(patientid, levels = patientid))

response_colors <- c(
  "CR" = "#00B050",
  "PR" = "#92D050",
  "SD" = "#FFC000",
  "PD" = "#FF0000"
)

waterfall_data
```

```
patientid pchg response
```


1	PT001	-65	CR
2	PT002	-55	CR
3	PT003	-48	PR
4	PT004	-42	PR
5	PT005	-38	PR
6	PT006	-35	PR
7	PT007	-30	PR
8	PT008	-28	PR
9	PT009	-25	PR
10	PT010	-22	PR
11	PT011	-18	PR
12	PT012	-15	PR
13	PT013	-12	PR
14	PT014	-10	PR
15	PT015	-8	PR
16	PT016	-5	SD
17	PT017	-2	SD
18	PT018	5	SD
19	PT019	8	SD
20	PT020	12	SD
21	PT021	15	SD
22	PT022	20	SD
23	PT023	25	SD
24	PT024	30	SD
25	PT025	35	SD
26	PT026	40	PD
27	PT027	45	PD
28	PT028	50	PD
29	PT029	55	PD
30	PT030	60	PD

88.3 2.3 Build the waterfall plot

- `geom_col()` for bars
- `geom_hline()` to draw RECIST cut-off lines (e.g. -30%, +20%)
- `coord_cartesian()` to fix y-axis limits

```
p2 <- ggplot(waterfall_data, aes(x = patientid, y = pchg, fill = response)) +
  geom_col(width = 0.8) +
```

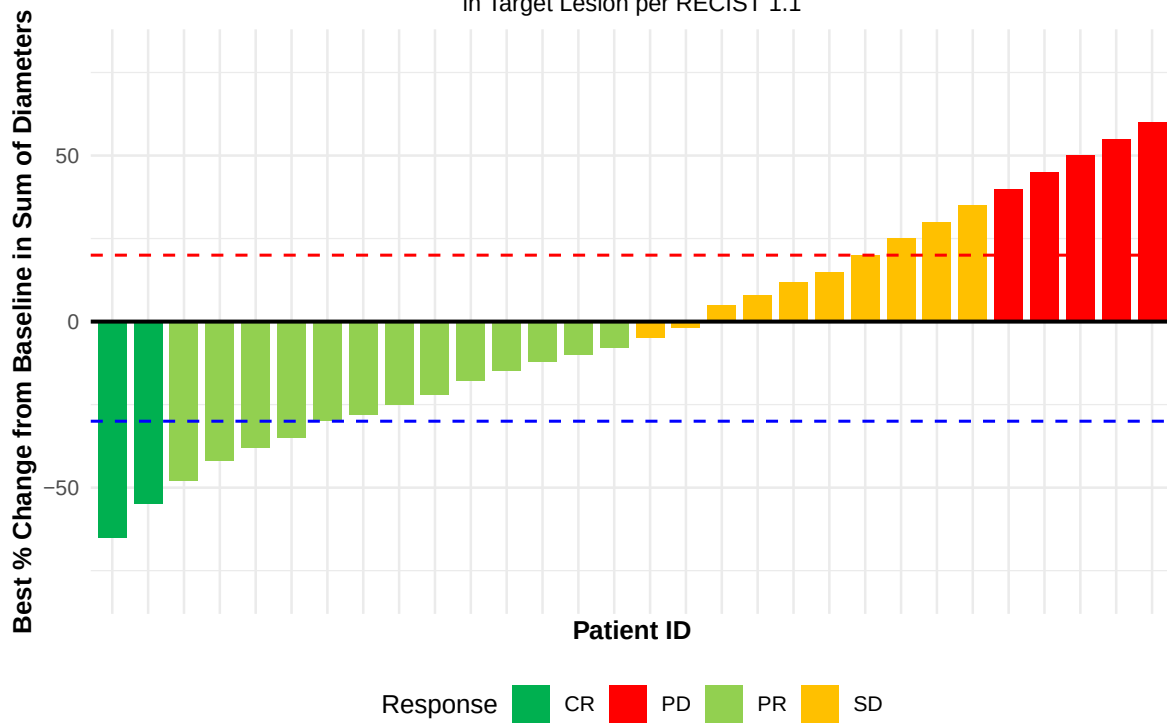
```

# Baseline line at 0% change
geom_hline(yintercept = 0, linetype = "solid", color = "black", size = 0.8) +
# Typical RECIST thresholds
geom_hline(yintercept = -30, linetype = "dashed", color = "blue", size = 0.6) +
geom_hline(yintercept = 20, linetype = "dashed", color = "red", size = 0.6) +
scale_fill_manual(values = response_colors) +
labs(
  title = "Figure 2
Waterfall Plot of Best Percentage Change from Baseline",
  subtitle = "in Target Lesion per RECIST 1.1",
  x = "Patient ID",
  y = "Best % Change from Baseline in Sum of Diameters",
  fill = "Response"
) +
theme_minimal() +
theme(
  plot.title = element_text(hjust = 0.5, face = "bold", size = 14),
  plot.subtitle = element_text(hjust = 0.5, size = 10),
  axis.title = element_text(face = "bold", size = 11),
  axis.text.x = element_blank(),
  axis.ticks.x = element_blank(),
  legend.position = "bottom"
) +
coord_cartesian(ylim = c(-80, 80))

```

p2

Figure 2
Waterfall Plot of Best Percentage Change from Baseline
 in Target Lesion per RECIST 1.1



In real ADaM:

- You'd compute `pchg` from ADTR using baseline and best post-baseline sum of diameters.

89 Figure 3 – Swimmer Plot: Duration of Response

Swimmer plots show **time on treatment** / **response** for each patient:

- Horizontal bar per patient: **treatment duration**
- Color: response category
- Symbols/markers: progression, death, etc.

89.1 3.1 Create swimmer data

We simulate:

- **start_time** / **end_time**: treatment/response duration (months)
- **response**: CR/PR/SD/PD
- **progression** / **death**: logical indicators

```
set.seed(123) # for reproducibility

swimmer_data <- data.frame(
  patientid = paste0("PT", sprintf("%03d", 1:20)),
  start_time = rep(0, 20),
  end_time = c(12, 15, 8, 20, 18, 14, 10, 22, 16, 19,
               7, 25, 13, 17, 11, 9, 21, 24, 15, 18),
  response = sample(c("CR", "PR", "SD", "PD"), 20, replace = TRUE),
  progression = c(TRUE, FALSE, TRUE, FALSE, TRUE, FALSE, TRUE, FALSE, TRUE, FALSE,
                  TRUE, FALSE, TRUE, FALSE, TRUE, FALSE, TRUE, FALSE, TRUE, FALSE),
  death = c(FALSE, TRUE, FALSE, FALSE, FALSE, TRUE, FALSE, FALSE, FALSE, TRUE,
            FALSE, FALSE, FALSE, TRUE, FALSE, FALSE, FALSE, TRUE, FALSE, FALSE)
)

# Sort by duration so longest bars at top
swimmer_data <- swimmer_data %>%
```

```

arrange(desc(end_time)) %>%
mutate(patientid = factor(patientid, levels = patientid))

swimmer_data

```

	patientid	start_time	end_time	response	progression	death
1	PT012	0	25	PR	FALSE	FALSE
2	PT018	0	24	CR	FALSE	TRUE
3	PT008	0	22	PR	FALSE	FALSE
4	PT017	0	21	PD	TRUE	FALSE
5	PT004	0	20	PR	FALSE	FALSE
6	PT010	0	19	CR	FALSE	TRUE
7	PT005	0	18	SD	TRUE	FALSE
8	PT020	0	18	SD	FALSE	FALSE
9	PT014	0	17	CR	FALSE	TRUE
10	PT009	0	16	SD	TRUE	FALSE
11	PT002	0	15	SD	FALSE	TRUE
12	PT019	0	15	SD	TRUE	FALSE
13	PT006	0	14	PR	FALSE	TRUE
14	PT013	0	13	PR	TRUE	FALSE
15	PT001	0	12	SD	TRUE	FALSE
16	PT015	0	11	PR	TRUE	FALSE
17	PT007	0	10	PR	TRUE	FALSE
18	PT016	0	9	SD	FALSE	FALSE
19	PT003	0	8	SD	TRUE	FALSE
20	PT011	0	7	PD	TRUE	FALSE

89.2 3.2 Build the swimmer plot

- `geom_segment()` draws horizontal bars
- Triangles (`shape = 17`) mark progression
- Crosses (`shape = 4`) mark death

```

p3 <- ggplot(swimmer_data, aes(y = patientid)) +
  # Treatment/response duration
  geom_segment(
    aes(x = start_time, xend = end_time,
        yend = patientid, color = response),

```

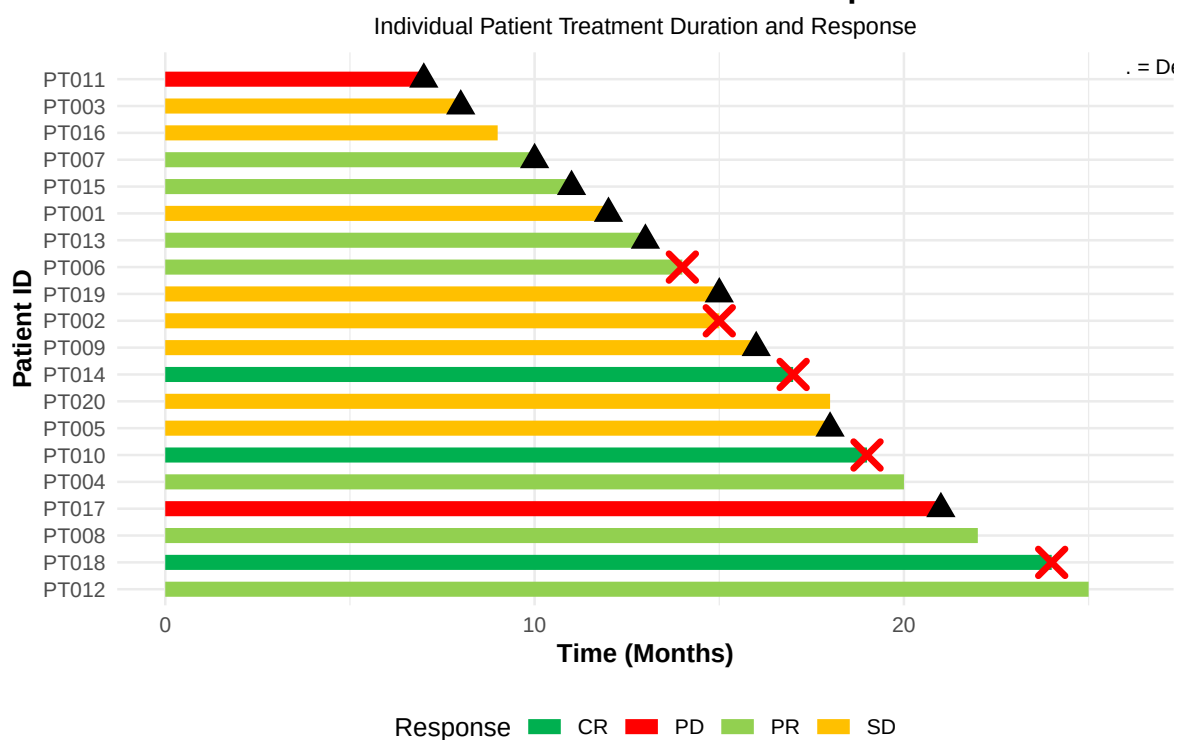
```

    size = 3
  ) +
  # progression (black triangle)
  geom_point(
    data = swimmer_data %>% filter(progression),
    aes(x = end_time),
    shape = 17, size = 4, color = "black"
  ) +
  # death (red cross)
  geom_point(
    data = swimmer_data %>% filter(death),
    aes(x = end_time),
    shape = 4, size = 4, color = "red", stroke = 2
  ) +
  scale_color_manual(values = c(
    "CR" = "#00B050", "PR" = "#92D050",
    "SD" = "#FFC000", "PD" = "#FF0000"
  )) +
  labs(
    title = "Figure 3
Swimmer Plot - Duration of Response",
    subtitle = "Individual Patient Treatment Duration and Response",
    x = "Time (Months)",
    y = "Patient ID",
    color = "Response"
  ) +
  theme_minimal() +
  theme(
    plot.title = element_text(hjust = 0.5, face = "bold", size = 14),
    plot.subtitle = element_text(hjust = 0.5, size = 10),
    axis.title = element_text(face = "bold", size = 11),
    legend.position = "bottom"
  ) +
  annotate(
    "text", x = 26, y = 21,
    label = " = Progression
= Death",
    hjust = 0, size = 3
  )

```

p3

Figure 3
Swimmer Plot – Duration of Response



In an ADaM workflow, these durations often come from ADTTE / custom response datasets (first response date, progression date, death date, etc.).

90 Figure 4 – Spider Plot: Tumor Trajectories Over Time

Spider plots show multiple **patients' tumor trajectories** on the same axes:

- x-axis: time (e.g. months from baseline scan)
- y-axis: relative tumor size (ratio vs baseline)

Each line represents one patient.

90.1 4.1 Create spider data

We create data for 15 patients, with tumor size values over time:

```
set.seed(123)

spider_data <- expand_grid(
  patientid = paste0("PT", sprintf("%03d", 1:15)),
  dur = seq(-10, 100, by = 10)
) %>%
  group_by(patientid) %>%
  mutate(
    # Simple random trend around baseline = 1
    size = 1 + rnorm(1, 0, 0.3) + (dur/100) * rnorm(1, -0.5, 0.4)
  ) %>%
  ungroup()

spider_data
```

```
# A tibble: 180 x 3
  patientid  dur  size
  <fct>      <dbl> <dbl>
1 PT001     -10 0.891
```



```

2 PT002      -10 1.51
3 PT003      -10 1.02
4 PT004      -10 1.24
5 PT005      -10 0.862
6 PT006      -10 1.40
7 PT007      -10 1.17
8 PT008      -10 0.812
9 PT009      -10 1.28
10 PT010     -10 1.28
# i 170 more rows

```

- At `dur = 0` we are around `size = 1` (baseline).
- Later timepoints can go up (> 1) or down (< 1).

90.2 4.2 Build the spider plot

```

p4 <- ggplot(
  spider_data,
  aes(
    x = dur, y = size,
    group = patientid,
    color = patientid
  )
) +
  geom_line(size = 1.2, alpha = 0.7) +
  geom_point(size = 2, alpha = 0.8) +
  # Horizontal line at baseline (1.0)
  geom_hline(
    yintercept = 1.0, linetype = "solid",
    color = "black", size = 0.8
  ) +
  scale_y_continuous(breaks = seq(0, 2, by = 0.20)) +
  scale_x_continuous(breaks = seq(-100, 100, by = 10)) +
  labs(
    title = "Figure 4
Relative Change in Tumor Size",
    x = "Months from Baseline Scan (Time 0)",
    y = "Tumor Size Relative to Baseline"
  ) +

```

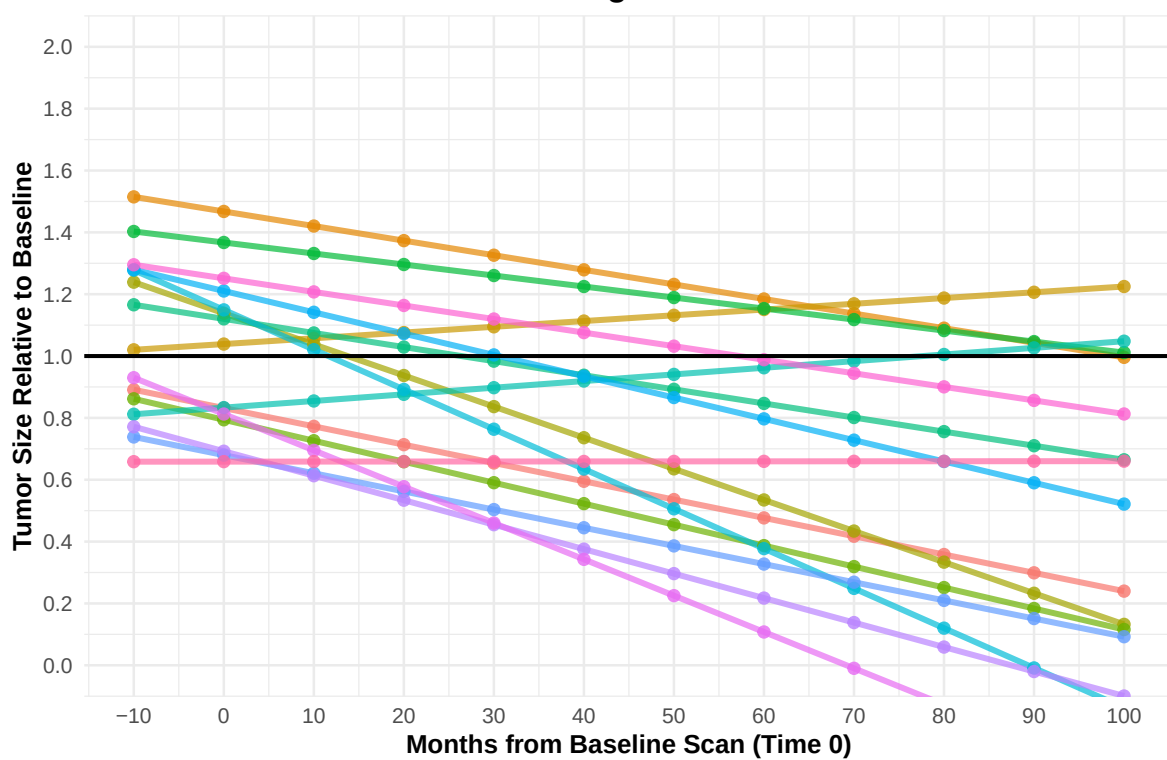
```

theme_minimal() +
theme(
  plot.title = element_text(hjust = 0.5, face = "bold", size = 14),
  axis.title = element_text(face = "bold", size = 11),
  legend.position = "none"
) +
coord_cartesian(xlim = c(-10, 100), ylim = c(0, 2))

```

p4

Figure 4
Relative Change in Tumor Size



In practice, this would also come from **ADTR** (longitudinal tumor measurements) with a derived `relative_size = AVAL / BASE`.

91 Figure 5 – Kaplan–Meier Survival Curve

Kaplan–Meier (KM) plots visualize **time-to-event endpoints** like PFS or OS:

- Step function for survival probability over time
- Separate curve for each treatment arm
- Optionally: confidence intervals and number-at-risk table

91.1 5.1 Create KM data

We simulate:

- **time**: event/censoring time
- **status**: 1 = event, 0 = censored
- **treatment**: Treatment A / Treatment B

```
set.seed(456)

km_data <- data.frame(
  time = c(rexp(100, 0.05), rexp(100, 0.03)),
  status = c(rbinom(100, 1, 0.7), rbinom(100, 1, 0.6)),
  treatment = rep(c("Treatment A", "Treatment B"), each = 100)
)

head(km_data)
```

	time	status	treatment
1	50.245343	1	Treatment A
2	41.407858	0	Treatment A
3	9.318211	1	Treatment A
4	4.601942	1	Treatment A

```
5 47.967306      1 Treatment A
6 16.705099      1 Treatment A
```

91.2 5.2 Fit the KM model and plot

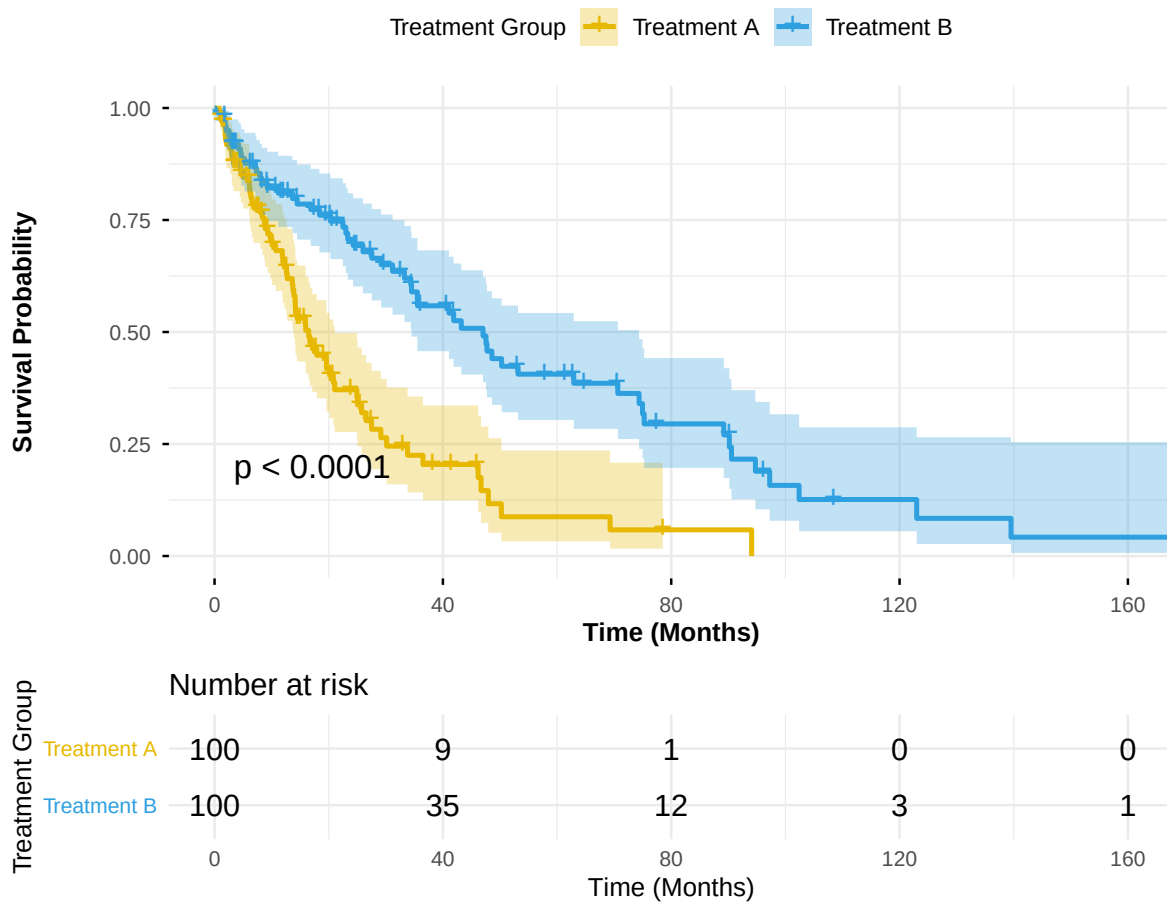
We use `survival::survfit()` and `survminer::ggsurvplot()`:

```
km_fit <- survfit(Surv(time, status) ~ treatment, data = km_data)

p5 <- ggsurvplot(
  km_fit,
  data = km_data,
  pval = TRUE,          # log-rank p-value
  conf.int = TRUE,      # confidence interval bands
  risk.table = TRUE,    # number at risk under the plot
  risk.table.height = 0.25,
  ggtheme = theme_minimal(),
  palette = c("#E7B800", "#2E9FDF"),
  title = "Figure 5
Kaplan-Meier Survival Curve",
  xlab = "Time (Months)",
  ylab = "Survival Probability",
  legend.title = "Treatment Group",
  legend.labs = c("Treatment A", "Treatment B"),
  font.main = c(14, "bold"),
  font.x = c(11, "bold"),
  font.y = c(11, "bold"),
  font.legend = c(10)
)

p5
```

Figure 5
Kaplan–Meier Survival Curve



In ADaM:

- This would typically be based on **ADTTE**, using variables like **AVAL** (time), **CNSR** (censor flag), and **TRT01A** or similar.

92 Figure 6 – Forest Plot for Subgroup Hazard Ratios

Forest plots summarize **effect estimates** (e.g. **hazard ratios**) across subgroups:

- Each row: one subgroup
- Point: HR
- Horizontal line: 95% CI
- Vertical reference line at $HR = 1$

Here we use the `forestploter` package and its example data.

92.1 6.1 Load example data and prepare

```
dt <- read.csv(system.file("extdata", "example_data.csv", package = "forestploter"))  
head(dt)
```

	Subgroup	Treatment	Placebo	est	low	hi	low_gp1	
1	All Patients	781	780	1.869694	0.13245636	3.606932	0.1507971	
2	Sex	NA	NA	NA	NA	NA	NA	
3	Male	535	548	1.449472	0.06834426	2.830600	1.9149515	
4	Female	246	232	2.275120	0.50768005	4.042560	0.6336414	
5	Age	NA	NA	NA	NA	NA	NA	
6	<65 yr	297	333	1.509242	0.67029394	2.348190	1.7679431	
	low_gp2	low_gp3	low_gp4	est_gp1	est_gp2	est_gp3	est_gp4	hi_gp1
1	0.35443249	0.3939730	1.1515801	1.181926	1.5163554	1.612725	1.949433	2.924330
2	NA	NA	NA	NA	NA	NA	NA	NA
3	0.09953409	0.3803214	0.3213258	3.266615	0.8291053	1.642294	1.607950	3.996409
4	2.57367694	1.0229365	0.3510777	1.971712	3.9719741	1.751471	1.742973	3.965617
5	NA	NA	NA	NA	NA	NA	NA	NA

6	0.57329716	0.8433183	2.1637576	2.447051	1.9990500	1.390913	3.136695	3.674451
	hi_gp2	hi_gp3	hi_gp4					
1	2.919964	3.485182	2.862447					
2	NA	NA	NA					
3	1.553593	2.573538	2.228632					
4	5.439582	3.674689	3.249342					
5	NA	NA	NA					
6	3.399146	2.346744	4.299211					

We:

- Indent subgroup labels when there is a number in **Placebo** column (for visual hierarchy).
- Replace NA with empty strings for display columns.
- Compute standard error (**se**) from CI.
- Add a blank column (" ") for the plot.
- Create a text column for **HR (95% CI)**.

```
# Indent subgroup if there is a number in the placebo column
dt$Subgroup <- ifelse(
  is.na(dt$Placebo),
  dt$Subgroup,
  paste0(" ", dt$Subgroup)
)

# NA to blank for display columns
dt$Treatment <- ifelse(is.na(dt$Treatment), "", dt$Treatment)
dt$Placebo <- ifelse(is.na(dt$Placebo), "", dt$Placebo)

# Standard error derived from CI on log scale
dt$se <- (log(dt$hi) - log(dt$est)) / 1.96

# Blank column used by forestploter for CI area
dt$` ` <- paste(rep(" ", 20), collapse = " ")

# Display column for HR (95% CI)
dt$`HR (95% CI)` <- ifelse(
  is.na(dt$se), "",
  sprintf("%.2f (%.2f to %.2f)", dt$est, dt$low, dt$hi)
)
```

```
)
```

```
head(dt)
```

```
      Subgroup Treatment Placebo      est      low      hi      low_gp1
1 All Patients      781      780 1.869694 0.13245636 3.606932 0.1507971
2      Sex              NA      NA      NA      NA
3      Male      535      548 1.449472 0.06834426 2.830600 1.9149515
4      Female      246      232 2.275120 0.50768005 4.042560 0.6336414
5      Age              NA      NA      NA      NA
6      <65 yr      297      333 1.509242 0.67029394 2.348190 1.7679431
      low_gp2 low_gp3 low_gp4 est_gp1 est_gp2 est_gp3 est_gp4 hi_gp1
1 0.35443249 0.3939730 1.1515801 1.181926 1.5163554 1.612725 1.949433 2.924330
2      NA      NA      NA      NA      NA      NA      NA      NA
3 0.09953409 0.3803214 0.3213258 3.266615 0.8291053 1.642294 1.607950 3.996409
4 2.57367694 1.0229365 0.3510777 1.971712 3.9719741 1.751471 1.742973 3.965617
5      NA      NA      NA      NA      NA      NA      NA      NA
6 0.57329716 0.8433183 2.1637576 2.447051 1.9990500 1.390913 3.136695 3.674451
      hi_gp2 hi_gp3 hi_gp4      se
1 2.919964 3.485182 2.862447 0.3352463
2      NA      NA      NA      NA
3 1.553593 2.573538 2.228632 0.3414741
4 5.439582 3.674689 3.249342 0.2932884
5      NA      NA      NA      NA
6 3.399146 2.346744 4.299211 0.2255292
      HR (95% CI)
1 1.87 (0.13 to 3.61)
2
3 1.45 (0.07 to 2.83)
4 2.28 (0.51 to 4.04)
5
6 1.51 (0.67 to 2.35)
```

92.2 6.2 Define forest theme and build the plot

We define a theme (fonts, refine color, arrow labels) and then call `forest()`:

```
tm <- forest_theme(
  base_size = 10,
  refline_col = "red",
```



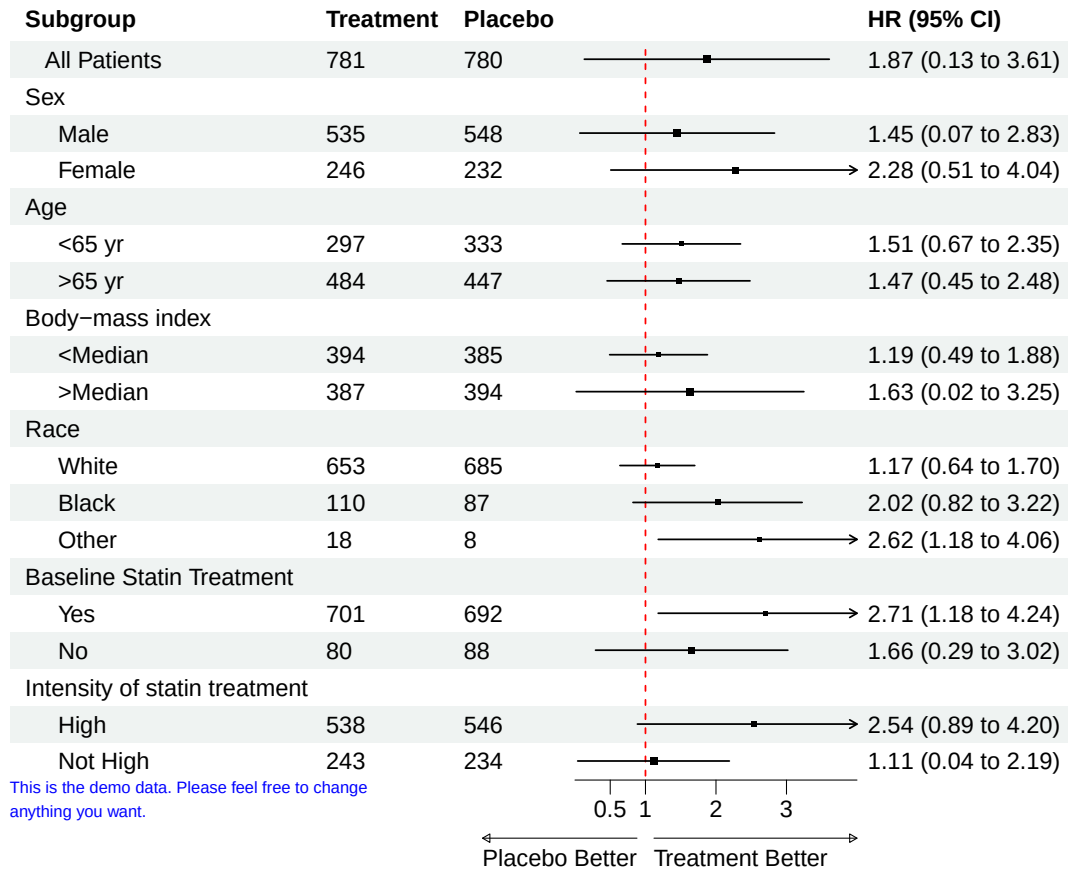
```

    arrow_type = "closed",
    footnote_gp = gpar(col = "blue", cex = 0.6)
)

p <- forest(
  dt[, c(1:3, 20:21)],
  est      = dt$est,
  lower    = dt$low,
  upper    = dt$hi,
  sizes    = dt$se,
  ci_column = 4,
  ref_line  = 1,
  arrow_lab = c("Placebo Better", "Treatment Better"),
  xlim      = c(0, 4),
  ticks_at  = c(0.5, 1, 2, 3),
  footnote  = "This is the demo data. Please feel free to change
anything you want.",
  theme     = tm
)

plot(p)

```



Interpretation:

- Values < 1 favor **treatment**, values > 1 favor **placebo** (depending on how you define HR).
- The arrow labels at the bottom make this direction explicit.

In real analyses, the columns **est**, **low**, **hi** would be generated from Cox models or other regressions per subgroup.

93 Summary and Next Steps

In this tutorial, you:

- Learned the **ggplot2 grammar of graphics** using **ADSL, ADLB, and ADTTE** style data
- Built common clinical plots:
 - Subject counts by arm
 - Histograms/boxplots of demographics
 - Longitudinal lab plots with summary statistics
 - Simple KM curves from ADTTE
- Customized scales, themes, coordinates, and annotations
- Combined **dplyr + ggplot2** for tidy ADaM workflows
- Wrote a small **reusable plotting function** tailored to ADaM

From here, you can:

- Swap the simulated data with your real **ADaM datasets**
- Extend functions for specific TLFs (e.g., safety labs, efficacy endpoints)
- Integrate these plots into **R Markdown/Quarto CSRs** or **Shiny dashboards** for interactive review.

Happy plotting with ggplot2 and ADaM!

94 Capstone: End-to-End Mini Workflow

This chapter ties everything together: **read data** → **derive ADSL** → **produce TLFs** → **render a report**.

94.1 Parameters

```
# You could parametrize paths via YAML; here we keep inline defaults.
dm_path <- "data/sdtm/dm.sas7bdat"
ex_path <- "data/sdtm/ex.sas7bdat"
```

94.2 1) Read (or Synthesize) SDTM

```
library(haven); library(dplyr); library(lubridate)
```

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

filter, lag

The following objects are masked from 'package:base':

intersect, setdiff, setequal, union

Attaching package: 'lubridate'

The following objects are masked from 'package:base':

date, intersect, setdiff, union

```
if (file.exists(dm_path)) {
  dm <- read_sas(dm_path)
} else {
  dm <- tibble::tibble(
    STUDYID = "XYZ123",
    USUBJID = sprintf("XYZ-%03d", 1:60),
    ARM = sample(c("Placebo", "Active"), 60, replace=TRUE),
    AGE = round(rnorm(60, 60, 8)),
    SEX = sample(c("M", "F"), 60, replace=TRUE),
    RANDDT = as.Date("2025-01-15") + sample(0:40, 60, replace=TRUE)
  )
}
if (file.exists(ex_path)) {
  ex <- read_sas(ex_path)
} else {
  ex <- tibble::tibble(
    USUBJID = dm$USUBJID,
    EXSTDTC = dm$RANDDT + sample(0:3, nrow(dm), replace=TRUE)
  )
}
```

94.3 2) Derive ADSL (Minimal Demo)

```
adsl <- dm |>
  left_join(ex, by="USUBJID") |>
  mutate(
    TRT01P = ARM,
    TRT01PN = as.integer(factor(ARM, levels=c("Placebo", "Active"))),
    TRT01A = TRT01P,
    TRT01AN = TRT01PN,
    SAFFL = "Y",          # demo only; define rules in real life
    FASFL = "Y"
  ) |>
  dplyr::select(STUDYID.x, USUBJID, TRT01P, TRT01PN, TRT01A, TRT01AN, AGE, SEX, EXSTDTC, SAF
```

[				
Table	1.	Baseline	by	Treatment
Table	1.	Baseline	by	Treatment
Description of Planned Arm	N	mean_age	sd_age	pct_female
Placebo	226	75.04867	8.503715	60.61947
Screen Failure	52	75.09615	9.699928	69.23077
Xanomeline High Dose	184	74.01087	7.939656	48.36957
Xanomeline Low Dose	181	75.29834	8.277778	60.77348

94.4 3) TLFs

```
library(gt); library(ggplot2); library(survival)
```

```
tbl1 <- adsl |>
  group_by(TRT01P) |>
  summarise(N=n(),
            mean_age = mean(AGE), sd_age = sd(AGE),
            pct_female = mean(SEX=="F")*100)
tbl1_gt <- gt(tbl1) |> tab_header(title="Table 1. Baseline by Treatment")
tbl1_gt
```

```
set.seed(123)
adsl$time <- rexp(nrow(adsl), rate=ifelse(adsl$TRT01P=="Active", 0.08, 0.1))
adsl$status <- rbinom(nrow(adsl), 1, 0.7)
fit <- survfit(Surv(time, status) ~ TRT01P, data=adsl)
# reuse plotting function from prior chapter
ggsurv <- function(fit) {
  ss <- summary(fit)
  dd <- data.frame(time=ss$time, surv=ss$surv, strata=rep(names(fit$strata), fit$strata))
  ggplot(dd, aes(x=time, y=surv, linetype=strata)) + geom_step() + theme_minimal() +
    labs(title="KM Curve (Toy)", x="Time", y="Survival", linetype="Treatment")
}
#ggsurv(fit)
```

94.5 4) Save Outputs

```
# Example: Save Table 1 as PNG  
#gtsave(tbl1_gt, "tlf-table1.png")
```

Challenge: Convert this chapter into a **parameterized report** (e.g., treatment subset or different cohort) and render multiple outputs.

95 Appendix: Tips, Profiles, .libPaths

95.1 Useful Profiles

Create ~/.Rprofile to set options (be careful on shared systems):

```
options(  
  repos = c(CRAN = "https://cloud.r-project.org"),  
  scipen = 999  
)
```

95.2 Custom Library Paths

```
# In .Rprofile or project-level .Rprofile  
.libPaths(c("/path/to/Rlibs", .libPaths()))
```

95.3 Format vs formatC (quick recap)

```
x <- c(123.456, 0.00123456)  
format(x, digits = 4)
```

```
[1] "1.235e+02" "1.235e-03"
```

```
format(x, nsmall = 2)
```

```
[1] "1.23456e+02" "1.23456e-03"
```

```
formatC(x, digits = 3, format = "f")
```

```
[1] "123.456" "0.001"
```


95.4 POSIXct vs POSIXlt

- **POSIXct**: seconds since epoch (numeric), compact, fast.
- **POSIXlt**: list-like with components (year, mon, mday...), easier to extract parts.

95.5 Recommended Packages

- tidyverse, lubridate, janitor, gt, gtsummary, survival, broom, here.
- Pharma/CDISC: admiral, tlf/tern, pharmaverse meta-packages (explore as you grow).

95.6 Short Glossary

- **SDTM**: Study Data Tabulation Model (FDA submission standard for raw domains).
- **ADaM**: Analysis Data Model (derived analysis-ready datasets).
- **TLF**: Tables, Listings, Figures for reporting.